

Introduction to the Java Universe

In the last few years, Java became the standard in programming due to its features and facilities. Java is completely object-oriented. Java programs are portable, can run on a wide variety of hardware platforms, can simultaneously work on multiple tasks and can be loaded dynamically via network.

The purpose of this book is to help you learn the essentials of Java programming in a systematic way, rather than in a "Learn Java in ... days" manner. It is based on the author's experience in teaching Java at some Romanian universities. The intended audience is programmers of all levels who need to learn and use Java.

The book is structured in three parts.

The first 7 chapters present the basics of Java programming: statements, classes and interfaces, methods, code organization, access and visibility, inheritance, exceptions and conversions.

In the second part (Chapters 8 - 11) more advanced facilities are included: standard utilities, input/output streams, graphic interfaces and threads as a basis for parallel and concurrent programming.

The last part of the book is dedicated to distributed programming. The main topics are object serialization, communication via sockets, remote method invocation and applets.

The book is both for novice and intermediate programmers. However, basic knowledge on programming languages, operating systems and Web communication is supposed.

The author welcomes your feedback about this book, especially if you spot errors; he can be contacted at hg@phobos.cs.unibuc.ro.

INTRODUCERE în UNIVERSUL JAVA

CIPRIAN COMȘA
Nr. - 106 -



EDITURA TEHNICĂ
București, 2002

Copyright © 2002, S.C. Editura TEHNICĂ S.A.
Toate drepturile asupra acestei ediții sunt rezervate editurii.

Adresă: S.C. Editura TEHNICĂ S.A.
Piața Presei Libere 1
33 București, România
cod 71341

Coperta colecției: Andrei MĂNESCU

Redactor : Adina IONESCU
Tehnoredactor: Andreea Cristina RĂDUȚ
Copertă: Andreea Cristina RĂDUȚ

Bun de tipar: 19.VI.2002; coli de tipar: 19
C.Z.U.: 621.377

ISBN 973-31-2095-2

PREFAȚĂ

În mod obișnuit, o prefață începe prin a convinge că subiectul tratat este important. Acest lucru este în cazul de față inutil. Java s-a impus rapid ca un standard în programare, așa cum la timpul lor au fost limbajele Fortran, Pascal, C și C++.

Java este un limbaj complet orientat pe obiecte. Programele Java sunt portabile, pot fi executate pe o largă gamă de platforme, pot executa simultan mai multe procese și pot fi încărcate dinamic prin rețele. Aceste caracteristici, dublate de o mare varietate de facilități puse la dispoziția utilizatorului, au făcut ca Java să devină un lider în multitudinea limbajelor de programare existente.

De fapt Java este mai mult decât un limbaj sau mediu de programare: este un mediu de executare din care putem întreprinde acțiuni specifice sistemelor de programare, putem lansa fire de executare, putem executa pe mașina proprie programe aflate pe o altă mașină și putem comanda unei alte mașini să execute diverse programe pentru date pe care i le punem la dispoziție.

Cartea se bazează pe cursurile ținute de autor timp de mai mulți ani la Universitatea din București, Academia de Studii Economice București, precum și la Universitatea Ovidius din Constanța. Observațiile studenților au fost totdeauna binevenite și luate în considerare.

Stilul imprimat cărții este preponderent didactic. Nu se urmărește dobândirea de cunoștințe aproximative în 8, 12 sau 16 lecții, ci se încearcă o introducere sistematică a noțiunilor, prin care cititorul să înțeleagă bine mecanismele folosite de Java. Accentul este pus în primul rând pe aceste mecanisme și nu pe simpla ilustrare a noțiunilor prin exemple. De aceea nu și-au găsit loc în această carte multe instrumente uzuale pentru un programator avansat în Java: crearea și exploatarea bazelor de date, arhive JAR, servlet-uri, interfețe grafice ce utilizează Swing, Java Beans, CORBA și multe altele. Am preferat să nu le includem decât să facem o prezentare superficială a lor. Unele dintre aceste subiecte pot face obiectul unei cărți de sine stătătoare și desigur binevenită.

S-a urmărit folosirea unui limbaj cât mai simplu, scopul fiind de a-l atrage pe cititor și nu de a-l convinge de erudiția autorului.

Direcțiile dezvoltate și alegerea exemplelor au inerent un caracter subiectiv. În această carte ele corespund domeniilor de interes ale autorului: tehnici de programare și programare concurrentă, paralelă și distribuită.

Programele au fost scrise în Java 2 SDK versiunea 1.3.0, folosind sistemul de operare Windows.

Cartea este structurată în trei părți.

Prima parte, cuprinzând 7 capitole, încearcă o prezentare sistematică a elementelor de bază ale limbajului Java. Sunt introduse noțiunile fundamentale de clasă, metodă, obiect, variabilă, operatori și instrucțiuni. Este prezentat apoi modul în care putem structura codul în unități de compilare și pachete. O atenție deosebită este acordată mecanismului de extindere a claselor, parte esențială a paradigmei programării orientate pe obiecte. Sunt studiate metodele și clasele abstracte, interfețele și metodele interne. Este explicat mecanismul tratării excepțiilor. În această primă parte cuvintele cheie apar îngroșate.

A doua parte conține elemente avansate de programare în Java. În Capitolul 8 sunt prezentate principalele facilități standard puse la dispoziție de Java. Capitolul 9 explică modul în care pot fi create și lansate firele de executare, cu aplicații la programarea paralelă și concurrentă. Fluxurile de intrare/ieșire fac obiectul următorului capitol; s-a urmărit o prezentare sistematică a lor, pentru a "naviga" ușor în foarte multe facilități puse la dispoziție. În capitolul 10 sunt studiate interfețele grafice, punându-se accent pe mecanismele de bază ale programării orientate pe evenimente.

Ultima parte a cărții este dedicată programării distribuite. După prezentarea mecanismului de serializare a obiectelor, este descris modul în care putem comunica în rețea cu ajutorul socket-urilor; sunt incluse mai multe aplicații standard, precum și modul de utilizare a semnăturilor digitale. În capitolul 14 este descris modelul RMI pentru invocarea la distanță a metodelor. Ultimul capitol tratează applet-urile, punând în evidență facilitățile, dar și restricțiile la care este supus lucrul cu aceste miniaplicații.

Cartea se adresează unui public foarte larg. Se cer cunoștințe minime în limbaje de programare (inclusiv recursivitate), sisteme de operare și navigare pe Internet. De exemplu un absolvent al clasei a X-a de la profilul Matematică-Informatică are o bază suficientă pentru abordarea cărții.

Autorul așteaptă cu interes opiniile cititorilor (mai ales pe cele critice!) la adresa: hg@phobos.cs.unibuc.ro.

Prof. univ. dr. Horia Georgescu

Februarie, 2002

CUPRINS

1. O PRIMĂ INCURSIUNE ÎN LIMBAJUL JAVA	11
1.1. Introducere	11
1.2. Un prim program scris în Java	14
1.3. O clasă folosită pentru intrările și ieșirile standard	15
1.4. Clase, câmpuri și metode	17
1.5. Simularea înregistrărilor în Java	21
1.6. Exemplu: crearea unei liste înlănțuite	22
1.7. Introducere în programarea orientată pe obiecte (OOP)	24
2. ELEMENTE DE BAZĂ ALE LIMBAJULUI JAVA	27
2.1. Caractere	27
2.2. Identificatori	28
2.3. Constante	28
2.4. Comentarii	29
2.5. Variabile	30
2.6. Operatori	32
2.7. Instrucțiuni	36
2.8. Despre recursivitate în limbajul Java	40
3. ALTE ELEMENTE DE BAZĂ ALE LIMBAJULUI JAVA. APLICAȚII	43
3.1. Lucrul cu tablouri în Java	43
3.2. Aplicația 1 : Parcurgerea în adâncime a unui graf neorientat	46
3.3. Referința this și blocuri de inițializare	49
3.4. Inițializarea claselor	50
3.5. Aplicația 2 : Generarea tuturor (n,k) - combinațiilor	51

4. PACHETE. ACCES ȘI VIZIBILITATE	55
4.1. Pachete	55
4.2. Vizibilitate și acces	60
5. EXTINDEREA CLASELOR	65
5.1. Cum se definesc clasele extinse	65
5.2. Din nou despre inițializare și constructori	67
5.3. Rescrierea metodelor și ascunderea câmpurilor	70
5.4. Cuvântul cheie <code>super</code>	73
5.5. Moștenire, polimorfism și legare dinamică	74
5.6. Din nou despre modificatori	76
6. METODE ȘI CLASE ABSTRACTE. INTERFEȚE. CLASE INTERNE	80
6.1. Metode și clase abstracte	80
6.2. Noțiunea de interfață	81
6.3. Extinderea interfețelor	83
6.4. Implementarea interfețelor	85
6.5. Inițializarea interfețelor	86
6.6. Rezolvarea în Java a problemei moștenirii multiple	87
6.7. Clonarea obiectelor	89
6.8. Clase interne	92
7. EXCEPȚII. CONVERSII	97
7.1. Excepții	97
7.2. Conversii	106
8. FACILITĂȚI STANDARD	109
8.1. Funcții matematice	109
8.2. Clasa <code>Random</code>	110
8.3. Clasele înfășurătoare pentru tipurile primitive	110
8.4. Aritmetica numerelor mari	112
8.5. Lucrul cu șiruri de caractere	113
8.6. Prelucrări asupra tablourilor	117
8.7. Organizări ale datelor	119
8.8. Lucrul cu clase. Clasele <code>Class</code> și <code>Method</code>	128
8.9. Clasa <code>System</code>	132

9. FIRE DE EXECUTARE. PROGRAMARE PARALELĂ ȘI CONCURENȚĂ	134
9.1. Crearea firelor de executare	135
9.2. Un prim mod de sincronizare	138
9.3. Despre programarea paralelă	139
9.4. Implementarea în Java a metodei arborelui binar	141
9.5. Despre programarea concurentă	144
9.6. Rezolvarea excluderii reciproce în Java	147
9.7. Primitive de sincronizare. Problema Producător - Consumator	150
9.8. Lucrul cu semafoare în Java	153
9.9. Alte facilități oferite de clasa <code>Thread</code>	158
10. FACILITĂȚI DE INTRARE/IEȘIRE	160
10.1. Schema generală. Fluxuri de intrare/ieșire	160
10.2. Fluxuri ce lucrează la nivel de octet	163
10.2.1. O structură simplificată de clase	163
10.2.2. O structură extinsă de clase	171
10.3. Fluxuri ce lucrează la nivel de caracter	178
10.3.1. O structură simplificată de clase	178
10.3.2. O structură extinsă de clase	182
10.4. Analizorul lexical <code>StreamTokenizer</code>	187
10.5. Fișiere cu acces direct. Clasa <code>RandomAccessFile</code>	190
10.6. Clasa <code>File</code>	192
11. INTERFEȚE GRAFICE	194
11.1. Un prim exemplu	194
11.2. Schema generală de lucru cu AWT	200
11.3. Containere. Fonturi și culori	207
11.4. Componente elementare	210
11.5. Gestionari de poziționare	225
11.6. Evenimente	230
12. SERIALIZAREA OBIECTELOR	233
12.1. Necesitatea serializării și deserializării obiectelor	233
12.2. Un prim exemplu	234
12.3. Ce se transmite la serializare?	236
12.4. Interfețe și clase folosite la serializare/deserializare	238
12.5. Serializarea claselor ce extind alte clase și implementează interfețe	240
12.6. Cazul claselor ale căror superclase sunt neserializabile	241

13. SISTEME DISTRIBUITE. SOCKET-URI	244
13.1. Sisteme distribuite	244
13.2. Comunicare prin socket-uri	249
13.3. Aplicații	252
13.4. Semnarea digitală a mesajelor	259
14. INVOCAREA LA DISTANȚĂ A METODELOR (RMI)	263
14.1. Introducere	263
14.2. Principiile modelului RMI	266
14.3. Facilități Java pentru invocarea la distanță	270
14.4. Apeluri inverse	273
14.5. Încărcarea dinamică a claselor	276
15. PE SCURT DESPRE APPLET-URI	283
15.1. Ce sunt applet-urile?	283
15.2. Facilități de lucru cu applet-urile	287
15.3. Lucrul cu imagini și fișere audio	291
15.4. Comunicarea între applet-uri	292
15.5. Restricții în lucrul cu applet-uri	297
BIBLIOGRAFIE SELECTIVĂ	298
INDEX	299

1 O PRIMĂ INCURSIUNE ÎN LIMBAJUL JAVA

1.1. Introducere

În anul 1990 firma *Sun Microsystems* a început cercetările pentru proiectarea unui nou limbaj de programare, care să aducă un plus de flexibilitate, să fie portabil și să înlăture inconvenientele constatate la limbajele de programare folosite la acel moment. La acest proiect a lucrat o întreagă echipă de cercetători, în care personajele principale au fost *James Gosling*, *Bill Joy* și *Patrick Naughton*.

Firma Sun a făcut publică specificația limbajului în anul 1995. De atunci limbajul Java s-a impus rapid și a cucerit o largă recunoaștere și utilizare.

Atragem atenția că, la fel ca în cazul altor limbaje de programare, prezentarea "secvențială" a limbajului Java nu este posibilă. De aceea, în mod inerent, la început multe aspecte vor fi prezentate incomplet, scopul fiind ca cititorul să ajungă cât mai repede în situația de a scrie, compila și executa programe.

Iată pe scurt avantajele utilizării limbajului Java, așa cum au fost ele evidențiate de Sun Microsystems, cu mențiunea că acest paragraf conține și elemente care nu pot fi explicate în câteva cuvinte, dar asupra cărora se va reveni ulterior.

♦ **Simplitate.** Deși sintaxa limbajului este bazată pe cea a limbajului C++, limbajul Java este considerabil mai simplu. Au fost eliminate multe cuvinte cheie, nu mai există un preprocesor, iar limbajul este îmbogățit cu o bibliotecă mult mai bogată de unelte de dezvoltare. Nu mai există funcții independente, variabile globale, instrucțiunea `goto` și lucrul liber cu pointeri. Pointerii sunt înlocuiți cu referințe, dar nu mai există o aritmetică a pointerilor. În acest mod sunt eliminate multe surse de erori de programare, ceea ce conduce la programe corecte și robuste.

- ♦ **Orientarea spre obiecte.** Limbajul Java a fost conceput de la început pentru a respecta principiile programării orientate pe obiecte. Ca și C++, Java folosește *clase* pentru organizarea logică a codului; clasa este unitatea de bază în elaborarea programelor. Nu există variabile sau funcții în afara claselor. La executare, programul creează *obiecte* de tipul claselor; aceste obiecte pot fi gândite ca niște ajutoare (persoane) care execută ele funcțiile (numite *metode*) ale clasei. Este admisă și larg utilizată moștenirea claselor, dar nu este permisă moștenirea multiplă (ea este suplinită de o altă facilități, numită *interfață*).
- ♦ **Java este proiectat pentru lucru în rețea.** Chiar de la început, limbajul Java a fost gândit pentru Internet. Deși este posibil să construim programe care folosesc numai resurse locale, Java pune la dispoziție o mulțime de facilități pentru programare în rețele, pentru a face conexiuni între calculatoare server și client și pentru executare în medii ca de exemplu un browser WWW.
- ♦ **Compilare prealabilă urmată de executare pe mașina gazdă.** Un program sursă Java trebuie mai întâi compilat; ca rezultat se obține un fișier "byte-code" (o secvență de instrucțiuni de asamblare pentru o mașină imagină, numită *mașina virtuală Java*), care nu depinde de mașina gazdă pe care va fi executat programul. Programul byte-code poate fi apoi transferat pe orice mașină. Fiecare mașină gazdă capabilă să execute programe Java dispune de un *interpretor*, care convertește reprezentarea "byte-code" în instrucțiuni mașină proprii, care apoi sunt executate; conversia are loc la lansarea executării și anume instrucțiune cu instrucțiune. În acest mod este asigurată portabilitatea și independența de platformă. Dezavantajul acestui mod de lucru constă în timpul mare de executare. O variantă pentru interpretor o constituie *compilatoarele JIT (Just In Time)*, care transformă programul byte-code nu instrucțiune cu instrucțiune, ci în întregime (la lansarea executării) într-o formă executabilă pe mașina gazdă, ceea ce conduce la o creștere semnificativă a vitezei de executare. În continuare vom lucra în varianta clasică, cea a utilizării unui interpretor.
- ♦ **Posibilitatea lansării mai multor fire de executare.** Limbajul permite lansarea mai multor fire de executare, permițându-se astfel executarea concurrentă a mai multor acțiuni. În acest mod este larg deschis drumul către programarea paralelă și concurrentă. La modul ideal, în momentul în care interpretorul detectează mai multe procesoare pe mașina gazdă, acestea sunt efectiv folosite pentru executarea concurrentă a mai multor acțiuni; în momentul de față această "detectare" nu este încă implementată în mod standard.
- ♦ **Colectorul de reziduuri.** Java permite crearea explicită de obiecte (de tipul unei clase). Distrugerea acestor obiecte este preluată de *colectorul de reziduuri (garbage collector)*, care marchează obiectele ce nu mai sunt folosite și

- eliberează spațiul ocupat de ele; eliberarea nu se face neapărat imediat, ci periodic sau atunci când spațiul disponibil curent nu mai poate satisface o nouă cerere de alocare de memorie.
- ♦ **Securitate ridicată.** Java asigură un grad de securitate ridicat, uneori cu un plus de efort din partea programatorului. Accesul la memorie este posibil numai prin verificarea prealabilă a drepturilor de acces. Lipsa pointerilor face ca accesarea unor zone de memorie pentru care accesul nu este autorizat să nu fie posibilă. Limbajul obligă programatorul să prevadă acțiunile ce trebuie întreprinse la diferitele erori (numite *excepții*) posibile. De exemplu, chiar o simplă operație de citire trebuie însoțită de codul ce trebuie executat la detectarea unei excepții; de asemenea se verifică permanent, la executare, valoarea indicelui unui tablou înainte de accesarea componentei respective.
- ♦ **Extensibilitate.** Limbajul Java permite includerea de *metode native*, adică de funcții scrise în alt limbaj (de obicei C++). Metodele native sunt legate dinamic la programul Java la momentul executării, rolul lor fiind în principal de a mări viteza de executare pentru anumite secvențe din program.
- ♦ **Claritate.** Așa cum s-a menționat, eliminarea unor facilități neesențiale din C++ conduce la programe mai ușor de înțeles. În plus se consideră că limbajul Java este ușor de învățat, cu condiția ca cel care îl prezintă să fi înțeles el însuși foarte bine mecanismele puse la dispoziție (ușor de spus, greu de realizat!). Trebuie însă menționat că dacă inițial limbajul a fost gândit a fi simplu și compact, acceptarea sa unanimă a condus la dezvoltarea sa continuă, astfel încât în momentul de față a devenit deosebit de complex.
- ♦ **Tipizare strictă.** Fiecare variabilă și expresie are un tip cunoscut chiar la momentul compilării.
- ♦ **Applet-uri.** Programele Java se împart în două categorii. O primă categorie este constituită de programele obișnuite, de sine stătătoare (*stand alone*), numite *aplicații*. A doua categorie o constituie *applet*-urile. Este folosit același limbaj, dar diferă modalitatea de lansare în executare. Când utilizatorul vizualizează o pagină Web ce include un applet, mașina la care este conectat transmite applet-ul mașinii gazdă, care este cea pe care se execută applet-ul (se presupune că mașina pe care lucrează utilizatorul are un interpretor Java). Applet-urile constituie o alternativă larg folosită pentru aplicațiile Java. Menționăm că applet-urile pot îngloba elemente de grafică, animație, sunet, jocuri etc., ceea ce la face foarte atractive.

În cele ce urmează ne vom îndrepta atenția asupra aplicațiilor Java, urmând ca applet-urile să fie descrise într-un capitol separat.

1.2. Un prim program scris în Java

Nu vom căuta să fim originali cu orice preț, ci vom prezenta un prim program "clasic":

```
import java.io.*;
class unu {
    public static void main(String[] sir) {
        for (int i=0; i<sir.length; i++)
            System.out.println(sir[i]);
    }
}
```

Deși scurt, programul necesită o serie întreagă de explicații.

Programul de mai sus este compus dintr-o singură clasă și anume cea cu numele unu; să presupunem că "programul sursă", numit în Java *unitate de compilare*, este Prog.java (extensia java este obligatorie). Scrierea programului poate fi realizată cu orice editor text.

Clasa unu conține doar o funcție (numită în Java *metodă*), cu numele main. Aceasta este metoda "principală": la executarea programului, sistemul detectează și execută metoda main a clasei al cărei nume coincide cu numele fișierului byte-code rezultat în urma compilării clasei ce conține metoda principală.

Declararea:

```
import java.io.*;
```

asigură accesul la diverse metode ce permit operații de intrare/ieșire (în programul nostru este folosită metoda println).

Metoda main trebuie declarată cu atributele (numite în Java *modificatori*) **public** și **static**. Semnificația lui **void** este următoarea: metoda nu întoarce vreo valoare. În metoda main apare în mod obligatoriu un parametru de tip tablou de șiruri de caractere. Cum tablourile sunt indexate în Java începând cu 0 și cum lungimea tabloului sir poate fi referită prin sir.length, instrucțiunea **for** tipărește pentru fiecare i luând valori în intervalul 0..sir.length-1 șirul de caractere sir[i]; este folosită metoda println, care realizează tipărirea la ieșirea standard (identificată prin System.out), urmată de trecerea la linia următoare.

Menționăm de asemenea că în Java se face distincție între literele mici și literele mari; de exemplu unu și Unu sunt doi identificatori diferiți.

Operatorul = utilizat mai sus este operatorul de atribuire.

Cuvintele cheie apar în text îngroșate, pentru a fi mai ușor identificate

Reamintim că numele unității de compilare este Prog.java. Cum extensia este formată din 4 caractere, se recomandă să nu folosim pentru editare, copiere etc. produse care au limitări asupra lungimii numelui și extensiei fișierelor (ca de exemplu Norton Commander). De altfel, lucrând din modul linie de

comandă, este totdeauna preferabil să utilizăm comenzile de bază ale sistemului de operare gazdă.

Compilarea se face folosind compilatorul javac.exe astfel:

```
javac Prog.java
```

În urma compilării ia naștere fișierul unu.class; mai general, pentru orice unitate de compilare (formată din mai multe clase), va lua naștere pentru fiecare clasă un fișier având numele clasei și extensia class). Pentru acest fișier putem comanda executarea sa apelând interpretorul java.exe astfel:

```
java unu lista de identificatori
```

De exemplu pentru programul de mai sus, după ce s-a trecut de faza de compilare, dacă dăm comanda:

```
java unu Unu Doi Trei
```

vectorul sir va fi format din trei componente, iar la ieșire va apărea:

```
Unu
Doi
Trei
```

Este indicat, dar nu obligatoriu (așa cum se vede din exemplul prezentat), ca numele unității de compilare să coincidă cu numele clasei ce conține metoda principală; în primele versiuni ale limbajului exista însă această obligativitate. În versiunile recente nici precizarea atributului **public** pentru metoda principală nu mai este obligatorie.

Modul de compilare și executarea a programelor Java descris mai sus este cel mai simplu posibil. El poate fi folosit utilizând *Java Development Kit*. Menționăm însă că în prezent există mai multe *medii de programare* Java ca de exemplu *Java Café* (Symantec), *Java Visual Age* (IBM), *Visual J++*, *Kawa*, *JBuilder* etc.

1.3. O clasă folosită pentru intrările și ieșirile standard

Prezentăm în continuare o clasă menită să realizeze citirea și scrierea de la/la dispozitivele de intrare/ieșire standard, într-un mod apropiat de folosirea fișierelor text în Pascal. Cititorul este invitat să se rezume doar la modul de utilizare a metodelor clasei.

```
import java.io.*;
```

```
public class IO {
    static InputStreamReader reader =
        new InputStreamReader(System.in);
    public static StreamTokenizer f =
        new StreamTokenizer(reader);
```

```
    public static double read() {
        try { f.nextToken(); }
```

```

catch (IOException e) { }
if (f.ttype == -2) return(f.nval);
else return(Double.NaN);
}

public static char readch() {
    try { return (char) System.in.read(); }
    catch (IOException e) { return '#'; }
}

public static String readString() {
    try { f.nextToken(); }
    catch (IOException e) { }
    if (f.ttype == '\"') return(f.sval);
    else return("*****");
}

public static void writeln(String s)
{ System.out.println(s); }

public static void write(String s) { System.out.print(s); }
}

```

Ne mărginim la a descrie efectul apelărilor metodelor acestei clase, cu precizarea că rezultatul compilării (fișierul `IO.class`) trebuie să se afle în același director cu programul ce folosește metodele clasei `IO`.

- `IO.read()` întoarce următorul număr zecimal fără exponent din fluxul de intrare de la dispozitivul standard de intrare (tastatura);
- `IO.readch()` întoarce următorul caracter;
- `IO.readString()` întoarce următorul șir de caractere (cuprins între ghilimele);
- `IO.write(s)` tipărește șirul `s` la dispozitivul standard de ieșire (ecranul);
- `IO.writeln(s)` tipărește șirul `s` și trece la rând nou.

Amănunte asupra acestei clase și în general asupra operațiilor de intrare/ieșire vor fi date într-un capitol separat. Scopul introducerii acestei clase este de a citi și scrie ușor la dispozitivele standard de intrare/ieșire numere, caractere și șiruri de caractere. Vom face totuși unele comentarii.

Așa cum am anunțat în primul paragraf al acestui capitol, limbajul Java asigură un grad de securitate ridicat, care însă presupune un mic efort suplimentar la redactarea programului. Concret, chiar o citire simplă de la dispozitivul standard de intrare trebuie însoțită de măsuri de precauție.

În clasa de mai sus am folosit, printre altele, metoda `read`, care nu are nici un parametru, a clasei `InputStream` din pachetul `java.io` al produsului Java (pentru moment, putem gândi un pachet de clase ca un director). Ca urmare a invocării `System.in.read()` este citit și întors un `byte` de la intrare, dacă nu s-a ajuns la sfârșit de fișier. Valoarea întoarsă este un număr întreg din intervalul

0.255. Interpretarea acestei valori ca fiind caracterul cu numărul de ordine respectiv ne conduce la obținerea caracterului care a fost citit. Interpretarea menționată se realizează printr-o conversie explicită la tipul `char`, realizată prin prefixarea apelului cu `(char)`. Rezumând, dacă `ch` este o variabilă de tip caracter, atunci prin:

```
ch = (char) System.in.read();
```

se obține în `ch` următorul caracter din fluxul de intrare, dacă există un astfel de caracter. Această instrucțiune trebuie plasată în contextul:

```
try {
    ... ; ch = (char) System.in.read(); ...
}
```

```
catch (IOException e) { ... }
```

pentru a spune explicit că, în cazul când intervine o eroare la citire, este executată secvența de instrucțiuni cuprinsă între acolade ce urmează lui `catch`.

1.4. Clase, câmpuri și metode

Clasa este unitatea de programare fundamentală în Java. Vom presupune pentru moment că o clasă nu poate să conțină altă clasă (de fapt există și așa numitele *clase interne*, despre care vom vorbi în capitolul 6). Orice clasă este formată din *câmpuri*, *metode* și *constructori*. Câmpurile și metodele unei clase formează împreună *membrii* acelei clase.

Câmpurile unei clase sunt variabile atașate clasei respective.

Metodele sunt funcțiile declarate în interiorul clasei. O metodă nu poate conține altă metodă. Numele funcției este prefixat cu tipul rezultatului întors de funcție (sau `void` dacă metoda nu întoarce vreo valoare) și urmat de lista parametrilor, cuprinsă între paranteze rotunde; cele două paranteze rotunde trebuie specificate chiar dacă nu există parametri. Lista tipurilor parametrilor, în ordinea lor de apariție, formează *signatura* metodei. În aceeași clasă pot apărea mai multe metode cu același nume, dar cu semnături diferite. Prin urmare, nu este permisă apariția a două metode cu același nume și aceeași semnătură, indiferent de tipul rezultatului întors.

Clasele sunt considerate tipuri. Entitățile al căror tip este o clasă se numesc *obiecte* ale clasei respective. Se mai spune că obiectele sunt *instanțieri* (*instanțe*) ale claselor.

Constructorii unei clase seamănă cu metodele, dar numele lor este obligatoriu numele clasei și nu întorc valori. Unul dintre constructori este automat invocat la crearea unui obiect de tipul clasei respective. Acțiunea constructorilor poate fi oricât de complexă, dar în principal sunt folosiți pentru inițializarea unor

câmpuri ale obiectului. Ca și pentru metode, pot exista mai mulți constructori, dar cu semnături diferite.

Această posibilitate de a exista mai mulți constructori, respectiv mai multe metode cu același nume, dar cu semnături diferite, poartă numele de *supraîncărcare* (*overloading*).

Obiectele sunt create cu ajutorul operatorului **new**. Un obiect poate folosi o variabilă pentru a păstra o *referință* către el. Mai multe variabile pot referi același obiect.

Exemplul 1. Dacă C este o clasă, atunci crearea unui obiect de tipul clasei C se poate face astfel:

1) **new C(...);**

prin care este creat un obiect anonim de tipul C;

2) C Ob; Ob = **new C(...);** sau prescurtat: C Ob = **new C(...);**

prin care este creat un obiect de tipul C la care ne putem referi prin variabila Ob (având evident tipul C).

La creare este invocat constructorul corespunzător (cu aceeași semnătură cu cea din lista de argumente). Dacă în clasa C nu există vreun constructor, se presupune că "există" totuși un constructor fără parametri, care nu prevede nici o acțiune. Drept urmare, la creare trebuie folosită forma: **new C();**

Variabila referință Ob poate fi folosită pentru accesarea câmpurilor și metodelor clasei. Astfel, dacă x este un câmp al clasei, el poate fi referit prin Ob.x, iar dacă met este o metodă a clasei, ea poate fi invocată prin Ob.met(...); (evident, este invocată acea metodă care are aceeași semnătură cu cea din lista de argumente). Mai spunem că Ob.x, respectiv Ob.met sunt calificări ale obiectului Ob cu câmpul x, respectiv metoda met.

Un obiect are o *comportare*, concretizată prin metodele clasei a cărei instanțiere este. Deci un obiect poate fi privit ca o persoană, care la creare are niște date (câmpurile) și modalități de a opera asupra lor (metodele). Invocarea unei metode pentru un obiect revine la a pune acel obiect (acea persoană) să execute codul metodei respective.

În afară de clase, Java permite declararea de *interfețe*, care diferă de clase prin aceea că pentru metode este precizată semnătura lor, dar nu este specificat corpul lor; cu alte cuvinte, metodele apar doar la stadiul de "intenție". De aceea nu pot fi create direct obiecte de tipul unei interfețe. Lucrul cu interfețe va fi prezentat într-un capitol separat. Am spus aici câteva cuvinte despre această noțiune deoarece anumite aspecte sunt comune atât claselor cât și interfețelor, iar unele dintre ele vor fi prezentate înainte de capitolul menționat.

O *variabilă* poate fi de un tip primitiv (numeric, caracter etc.) sau o referință. Vom vedea că variabilele folosite în Java pentru tablouri sunt variabile de tip referință (tablourile sunt obiecte!). De asemenea, șirurile de caractere (tipul

String) sunt referințe; pentru ele se poate folosi operatorul + cu semnificația de concatenare.

O variabilă de tip referință poate avea valoarea **null**, care indică o referință către "nimic"; drept urmare variabila nu conține o referință validă, deci nu poate fi folosită pentru a accesa câmpuri sau invoca metode.

O variabilă de tip referință, a cărei valoare nu este **null**, poate fi gândită ca o adresă la o zonă de memorie ce conține în esență câmpurile obiectului respectiv și referințe la codurile metodei clasei.

Precizare. În limbaje ca de exemplu C++ se lucrează cu pointeri. Aceștia nu trebuie confundați cu referințele din Java, deoarece singurele operații asupra variabilelor de tip referință sunt atribuirile și comparările, deci nu există o aritmetică a referințelor.

La apelarea (invocarea) oricărei funcții (metode), *transmiterea argumentelor se face prin valoare*. Vom explica mai amănunțit acest mod de transmitere, deoarece în unele cărți se afirmă în mod greșit că pentru referințe se folosește apelul prin referință.

La invocarea unei metode este creată câte o variabilă locală pentru fiecare parametru al metodei. Înainte de executarea metodei, variabila parametru este inițializată cu valoarea argumentului corespunzător din invocare. De exemplu, dacă argumentul este de tip referință, variabila parametru corespunzătoare este inițializată cu respectiva referință; drept urmare argumentul și variabila parametru sunt referiri la același obiect.

O metodă poate modifica câmpurile obiectului și valoarea variabilelor parametru. Aceasta nu va conduce însă la modificarea valorii argumentului (eventual numai câmpurile sale sunt modificate, dar nu referința la obiect).

Aceeași relație există între o variabilă parametru și argumentul corespunzător, când tipul acestuia este un tip primitiv.

Exemplul 2. Să considerăm următorul program (unitate de compilare):

```
class C {
    int i; static int i0;
    C() { i0 = 100; scrie(); }
    C(int j) { i = i0+j; }
    void scrie() { IO.writeln(i0 + " " + i); }
    void modif() { i = -i; }
    void modif(C Ob) { i = Ob.i; }
}

class Obiect {
    public static void main(String[] args) {
        new C();
        C Ob1 = new C(1); Ob1.scrie();
        C Ob2 = new C(2); Ob2.scrie();
        IO.writeln(" "+ (Ob1==Ob2) );
        C Ob3 = Ob1; Ob3.modif(); Ob3.scrie();
    }
}
```

```

IO.println(""+ (Ob1==Ob3) );
Ob2.modif(Ob1); Ob2.scrie();
IO.println(""+ (Ob1==Ob2) );
}

```

În care operatorul `==` este operatorul de egalitate, iar `+` este fie operatorul de adunare, fie cel de concatenare (în situația în care cel puțin un membru este un șir de caractere, caz în care și ceilalți membri sunt convertiți la șiruri de caractere).

În acest program apare o noțiune nouă și anume modificatorul `static` cu care este declarat câmpul întreg `i0` din clasa `C`. Aceasta face ca `i0` să fie comun tuturor obiectelor de tipul `C`. Existența unui câmp static nu depinde de obiecte ce instanțiază clasa. Câmpul respectiv poate fi adresat din afara clasei prin numele clasei calificat cu numele câmpului (`C.i0`); evident, pentru orice obiect `Ob` de tipul `C`, câmpul poate fi adresat în continuare prin `Ob.i0`.

Analog, modificatorul `static` poate fi folosit și pentru metodele unei clase. Din afara clasei, metoda poate fi invocată prin numele clasei calificat cu numele metodei și urmat de lista argumentelor. Metodele statice sunt în general folosite pentru modificarea câmpurilor statice ale clasei. Cunoaștem deja un exemplu de folosire a acestui modificator: el însoțește "metoda principală" `main`; acest lucru este necesar deoarece nu este creat un obiect de tipul clasei din care face parte această metodă, ci este folosit un "încărcător de clasă", care trebuie să aibă acces la metoda `main`.

Mai menționăm că, în mod implicit (prin lipsă), câmpurile unui obiect primesc la creare o valoare (0 pentru tipuri numerice); de exemplu la crearea unui obiect de tipul `C`, câmpul `i` are implicit valoarea 0.

La executarea programului, vor fi tipărite în ordine următoarele rezultate:

```

100 0    ca urmare a executării instrucțiunii new C(), care inițializează
         câmpul static i0 cu valoarea 100 și tipărește valorile lui i0 și i; să
         observăm că a fost folosit constructorul fără parametri;
100 101   ca urmare a creării obiectului Ob1 prin invocarea constructorului cu
         un parametru și a executării metodei scrie;
100 102   ca urmare a creării obiectului Ob2 prin invocarea constructorului cu
         un parametru și a executării metodei scrie;
false     deoarece la fiecare creare a unui obiect prin new se obține o nouă
         referință, deci Ob1 este diferit de Ob2;
100 -101  ca urmare a invocării metodelor modif (fără parametru) și scrie de
         către obiectul Ob3; să remarcăm faptul că referințele Ob1 și Ob3
         coincid. Ele sunt referințe către același obiect; mai spunem că Ob3
         este un "alias" al lui Ob1;
true      din motivele expuse mai sus;
100 -101  ca urmare a invocării metodelor modif (cu argumentul Ob1) și scrie
         de către obiectul Ob2; câmpul i al lui Ob2 primește valoarea
         câmpului i a lui Ob1. Reamintim că dacă în această metodă modif

```

am fi adăugat o instrucțiune de tipul `Ob.i=7`, aceasta ar fi modificat un câmp al obiectului referit prin `Ob1`, dar nu valoarea referinței `Ob1`; deoarece referințele `Ob1` și `Ob2` sunt diferite (chiar dacă obiectele respective au aceleași valori ale câmpurilor).

1.5. Simularea înregistrărilor în Java

În limbajul Java nu există structuri de date eterogene (înregistrări), adică ceva analog cu tipul `record` din Pascal. Ținând cont că la invocarea metodelor se face apel prin valoare, putem să ne punem întrebarea cum poate o metodă să întoarcă mai multe rezultate. În Java, înregistrările pot fi simulate folosind clase (eventual fără metode). Acest lucru este ilustrat de următorul program, care arată cum o metodă poate întoarce mai multe valori:

```

class Record {
    int i; boolean b;
    Record(int ii, boolean bb) { i = ii; b = bb; }
}

class C {
    Record metoda (int i) {
        Record R = new Record(i+1,i<0);
        return R;
    }
}

class Doi {
    public static void main(String[] sir) {
        int i; Record R;
        i = (int) IO.read();
        C Ob = new C(); R = Ob.metoda(i);
        IO.println(R.i + " " + R.b);
    }
}

```

Legat de acest program simplu se impun mai multe precizări:

- comanda de compilare a clasei `Doi` va atrage automat după sine compilarea claselor `Record` și `C` (în ipoteza, pe care o folosim în continuare, că plasăm toate fișierele sursă în același director);

- clasa `Record` nu are metode, ceea ce este permis, dar nu vine în concordanță cu programarea orientată pe obiecte, mai precis cu caracteristica de încapsulare, care presupune includerea în aceeași clasă a unor date și a metodelor care le prelucrează;

- după citirea unui număr real de la intrare, trebuie ca înainte de a fi atribuit unei variabile întregi, să fie convertit explicit la acest tip, ceea ce am și făcut pentru variabila `i`; precizăm că la conversie se face *trunchiere*;

- deoarece câmpurile R.i și R.b ale obiectului R nu sunt de tip șir de caractere, în apelul `IO.writeln(...)` trebuie să apară, în concatenare (realizată prin operatorul `+`) cel puțin un șir de caractere; drept urmare ceilalți membri ai concatenării sunt convertiți implicit la șiruri de caractere;

- dacă este citită valoarea 5, la ieșire va apărea: 6 false
- dacă este citită valoarea -5, la ieșire va apărea: -4 true

1.6. Exemplu: crearea unei liste înlănțuite

Exemplul 4. Prezentăm o modalitate de a crea și prelucra o listă înlănțuită, ale cărei elemente alegem să fie caractere.

După cum știm, idea de bază este de a forma "celule" care să conțină un element efectiv și "adresa" următoarei celule. Pentru aceasta vom defini o clasă elem formată din:

- câmpurile `c` (de tip caracter) și `leg` (de tip elem);
- câmpurile statice `p` și `u`, conținând referințele spre primul și ultimul element al listei;
- un constructor fără parametri, care este folosit pentru crearea în metoda principală a obiectului `Ob`;
- un constructor, care are un parametru de tip caracter; la crearea unui nou obiect de tip elem, obiectul va primi în câmpul `c` caracterul transmis la invocarea constructorului, iar câmpul `leg` va deveni în mod implicit referința nulă `null`;
- metoda `create`, care creează un prim element al listei (memorând referința la el în variabilele `p` și `u`) și apoi în mod repetat, până când este citit caracterul '\$', obiectul `u` creează, folosind metoda `adaug`, un nou obiect pentru noul caracter citit și actualizează referința `u` pe baza rezultatului întors;
- metoda `adaug`, care creează un nou obiect și întoarce o referință către el;
- metodele `direct` și `invers` ce au un parametru `x` de tip elem și întorc un rezultat de tip șir de caractere ce corespunde parcurgerii în ordine normală (de la stânga la dreapta), respectiv în ordine inversă, a șirului de caractere ce începe de la "adresa" `x`.

În clasa `Lista`, în metoda principală, este invocată metoda `create` și apoi metodele `direct` și `invers`, prin intermediul unui obiect `Ob` de tipul elem, ultimele două având ca argument câmpul static `p` al clasei elem.

Programul corespunzător este prezentat în continuare.

```
class elem {
    char c; elem leg; static elem p,u;
    elem() { }
    elem(char ch) { c = ch; }
    elem adaug(char ch) {
        elem x = new elem(ch); leg = x; return x;
    }
}
```

```
void create() {
    char ch = IO.readch(); p = new elem(ch); u = p;
    ch = IO.readch();
    while (ch!='$') { u = u.adaug(ch); ch = IO.readch(); }
}

String direct(elem x) {
    if (x==null) return "";
    else return x.c + direct(x.leg);
}

String invers(elem x) {
    if (x==null) return "";
    else return invers(x.leg)+x.c;
}

class Lista {
    public static void main(String[] s) {
        elem Ob = new elem();
        Ob.create();
        IO.writeln( Ob.direct(elem.p) );
        IO.writeln( Ob.invers(elem.p) );
    }
}
```

unde operatorul `!=` are sensul "diferit de". Să observăm că metodele `direct` și `invers` puteau fi invocate și cu argumentul `Ob.p`.

Să analizăm pe un exemplu modul de funcționare a programului de mai sus. Să presupunem că la intrare apare: `abc$c1`

Este citit caracterul 'a' și este creat un nou obiect de tipul elem, al cărui câmp `c` este caracterul 'a' și al cărui câmp `leg` este `null`. Variabila `u` de tip elem păstrează referința către acest obiect. Pentru claritate, să notăm cu P_1 acest obiect. Variabilele `p` și `u` memorează o referință la obiectul P_1 .

În continuare este citit caracterul 'b'. Apoi obiectul P_1 (referit prin `u`) este "pus la lucru". El creează un nou obiect de tip elem, al cărui câmp `c` este caracterul 'b' și al cărui câmp `leg` este `null`; fie P_2 acel obiect. Câmpul `leg` al lui P_1 este actualizat și devine referința la P_2 . Variabila `u` va conține acum o referință la obiectul P_2 .

În continuare este citit caracterul 'c'. Apoi obiectul P_2 este "pus la lucru". El creează un nou obiect de tip elem, al cărui câmp `c` este caracterul 'c' și al cărui câmp `leg` este `null`; fie P_3 acel obiect. Câmpul `leg` al lui P_2 este actualizat și devine referința la P_3 . Variabila `u` va conține acum o referință la obiectul P_3 .

Cum următorul caracter citit de la intrare este '\$', crearea listei de obiecte este încheiată.

Să analizăm efectul executării instrucțiunii `IO.writeln(Ob.direct(p));`

care prevede scrierea la ieșire a șirului de caractere întors de invocarea `p.direct(p)`.

Întrucât variabila `p` memorează o referință la obiectul `P1`, obiectul `P1` va invoca metoda `direct` cu parametrul `P1`. Este întors șirul format din caracterul 'a' la care se concatenează șirul întors de invocarea metodei `direct` efectuată pentru referința `P2`. Acest apel întoarce șirul format din caracterul 'b', la care se concatenează șirul întors de invocarea metodei `direct` efectuată pentru referința `P3`. Cum această din urmă invocare întoarce șirul vid (deoarece câmpul `leg` al lui `P3` conține `null`), rezultă că la ieșire va apărea:
abc

În mod analog, executarea instrucțiunii:
`IO.writeln(Ob.invers(p));`
face ca la ieșire să apară:
cba

Vom face în continuare câteva precizări suplimentare privind colectorul de reziduuri. Interpretorul Java este capabil de a detecta momentul în care un obiect nu mai este referit și îl poate "distruge", sarcină preluată de colectorul de reziduuri. Așa cum am menționat, aceasta nu implică eliberarea imediată a spațiului alocat obiectului. Este posibil ca interpretorul Java să termine executarea programului înaintea colectorului de reziduuri (acesta nu a eliberat încă toate spațiile inutile); în această situație, resursele rămase alocate vor fi eliberate de sistemul de operare.

De asemenea menționăm că orice aplicație Java presupune executarea concurentă a cel puțin două fire de executare: cel care execută programul utilizatorului și cel ce gestionează colectorul de reziduuri.

1.7. Introducere în programarea orientată pe obiecte (OOP)

Începem prin recapitularea unor noțiuni deja prezentate:

- clasa este unitatea de bază în elaborarea programelor;
- orice obiect este o instanță a unei clase;
- un obiect încapsulează *stări* (valorile câmpurilor clasei) și *comportament* (metodele clasei);
- un obiect își etalează comportamentul în momentul în care este invocată o metodă prin intermediul său.

Este foarte important a înțelege *deosebirea între apelarea unei funcții* (de exemplu în limbajul Pascal) și *invocarea unei metode prin intermediul unui obiect*.

La apelarea unei funcții controlul este deținut în continuare de unicul "executant" al programului.

La invocarea unei metode prin intermediul unui obiect, acelui obiect îi revine sarcina de a executa acțiunile prevăzute în metodă. Acțiunea constă deci în transmiterea unui mesaj obiectului, care este pe post de receptor. Mesajul conține:

- numele obiectului (receptorului);

- acțiunea ce trebuie întreprinsă (numele metodei);
- informații suplimentare necesare pentru întreprinderea acțiunii (argumentele cu care este invocată metoda).

În plus, după cum vom vedea, uneori receptorul este cunoscut doar în timpul executării.

Să observăm că nu este necesar să fie cunoscută modalitatea în care obiectul își duce la bun sfârșit acțiunea cerută; de exemplu clasa a cărei instanțiere este obiectul poate fi furnizată nu ca unitate de compilare, ci ca rezultat al compilării unității. Mai trebuie menționat că invocarea unei metode cu același nume și aceeași listă de parametri pentru un alt obiect, instanță a unei alte clase, va conduce la o altă acțiune.

Cele de mai sus au implicații importante:

- *ascunderea (secretizarea) informațiilor* : sunt cunoscute rezultatul compilării unei clase, constructorii și câmpurile claselor, precum și antetul metodelor împreună cu o informație generală asupra acțiunii metodelor (ce realizează metoda și nu cum are loc această realizare);

- *delegarea responsabilităților* : clasa de al cărei tip este obiectul este responsabilă de corecta desfășurare a activităților;

- *încrederea în soft-ul scris de alții* : odată cu folosirea unei clase trebuie să avem totală încredere în corectitudinea ei, deci în soft-ul pus la dispoziție de alți programatori; nu are sens ca la scrierea unui nou program să "inventăm din nou lumea", în loc să folosim soft deja existent și verificat.

- *dezvoltarea componentelor reutilizabile* : este o consecință a celor de mai sus și constă printre altele în dezvoltarea unor clase deja existente prin *extinderea* lor (adăugarea unor noi câmpuri și metode, eventual redefinirea unor metode, dar cu posibilitatea de a folosi și câmpurile și metodele clasei inițiale).

Este momentul de a enunța un principiu fundamental al programării orientate pe obiecte: *Un program orientat pe obiecte este o comunitate de agenți (obiecte) care își desfășoară fiecare activitatea lui, dar și interacționează între ele*. Interacțiunea constă în esență în faptul că fiecare obiect poate cere altui obiect să întreprindă anumite acțiuni.

Exemplul din paragraful precedent ilustrează doar parțial principiul enunțat anterior, datorită secvențialității inerente în lucrul cu liste. Totuși merită accentuat un aspect legat de variabila `leg` ce apare în metoda `adaug` și anume că este vorba de câmpul `leg` al obiectului prin care a fost invocată metoda, adică al obiectului `u`. Acest aspect trebuie să stea în permanență în atenția programatorului: totdeauna trebuie să știm care este "obiectul curent", adică cel care execută activitatea curentă.

Prezentăm în continuare un exemplu mai sugestiv.

Exemplul 5. Deși vom vorbi despre interfețe grafice doar în a doua parte a cărții, cu puțină imaginație putem înțelege cele ce urmează. Să considerăm o clasă

\b = \u0008 (backspace) \f = \u000c (form-feed)
 \r = \u000d (return) \" = \u0022 (ghilimele)

Motivația utilizării setului Unicode este integrarea în limbaje de tip HTML care să permită circulația pe Internet.

2.2. Identificatori

În Java identificatorii încep cu o literă (inclusiv '_' și '\$') și pot continua cu litere sau / și cifre; este totuși indicat să nu includem în identificatori caracterul '\$', deoarece el este folosit de compilatorul Java pentru a genera identificatori interni, despre care nu vom vorbi aici. Prin "litere" înțelegem atât pe cele obișnuite (literele mici și mari din alfabetul englez), cât și pe cele din alte limbi. Un identificator poate avea orice lungime; de aceea este indicat să folosim identificatori cât mai sugestivi pentru rolul pe care îl joacă, timpul "pierdut" la scrierea programului fiind recuperat din plin atunci când revedem programul peste câteva luni sau când trebuie ca el să fie înțeles de alte persoane.

Anumiți identificatori au o semnificație predefinită, ce nu poate fi modificată, și nu pot fi folosiți în alt context decât cel intenționat de către autorii limbajului. Ei se numesc *cuvinte cheie*. O primă listă de cuvinte cheie este următoarea:

abstract	boolean	break	byte	case
catch	char	class	continue	default
do	double	else	extends	final
finally	float	for	if	implements
import	instanceof	int	interface	long
new	null	package	private	protected
public	return	short	static	super
switch	synchronized	this	throw	while
throws	try	void		

Așa cum am precizat în capitolul anterior, în programe sau secvențe de cod aceste cuvinte cheie vor apărea îngroșate pentru a veni în ajutorul cititorilor.

2.3. Constante

În limbajul Java, constantele mai poartă și numele de *literal*. Vom prezenta doar o parte a acestor constante (de exemplu nu vom vorbi de literalii întregi octali și hexazecimali):

1) *Literalii întregi* sunt de două tipuri:

- *normali*, reprezentați pe 32 de biți și pentru care scrierea este cea uzuală;

- *lungi*, reprezentați pe 64 de biți, identificați prin faptul că au sufixul 'l' sau 'L'.

2) *Literalii reali (în virgulă mobilă)*:

- *dubli*, reprezentați pe 64 de biți, pentru care scrierea este cea uzuală. Este vorba de numere zecimale ce conțin opțional punctul zecimal și pot fi urmate de un exponent prefixat cu e sau E. De exemplu următorii trei literalii reprezintă aceeași valoare:

21. 2.1e1 .21E2

- *normali*, reprezentați pe 32 de biți. Acești literalii au aceeași formă ca cei dubli, dar au în plus sufixul 'f' sau 'F'.

Conform standardului IEEE 754, sunt incluși și literalii pentru "nu este un număr", -∞ și +∞:

Double.NaN, Double.NEGATIVE_INFINITY și Double.POSITIVE_INFINITY
 (pentru aritmetica pe 8 octeți);

Float.NaN, Float.NEGATIVE_INFINITY și Float.POSITIVE_INFINITY
 (pentru aritmetica pe 4 octeți).

Valoarea NaN apare ca rezultatul unor operații ca de exemplu împărțirea la zero; ea nu este "ordonată": compararea ei cu orice valoare produce rezultatul false. În schimb valorile corespunzătoare lui -∞ și +∞ sunt ordonate: pentru orice literal "efectiv" val, avem

-∞ < val < +∞.

Cele mai mici valori pozitive pentru literalii normali și dubli sunt:

Float.MIN_VALUE=1.43-45f și Double.MIN_VALUE=5e-324

iar cele mai mari valori posibile pentru literalii în virgulă mobilă sunt:

Float.MAX_VALUE=3.4028e+38f și Double.MAX_VALUE=1.79769e+308.

3) *Literalii booleani* sunt true și false.

4) *Literalii de tip caracter* au forma 'c', unde c este un caracter sau o secvență Escape; aceste secvențe au fost prezentate mai sus.

5) *Literalii de tip șir de caractere* sunt cuprinși între ghilimele.

Menționăm că șirurile de caractere sunt instanțe ale clasei String declarată standard în Java. Iată câteva exemple:

" " (șirul vid) ; "ab\nn" (unde apare secvența Escape '\n').

6) *Literalul referință* este null.

2.4. Comentarii

Comentariile sunt folosite în principal pentru explicarea acțiunilor întreprinse de cod; ele sunt ignorate de compilator.

Există trei tipuri de comentarii:

- *de sfârșit de linie*: încep cu // și se termină la sfârșitul liniei;
- *generale*: încep cu /* și se termină la prima succesiune */.
- *de documentare*: încep cu /** și se termină la prima succesiune */.

Acest tip de comentariu poate fi plasat doar imediat înaintea unei clase, unui membru al unei clase sau a unui constructor. Utilitarul javadoc este capabil să colecteze comentariile de documentare din codul sursă al claselor și să le introducă în documente HTML.

Din cele de mai sus rezultă că nu există comentarii în comentarii.

Șirul /**/ este considerat un comentariu de documentare, în timp ce /* */ este un comentariu general.

2.5. Variabile

Forma generală folosită pentru declararea variabilelor este:

< șir de modificatori > < tip > < listă de identificatori > ;

unde prezența modificatorilor este opțională. Informațiile din șirul de modificatori sunt separate prin spații albe, iar cele din lista de identificatori prin virgulă. Dacă dorim să specificăm și o valoare inițială pentru un identificator, acesta trebuie urmat de semnul = și de un literal, ca de exemplu în: `int a = 3;`

Menționăm pentru moment doar următorii modificatori:

- *modificatori de acces* (**public**, **private**, **protected**); dacă nu este specificat nici unul dintre acești modificatori, se consideră că este atașat un modificator implicit;

- **static**;
- **final**.

Modificatorii pot însoți nu numai câmpurile, ci și clasele și metodele, dar despre acest aspect vom vorbi într-un capitol următor.

La declararea câmpurilor unei clase se recomandă precizarea modificatorilor în ordinea de mai sus.

Așa cum am menționat în capitolul anterior, modificatorul **static** specifică faptul că respectivul câmp este unic pe clasă, deci comun tuturor obiectelor ce sunt instanțe ale clasei (joacă rolul de memorie comună pentru toate obiectele ce instanțiază clasa); câmpul static poate fi referit din afara clasei ca orice câmp, dar și prin precalificare cu numele clasei.

O declarație poate să apară oriunde în cadrul unei clase, metode sau a unui bloc de inițializare, deci nu neapărat la începutul lor.

Limbajul Java atribuie valori inițiale implicite câmpurilor claselor. Ele sunt:

`false` (pentru **boolean**); `'u0000'` (pentru **char**); `null` (pentru referințe); `0` (pentru orice tip întreg); `+0.0f` sau `+0.0d` (pentru **float** și **double**),

unde tipurile menționate sunt descrise în continuare. În schimb nu este atribuită vreo valoare inițială prin lipsă variabilelor locale din metode, constructori și blocuri de inițializare și de aceea aceste variabile trebuie să primească valori înainte de a fi utilizate; nerespectarea acestei reguli conduce la semnalarea unei erori.

O variabilă declarată cu modificatorul **final** este inițializată implicit sau explicit prin declararea sa; această valoare nu mai poate fi schimbată ulterior.

Atenție: pentru o variabilă de tip referință, această regulă înseamnă că ea nu își mai poate modifica valoarea; nu rezultă însă de aici că nu poate fi modificată valoarea unui câmp al obiectului respectiv, așa cum arată următorul program:

```
class C { int a; }

class Clasa {
    final C Ob = new C();
    void modify(int i) { Ob.a = i; }
}

class Final {
    static public void main(String[] args) {
        Clasa Ob = new Clasa();
        Ob.modify(1); IO.write(Ob.Ob.a + " ");
        Ob.modify(2); IO.println(Ob.Ob.a + " ");
    }
}
```

care produce la ieșire valorile 1 și 2.

O variabilă are un nume (care este un identificator) și un tip, despre care știm că este un tip primitiv sau un tip referință. Enumerăm în continuare *tipurile primitive* (de bază).

1) *Tipul boolean*. Variabilele de acest tip pot lua doar valorile `true` și `false`.

2) *Tipul char*. Variabilele de acest tip sunt reprezentate pe 16 biți și pot primi ca valoare orice simbol din codul Unicode. O variabilă de tip caracter poate fi folosită oriunde poate apărea o valoare întreagă: este considerat numărul său de ordine în setul de caractere Unicode.

3) *Tipuri întregi*. Ele sunt următoarele:

byte (octet)	short (întreg scurt)	int (întreg)	long (întreg lung)
reprezentate în ordine pe 1, 2, 4, 8 octeți. Valorile maxime sunt în ordine:			
255	32767	$\leq 2 \times 10^9$	$\leq 9 \times 10^{18}$

Precizăm că în calcule tipurile **byte** și **short** sunt convertite la **int**.

4) *Tipurile în virgulă mobilă float și double*, reprezentate respectiv pe 4 și 8 octeți.

Fiecărui tip primitiv din Java îi corespunde o clasă definită standard în Java: Boolean, Character, Byte, Short, Integer, Long, Float și Double. În aceste clase apar constante și metode utile. Am prezentat deja câteva constante din aceste clase în discuția asupra literalilor în virgulă mobilă. Mai menționăm metoda `isNaN` din clasele Float și Double: metoda primește o valoare de tipul corespunzător și întoarce o valoare booleană, care este `true` dacă și numai dacă argumentul nu este o valoare numerică validă; am folosit această metodă în clasa 10 prezentată în primul capitol.

2.6. Operatori

Menționăm că Java pune la dispoziția utilizatorului aproximativ 40 de operatori, dintre care unii sunt descriși în continuare. Trebuie precizat că programatorul poate defini metode noi, dar nu și operatori noi.

• Operatori aritmetici

Acești operatori sunt următorii:

+ - * / %

De asemenea este folosit operatorul unar - pentru schimbarea semnului, precum și operatorul unar + (introdus pentru simetrie).

În aritmetica întreagă trebuie să ținem cont de următoarele reguli:

- orice valoare ce depășește limita admisă este redusă modulo această limită; de aceea nu există depășiri (*overflow* și *underflow*);
- împărțirea întreagă se face prin trunchiere; de exemplu $-8/3 == -2$, iar $8/3 == 2$;

- operatorul % este definit prin:

$(x/y) * y + x \% y = x$

De exemplu $8 \% 3 == 2$, iar $-8 \% 3 == -2$.

• Operatorii de incrementare și decrementare

Este vorba de operatorii unari ++ și --, care pot fi aplicați operanzilor numerici (întregi sau în virgulă mobilă) atât în formă prefixată cât și în formă postfixată.

Expresiile `x++` și `++x` sunt echivalente cu `x=x+1`, cu două excepții:

- sunt *atomice* (considerate ca o unică operație, nu ca o incrementare urmată de o atribuire). Acest aspect este relevant doar când mai multe fire de execuție acționează asupra unei aceleiași variabile;
- operandul `x` este evaluat *o singură dată*. Această excepție are în vedere instrucțiuni de tipul `a[indice()]=a[indice()]+1`, în care este folosită o metodă ce întoarce un întreg; astfel, în exemplul de mai sus, invocarea metodei

îndice va avea loc o singură dată (în caz contrar este posibil ca în membrul stâng și cel drept să nu se refere la aceeași componentă a tabloului `a`, de exemplu dacă metoda întoarce în mod aleator un indice).

Diferența între `x++` și `++x` constă în faptul că incrementarea este realizată după, respectiv înainte de utilizarea lui `x` în contextul în care apare. Astfel, dacă valoarea curentă a lui `x` este 4, atunci:

- evaluarea expresiei `2 * ++x` conduce la rezultatul 10,

- evaluarea expresiei `2 * x++` conduce la rezultatul 8,

după care valoarea lui `x` va fi în ambele cazuri 5.

Evident, operatorul -- se supune unor reguli analoage.

O construcție de tipurile `x++`, `x--`, `++x`, `--x` reprezintă o expresie aritmetică, dar și o *instrucțiune* (valoarea expresiei nu este folosită, însă `x` este incrementat sau decrementat).

Operatorii de mai sus pot fi aplicați și pentru tipul `char`, semnificând trecerea la precedentul, respectiv următorul caracter din setul de caractere Unicode.

• Operatorii de atribuire

Pe lângă operatorul = folosit standard pentru atribuire, mai pot fi folosiți și operatorii:

`+=` `-=` `/=` `*=` `%=`

care reprezintă scrieri prescurtate. Astfel `x+=y` este echivalent cu `x=x+y`, cu excepția faptului că evaluarea adresei lui `x` se face o singură dată (vezi paragraful precedent). Desigur, operatorul `%=` va fi folosit doar pentru aritmetica întreagă.

• Operatorii relaționali

Este vorba de următorii operatori:

> (mai mare ca)	>= (mai mare sau egal cu)	== (egal cu)
< (mai mic ca)	<= (mai mic sau egal cu)	!= (diferit de)

• Operatori booleeni

Este vorba de următorii operatori:

|| (disjuncția logică, sau) && (conjuncția logică, și) ! (negație)
cu mențiunea că la evaluare se face *scurtcircuitare* (pe scurt, dacă la evaluarea unei expresii booleene devine la un moment dat clară valoarea ei, nu se mai continuă în mod inutil evaluarea; de exemplu dacă expresia este disjuncția logică a trei termeni și valoarea primului este `true`, nu se mai evaluează ceilalți termeni).

• Operatori pe biți

Este vorba de operatorii binari `|` și `&` care realizează disjuncția logică și conjuncția logică la nivel de bit, pentru fiecare pereche de biți de pe aceeași poziție din reprezentarea operanzilor.

• Operatorul condițional `?:`

Acest operator se utilizează în expresii sub forma:

`( cond ? e1 : e2 )`

a cărei valoare este `e1` dacă `cond` este `true`, respectiv `e2` dacă `cond` este `false`; `cond` trebuie să fie o expresie de tip boolean.

Pentru a nu intra în amănunte, vom impune deocamdată regula ca `e1` și `e2` să aibă același tip.

• Operatori postfix

Includem aici:

- cuprinderea între paranteze a indicilor (cu `[]`);
- operatorul de calificare `.` folosit pentru accesarea membrilor claselor;
- parantezele rotunde folosite pentru specificarea listei de parametri din invocări;
- operatorii postfix de incrementare/decrementare `++` și `--` de mai sus.

• Operatorul de conversie a tipurilor

Ne mărginim la a preciza că o conversie explicită de tip are forma:

`(tip) expresie`

urmând să discutăm ulterior condițiile în care poate avea loc o astfel de conversie.

• Operatorul `+` pentru lucrul cu șiruri

Acest operator este folosit pentru concatenarea șirurilor. Trebuie menționat că dacă un membru al unei sume este un șir de caractere, atunci are loc o conversie implicită a celorlalți membri ai sumei (devenită acum concatenare) la șiruri de caractere; această facilități a fost de altfel des folosită în programele prezentate anterior, de exemplu în cadrul invocării `IO.write(...)`. Adăugăm doar că printre membrii sumei pot apărea și variabile referință!

• Operatorii pentru referințe

Sunt utilizați următorii operatori:

- accesul la câmpuri (prin calificare);
- invocarea metodelor (prin calificare);
- operatorul de conversie;
- operatorii `==` și `!=`;
- operatorul condițional;
- operatorul `instanceof`, folosit în contextul:

`Ob instanceof Clasa`

care produce o valoare booleană ce este `true` dacă și numai dacă obiectul `Ob` este diferit de `null` și este o instanță a clasei `Clasa` sau poate fi convertit la tipul `Clasa` (despre conversiile de tip vom vorbi într-un capitol aparte).

• Precedența operatorilor

Ordinea în care are loc efectuarea prelucrărilor determinate de operatori este dată în următorul tabel de priorități ale operatorilor (de la prioritate maximă la prioritate minimă):

- operatorii postfix
- operatorii unari de incrementare/decrementare, operatorii `+` și `-` unari, operatorul de negație `!`
- operatorul `new` de creare de obiecte și cel de conversie: `(tip) expresie`
- operatorii multiplicativi: `*` `/` `%`
- operatorii aditivi: `+` `-`
- operatorii relaționali și `instanceof`
- operatorii de egalitate: `==` `!=`
- operatorul `&`
- operatorul `|`
- conjuncția logică `&&`
- disjuncția logică `||`
- operatorul condițional `( ? : )`
- operatorii de atribuire.

Observații:

- la prioritate egală, operatorii "vecini" acționează conform regulilor de asociativitate prezentate în continuare;
 - utilizarea parantezelor este mai puternică decât prioritatea operatorilor.
- Astfel, spre deosebire de `x+y*z`, în `(x+y)*z` prima operație care va fi executată este adunarea.

• Asociativitate

Regula generală o constituie asociativitatea la stânga. Fac excepție următorii operatori, pentru care este folosită asociativitatea la dreapta:

- operatorii unari;
- operatorii de atribuire.
- operatorul (? :).

Exemple :

1) În expresia $x-y+z$ întâi se va efectua scăderea și apoi adunarea;

2) Instrucțiunea:

```
x = y = z = 0;
```

este echivalentă cu:

```
x = ( y = ( z = 0 ) );
```

și are ca efect atribuirea valorii 0 variabilelor x, y, z . Explicația constă în aceea că efectuarea unei atribuirii conduce și la obținerea unui rezultat (valoarea care a fost atribuită) și care poate fi folosită ca în exemplul de mai sus.

3) Secvența de program:

```
int a=5, b=10, c=15;
```

```
a += b --c % 4;
```

```
IO.println(a + " " + b + " " + c);
```

produce la ieșire:

```
12 7 3
```

2.7. Instrucțiuni

Limbajul Java prevede mai multe tipuri de instrucțiuni:

- **Instrucțiunea compusă** grupează mai multe instrucțiuni prin includerea lor între acolade; ia naștere astfel un *bloc*. Un bloc este considerat a fi o unică instrucțiune.

- **Instrucțiunea de declarare** asociază unei variabile un anumit tip și eventual îi atribuie o valoare inițială; variabila devine *locală* celui mai interior bloc care o conține, în sensul că există atâta timp cât se execută instrucțiuni ale blocului. O instrucțiune de declarare poate să apară oriunde în interiorul unui bloc. Înainte de utilizarea lor, variabilele locale trebuie să fi primit valori fie prin inițializare la declarare, fie printr-o instrucțiune de atribuire (în caz contrar va fi semnalată o eroare la compilare).

- **Invocarea unei metode**, indiferent dacă ea întoarce sau nu o valoare.

- **Incrementările**, ce folosesc forma prefixată sau postfixată a lui ++ și --.

- **Instrucțiunea de atribuire** conține semnul =, eventual prefixat cu un operator.

- **Instrucțiunea vidă** este formată numai din ; și care nu prevede vreo prelucrare.

- **Instrucțiunea prin care este creat un obiect** folosește în acest scop operatorul **new**.

- **Instrucțiunile ce controlează ordinea de executare**, ca de exemplu: **if-else**, **for**, **while** etc. Ele vor fi prezentate detaliat în continuare.

Trecem la descrierea instrucțiunilor ce controlează ordinea de executare. Instrucțiunile care apar în cadrul lor vor fi notate prin **instr**, **instr1** etc.

- **Instrucțiunea if-else**. Ea are una dintre formele:

```
if (expresie_booleană) instr1
```

sau

```
if (expresie_booleană) instr1
```

```
else instr2
```

Este evaluată expresia booleană. Dacă rezultatul este **true**, atunci se execută **instr1**, iar în caz contrar se execută **instr2** (sau nimic pentru forma scurtă).

Clauza **else** este asociată celui mai recent **if** căruia nu i s-a asociat un **else**. Astfel în :

```
if (a==0) b=1; if (c==1) b=2; else b=3;
```

clauza **else** este asociată celui de al doilea **if**. Pentru a face lucrurile clare, este indicat ca fiecare **if** să fie însoțit de un **else** (urmat eventual de instrucțiunea vidă).

- **Instrucțiunea while** are forma:

```
while (expresie_booleană) instr
```

Se începe cu evaluarea expresiei booleene și se execută în mod repetat instrucțiunea **instr** atâta timp cât expresia booleană are valoarea **true**.

- **Instrucțiunea do-while** are forma:

```
do instr while (expresie_booleană);
```

Se începe cu executarea instrucțiunii **instr** și se reia executarea ei atâta timp cât expresia booleană are valoarea **true**.

- **Instrucțiunea for** are forma:

```
for (inițializări; expresie_booleană; actualizări) instr
```

unde:

- inițializările și actualizările (de obicei incrementări și/sau decrementări) sunt liste de instrucțiuni separate prin virgulă; listele pot fi eventual vide;
- expresia booleană poate și ea lipsi, caz în care se presupune că ea are implicit valoarea **true**.

Efectul instrucțiunii poate fi descris astfel:

```
inițializări;
while (expresie_booleană) {
    instr; actualizări
}
```

Cadrul tipic de utilizare este în iterații. De exemplu, suma primelor n numere naturale poate fi obținută astfel:

```
s=0;
for (i=0; i<n; i++) s+=i;
```

Mai prezentăm încă un exemplu: deși n-am vorbit încă despre tablouri, menționăm că pentru inversarea ordinii elementelor unui tablou (unidimensional) a este suficientă o instrucțiune **for** (indexarea elementelor tablourilor începe cu 0):

```
for(i=0,j=a.length-1; i<j; i++,j--)
    { b=a[i];a[i]=a[j];a[j]=b;}
```

Un caz limită este:

```
for( ; ; ) instr
```

care desemnează repetarea "la infinit" a executării lui *instr*. Evident, în cazul folosirii acestei forme, asigurarea ieșirii din ciclu cade în sarcina instrucțiunii *instr*.

În inițializările din **for** este permis să facem declarații, ca de exemplu:

```
for(int i=1;i<=n;i++) s+=i;
```

dar numai dacă *i* nu este deja declarat; precizăm că în această situație variabila *i* este locală instrucțiunii **for**.

Înainte de a prezenta alte instrucțiuni, menționăm că o instrucțiune poate fi etichetată. Etichetarea ei se face astfel:

```
eticheta:instr
```

unde *eticheta* este un identificator. În acest mod va fi posibil să facem referire (trimitere) la instrucțiunea respectivă.

• **Instrucțiunea break** este folosită pentru a ieși dintr-un bloc. O primă formă este:

```
break;
```

prin care se iese din cea mai interioară instrucțiune **for**, **while**, **do** sau **switch**, sau bloc ce conține această instrucțiune.

Forma:

```
break eticheta;
```

unde *eticheta* este eticheta unui bloc sau a uneia dintre cele 4 tipuri de instrucțiuni menționate, ce conține instrucțiunea **break**. Ca efect, controlul este transferat la instrucțiunea cu eticheta respectivă. Se poate astfel ieși de exemplu din două instrucțiuni **for** imbricate. Să remarcăm faptul că nu este posibil să trecem controlul oricărei alte instrucțiuni (ca pentru **goto** din alte limbaje), deci transferul este controlat și respectă structurarea programului.

• **Instrucțiunea continue** are forma:

```
continue;
```

și este folosită în cadrul instrucțiunilor **while**, **do** și **for**. Efectul ei este saltul la condiția care controlează reluarea acestor instrucțiuni de ciclare.

Ca și în cazul instrucțiunii **break**, poate fi folosită și varianta etichetată, prin care se face salt la condiția de control a unei instrucțiuni **while**, **do** sau **for** cu eticheta respectivă, care o "acoperă" (include) la un nivel superior.

• **Instrucțiunea switch** evaluează o expresie întreagă, a cărei valoare este folosită pentru a detecta o secvență de instrucțiuni ce urmează a fi executată. O primă formă a ei este:

```
switch (expresie) {
    case val1 : secvență_instrucțiuni1
    . . .
    case valk : secvență_instrucțiunik
    default   : secvență_instrucțiuni
}
```

unde:

- tipul expresiei poate fi numai **char**, **byte**, **short** sau **int**;

- *val₁*, ..., *val_k* sunt constante (literali sau câmpuri statice finale inițializate cu expresii constante) de același tip cu al expresiei;

- alternativa **default** este opțională.

Executarea instrucțiunii are loc după cum urmează. Este calculată valoarea expresiei, care este apoi comparată pe rând cu *val₁*, ..., *val_k*, până când se detectează prima dată egalitate. Urmează executarea secvenței de instrucțiuni corespunzătoare, dar și a celor care urmează; executarea acestora poate fi limitată prin folosirea instrucțiunii **break**. Dacă nu se detectează nici o egalitate, atunci se execută secvența de instrucțiuni corespunzătoare lui **default** (dacă această alternativă este prezentă) sau nu se execută nimic (în caz contrar).

Mai general, o alternativă poate avea mai multe valori, ca în exemplul următor:

```
case 1 : case 3 : case 5 : a = b; b = c; c = a;
```

Exemplu. Programul următor:

```
class Switch {
    static void numara(int i) {
        switch (i) {
            case 1 : IO.write("unu ");
            case 2 : IO.write("doi ");
            case 3 : IO.writeln("trei");
        }
    }

    public static void main (String[] args) {
        numara(3); numara(2); numara(1);
    }
}
```

va produce la ieșire:

```
trei
doi trei
unu doi trei
```

• **Instrucțiunea return** are una dintre formele:

```
return;
return expresie;
```

și furnizează valoarea care va fi întoarsă (în cazul când metoda prevede întoarcerea unei valori) și termină executarea metodei în care apare.

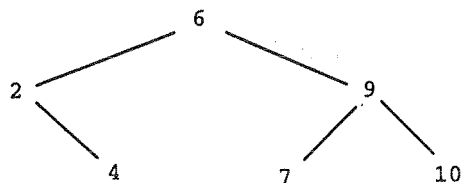
Putem folosi `return` și pentru a ieși dintr-un constructor sau dintr-un bloc de inițializare static.

2.8. Despre recursivitate în limbajul Java

Acest paragraf este menit a face mai puțin aridă prezentarea de mai sus și de a face unele precizări suplimentare.

O metodă poate invoca o altă metodă (eventual pe ea însăși). Invocarea se poate face pe baza obiectului curent sau pe baza unui alt obiect (câmp al clasei sau parametru al metodei).

Programul următor implementează *metoda sortării cu ajutorul arborilor de sortare*. Reamintim că un arbore de sortare este un arbore binar (fiecare vârf are cel mult doi descendenți) în care valoarea atașată fiecărui vârf este mai mare decât valorile vârfurilor din subarboarele stâng și mai mică decât valoarea vârfurilor din subarboarele drept. Un exemplu de arbore de sortare este următorul:



Arborele s-a obținut în urma citirii de la intrare a valorilor 6, 9, 2, 10, 4, 7. Dacă următoarea valoare citită va fi 8, se va crea un vârf ca descendent drept al celui ce conține valoarea 7, iar noul vârf va conține (va fi etichetat cu) valoarea 8.

Este folosită clasa `IO` descrisă în capitolul anterior. Esența programului este cuprinsă în clasa `elem`.

Câmpurile clasei sunt variabila întreagă `v` și referințele `st` și `dr` de tip `elem`, corespunzătoare unei înregistrări asociate unui vârf, care memorează valoarea asociată vârfului și adresele descendenților stâng și drept. De asemenea apare câmpul `rad` cu modificatorul `static`, deoarece el reprezintă referința la rădăcina arborelui, deci în mod natural este comun tuturor obiectelor de tipul `elem`.

Constructorul `elem` cu un parametru inițializează câmpurile obiectului care a produs invocarea sa cu valoarea asociată și cu referințele nule, adică este creată o frunză în arbore.

Constructorul fără nici un parametru nu prevede nici o acțiune. Știm din primul capitol că în lipsa vreunui constructor prezența sa este implicită, dar cum clasa `elem` include un constructor, el trebuie specificat explicit.

Metoda `creare` creează arborele de sortare astfel:

- este citit un număr întreg și este creat arborele (frunza) ce conține acea valoare; "adresa" sa este întoarsă în referința statică `rad`;
- atâta timp cât la intrare urmează valori numerice (prima valoare nenumerică este detectată cu ajutorul metodei `isNaN`), ele sunt citite și adăugate, prin invocarea metodei `adaug`, arborelui de sortare curent.

Metoda `adaug` primește referința către un vârf și o valoare întreagă. Este creată o nouă frunză cu ajutorul constructorului `elem` și este întoarsă "adresa" ei astfel încât frunza să devină descendentul stâng sau drept al unei frunze din arborele de rădăcină `x`. Determinarea vârfului la care este "atărnată" noua frunză se face recursiv prin invocarea metodei pentru descendentul stâng sau cel drept, după cum valoarea este mai mică sau mai mare decât decât informația din vârful primit ca parametru; valoarea întoarsă este atribuită respectivului descendent.

Metoda `parcurs` întoarce un șir ce conține, în inordine, valorile din subarboarele de rădăcină `x` (parametrul metodei).

Clasa principală `ArbSort`, prin metoda `main`, creează o instanță `Ob` a clasei `elem` și, prin intermediul acestui obiect, invocă metodele `creare` și `parcurs` ale acestei clase. Pentru metoda `parcurs` este prevăzut argumentul `elem.rad`, adică rădăcina arborelui de sortare.

Pentru o mai bună înțelegere, să presupunem că obiectul `rad` are ca descendent stâng obiectul `rads`, iar ca descendent drept obiectul `radd`. În cazul adăugării unei valori `val` în arborele de rădăcină `rad`, lucrurile se petrec după cum urmează. Obiectul `rad` este pus să execute metoda sa `adaug`. Dacă `val` este mai mic sau egal cu câmpul numeric al obiectului `rad`, atunci `rad` "comandă" obiectului `rads` să execute metoda sa `adaug`, adică să insereze valoarea `val` în subarboarele de rădăcină `rads`. Cazul complementar este tratat analog.

```

class elem {
    int v; elem st,dr; static elem rad;
    elem() { }
    elem(int val) { v = val; }

    void creare() {
        double val;
        for ( ; ; ) {
            val = IO.read();
            if ( Double.isNaN(val) ) break;
            else rad = adaug(rad, (int) val);
        }
    }

    elem adaug(elem x, int val) {
        if (x==null) x = new elem(val);
    }
}
  
```

```

    else if (val<x.v) x.st=adaug(x.st,val);
        else x.dr=adaug(x.dr,val);
    return x;
}

String parcurg(elem x) {
    if (x==null) return "";
    else return parcurg(x.st) + x.v + " " + parcurg(x.dr);
}

}

class ArbSort {
    public static void main(String[] arg) {
        elem Ob = new elem();
        Ob.creare(); IO.writeln(Ob.parcurg(elem.rad));
    }
}

```

Observăm că este respectat unul dintre principiile programării orientate spre obiecte și anume *încapsularea*: clasa elem "pune la un loc" câmpuri și operații (metode) prin care aceste câmpuri sunt prelucrate.

Programul de mai sus se pretează la o scurtă (și incompletă) discuție asupra noțiunii de *tip* de date.

Să considerăm de exemplu tipul primitiv `int`. În multe locuri, chiar acest tip simplu este prezentat incorect: se reduce noțiunea de tip la mulțimea valorilor posibile. În realitate este vorba nu numai de această mulțime, ci și de prelucrările ce se pot efectua asupra valorilor de tipul respectiv (adunare, scădere etc.).

Lucrurile devin mai complexe pentru clase. O clasă este un tip caracterizat prin:

- câmpuri (cu tipurile lor);
- constructori (cu signaturile și corpurile lor);
- metode (cu signaturile și corpurile lor, precum și cu tipul rezultatului întors);
- drepturile de acces la fiecare constructor, câmp și metodă;
- drepturile de moștenire referitoare la câmpuri și metode.

Despre drepturile de acces și moștenire vom discuta în capitolele următoare.

3 ALTE ELEMENTE DE BAZĂ ALE LIMBAJULUI JAVA. APLICAȚII

3.1. Lucrul cu tablouri în Java

Prezentăm în acest paragraf elementele de bază legate de tablouri și lucrul cu ele.

Un *tablou* este o secvență de componente de același tip. Acest tip poate fi un tip primitiv sau un tip referință (deci putem lucra cu tablouri de obiecte).

Un tablou a poate fi declarat folosind una dintre următoarele modalități:

```

tip[] a;
tip a[];

```

unde tip este tipul componentelor tabloului. În continuare vom folosi numai prima formă.

Declararea unui tablou nu are drept consecință crearea sa. Crearea tabloului a declarat mai sus trebuie făcută explicit, prin:

```

a = new tip[n];

```

unde n este o constantă sau o variabilă întreagă ce a primit o valoare strict pozitivă.

Cele de mai sus devin clare dacă specificăm faptul (menționat și în primul capitol) că *un tablou este un tip referință*. Prin creare se obține un obiect de tip tablou (obiect numit prin abuz de limbaj tot tablou).

Componentele tabloului pot fi referite prin `a[i]`, cu i luând valori în intervalul `0..n-1`; dacă i nu este în acest interval, va fi semnalată o eroare la executare. Lungimea tabloului poate fi referită prin `a.length`.

Declararea și crearea pot fi făcute și simultan prin:

```

tip[] a = new tip[n];

```

sau printr-o inițializare efectivă, ca de exemplu:

```

int[] a = {0,3,2,5,1}

```

prin care, evident, `a.length` devine 5.

Câmpul `length` al unui (obiect de tip) tablou este un câmp constant (cu modificatorii `public` și `final`) de tip `int`; deci, odată creat, un obiect tablou nu

își poate schimba dimensiunea (numărul de componente). Pe de altă parte, variabilei referință la tablou i se poate asocia o referință la un tablou de același tip.

Exemplul 1. Următoarea secvență de program:

```
tip[] a ;
a = new tip[10]; ...
a = new tip[20]; ...
```

este corectă. Evident, noul tablou nu are nici o legătură cu cel vechi (în particular nu se păstrează valoarea nici unei componente a tabloului vechi).

Exemplul 2. Pentru interschimbarea conținutului a două tablouri se poate proceda la fel ca pentru interschimbarea valorilor a două variabile primitive:

```
int[] a = {1,2,3,4}, b = {11,12,13}, c;
c=a; a=b; b=c;
```

deci practic se interschimbă referințele la cele două tablouri.

Exemplul 3. Deoarece un tablou este un tip referință, transmiterea sa ca argument la invocarea unei metode se supune regulilor deja cunoscute referitoare la apelul prin valoare. Astfel, executarea următorului program:

```
class C {
    void met(int[] a) {
        a[0]=9;
        a = new int[5]; for (int i=0; i<5; i++) a[i]=0;
    }
}

class Tablou2 {
    public static void main(String[] s) {
        int[] a = {1,2,3,4};
        C Ob = new C(); Ob.met(a);
        for (int i=0 ; i<a.length; i++) IO.write(a[i]+" ");
    }
}
```

va produce la ieșire:

```
9 2 3 4
```

Exemplul 4. Următorul program este menit să reamintească efectul modificatorului **final** pentru variabile de tip referință, în particular pentru tablouri:

```
class A {
    static final int[] x = {1,2};
    void modify() { x[0] = 11; }
}

class TabFinal {
    public static void main (String[] args) {
        A a = new A(); a.modify();
        for (int i=0; i<A.x.length; i++) IO.write(A.x[i]+" ");
    }
}
```

```
}
}
```

și va produce la ieșire:

```
11 2
```

Mai precizăm că un tablou de caractere nu este un șir de caractere.

De asemenea sunt permise tablouri având ca tip o interfață sau o clasă abstractă. Precizările necesare apar în capitolele ce urmează.

Limbajul Java permite lucrul cu tablouri multidimensionale. De exemplu expresia `C[][]` este un tip ce reprezintă tablouri bidimensionale ale căror componente au tipul `C`.

Tablourile multidimensionale trebuie gândite ca tablouri unidimensionale ale căror elemente sunt tablouri unidimensionale etc. De aceea referirea la un element al unui tablou multidimensional a se face prin:

```
a[indice1]...[indicen].
```

Este suficient să reducem discuția la tablouri bidimensionale, generalizarea fiind imediată. Să considerăm următorul exemplu:

```
int[][] a = new int[3][];
a[0] = new int[3];
a[1] = new int[4];
a[2] = new int[2];
```

Ia naștere astfel un tablou de forma:

ceea ce arată că în Java tablourile nu sunt neapărat dreptunghiulare; aceasta conduce desigur la economie de spațiu.

Evident, `a[1].length=4`.

La aceeași structură se poate ajunge și printr-o inițializare efectivă:

```
int[][] a = { {0,1,2}, {1,2,3,4}, {2,3} };
```

care în plus atribuie valori elementelor tabloului.

Dacă în referirea la un element al unui tablou unul dintre indici nu este în intervalul corespunzător, va apărea eroarea (numită în Java *excepție*) cu numele: `IndexOutOfBoundsException`.

Reamintim că unei referințe la un tablou i se poate asocia o referință la un alt tablou. Astfel, pentru exemplul de mai sus poate apare în continuare:

```
a[1] = new int[7];
```

ceea ce permite să schimbăm "o parte" dintr-un tablou fără a defini un altul și a copia părțile comune. De asemenea putem de exemplu să interschimbăm două "linii" ale tabloului de mai sus prin:

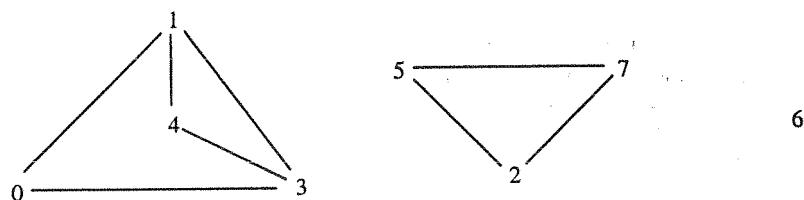
```
int[] temp;
temp = a[0]; a[0] = a[2]; a[2] = temp;
```

3.2. Aplicația 1 : Parcurgerea în adâncime a unui graf neorientat

Parcurgerea grafurilor apare în multe probleme din teoria grafurilor. Ea constă în a pleca dintr-un vârf al grafului și de a trece, mergând pe muchii, prin toate celelalte vârfuri. Un vârf poate fi atins de mai multe ori, dar este pusă în evidență numai o singură dată atingerea sa, numită *vizitare*, ce corespunde unei prelucrări a vârfului respectiv, prelucrare specifică fiecărei probleme concrete.

Evident, plecând dintr-un vârf putem atinge doar vârfurile din componenta sa conexă. De aceea în continuare se va trece la un vârf ce nu a fost atins, deci din altă componentă conexă.

Graful este reprezentat prin lista vecinilor fiecărui vârf. Vom folosi pentru exemplificare următorul graf:



pentru care listele vecinilor vârfurilor sunt:

$L_0 = \{3, 1\};$ $L_1 = \{3, 0, 4\};$ $L_2 = \{5, 7\};$ $L_3 = \{0, 4, 1\};$
 $L_4 = \{3, 1\};$ $L_5 = \{2, 7\};$ $L_6 = \emptyset;$ $L_7 = \{2, 5\}.$

Prezentăm în continuare algoritmul *DF* (Depth First) de parcurgere în adâncime a unui graf neorientat cu n vârfuri.

1) se citește valoarea n ;

2) se citesc listele vecinilor vârfurilor;

3) într-un tablou boolean *atins* de dimensiune n se inițializează elementele sale cu false, pentru a marca faptul că nici un vârf nu a fost încă vizitat; la vizitarea unui vârf, elementul respectiv va primi valoarea true;

4) se execută instrucțiunea (scrisă în pseudocod):

```
for i=0, n-1
  if !atinsi then DF(i)
endif
endfor
```

unde metoda *DF* are corpul:

```
scrie valoarea lui i; atinsi ← true;
for toți j ∈ Li (în ordinea în care apar în Li)
  if !atinsj
    then DF(j)
  endif
endif
endfor
```

Drept rezultat, vârfurile vor apărea în ordinea:

0 3 4 1 2 5 7 6

Programul care urmează implementează algoritmul descris mai sus, memorând listele vecinilor vârfurilor într-un tablou bidimensional. Nu sunt deci utilizate liste înlănțuite.

Clasa principală este *ParcDF*. Metoda *main* creează un obiect *Ob* al clasei *df*, iar apoi îl folosește pentru invocarea metodei *parcure* al acestei clase. Este folosită și clasa *IO* prezentată anterior.

Clasa *df* utilizează în principal următoarele câmpuri:

- *n* = numărul de vârfuri din graf;

- *mat* = matricea (tabloul bidimensional) care va conține listele vecinilor fiecărui vârf;

- *atins* = tablou care memorează pentru fiecare vârf dacă el a fost sau nu vizitat la momentul curent al parcurgerii.

Constructorul clasei nu are parametri și are rolul de a face inițializări. Se începe prin citirea numărului n de vârfuri ale grafului. Sunt create tabloul *temp* de dimensiune n și tabloul *mat* de aceeași dimensiune, ale cărui elemente sunt tablouri. În continuare, pentru fiecare vârf i se citesc în *temp* vecinii săi până când este detectat la intrare altceva decât un număr; pentru aceasta, la intrare lista vecinilor unui vârf se va termina (de exemplu) cu o literă; prin program acest "delimitator" este detectat de funcția standard *Double.isNaN*. Numărul vecinilor lui i este calculat în *ntemp*, iar vecinii sunt citați în primele *ntemp* componente ale lui *temp*. Apoi este creat vectorul *mat[i]* de lungime *ntemp* și sunt mutate în el primele *ntemp* componente ale lui *temp*.

Este evident că tabloul bidimensional *mat* nu este în general unul dreptunghiular, deoarece numărul vecinilor unui vârf poate să difere de numărul vecinilor altui vârf, așa cum se întâmplă și în exemplul considerat.

Metoda *parcure* mai folosește variabila *cc* a cărei valoare reprezintă numărul ultimei componente conexe determinate. Este folosită și apelată recursiv metoda *DF*, a cărei acțiune a fost descrisă anterior.

Să observăm că din afara clasei *df* știm doar că ea conține un constructor și metoda *parcure*, ambele fără nici un parametru. În acest mod toată esența problemei este încapsulată și ascunsă în clasa *df*, urmându-se unul dintre imperativele programării orientate spre obiecte.

```
class df {
  int[][] mat; int n; boolean[] atins;

  df() {
    double d; int[] temp; int ntemp;
    IO.write("n = "); n = (int) IO.read();
    mat = new int[n][]; temp = new int[n];
```

```

for (int i=0; i<n; i++) {
    IO.write("Vecinii lui " + i + " : ");
    ntemp = 0; d = IO.read();
    while ( ! Double.isNaN(d) )
        { temp[ntemp++] = (int) d; d = IO.read(); }
    mat[i] = new int[ntemp];
    for (int j=0; j<ntemp; j++) mat[i][j] = temp[j];
}
IO.writeln("*****");

void listare() {
    for (int i=0; i<n; i++) {
        IO.write(i + " :\t");
        for (int j=0; j<mat[i].length; j++)
            IO.write(mat[i][j] + " ");
        IO.writeln();
    }
    IO.writeln("*****");
}

void parcurg() {
    atins = new boolean[n];
    int cc=0, i;
    for (i=0; i<n; i++) atins[i] = false;
    for (i=0; i<n; i++)
        if ( ! atins[i] ) {
            IO.write("Comp. conexa " + ++cc + " :\t");
            DF(i); IO.writeln();
        }
}

private void DF(int i) {
    int j,k;
    atins[i] = true; IO.write(i + " ");
    for (j=0; j<mat[i].length; j++) {
        k=mat[i][j]; if ( ! atins[k] ) DF(k);
    }
}

}

class ParCDF {
    public static void main(String[] s) {
        df Ob = new df(); Ob.listare(); Ob.parcarg();
    }
}

```

Pentru exemplul considerat, programul de mai sus va produce la ieșire:

```

Comp. conexa 1 :  0 3 4 1
Comp. conexa 2 :  2 5 7
Comp. conexa 3 :  6

```

3.3. Referința *this* și blocuri de inițializare

Într-o metodă care nu este declarată cu modificatorul **static**, putem utiliza referința obiect **this** pentru a ne referi la obiectul curent, adică cel care a invocat metoda. Prezentăm doar trei situații în care această referință specială este de obicei folosită:

1) În interiorul constructorilor:

```

class clasa {
    ... int i; ...
    clasa(int i) { this.i=i; }
    ...
}

```

prin care câmpul *i* al clasei *clasa* primește valoarea *i* transmisă la crearea obiectului prin utilizarea constructorului de mai sus. În acest mod este clară distincția între câmpul *i* și parametrul *i*. Evident, am putea renumi parametrul:

```

clasa (int ii) { i=ii; }

```

ceea ce poate conduce însă uneori la confuzii privind semnificația variabilelor.

2) La apelarea unei metode, când dorim să transmitem ca parametru obiectul curent. Aceasta este util de exemplu când vrem să creăm un nou obiect care să păstreze referința către cel curent:

```

class Clasa {
    int i,j; Clasa Ob;
    ...
    Clasa(int i, Clasa Ob) {this.i=i; this.Ob=Ob;}
    ...
    Ob=new Clasa(j, this);
    ...
}

```

3) Când dorim ca rezultatul întors de o metodă să fie obiectul curent.

Este suficient să includem instrucțiunea:

```

return this;

```

Un al doilea aspect de care ne vom ocupa în acest paragraf îl constituie *blocurile de inițializare*.

O clasă poate avea unul sau mai multe blocuri de inițializare. Ele trebuie declarate cu modificatorul **static**:

```

static { ... }

```

Blocurile statice pot opera numai asupra unor entități statice. În principal, ele sunt folosite pentru inițializarea unor câmpuri statice ale clasei cărora dorim să le atribuim de la început alte valori decât cele implicite. Inițializările statice, realizate în cadrul blocurilor de inițializare, se fac în ordinea în care apar acestea în cadrul clasei. Într-un bloc static nu putem folosi **return** sau **this**.

Declararea câmpurilor asupra cărora acționează un bloc de inițializare trebuie, în mod logic, să preceadă acel bloc. Astfel, în exemplul următor:

```
class C {
    static int[] a = new int[10];
    static {
        for (int i=0; i<a.length; i++) a[i]=1;
    }
    ...
}
```

este clar că blocul de inițializare trebuie *precedat* de crearea vectorului a. Executarea blocului static inițializează componentele vectorului a cu 1.

3.4. Inițializarea claselor

Discuția de mai sus asupra blocurilor de inițializare ridică următoarea problemă: când sunt ele executate? Pentru aceasta trebuie vorbit întâi despre *inițializarea claselor*.

Începem prin a preciza că un câmp se numește *constant* dacă este declarat cu modificatorii **static** și **final** și în plus în declarație este inițializat cu o expresie constantă. O *expresie constantă* poate conține numai:

- literal având tipuri primitive sau tipul String;
- conversii la un tip primitiv sau la tipul String;
- operatorii următori:

+ - ! * / % , > <= >= && || operatorul (? :)
(deci nu ++, -- sau instanceof)

- variabile (eventual precalificate) declarate cu **final** și inițializate cu expresii constante.

Expresiile constante sunt evaluate în faza de compilare. De asemenea câmpurile constante sunt inițializate la compilare.

O clasă este *inițializată* la prima utilizare *activă* a ei, adică atunci când:

- este invocată o metodă declarată în clasa respectivă;
- este invocat un constructor al clasei;
- este creat un tablou ale cărui elemente sunt de tipul clasei;
- este folosit un câmp neconstant declarat în clasă sau se atribuie o valoare unui astfel de câmp.

După cum vom vedea în capitolul referitor la extinderea claselor, înainte ca o clasă să fie inițializată este inițializată superclasa sa.

Când o clasă este inițializată, are loc inițializarea câmpurilor și blocurilor statice, exact în ordinea în care apar. În consecință o inițializare poate conține referiri numai la câmpuri declarate anterior. Astfel secvența de declarații:

```
static int i = j;
static int j = 1;
```

este incorectă și va fi semnalizată o eroare la compilare.

Regula de mai sus nu se extinde însă asupra metodelor. Astfel următorul program este corect:

```
class C {
    static int scrie() { return j; }
    static int i = scrie();
    static int j = 1;
}

class Init {
    public static void main(String[] sir) {
        IO.println(C.i + "");
    }
}
```

și produce la ieșire valoarea 0, deoarece inițializarea cu 1 a variabilei j are loc după ce i a primit valoare prin invocarea metodei scrie.

3.5. Aplicația 2 : Generarea tuturor (n, k) - combinărilor

Fie date numerele naturale n, k cu $n \geq k > 0$. Numim (n, k) - *combinare* un k -uplu $(a_1, \dots, a_k) \in \{1, 2, \dots, n\}^k$ cu $a_1 < a_2 < \dots < a_k$. Fie $C(n, k)$ mulțimea lor. Se știe că numărul (n, k) - combinărilor este C_n^k , adică $|C(n, k)|$. Ne propunem în continuare să scriem un program care să producă toate (n, k) - combinările.

Evident, nu este vorba de a calcula C_n^k , ci de a produce elementele lui $C(n, k)$; altfel spus, cerințele problemei nu sunt de natură cantitativă, ci calitativă. Totuși, aparent paradoxal, vom pleca de la formulele cantitative:

$$C_n^0 = 1 \quad (1)$$

$$C_n^k = C_{n-1}^{k-1} + C_{n-1}^k \text{ pentru } n > k \quad (1')$$

Formulele cantitative de mai sus sunt ușor de demonstrat prin inducție și apar în orice culegere de probleme ce acoperă clasa a X-a. Ce nu apare în aceste culegeri, lăsând să se creadă că este vorba numai de artificii de calcul, este aspectul lor calitativ. Mai precis, semnificația lui (1') este:

$$C(n, k) = \bigcup_{i=k}^n C(i-1, k-1) \quad (2)$$

unde $C(i-1, k-1)$ este mulțimea obținută prin adăugarea elementului i la fiecare dintre vectorii de lungime $k-1$ din $C(i-1, k-1)$. Vom nota acest lucru prin:

$$C(i-1, k-1) = C(i-1, k-1) \cdot i.$$

Să exemplificăm cele de mai sus pentru $n=4, k=2$.

$$\begin{aligned} C(4, 2) &= C(1, 1) \cdot 2 \cup C(2, 1) \cdot 3 \cup C(3, 1) \cdot 4 = \\ &= \{(1)\} \cdot 2 \cup \{(1), (2)\} \cdot 3 \cup \{(1), (2), (3)\} \cdot 4 = \\ &= \{(1, 2)\} \cup \{(1, 3), (2, 3)\} \cup \{(1, 4), (2, 4), (3, 4)\} = \\ &= \{(1, 2), (1, 3), (2, 3), (1, 4), (2, 4), (3, 4)\} \end{aligned}$$

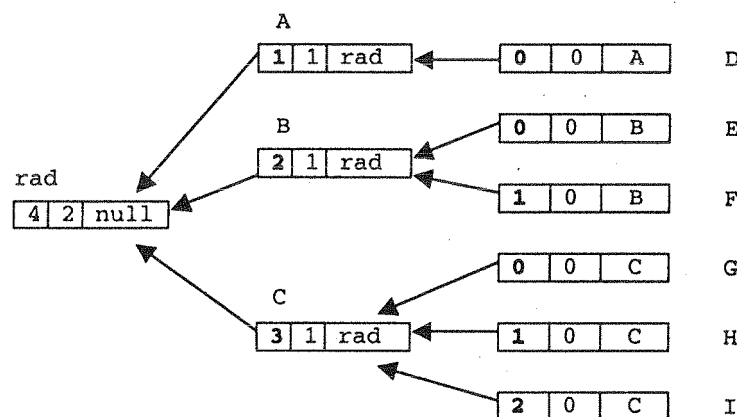
Să mai observăm că elementul care se concatenează unui $C(i, k)$ este $i+1$, deci această din urmă valoare este bine determinată și nu mai trebuie "ținută minte" explicit.

Este clar că:

- dezvoltarea nodurilor din arbore se oprește când k devine 0;
- $(4, 2)$ - combinațiile se regăsesc parcurgând drumul de la frunze către rădăcină.

Plecând de la cele de mai sus, vom construi un arbore de obiecte pentru care legăturile merg spre tatăl lor. Câmpurile obiectelor vor fi constituite din valorile n, k proprii și referința către obiectul precedent.

Dorim deci să obținem următorul arbore de obiecte (în care obiectele au primit nume doar pentru claritate):



arbore în care dacă parcurgem drumurile de la frunze către rădăcină, câmpurile n ale obiectelor de pe aceste drumuri (fără a considera însă rădăcina) produc o (n, k) - combinatie. De exemplu plecând de la obiectul H, vom obține $(2, 4)$.

Putem trece acum la descrierea programului.

Metoda principală construiește un obiect Ob de tipul clasei elem pe baza căruia este invocată metoda prel a clasei elem.

Clasa elem cuprinde câmpurile n, k și tata (tata indică spre tatăl obiectului curent în structura de arbore). Constructorul fără parametri nu prevede vreo acțiune, iar constructorul cu trei parametri creează un obiect și stabilește legătura către tatăl său.

Metoda prel citește în câmpurile statice $n0$ și $k0$ valorile inițiale ale lui n și k , iar apoi creează obiectul rad prin intermediul căruia invocă metoda p.

Metoda p construiește descendenți ai nodurilor arborelui atâta timp cât $k > 0$; dacă s-a ajuns la $k=0$, atunci se invocă metoda print pentru parcurgerea

drumului de la frunză spre rădăcină și implicit pentru listarea (n, k) - combinații asociate.

Să considerăm de exemplu situația în care obiectul E invocă metoda p (fără parametri). Cum $k=0$, obiectul E va tipări valoarea lui cnk (vezi mai jos) urmată de rezultatul întors prin executarea metodei print fără parametrii. Ca urmare va fi întors șirul format din 1 (valoarea expresiei $n+1$, unde n este câmpul lui E, acesta fiind obiectul ce realizează invocarea), concatenat cu rezultatul invocării B.print(). Dar acest din urmă rezultat este șirul format din 3 (valoarea expresiei $n+1$, unde n este câmpul lui B), urmat de rezultatul invocării rad.p(), care este șirul vid deoarece câmpul tata al obiectului rad este null. În consecință la ieșire va apărea combinația $(1, 3)$.

În clasă a mai fost folosit câmpul static cnk ce numără, pentru verificare, câte (n, k) - combinații au fost generate. Așa cum am văzut, metoda principală va tipări și această valoare.

Programul este următorul:

```

class elem {
    int n,k; elem tata;
    static int cnk;

    elem(int nn, int kk, elem x) { n = nn; k = kk; tata = x; }

    String print() {
        if (tata==null) return ("");
        else return n+1 + " " + tata.print();
    }

    void p() {
        elem x=null; int i;
        if (k == 0) IO.writeln(++cnk + "\t: " + print());
        else for (i = k; i<=n; i++) {
            x = new elem(i-1, k-1, this); x.p();
        }
    }
}

class Comb {
    public static void main(String[] arg) {
        IO.write("n,k = ");
        int n= (int) IO.read(); int k= (int) IO.read();
        elem prim = new elem(n,k,null);
        prim.p();
        IO.writeln("n= " + n + " k= " + k + " cnk= " + elem.cnk);
    }
}
  
```

Atragem din nou atenția asupra importanței cunoașterii exacte a obiectului curent, așa cum s-a văzut la interpretarea corectă a semnificației lui n în metoda print.

un pachet fără nume, constituit din aceste unități; dar în acest mod clasele pachetului fără nume pot fi accesate numai din interiorul pachetului, ceea ce evident reprezintă un dezavantaj.

Ne îndreptăm în continuare atenția asupra pachetelor cu nume. Pachetele sunt organizate în directoare în sistemul de fișiere al mașinii gazdă.

Un pachet poate conține:

- subpachete ale pachetului;
- tipuri (clase sau interfețe) declarate în unitățile de compilare ale pachetului (având toate aceeași declarație **package**).

Pentru a crea într-un director un pachet, al cărui nume dorim să fie pachet, trebuie creat un subdirector cu numele pachet și apoi introduse în subdirector unitățile de compilare ale pachetului.

Fiecare unitate de compilare din pachetul pachet trebuie să înceapă cu declarația:

```
package pachet;
```

care dă informații compilatorului asupra numelui pachetului din care face parte unitatea de compilare.

Dacă o unitate de compilare face referire la o clasă C din pachetul pachet, atunci ca prima linie a acestui fișier trebuie plasată declarația **import** :

```
import pachet.C;
```

Dacă dorim acces la toate clasele pachetului, putem folosi următoarea formă pentru declarație:

```
import pachet.*;
```

Structura de pachete este ierarhică (formează un arbore); un pachet P1 poate avea un subpachet P2. Această structură trebuie urmată și în declarațiile **import**, în care trebuie precizat P1 . P2 (vezi exemplul de mai jos).

O clasă sau interfață dintr-un pachet poate fi accesată din afara pachetului doar dacă are modificatorul **public**. Aceeași regulă este valabilă pentru accesarea directă a membrilor unei clase a pachetului. Pentru constructori trebuie procedat la fel, afară de cazul în care se folosește constructorul implicit, care este considerat ca având modificatorul **public**.

Exemplul 1. Considerăm următoarea structură de directoare:

C:

HG

A

B

B

UTIL

unde directorul HG conține pachetul A, care are ca subpachet pe B. În A se află unitățile de compilare Unu.java, Doi.java și Trei.java, în subdirectorul B al

lui A se află unitatea de compilare Patru.java, iar în subdirectorul B al lui X se află unitatea de compilare Test.java, descrise în cele ce urmează. Cum toate unitățile de compilare folosesc clasa IO.java, unitatea cu acest nume este plasată în pachetul UTIL.

Unitatea de compilare Unu.java:

```
package A;
import UTIL.*;
public class Unu {
    int k=1;
    public void met() { IO.writeln(k + " Unu"); }
}
```

Unitatea de compilare Doi.java:

```
package A;
import UTIL.*;
public class Doi {
    public static int k=2;
    public static void met() { IO.writeln(" Doi"); }
}
```

Unitatea de compilare Trei.java:

```
package A;
import UTIL.*;
public class Trei {
    static void met() { IO.writeln("Trei"); }
}
```

Unitatea de compilare Patru.java:

```
package A.B;
import UTIL.*;
public class Patru {
    public static void met() { IO.writeln("Patru"); }
}
```

Unitatea de compilare Test.java:

```
import A.*;
import A.B.*;
import UTIL.*;
class Test {
    public static void main(String[] s) {
        Unu Ob = new Unu(); Ob.met();
        System.out.print(Doi.k); Doi.met();
        Patru.met();
    }
}
```

În plus, în unitatea IO.java vom adăuga ca primă linie:

```
package UTIL;
```

Înainte de a descrie modul în care este executată aplicația, sunt necesare câteva precizări suplimentare relative la comenzile `java` și `javac` și la variabila de mediu `CLASSPATH`.

La executarea unor comenzi ale mașinii virtuale Java, ca de exemplu `javac` și `java` (ce vor fi descrise mai jos), are loc o *căutare de clase*, adică de fișiere cu extensiile `.java` și `.class`. Pentru precizarea locului unde vor fi căutate aceste clase, Java prevede variabila de mediu `CLASSPATH`. Valoarea variabilei (numită și *calea pentru cod*) este o mulțime de căi, referită în continuare prin `path`; căile din `path` sunt separate între ele prin caracterul `' ; '`.

Calea pentru cod `path` poate fi setată la pornirea sistemului; de exemplu pentru Windows 98, setarea se poate face în `autoexec.bat`. Este considerat implicit și directorul curent. Apoi variabilei `CLASSPATH` i se poate atribui o valoare prin comanda:

```
SET CLASSPATH=path
```

sau prin opțiunea `classpath` din comenzile `java` și `javac`.

În comanda `SET`, este indicat să precizăm calea cu specificarea partiției (de exemplu `C:\...`) pentru ca ea să fie regăsită chiar dacă lucrăm din altă partiție. În unele situații este util să precizăm o cale vidă:

```
SET CLASSPATH=
```

Ordinea în care apar căile în `path` determină ordinea de căutare a claselor.

Atribuirea unei valori variabilei `CLASSPATH` anulează atribuirea precedentă. În particular, pentru păstrarea directorului curent trebuie inclus explicit caracterul `'.'`.

Precizările de mai sus sunt valabile și pentru setările prin opțiunea `classpath` din comenzile `javac` și `java`, cu observația că în acest caz noua setare este valabilă numai pentru comanda respectivă.

Executarea comenzii `javac` are ca efect compilarea uneia sau a mai multor unități de compilare, ce conțin definiții de clase sau interfețe. Sunt compilate toate tipurile (clase sau interfețe) din unitățile respective, dar și toate tipurile referite din ele și care sunt regăsite în calea pentru cod.

O formă restrânsă a comenzii este:

```
javac opt unit
```

unde atât opțiunile din lista `opt`, cât și unitățile de compilare din lista `unit` sunt separate între ele prin blankuri.

Dintre opțiuni vom menționa numai următoarele două:

```
-d dir
```

```
-classpath path sau, prescurtat: -cp path
```

Prima opțiune permite plasarea fișierelor byte-code, rezultate în urma compilării, în directorul `dir`; în lipsa acestei opțiuni, drept `dir` este considerat directorul curent.

A doua opțiune permite precizarea unei noi căi de cod, în care se va încerca regăsirea (căutarea) claselor. Căutarea include atât fișierele cu extensia

`.java`, cât și cele cu extensia `.class`. La căutarea unui tip (clasă sau interfață) tip deosebim trei cazuri:

- 1) este găsit doar tip `.class`: este folosit acest fișier byte-code;
- 2) este găsit doar tip `.java`: este compilat acest tip și este folosit rezultatul compilării;
- 3) sunt găsite atât tip `.class` cât și tip `.java`: este folosit tip `.class` doar dacă acest fișier este mai recent decât tip `.java`.

Executarea comenzii `java` pornește o aplicație Java. Prezentăm aici doar o formă restrânsă a ei:

```
java opt clasa
```

unde `opt` este o listă de opțiuni, iar `clasa` este o clasă care conține metoda principală declarată prin:

```
public static void main(String[] sir)
```

Ca efect este încărcată clasa și invocată metoda principală.

Dintre opțiuni menționăm doar:

```
-classpath path sau, prescurtat: -cp path
```

```
-Dprop=val
```

Prima opțiune de mai sus precizează o nouă cale pentru cod.

A doua opțiune setează valoarea `val` pentru o proprietate `prop` a sistemului. Vom folosi această opțiune în capitolul referitor la invocarea la distanță a metodelor, unde vom face și precizările de rigoare.

Revenim la exemplul considerat, prezentând o modalitate de executare a aplicației.

Dacă ne aflăm în directorul `C:\HG\B`, vom compila fișierul sursă `Test.java` cu comanda:

```
javac -classpath .. Test.java
```

Nu este necesar ca unitățile aflate în diferitele pachete să fie anterior compilate.

Executarea va fi comandată prin:

```
java -classpath .. Test
```

prin care la ieșire va apărea:

```
1 Unu
```

```
2 Doi
```

```
Patru
```

A fost folosită opțiunea de compilare `classpath`. Explicațiile sunt următoarele:

- am presupus în mod tacit că, la pornirea sistemului, în calea pentru cod a fost introdusă calea `c:\java\bin`. În caz contrar, trebuie asigurat explicit accesul (atât pentru compilare, cât și pentru executare) la "clasele esențiale" furnizate de Java; aceasta se poate realiza de exemplu introducând în opțiunea `classpath` și calea `c:\java\lib\`. Reamintim că la începutul capitolului s-a făcut presupunerea că Java a fost instalată în `c:\java`.

- informațiile din **package** și **import** urmăresc ca, mergând în sus în arborele determinat de structura de subdirectoare, să se ajungă la un subdirector comun (în exemplul nostru este vorba de `C:\HG`);

- la executare trebuie precizat și directorul curent, pentru a avea acces la fișierul `Test.class`.

Evident, subdirectorul comun menționat mai sus poate fi specificat și explicit în comenzi:

```
javac -classpath c:\HG\ Test.java
java -classpath .;c:\HG\ Test
```

Dacă din metoda principală a clasei `Test` am fi inclus invocarea:

```
Trei.met();
```

s-ar fi obținut o eroare la compilare, deoarece în clasa `Trei` metoda `met` nu are modificatorul **public**. De asemenea câmpul static `k` din clasa `Doi` trebuie să aibă neapărat modificatorul **public** pentru a putea fi referit din afara pachetului `A`; analog, o referire la câmpul `Ob.k` din clasa `Test` este incorectă.

Menționăm că nu este neapărat nevoie să compilăm în prealabil unitățile de compilare din directorul `A` și subdirectorul său `B`, acest lucru fiind făcut automat la compilarea clasei `Test`. De asemenea atragem atenția că dacă unitatea de compilare `Patru.java` ar fi inclusă și în pachetul `A`, la executare s-ar fi semnalat o eroare, datorită ambiguității invocării `Patru.met()` din clasa `Test.java`.

Un director poate conține mai multe pachete. De exemplu, pentru ca directorul `X` să conțină și un pachet cu numele `AA`, trebuie creat un subdirector `AA` al lui `X`, în care să introducem unitățile de compilare ale pachetului `AA`.

Încheiem acest paragraf cu observația (evidentă) că dacă folosim un mediu de programare (de exemplu *Kawa*), compilarea și executarea devin mult mai ușor de efectuat, dar ideile de bază rămân aceleași.

4.2. Vizibilitate și acces

Pachetele, clasele, metodele, câmpurile și variabilele sunt identificate prin numele lor. Problema care se pune este de a stabili din ce loc putem face referire la aceste nume (cu semnificația dorită), adică de a preciza *domeniul lor de vizibilitate*.

Cu acest prilej vom face o discuție mai amănunțită referitoare la modificatori, reluând și unele aspecte deja prezentate.

Discuția va fi restrânsă la stadiul actual al prezentării, care face abstracție de facilitatea de extindere a claselor, și va fi reluată ulterior.

• Variabile locale și etichete

Prin *bloc* înțelegem o secvență (eventual vidă) de instrucțiuni, cuprinsă între acolade. Un bloc *este o instrucțiune* și de aceea poate fi folosit în orice loc unde poate să apară o instrucțiune.

De exemplu orice metodă și orice constructor sunt formate dintr-un antet urmat de un bloc. În schimb ceea ce urmează după modificatorii și numele unei clase nu constituie un bloc, deși apar acoladele deschise și închise: conținutul unei clase nu este o succesiune de instrucțiuni.

Orice bloc poate conține la rândul său un alt bloc, putând astfel lua naștere o structură imbricată de blocuri.

Prin *variabilă locală* înțelegem o variabilă declarată în interiorul unui bloc, inclusiv într-o instrucțiune **for**.

Domeniul de vizibilitate al unei variabile locale este constituit din partea din blocul în care a fost declarată, ce urmează declarării. Prin urmare o variabilă locală nu poate fi referită înainte de a fi declarată. Variabilele locale există numai pe perioada în care blocul în care sunt declarate este în curs de executare.

Nu putem folosi facilitatea de imbricare a blocurilor pentru a redeclara o variabilă locală. O variabilă locală nu poate fi redefinită nici în blocul în care a fost declarată, nici într-un bloc inclus în acesta.

Regulile care guvernează domeniul de vizibilitate al etichetelor sunt aceleași ca pentru variabilele locale.

• Parametrii metodelor și constructorilor

Numele parametrilor dintr-o metodă sau dintr-un constructor sunt vizibile doar în interiorul acestora, adică în blocul care urmează antetului metodei sau constructorului. Este însă posibil ca într-un bloc dintr-o metodă sau dintr-un constructor să redeclaram numele unui parametru; în acest caz, în partea din bloc ce urmează redeclării, numele respectiv este considerat ca având semnificația dată de redeclare.

Cu alte cuvinte, domeniul de vizibilitate al unui parametru este constituit din corpul metodei sau constructorului în care apare, mai puțin blocurile (evident disjuncte) în care este redeclarat.

• Vizibilitatea în interiorul unei clase

Știm că o clasă este constituită din câmpuri, constructori (folosiți la crearea de obiecte) și metode (folosite la invocarea lor). Din orice constructor și din orice metodă putem să ne referim la orice câmp sau metodă a clasei. De asemenea dintr-un constructor putem apela alt constructor prin **this** urmat, între paranteze, de o listă de argumente ce respectă semnatura noului constructor.

Ordinea în care apar câmpurile nestatice și metodele în cadrul clasei nu este relevantă (de exemplu nu contează dacă un câmp static este declarat la începutul sau la sfârșitul clasei).

Totuși, dacă numele unui câmp este redeclarat într-o metodă (este folosit ca nume al unui parametru sau declarat într-un bloc), atunci declararea cea mai interioară este cea care primează când se face o referire la acel nume (identificator).

Putem spune că un parametru poate *ascunde* un câmp, iar o variabilă locală poate *ascunde* un parametru și/sau un câmp.

După cum vom vedea în continuare, numele unei metode nu poate fi ascuns.

• Modificatorul **final**

Modificatorul **final** poate fi atașat atât variabilelor (locale sau câmpuri), cât și metodelor.

În cazul variabilelor locale, variabilei *i* se atribuie valoare la declarare sau înainte de a fi folosită; variabila nu poate primi de două ori valoare. Menționăm că **final** este singurul modificator ce poate fi atașat unei variabile locale.

Unui câmp final trebuie să i se atribuie valoare fie prin inițializare, fie prin constructori. Când câmpul este static, există posibilitatea de a i se atribui o valoare printr-un bloc de inițializare static:

Exemplul 2. Următorul program:

```
class Clasa {
    static final int x; static { x=-1;}

    public static void met() {
        IO.write(x);
        final int x; x=3; IO.println(" " + x);
    }
}

public class temp {
    public static void main (String[] s) {
        Clasa Ob = new Clasa(); Ob.met();
    }
}

produce la ieșire valorile:
-1 3
```

Menționăm că în general modificatorul **final** este folosit împreună cu **static**, pentru a trata un câmp ca o "constantă cu nume".

Efectul modificatorului **final** asupra metodelor și claselor va fi prezentat atunci când vom vorbi despre extinderea claselor.

• Modificatorul **static**

Modificatorul **static** poate fi folosit la declararea câmpurilor și metodelor unei clase, precum și pentru a anunța un bloc de inițializare static.

O variabilă locală nu poate fi declarată cu **static**.

Fie *c* un câmp declarat cu **static** în clasa *Clasa*. Această declarație are două consecințe:

- câmpul *c* este comun tuturor obiectelor de tipul *Clasa*;
- referirea la câmp din exteriorul clasei se face fie conform regulii generale (prin crearea și utilizarea unei instanțe a clasei *Clasa*), fie direct, prin precalificarea câmpului cu numele clasei (*Clasa.c*).

După cum s-a precizat anterior, inițializarea câmpurilor statice se face la inițializarea clasei.

Fie acum *met* o metodă declarată cu modificatorul **static** în clasa *Clasa*; ea se numește *metodă statică* sau *metodă de clasă*. Invocarea ei se poate face fie conform regulii generale (prin crearea și utilizarea unei instanțe a clasei *Clasa*), fie direct, prin precalificarea metodei cu numele clasei: *Clasa.met(...)*;

De aceea într-o metodă statică nu poate fi folosită referința **this**.

O metodă statică poate accesa numai câmpurile și metodele statice ale clasei din care face parte sau a altei clase. Metodele statice (de clasă) sunt folosite de obicei pentru a efectua prelucrări asupra câmpurilor statice ale clasei. Putem "ocoli" însă această regulă de exemplu astfel:

- la invocarea metodei îi transmitem ca parametru o referință la un obiect;
- în metodă construim explicit un obiect de tipul clasei la ale cărei metode și câmpuri dorim să ne referim.

Lucrul cu blocuri de inițializare statice a fost descris anterior.

• Modificatorii de acces

Ne mărginim în acest paragraf la o succintă prezentare a acestor modificatori. O discuție mai detaliată va fi făcută după ce vom vorbi despre extinderea claselor.

Toate câmpurile și metodele unei clase sunt accesibile (pot fi referite) din interiorul clasei.

Rolul modificatorilor **public**, **protected**, **private** și cel implicit (numit uneori *package*) referitor la accesul din exteriorul unei clase la membrii săi este sintetizat în următorul tabel:

Modificator	Membrii clasei cu acest modificator sunt accesibili ...
public	de oriunde clasa este accesibilă
protected	din cod din același pachet
implicit (<i>package</i>)	din cod din același pachet
private	numai din interiorul clasei

Reamintim că modificatorii de mai sus joacă un rol nu numai în privința accesului, dar și pentru facilitarea de extindere a claselor, ceea ce va face diferența între modificatorul **protected** și cel implicit.

• Determinarea semnificației unui nume

Pentru un identificator folosit ca nume de variabilă (locală sau câmp), metodă sau clasă, semnificația sa se obține la prima căutare încununată de succes din următorul șir de căutări succesive:

- 1) variabilele locale declarate într-un bloc sau un ciclu **for**;
- 2) numai dacă suntem în interiorul unei metode sau a unui constructor: parametrii metodei sau constructorului;
- 3) membrii clasei, inclusiv cei rezultați prin extinderea claselor (vezi capitolul următor)
- 4) tipuri importate explicit;
- 5) tipuri declarate în același pachet;
- 6) tipuri importate implicit;
- 7) pachete disponibile pe sistemul gazdă.

Mai mult, se face distincție clară între tipurile de identificatori: nume de variabile (locale), de pachete, de clase, de metode și de câmpuri; iată de ce numele unei metode nestatice nu poate fi ascuns. Aceasta permite să folosim un același identificator pentru a numi una sau mai multe astfel de entități.

Exemplul 3. Următoarea clasă este corectă din punct de vedere sintactic, compilatorul nesemnălând vreo eroare:

```
class obsesie {
    obsesie obsesie;
    obsesie() { obsesie = new obsesie(); }
    obsesie obsesie (obsesie obsesie) {
        obsesie:
        while (obsesie != null)
            if ( obsesie.obsesie(obsesie)==obsesie ) break obsesie;
        return obsesie;
    }
}
```

Nu recomandăm însă să abuzăm în acest mod de distincția menționată. Clasa de mai sus poate fi scrisă sub următoarea formă mai clară și "neobsesivă":

```
class obsesie {
    obsesie C;
    obsesie() { C = new obsesie(); }
    obsesie met (obsesie param) {
        et:
        while (param != null) {
            if ( param.met(param)==param ) break et;
        }
        return param;
    }
}
```

5 EXTINDEREA CLASELOR

Așa cum am menționat de la început, limbajul Java a fost conceput ca un limbaj orientat pe obiecte. Am prezentat deja una dintre caracteristicile orientării spre obiecte și anume *încapsularea* (datele împreună cu operațiile asupra lor sunt "puse la un loc" și anume într-o clasă).

Vom studia acum modul în care Java tratează alte aspecte ale programării orientate pe obiecte: *extinderea claselor*, *polimorfismul* și *legarea dinamică*.

5.1. Cum se definesc clasele extinse

Limbajul Java pune la dispoziție și o altă facilități importantă legată de OOP (programarea orientată pe obiecte): *posibilitatea de extindere a claselor*. Foarte pe scurt, deci incomplet și nu în întregime riguros, aceasta constă în:

- o clasă poate fi extinsă, adăugându-se noi câmpuri și noi metode, care permit considerarea unor atribute suplimentare (dacă ne gândim la câmpuri) și unor operații noi (asupra câmpurilor "inițiale", dar și asupra câmpurilor nou adăugate);
- unele metode ale clasei pe care o extindem pot fi redefinite, iar anumite câmpuri ale acestei clase pot fi "ascunse";
- un obiect având ca tip clasa extinsă poate fi folosit oriunde este așteptat un obiect al clasei care a fost extinsă.

La prima vedere, rezolvarea problemelor de mai sus se poate face simplu: modificăm clasa, introducând sau/și modificând câmpurile și metodele clasei. În practica programării, această soluție este de neconceput. Pe de o parte se pot introduce erori, iar pe de altă parte utilizatorii clasei vechi nu vor mai putea folosi clasa așa cum o făceau înainte și cum vor în continuare să o facă, nefiind interesați de noile facilități; clasa (firma care a elaborat-o) își va pierde astfel vechii clienți. De aceea vechea clasă trebuie să rămână nemodificată, iar actualizarea ei trebuie făcută prin mecanismul de extindere a claselor, prezentat în continuare.

Să reținem deci ca o regulă generală de programare faptul că nu trebuie modificată o clasă testată și deja folosită de mulți utilizatori; cu alte cuvinte, nu trebuie modificat "contractul" ce a dus la scrierea clasei.

Exemplul 1. Să presupunem că dorim să urmărim mișcarea unui punct în plan. Vom începe prin a considera clasa:

```
class Punct {
    int x,y; Punct urm;
    Punct(int x, int y) { this.x=x; this.y=y; }
    void Origine() { x=0; y=0; }
    Punct Miscare(int x, int y) {
        Punct p = new Punct(x,y); urm=p; return p;
    }
}
```

Clasa Punct conține:

- câmpurile x, y, urm;
- constructorul Punct cu semnatura (int, int);
- metoda Origine cu semnatura () și metoda Miscare cu semnatura (int,int).

Vom extinde clasa Punct astfel încât punctul să aibă și o culoare:

```
class Pixel extends Punct {
    String culoare;
    Pixel(int x, int y, String culoare) {
        super(x,y); this.culoare=culoare;
    }
    void Origine() { super.Origine(); culoare="alb"; }
}
```

Clasa Pixel conține:

- câmpurile x,y,urm (moștenite de la clasa Punct) și culoare (care a fost adăugat prin extindere);
- constructorul Pixel cu semnatura (int,int,String);
- metoda Miscare cu semnatura (int,int), moștenită de la clasa Punct, precum și metoda Origine cu semnatura (), care *redefinește* metoda Origine a clasei Punct.

Este adoptată următoarea terminologie: clasa Punct este *superclasă* a lui Pixel, iar Pixel este *subclasă* (clasă extinsă) a lui Punct.

În Java, clasele formează o structură de arbore în care rădăcina este clasa Object, a cărei definiție apare în pachetul java.lang. Orice clasă extinde direct (implicit sau explicit) sau indirect clasa Object. Știm că în informatică arborii "cresc în jos", deci rădăcina (clasa Object) se află pe cel mai de sus nivel.

Termenii de superclasă și subclasă se referă exclusiv la relația tată ↔ fiu, conform relației de extindere, în arborele de clase. *Este incorrect de a interpreta acești termeni în sensul de incluziune!*

În exemplul de mai sus, apelurile `super.Origine()` și `super(x,y)` se referă respectiv la metoda Origine și la constructorul din superclasa Punct a lui Pixel. Vom reveni însă cu toate detaliile legate de `super`.

Fie Sub o clasă ce extinde clasa Super. Sunt importante următoarele precizări:

- obiectele de tip Sub pot fi folosite oriunde este așteptat un obiect de tipul Super. De exemplu dacă un parametru al unei metode este de tipul Super, putem invoca acea metodă cu un argument de tipul Sub;
- spunem că Sub *extinde comportarea* lui Super;
- o clasă poate fi subclasă a unei singure clase (*moștenire simplă*) deoarece, reamintim, clasele formează o structură de arbore. Drept consecință o clasă nu poate extinde două clase, deci nu există *moștenire multiplă* ca în alte limbaje (există însă mecanisme pentru a simula această facilități și anume *interfețele*);
- o metodă din superclasă poate fi rescrisă în subclasă. Spunem că metoda este *redefinită* (dacă nu este statică) sau *ascunsă* (dacă este statică). Evident, ne referim aici la o rescriere a metodei folosind aceeași semnătură, pentru că dacă scriem în subclasă o metodă cu o semnătură diferită de semnăturile metodelor cu același nume din superclasă, atunci va fi vorba de o metodă nouă. La rescriere nu putem modifica tipul valorii întoarse de metodă. Accesul la metoda cu aceeași semnătură din superclasă se face, după cum vom vedea, prin precalificare cu `super`;
- un câmp redeclarat în Sub *ascunde* câmpul cu același nume din Super;
- câmpurile neascunse și metodele nerescrise sunt automat *moștenite* de subclasă (în exemplul de mai sus este vorba de câmpurile x și y și de metoda Miscare);
- apelurile `super` din constructorii subclasei trebuie să apară ca primă acțiune.

5.2. Din nou despre inițializare și constructori

Relativ la blocurile de inițializare, adăugăm la cele deja știute faptul că înainte de inițializarea unei clase sunt inițializate superclasele ei, dacă n-au fost deja inițializate.

Exemplul 2. Să considerăm clasele:

```
class Super {
    static { IO.println("Bloc_static_Super"); }
}

class Sub extends Super {
    static { IO.println("Bloc_static_Sub"); }
}
```

La crearea unui obiect prin `new Sub()`; vor fi tipărite următoarele două rânduri:
 Bloc_static_Super
 Bloc_static_Sub

Exemplul 3. În schimb programul:

```
class Super { static int i = 5; }

class Sub extends Super {
    static { IO.println("Sub"); }
}

class Test {
    public static void main(String[] s) {
        IO.println(Sub.i + "");
    }
}
```

va tipări valoarea 5: clasa Sub nu este inițializată, deoarece se face referire la un câmp declarat în clasa Super, deci nu este vorba de o utilizare activă a clasei Sub.

Suntem acum în măsură să prezentăm mai complet și acțiunile întreprinse la crearea unui obiect.

Prima acțiune constă în a atribui câmpurilor valorile inițiale implicite:

0 - pentru tipuri numerice
 \u0000 - pentru tipul **char**
 false - pentru tipul **boolean**
 null - pentru referințe (la obiecte, inclusiv tablouri și String).

În continuare este invocat constructorul corespunzător (determinat de semnătură, în același mod ca pentru metode); acțiunea sa cuprinde trei faze:

- 1) Invocarea unui constructor al superclasei și anume:
 - explicit, cu **super(...)**;
 - implicit: este vorba de constructorul fără argumente.
- 2) Executarea, în ordinea în care apar, a inițializărilor directe ale câmpurilor;
- 3) Executarea corpului constructorului.

Observații:

- un constructor poate invoca un alt constructor și anume prin **this(...)**; această invocare poate avea loc numai ca primă acțiune a constructorului (deci nu pot fi invocați doi constructori). În schimb acest mecanism nu poate fi folosit pentru a invoca, din corpul unei metode, un constructor;

- un constructor poate invoca o metodă, caz în care se consideră implementarea metodei corespunzătoare tipului *real* al obiectului (vezi subcapitolul următor).

- dacă din constructorul Pixel din *Exemplul 1* am elimina invocarea **super(x,y)**, compilatorul ar semnală o eroare deoarece, conform mecanismului descris mai sus, constructorul Pixel va invoca implicit constructorul fără argumente al superclasei, care însă nu există (el există implicit doar dacă nu s-a specificat un alt constructor).

Exemplul 4. Să considerăm clasele:

```
class a {
    int va=1, v;
    a() { v=va; }
}

class b extends a {
    int vb=2;
    b() { v=va+vb; }
}
```

La crearea unui obiect de tipul b, prin **new b()**, au loc în ordine următoarele acțiuni:

- câmpurile v, va, vb primesc valoarea implicită 0;
- este invocat constructorul b;
- este invocat constructorul a (al superclasei);
- are loc inițializarea directă a lui va, care primește valoarea 1;
- este executat corpul constructorului a și ca urmare v primește valoarea 1;
- are loc inițializarea directă a lui vb, care primește valoarea 2;
- este executat corpul constructorului b și ca urmare v primește valoarea 3.

Dacă o clasă este declarată cu modificatorul **public**, constructorul implicit are automat acest modificator.

Constructorii unei clase nu sunt considerați membri ai clasei. De aceea ei nu pot fi moșteniți sau redeclarați.

Constructorilor li se pot atașa modificatori de acces la fel ca metodelor (vezi subcapitolele următoare). Dar, spre deosebire de metode, un constructor nu poate fi declarat cu alți modificatori decât cei de acces.

Într-un constructor poate fi folosită instrucțiunea **return**, dar neînsoțită de o expresie.

Dacă un constructor se invocă (direct sau indirect) pe el însuși, la compilare este semnalată o eroare.

Într-o invocare explicită a unui constructor, argumentele nu pot fi instanțieri de câmpuri declarate în aceeași clasă sau într-o superclasă; de asemenea nu este permisă utilizarea lui **this** și **super** în expresii.

Exemplificăm în continuare acest aspect pentru câmpuri.

Exemplul 5. Să considerăm următoarea clasă:

```
class C {
    int x; int y;
    C() { this(x); }
    C(int x) { y = x; }
}
```

În clasa C, constructorul fără parametri invocă pe cel cu un parametru. Va fi semnalată o eroare la compilare. În schimb clasa va fi corectă dacă de exemplu câmpul x ar fi fost declarat cu modificatorul **static**.

5.3. Rescrierea metodelor și ascunderea câmpurilor

Câmpurile pot fi *ascunse* prin redeclararea lor într-o subclasă. Câmpul cu același nume din superclasă nu mai poate fi accesat direct prin numele său, dar ne putem referi la el folosind **super** (vezi subcapitolul următor) sau o referință la tipul superclasei.

O metodă declarată într-o clasă poate fi rescrisă într-o subclasă prin declararea ei în subclasă cu același nume, aceeași semnătură și același tip al valorii întoarse.

În metoda rescrisă putem schimba modificatorul de acces, cu condiția ca dreptul de acces să crească; reamintim că modificatorii de acces, în ordine de la cel mai restrictiv la cel mai permisiv, sunt: **private**, **protected**, cel implicit (*package*) și **public**.

Spunem că metodele rescrise sunt *ascunse* (dacă e vorba de metode statice) sau *redefinite* (în cazul metodelor nestatice). Nu este vorba numai de o diferență de terminologie, deoarece metodele statice pot fi rescrise (ascunse) numai de metode statice, iar metodele nestatice pot fi rescrise (redefinite) numai de metode nestatice. Alte precizări vor fi făcute în continuare.

Fie A o clasă și fie B o subclasă a sa. Să considerăm următoarele patru acțiuni echivalente din punctul de vedere al obiectului a ce ia naștere:

- 1) A a; a = new B(...);
- 2) A a = new B(...);
- 3) A a; B b; b = new B(...); a = b;
- 4) A a; B b; b = new B(...); a = (A) b;

Să observăm că este vorba de o conversie de la o subclasă la o superclasă, conversie numită *upcasting*; ea poate fi implicită (ca în primele trei acțiuni) sau explicită (ca în cea de a patra acțiune).

Vom spune că obiectul a are *tipul declarat* A și *tipul real* B, ceea ce pune în evidență noțiunea de *polimorfism*.

Tipul real al unui obiect coincide cu tipul său declarat (este cazul obiectului b) sau este o subclasă a tipului declarat (vezi obiectul a).

Fie camp un câmp al clasei A, ce este redeclarat (ascuns) în subclasa B. Dacă obiectul a face referire la câmpul camp, atunci este vorba de câmpul declarat în clasa A, adică este folosit tipul declarat al obiectului.

Fie met o metodă a clasei A, care este rescrisă în subclasa B. La invocarea metodei met de către obiectul a, este folosită fie implementarea corespunzătoare metodei ascunse (dacă este statică), fie cea corespunzătoare metodei redefinite (dacă este nestatică). Cu alte cuvinte, pentru metode statice este folosit tipul declarat (la fel ca pentru câmpuri), iar pentru metode nestatice este folosit tipul real al obiectului. În ambele cazuri, se pleacă de la tipul indicat mai sus și se merge în sus spre rădăcină (clasa Object) în arborele de clase până când se ajunge la primul tip în care apare metoda respectivă (evident cu semnătura corespunzătoare); inexistența unui astfel de tip este semnalată chiar în faza de compilare.

Exemplul 6. Următorul program:

```
class Super {
    static void met1() { IO.writeln("static_Super"); }
    void met2() { IO.writeln("Super"); }
}

class Sub extends Super {
    static void met1() { IO.writeln("static_Sub"); }
    void met2() { IO.writeln("Sub"); }
}

class Test1 {
    public static void main(String[] s) {
        Super Ob = new Sub(); Ob.met1(); Ob.met2();
    }
}

produce la ieșire:
static_Super
Sub
```

Exemplul 7. Considerăm o nouă formă a claselor din Exemplul 2:

```
class A {
    int va=1, v;
    A() { v=va; IO.write(""+met()); }
    int met() { return va; }
}

class B extends A {
    int vb=2, v;
    B() { v=va+vb; }
    int met() { return vb; }
}

class Constr {
    public static void main (String[] s) {
        A a = new B(); IO.writeln("\t"+a.met());
    }
}
```

Rezultatele tipărite de programul de mai sus sunt 0 și 2, deoarece:

- la invocarea metodei `met` din cadrul constructorului clasei `A` este folosit tipul real al obiectului, adică este întors rezultatul furnizat de metoda `met` redeclarată în `B`; la momentul invocării, valoarea curentă a lui `vb` este 0 (vezi subcapitolul precedent).

- invocarea metodei `met` în metoda principală, realizată de obiectul `a`, se va referi tot la metoda `met` din clasa `B`.

Dacă însă în clasa `B` am înlocui declararea `int vb=2` a lui `vb` prin:

```
static int vb=1;
static { vb=2; }
```

atunci la ieșire vor apărea numerele 2 și 2.

Exemplul 8. Să considerăm următorul program:

```
class A {
    String s="Super";
    void scrie() { IO.println("A : "+s); }
}

class B extends A {
    String s="Sub";
    void scrie() { IO.println("B : "+s); }
}

class AB1 {
    public static void main(String[] s) {
        B b = new B(); A a = b;
        a.scrie(); b.scrie();
        IO.println(a.s + "\t" + b.s);
    }
}
```

La executare sunt tipărite următoarele:

```
B : Sub
B : Sub
Super      Sub
```

adică rezultatele așteptate, ținând cont că:

- pentru obiectul `b`: tipul declarat coincide cu cel real și este `B`;
- pentru obiectul `a`: tipul declarat este `A`, iar tipul real este `B`.

Să considerăm o clasă `C` și două subclase `X` și `Y` ale sale, precum și următoarea secvență de instrucțiuni:

```
C Ob;
... Ob = new X(...);
... Ob = new C(...);
... Ob = new Y(...);
...
```

în care `Ob` are mai întâi tipul real `X`, apoi tipul real `C`, apoi tipul real `Y`. Spunem că `Ob` este o variabilă *polimorfică*, deoarece are pe rând forma (comportamentul) a mai multor clase.

Din exemplul de mai sus rezultă clar că doar la momentul executării putem determina necesarul de memorie pentru obiectul `Ob`, ceea ce justifică de ce la crearea obiectele sunt "stocate" în heap-ul memoriei. Mai precis, la crearea unui obiect este alocat spațiu pentru toate câmpurile nestatice din clasă, inclusiv pentru cele ascunse.

5.4. Cuvântul cheie `super`

Putem folosi cuvântul cheie `super` în orice metodă nestatică a unei clase extinse, și anume sub una dintre formele:

- `super(...)` : pentru a invoca un constructor al superclasei clasei curente;
- `super.met(...)` : pentru a invoca o metodă a superclasei, metodă ce a fost redefinită în clasa curentă;
- `super.c` : pentru a accesa un câmp `c` al superclasei, câmp ce a fost ascuns în clasa curentă prin redefinirea sa.

Observăm că la accesarea unui câmp sau a unei metode, `super` acționează ca referință la obiectul curent ca instanțiere a superclasei sale.

Exemplul 9. Programul următor:

```
class A {
    String s="Super";
    void scrie() { IO.println("A : "+s); }
    void metoda() { IO.println("A:"); }
}

class B extends A {
    String s="Sub";
    void scrie() { IO.println("B : "+s+" "+super.s); }
    void metoda() { scrie(); super.scrie(); super.metoda(); }
}

class AB2 {
    public static void main(String[] x) {
        B b = new B(); b.metoda(); IO.println(b.s);
        IO.println("*****");
        A a; a = b; a.metoda(); IO.println(a.s);
        IO.println("*****");
        A c = new A(); c.metoda(); IO.println(c.s);
    }
}

)
produce la ieșire următorul rezultat:
```

```

B : Sub Super
A : Super
A:
Sub
*****
B : Sub Super
A : Super
A:
Super
*****
A:
Super

```

Sunt incorecte, neîncadrându-se în formele prezentate mai sus de folosire a lui **super**, construcții de forma **b.super.metoda()** (care ar intenționa ca pentru obiectul **b** să invocăm metoda **metoda** din superclasa clasei de tipul căreia este) sau urcarea mai multor niveluri în arborele de clase prin **super.super**. Pentru a avea acces la un câmp ascuns aflat cu două niveluri mai sus în arborele de clase putem proceda de pildă ca în exemplul următor.

Exemplul 10. Executarea programului:

```

class A {
    int x=1;
    void met() { IO.write("\t"+x); }
}

class B extends A {
    boolean x=false;
    void met() { IO.write("\t"+x); super.met(); }
}

class C extends B {
    double x=3.14;
    void met() { IO.write("\t"+x); super.met(); }
}

class SubSup {
    public static void main (String[] s) {
        C Ob = new C(); Ob.met();
    }
}

```

produce la ieșire:

```

3.14 false    1

```

5.5. Moștenire, polimorfism și legare dinamică

Reamintim următoarele:

- orice obiect care instanțiază o clasă poate fi folosit în orice cod menit să lucreze cu instanțieri ale superclasei sale;
- orice obiect care instanțiază o clasă poate folosi (în modurile descrise mai sus) codul supraclasei.

La aceste două caracteristici legate de moștenire (extinderea claselor), Java adaugă *polimorfismul* și *legarea dinamică*, între care există o strânsă relație.

Polimorfismul constă în posibilitatea ca o invocare **Ob.met(...)** să aibă drept consecință invocarea unei anumite metode cu numele **met**, în funcție de tipul real al obiectului **Ob**. Unii autori consideră că și cele două facilități enunțate mai sus fac parte din conceptul de polimorfism.

Legarea dinamică identifică metoda **met** respectivă la executarea programului. Polimorfismul și legarea dinamică sunt două caracteristici esențiale ale programării orientate pe obiecte și vor fi analizate în acest subcapitol.

Așa cum am precizat în subcapitolul precedent, fiecare obiect are un tip declarat și un tip real; tipul real coincide cu cel declarat sau este un subtip al acestuia. La invocarea metodelor nestatice se folosește tipul real, iar la invocarea metodelor statice și la referirea câmpurilor se folosește tipul declarat.

Menționăm că în literatura consacrată programării orientate pe obiecte nu este agreată opțiunea Java de a folosi pentru câmpuri tipul declarat și nu pe cel real.

Pentru metode nestatice se folosește tipul real al obiectelor care le invocă deoarece:

- cronologic, superclasa a fost scrisă înaintea clasei și este de presupus că a fost folosită de mai mulți utilizatori, care vor în continuare să lucreze cu ea;
- în metodele noi clase se pot face referiri la câmpuri nou definite, ceea ce nu este posibil pentru metodele superclasei.

Utilizarea tipului declarat pentru metodele statice se datorează următorului fapt: compilatorul "face tot ce poate" pentru a detecta metoda ce trebuie invocată.

Java folosește *legarea dinamică*, adică identificarea metodei nestatice concrete care este folosită la o invocare se face la executare și nu la compilare. De aceea legarea dinamică se mai numește *legare târzie*.

Necesitatea legării dinamice apare din următorul exemplu:

Exemplul 11. Fie **A** o clasă în care este definită o metodă **met**, iar **B** o subclasă a sa în care metoda **met** este redefinită. Secvența de instrucțiuni:

```

double d = IO.read(); A Ob;
if (d>0) Ob = new A(...);
else Ob = new B(...);
Ob.met(...);

```

arată că doar la momentul invocării metodei devine clar care dintre cele două metode **met** va fi invocată.

5.6. Din nou despre modificatori

Reluăm discuția despre modificatori, pentru a include și aspectele legate de extinderea claselor.

Lista completă a modificatorilor folosiți în Java este următoarea:

- modificatorii de acces (**public**, **protected**, **private** și cel implicit);
- **abstract**, **static**, **final**, **synchronized**, **native**, **transient**, **volatile**.

Ei pot fi folosiți astfel:

- pentru *clase*: **public** și cel implicit, **abstract**, **final**;
- pentru *interfețe*: modificatorii de acces;
- pentru *constructori*: modificatorii de acces;
- pentru *câmpuri*: modificatorii de acces, **final**, **static**, **transient**, **volatile**;
- pentru *metode*: modificatorii de acces, **final**, **static**, **abstract**, **synchronized**, **native**.

Despre modificatorul **abstract** vom vorbi în subcapitolul următor.

Despre interfețe, ca și despre unii dintre modificatorii **transient**, **volatile**, **synchronized** și **native**, vom vorbi în capitolele care urmează.

Modificatorul **final** poate fi asociat (în afara variabilelor locale și câmpurilor) și metodelor și claselor. El trebuie înțeles în sensul de "variantă finală":

- o metodă cu acest modificator nu poate fi rescrisă într-o subclasă;
- o clasă cu acest modificator nu poate fi extinsă.

Metodele declarate cu **private** sau/și **static** sunt echivalente cu **final**, din punctul de vedere al redefinirii: nu pot fi redefinite.

Dacă o metodă este finală, "ne putem baza" pe implementarea ei (bineînțeles dacă nu invocă metode nefinale). În unele situații, este bine să marcăm toate metodele cu **final**, dar nu și clasa.

• Precizări asupra noțiunii de membru al unei entități

Membrii unui pachet sunt clasele ce intră în componența sa.

Membrii unei clase sunt:

- cei declarați în corpul clasei;
- cei moșteniți de la superclasa sa directă (doar clasa **Object** nu are o superclasă);
- cei moșteniți de la superinterfețele directe ale clasei.

Membrii unui tablou sunt:

- cei moșteniți de la superclasa implicită **Object**;
- câmpul **length**, care este un câmp constant (are implicit modificatorii **public** și **final**) al fiecărui tablou; tipul său este **int**, iar câmpul memorează numărul de componente ale tabloului.

Membrii unei interfețe (despre interfețe vom vorbi în capitolul următor) sunt:

- membrii declarați în corpul interfeței;
- membrii moșteniți de la orice superinterfață directă.

În discuția anterioară din capitolul 3 asupra vizibilității, la determinarea semnificației unui identificator, am rămas datori cu explicarea celei de a treia căutări: ea are în vedere luarea în considerație a membrilor clasei, deci și a membrilor moșteniți de la superclase.

• Despre modificatorii de acces

Referitor la modificatorii de acces, elementele noi ce intervin la extinderea claselor se referă la moștenirea lor în subclase.

Accentuăm faptul că *accesul* este diferit de *domeniul de vizibilitate*. Accesul se referă la porțiunea din cod din care o entitate poate fi referită, pe când domeniul de vizibilitate reprezintă porțiunea din cod în care o entitate declarată poate fi referită și recunoscută ca având tipul declarat respectiv.

Descriem în continuare influența modificatorilor.

I. Accesibilitatea unui pachet este determinată de sistemul de operare gazdă.

II. Dacă o clasă sau interfață este declarată cu **public**, atunci ea poate fi accesată din orice cod ce poate accesa pachetul în care este declarată. În caz contrar (dacă nu apare modificatorul **public**), ea poate fi accesată doar din pachetul în care ea este declarată.

III. Un membru (câmp sau metodă) al unui tip referință (clasă, tablou sau interfață) ca și un constructor al unei clase este accesibil numai dacă tipul referință este accesibil și membrul sau constructorul permite (prin modificatorii săi) accesul. Acest al doilea aspect poate fi detaliat astfel:

III.1. Dacă membrul sau constructorul este declarat cu **public**, accesul este permis. Membrii interfețelor sunt implicit declarați cu **public**.

III.2. Dacă membrul sau constructorul este declarat cu **protected**, accesul este permis numai din:

III.2.1. pachetul ce conține clasa în care entitatea respectivă este declarată sau

III.2.2. o subclasă a clasei în care entitatea respectivă este declarată (a se vedea și precizările ce urmează).

III.3. Dacă membrul sau constructorul nu este declarat, deci are modificatorul implicit, accesul este permis numai din pachetul în care entitatea respectivă este declarată.

III.4. Dacă membrul sau constructorul în care entitatea respectivă este declarată cu **private**, accesul este permis doar din interiorul clasei în care entitatea respectivă este declarată.

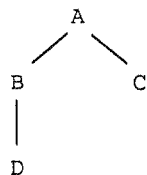
Membrii unei clase sunt moșteniți astfel:

- în subclase, dacă au modificatorul **public** sau pe cel implicit;
- în subclase din același pachet, dacă au modificatorul implicit;
- nu sunt moșteniți, dacă au modificatorul **private**.

Cele de mai sus sunt sintetizate în următorul tabel:

Modificator	Membrii clasei cu acest modificator sunt accesibili ...	Membrii clasei cu acest modificator ...
public	de oriunde clasa este accesibilă	sunt moșteniți în subclase
protected	din cod din același pachet și din subclasele clasei respective	sunt moșteniți în subclase
Implicit (<i>package</i>)	din cod din același pachet	sunt moșteniți numai în subclase din același pachet
private	numai din interiorul clasei	nu sunt moșteniți în subclase

Precizăm efectul lui **protected**: un membru declarat cu **protected** poate fi accesat dintr-o clasă numai prin intermediul unor obiecte al căror tip este un descendent al clasei în arborele de clase. Să considerăm următorul subarbore de clase:



Presupunem că în clasa A apare un câmp **x** declarat cu **protected**. Să presupunem că din B facem o referire la acest câmp prin **c.x**, unde **c** este un obiect de tipul C. Această referire nu este permisă (exceptând cazul în care clasele B și C apar în același pachet) deoarece ieșim din subarboarele din care se face referirea.

Restricția de mai sus nu mai funcționează pentru membrii declarați cu **protected**, dar și cu **static**: orice metodă din B, C, D poate accesa câmpul **x** prin referințe având oricare dintre tipurile A, B, C, D.

Și constructorilor le putem atașa modificatori de acces. Un caz limită este cel în care într-o clasă C există cel puțin un constructor și toți constructorii sunt declarați cu **private**. În acest caz nu putem construi direct din afara clasei un

obiect având ca tip această clasă. Astfel de clase sunt folosite de obicei pentru a "stoca" metode și câmpuri statice, un exemplu fiind clasa **System**. Dar aceasta nu înseamnă că nu putem folosi o construcție indirectă: din afara clasei putem invoca de exemplu o metodă statică a lui C ce construiește un obiect de tipul C și îl întoarce ca rezultat:

```

class C {
    private C() {}
    static C met() { return new C(); }
    void scrie() { IO.writeln("O.K."); }
}

class Constrf {
    public static void main (String[] s) {
        C Ob = C.met(); Ob.scrie();
    }
}
  
```

6 METODE ȘI CLASE ABSTRACTE. INTERFEȚE. CLASE INTERNE

6.1. Metode și clase abstracte

Se întâmplă frecvent ca atunci când lucrăm cu clase extinse, pentru o clasă să nu putem preciza implementarea unei metode, deoarece în subclasele sale ea va fi specifică fiecăreia dintre ele. Atunci metodele din această categorie vor fi marcate cu cuvântul cheie **abstract** și se vor reduce la antetul lor; în același timp și clasa trebuie însoțită de atributul **abstract**. Fiecare metodă abstractă trebuie implementată, adică (re)definită în orice subclasă care nu este la rândul său abstractă.

Nu este posibil să creăm obiecte ca instanțe ale unei clase abstracte. Pe de altă parte, într-o subclasă putem să redefinim o metodă a superclasei transformând-o într-o metodă abstractă; aceasta are sens de exemplu dacă în subarboarele având ca rădăcină subclasa, comportamentul materializat prin acea clasă nu mai este valabil în subarboare și este specific fiecărei extensii a subclasei.

Exemplul 1. Următoarea clasă are ca scop să măsoare timpul necesitat de executarea unei metode oarecare fără parametri și care nu întoarce vreun rezultat:

```
abstract class C {
    abstract void met();
    public long timp() {
        long t0 = System.currentTimeMillis();
        met();
        return System.currentTimeMillis() - t0;
    }
}
```

unde a fost folosită metoda statică `currentTimeMillis()` a clasei `System`, metodă care întoarce timpul curent în milisecunde. Într-o clasă ce extinde `C`, se poate redefini metoda `met` și apoi, folosind un obiect ce este o instanțiere a noii clase, putem apela metoda `timp` pentru a determina timpul de executare a metodei `met`, ce are acum o implementare precisă:

```
class C1 extends C {
    void met () {
        int x=0;
        for (int i=0; i<1000000; i++) x=x+1-1;
    }
}

class Abstr {
    public static void main(String[] s) {
        C1 Ob = new C1();
        IO.writeln( "durata="+Ob.timp() );
    }
}
```

Legat de exemplul de mai sus, nu trebuie să ne mire că e posibil ca la executări diferite să obținem timpi diferiți: aceasta este o consecință a faptului că pe perioada executării programului nostru sistemul gazdă "mai face și altceva", sau că a intervenit colectorul de reziduuri.

6.2. Noțiunea de interfață

Așa cum s-a menționat anterior, este posibil să declarăm clase abstracte în care toate metodele sunt abstracte. Acest lucru poate fi realizat și prin intermediul *interfețelor*. În plus, așa cum vom vedea în continuare, ele reprezintă mecanismul propus de Java pentru tratarea problemelor ridicate de moștenirea multiplă.

Moștenirea multiplă este facilitatea oferită de unele limbaje de programare ca o clasă să moștenească (prin extindere) membri ai mai multor clase. Evident, această caracteristică importantă a programării orientate pe obiecte nu este neglijată în Java. Modul în care această facilitate poate fi implementată în Java va fi tratat în continuare.

Interfețele reprezintă o modalitate de a declara un tip constând numai din constante și din metode abstracte. Ca sintaxă, o interfață este asemănătoare unei clase, cu deosebire că în loc de **class** trebuie precizat **interface**, iar metodele nu au corp, acesta fiind înlocuit cu `;`.

Putem spune că interfețele reprezintă numai proiecte, pe când clasele sunt o combinație de proiecte și implementări.

Ca și în cazul claselor abstracte, este evident că nu pot fi create obiecte de tipul unei interfețe.

Câmpurile unei interfețe au în mod implicit modificatorii **static** și **final**, deci sunt constante.

Metodele interfețelor sunt totdeauna publice și au în mod implicit modificatorul **abstract**. În plus ele nu pot fi statice, deoarece fiind abstracte, nu pot fi specifice claselor.

Orice interfață este gândită pentru a fi ulterior *implementată* de o clasă, în care metodele interfeței să fie redefinite, adică să fie specificate acțiunile ce trebuie întreprinse. Faptul că o clasă C implementează o interfață I trebuie specificat prin inserarea informației `implements I` în antetul clasei.

Exemplul 2. Prezentăm un prim exemplu de implementare a unei interfețe.

```
interface I {
    char c='a'; int i=0;
    void scrie();
}

class C implements I {
    public void scrie() { IO.println(c+" "+i); }
}
```

Precizăm următoarele:

- o interfață poate extinde oricâte interfețe. În acest mod interfețele permit moștenirea unor "contracte" (numele și signaturile unor metode) fără moștenirea implementării;

- dacă o clasă implementează doar unele din metodele unei interfețe, atunci ea trebuie declarată cu `abstract`;

- spre deosebire de interfețe (care sunt limitate la constante și anunțuri de metode), clasele abstracte pot avea implementări parțiale, metode statice, membri cu modificatorul `protected`, câmpuri care nu sunt finale etc.;

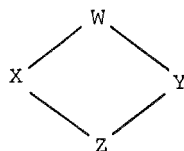
- o clasă poate implementa oricâte interfețe, dar poate extinde o singură clasă.

Dacă o clasă extinde o altă clasă și implementează una sau mai multe interfețe, atunci trebuie anunțată întâi extinderea și apoi implementarea, ca în exemplul următor:

```
class C extends B implements I1,I2,I3;
```

Așa cum vom vedea în continuare, facilitatea ca interfețele și clasele să poată extinde oricâte interfețe reprezintă modalitatea prin care Java rezolvă problema moștenirii multiple.

Dacă s-ar permite ca o clasă să extindă mai multe clase, ar apărea neclarități legate de semnificația numelor. Astfel, dacă ar fi posibilă următoarea structură de clase extinse:



și în clasele W, X și Y ar fi definit (respectiv redefinit) un câmp c, atunci pentru un obiect Ob de tipul Z semnificația lui Ob.c ar fi neclară.

Faptul că în Java o clasă poate extinde cel mult o clasă nu rezolvă însă automat problema conflictelor de nume, problemă care va fi tratată în continuare.

Suntem acum în măsură să completăm discuția despre supertipuri și subtipuri.

Supertipurile unei clase sunt clasa pe care o extinde, interfețele pe care le implementează, precum și supertipurile acestei clase și acestor interfețe. Drept urmare o referință la un obiect de tipul clasei poate fi folosit oriunde este așteptat un obiect al oricărui supertip al său (vezi și discuția despre *upcasting* din capitolul referitor la extinderea claselor).

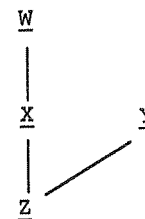
Mai menționăm că putem declara variabile având ca tip numele unei interfețe. Unei astfel de variabile îi poate fi atribuit un obiect care implementează interfața (are ca tip real o clasă ce implementează interfața).

Un exemplu ce ilustrează aceste elemente va fi prezentat în paragraful destinat implementării interfețelor.

6.3. Extinderea interfețelor

Ca și clasele, interfețele pot fi extinse. O interfață I poate extinde oricâte interfețe, în acest mod adăugându-se la I noi constante și (anunțuri de) metode.

Este permis ca o interfață ce extinde altă interfață să conțină o constantă cu același nume. De exemplu pentru următoarea structură de interfețe:



este posibil ca în una sau mai multe dintre interfețe să fie declarată o constantă c. Deosebim două cazuri:

1) Constanta c este redeclarată în interfața Z : o referire la c constituie o referire la constanta c din Z. Putem face însă referire și la constantele din celelalte interfețe prin X.c, Y.c și W.c.

2) Constanta c nu este redeclarată în interfața Z : o referire la c este corectă dacă există un unic drum de interfețe ce "coboară" în Z, drum în care c poate fi declarată de mai multe ori; în acest caz referirea are ca obiect cea mai "recentă" declarație a câmpului, adică cea din interfața cea mai apropiată de Z. Dacă c este declarat pe mai multe drumuri de interfețe ce "coboară" în Z, compilatorul va semnaliza că este vorba de o referire ambiguă. Dacă în Z constanta c nu poate fi regăsită pe nici un drum de interfețe ce ajunge în Z, atunci va fi semnalată din nou eroare.

Rămânând la structura de interfețe de mai sus, să presupunem acum că în X și în Y (sau în supertipuri ale lor) apare o metodă cu același nume. Deosebim situațiile:

- 1) dacă metodele au semnături diferite, vor fi moștenite ambele metode;
- 2) dacă metodele au aceeași semnătură și același tip pentru valoarea întoarsă, va fi moștenită o singură metodă;
- 3) dacă metodele au aceeași semnătură, dar tipurile valorilor întoarse diferă, atunci moștenirea nu va fi posibilă (eroare de compilare).

Este posibil ca de exemplu în X și în Z (sau, mai general, într-un drum de interfețe extinse) să anunțăm câte o metodă cu același nume și aceeași semnătură. Atunci o clasă care implementează interfața Z va implementa ambele metode.

Exemplul 3. Programul următor:

```
interface W { char c='a'; }
interface X extends W { int c=1; void met(); }
interface Y { boolean c=true; }
interface Z extends X,Y { int c=99; void met(); }

class C implements X {
    public void met() { IO.writeln("+++++"); }
}

class D implements Z {
    public void met() { IO.writeln("*****"); }
}

class Interfetel implements Z {
    public void met() { }
    public static void main (String[] s) {
        C Ob1 = new C(); Ob1.met();
        D Ob2 = new D(); Ob2.met();
        int c=-88; // met();
        IO.writeln(c+" "+W.c+" "+X.c+" "+Y.c+" "+Z.c );
    }
}
```

produce la ieșire:

```
+++++
*****
-88 a 1 true 99
```

Se impun următoarele observații:

1) În clasele C și D, redefinirea metodei met s-a făcut cu precizarea modificatorului **public**. Acest lucru este obligatoriu, pentru că altfel ar fi fost folosit modificatorul implicit, dar o redefinire nu poate fi mai restrictivă în privința drepturilor de acces;

2) Din metoda principală nu putem invoca implementarea lui met din Interfetel, deoarece o metodă statică poate accesa numai câmpurile și metodele statice ale clasei; pe de altă parte însă, o metodă anunțată într-o interfață nu poate fi redefinită printr-o metodă statică, deci nu putem preciza atributul **static** pentru metoda met din Interfetel.

6.4. Implementarea interfețelor

În subcapitolul de față prezentăm modul în care sunt rezolvate conflictele de nume ce pot apărea la implementarea interfețelor.

Fie C o clasă care implementează una sau mai multe interfețe și extinde eventual o clasă.

Presupunem mai întâi că în unul sau mai multe dintre aceste tipuri apare un câmp (constantă în cazul interfețelor) cu același nume c. Atunci discuția este analoagă cu cea referitoare la extinderea interfețelor, cu următoarea adăugire: dacă c este declarat atât în Z, cât și în clasa pe care o extinde, va trebui să specificăm **super.c** pentru câmpul din superclasă.

Trecem acum la cazul metodelor. Moștenirea metodelor interfețelor are loc în modul descris la extinderea interfețelor. Drept urmare, toate metodele moștenite din interfețe trebuie implementate.

Dacă o metodă cu același nume și aceeași semnătură apare în superclasă și în interfețele implementate, atunci:

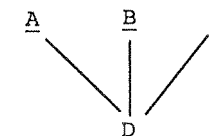
- dacă precizăm în C o implementare a metodei, ea va constitui o redefinire a metodei din superclasă; la aceasta din urmă putem face însă apel prin **super**;
- dacă în C nu este precizată o implementare a metodei, atunci metoda (moștenită) din superclasă constituie implementarea metodei din interfețe, cu condiția ca ea să aibă modificatorul **public**.

Exemplul 4. Programul următor:

```
interface A {
    int x=1; void scrie();
}

interface B {
    int x=2; void scrie();
}

class C {
    int x=3;
    public void scrie() {
        IO.write(A.x+" "+B.x+" "+x);
    }
}
```



```

class D extends C implements A,B {
    int x=4;
}

class Interfete2 {
    public static void main(String[] w) {
        D Ob = new D(); Ob.scrie(); IO.writeln(" "+Ob.x);
        A Ob1 = new D(); Ob1.scrie(); IO.writeln(" "+Ob1.x);
    }
}

```

produce la ieșire:

```

1 2 3 4
1 2 3 1

```

Sunt necesare explicații doar referitor la obiectul Ob1. El are tipul declarat A și tipul real D, ceea ce face ca invocarea metodei `scrie` prin intermediul acestui obiect să se refere la metoda `scrie` din clasa C, iar referința Ob1.x să se adreseze constantei x din interfața A.

6.5. Inițializarea interfețelor

Cum câmpurile unei interfețe sunt statice, este importantă ordinea în care ele sunt declarate, așa cum am atras atenția la inițializarea claselor.

Inițializarea unei interfețe nu implică inițializarea superinterfețelor sale. Exemplul care urmează face mai clar acest aspect.

Exemplul 5. Următorul program:

```

interface X {
    int x = 1, xx = InitInter.out("xx",2);
}

interface Y extends X {
    int y = InitInter.out("y",3), yy = InitInter.out("yy",4);
}

interface Z extends Y {
    int z = InitInter.out("z",5);
}

class InitInter {
    public static void main(String[] s) {
        IO.writeln(Y.x + " "); IO.writeln(Z.y + " ");
    }

    static int out(String s, int i) {
        IO.writeln(s + " = " + i); return i;
    }
}

```

produce la ieșire:

```

1
y = 3
yy = 4
3

```

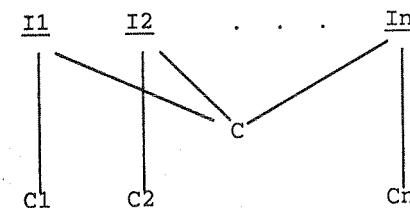
din următoarele motive:

- Y.x din metoda principală este o referire la un câmp constant, deci nu conduce la inițializarea interfeței X;
- Z.y din metoda principală este o referire la un câmp neconstant declarat în Y și ca urmare această interfață este inițializată. Aceasta nu conduce însă la inițializarea superinterfeței X. De asemenea nici interfața Z nu este inițializată, deși numele ei apare în referirea Z.y.

6.6. Rezolvarea în Java a problemei moștenirii multiple

Să presupunem că plecând de la clasele C1, C2, ..., Cn dorim să construim o nouă clasă care să moștenească unele dintre metodele lor. Java permite doar moștenire simplă, deci va fi necesar să apelăm la interfețe.

Modalitatea de rezolvare a problemei moștenirii multiple pe care o prezentăm în continuare este independentă de numărul de clase moștenite. Vom folosi următoarea structură de interfețe și clase:



În această structură clasele C1, C2, ..., Cn implementează respectiv metodele anunțate în interfețele I1, I2, ..., In, iar clasa C implementează toate interfețele I1, I2, ..., In. Este suficient să prezentăm modul în care clasa C moștenește implementările din C1 ale metodelor anunțate în I1, lucrurile decurgând analog pentru celelalte interfețe și clase pe care le implementează. De aceea în continuare vom presupune că C trebuie să extindă doar clasa C1; evident, presupunerea este făcută doar pentru o prezentare mai concisă, deoarece pentru extinderea unei singure clase nu apare necesitatea folosirii interfețelor.

În clasa C vom declara și crea un obiect Ob1 de tipul C1 (se presupune că în clasa C se știe ce implementare a interfeței I1 trebuie folosită). Atunci pentru fiecare metodă met implementată de C1 introducem în clasa C metoda met cu aceeași semnătură și având una dintre formele:

```

tip met(...) { return Ob1.met(...); }
void met(...) { Ob1.met(...); }

```

după cum metoda întoarce sau nu o valoare.

Exemplul 6. Programul următor:

```

interface X {
    void x1();
    int x2();
}

class CX implements X {
    public void x1() { IO.write("x1 "); }
    public int x2() { return 1; }
}

class C implements X {
    X ObX= new CX(); // Clasa C "stie" ca trebuie sa foloseasca
                    // implementarea CX a interfetei X
    public void x1() { ObX.x1(); }
    public int x2() { return ObX.x2(); }
}

class Mult1 {
    public static void main (String[] s) {
        C ObC = new C();
        ObC.x1(); IO.writeln(" "+ObC.x2());
    }
}

```

produce la ieșire :

x1 1

Să reamintim însă că putem declara variabile având ca tip numele unei interfețe și putem atribui unei astfel de variabile un obiect care implementează interfața. Această facilități permite o mai mare libertate de acțiune. Astfel, în exemplul de mai sus nu mai este necesar ca C să știe care este clasa ce implementează pe I1, ci poate afla acest lucru prin intermediul unui constructor; cu alte cuvinte, la crearea unei instanțieri a lui C, putem preciza ce implementare a lui I1 să folosească. Mai mult, pentru redefinirea unor metode diferite ale aceleiași interfețe, putem folosi implementări diferite.

Aceste aspecte sunt ilustrate în *Exemplul 7* de mai jos, în care interfața X este implementată de clasele CX1 și CX2, dar și de clasa C ale cărei metode folosesc același mecanism: invocă în fapt metodele cu același nume ale claselor CX1 și CX2 prin instanțieri ale acestora.

Exemplul 7. În urma executării programului:

```

interface X {
    void x1();
    int x2();
}

```

```

class CX1 implements X {
    public void x1() { IO.write("x1 "); }
    public int x2() { return 1; }
}

class CX2 implements X {
    public void x1() { IO.write("x2 "); }
    public int x2() { return 2; }
}

class C implements X {
    X Ob1,Ob2;
    C(X Ob1, X Ob2) { this.Ob1=Ob1; this.Ob2=Ob2;}
    public void x1() { Ob1.x1(); }
    public int x2() { return Ob2.x2(); }
}

class Mult2 {
    public static void main (String[] s) {
        X Ob1 = new CX1(); X Ob2 = new CX2();
        C ObC = new C(Ob1,Ob2);
        ObC.x1(); IO.write(" "+ObC.x2());
    }
}

```

se obține la ieșire:

x1 2

Dacă însă am înlocui a doua linie din metoda principală cu:

```
X Ob1 = new CX2(); X Ob2 = new CX1();
```

la ieșire s-ar obține:

x2 1

Să remarcăm că în metodele x1 și x2 ale clasei C nu apare nici un indiciu asupra tipului real al obiectelor care sunt delegate să execute metodele cu aceleași nume ale claselor pe care le instanțiază. De asemenea, posibilitatea de a folosi mai multe implementări pentru o interfață și de a folosi una sau alta dintre ele în funcție de context (de exemplu în funcție de valoarea unei variabile) arată pe de o parte că introducerea interfețelor oferă utilizatorului un instrument puternic și flexibil de lucru, iar pe de altă parte justifică încă o dată de ce Java utilizează legarea dinamică (târzie).

6.7. Clonarea obiectelor

Subcapitolul de față exemplifică utilizarea interfețelor pentru problema clonării obiectelor.

În multe cazuri suntem puși în situația ca plecând de la un obiect Ob1 deja construit, să trebuiască să construim un alt obiect Ob2 care să copieze pe Ob1, urmând ca în continuare cele două obiecte să-și desfășoare independent acțiunile. Cu alte cuvinte, vrem să creăm o nouă instanță Ob2 care să cunoască tot ce cunoaște Ob1 la un anumit moment de timp (spunem că Ob2 *clonează* pe Ob1), dar în continuare cele două obiecte să acționeze independent.

Încercarea de a rezolva problema prin instrucțiunea de atribuire Ob2=Ob1 este sortită eșecului. Într-adevăr, prin această atribuire variabilele Ob1 și Ob2 devin două referiri la același obiect.

Pentru realizarea clonării obiectelor, Java pune la dispoziție interfața Cloneable și metoda clone fără parametri a clasei "rădăcină" Object.

Metodei clone este declarată prin:

```
public Object clone() throws CloneNotSupportedException
și întoarce un obiect nou, în ale cărui câmpuri sunt copiate câmpurile obiectului
curent. Metoda clone poate lansa excepția menționată.
```

Interfața publică Cloneable din pachetul java.lang nu are metode; rolul ei este de a stabili dacă o clasă este sau nu clonabilă. Ea trebuie să fie implementată de clasa în care este invocată metoda clone. Modul tipic de lucru este următorul: dacă C este clasa ale cărei obiecte urmează a fi clonate, vom specifica faptul că ea implementează interfața Cloneable și vom include o metodă care invocă pe clone.

Metoda clone a clasei Object verifică dacă clasa a cărei instanțiere este obiectul curent implementează Cloneable. În caz afirmativ, este întors un obiect nou ale cărui câmpuri sunt inițializate ca având aceleași valori ca și câmpurile obiectului curent. În caz negativ, este lansată excepția specificată mai sus.

Exemplul 8. Programul următor:

```
class C implements Cloneable {
    int x;
    void inc() { x++; }
    C Clone() {
        try { return (C) clone(); }
        catch (CloneNotSupportedException e) { return null; }
    }
}

class Copiel {
    public static void main (String[] s) {
        C Ob1 = new C(); C Ob2 = Ob1.Clone();
        IO.writeln(Ob1.x+" "+Ob2.x);
        Ob1.inc(); Ob2.inc(); Ob2.inc();
        IO.writeln(Ob1.x+" "+Ob2.x);
    }
}
```

produce la ieșire:

```
0 0
1 2
```

Sunt necesare următoarele precizări:

- este obligatorie conversia explicită a rezultatului întors de clone la tipul dorit;

- am inclus invocarea lui clone într-o metodă Clone din clasa C deoarece acolo îi este în mod natural locul (este o prelucrare ce se referă la clasa C); pe de altă parte nu puteam să invocăm clone din metoda principală, deoarece nu este statică;

- despre try și catch vom vorbi pe larg în capitolul următor.

Exemplul 9. Dorim să clonăm un obiect care gestionează o stivă. Vrem ca noul obiect să plece de la același conținut al stivei ca și cel vechi, dar în continuare să lucreze independent, adică stivele gestionate de cele două obiecte să fie distincte.

Stiva este memorată în tabloul v și gestionată prin intermediul metodelor pop (extragere) și push (adăugare în stivă). Constructorul clasei Stiva creează vectorul de lungimea transmisă ca parametru și inițializează câmpul top cu -1 (top este poziția corespunzătoare vârfului stivei). Metoda Clone are același rol ca în programul precedent, iar metoda scrie produce la ieșire conținutul stivei.

Pentru moment vom face abstracție de comentariul de sfârșit de linie din metoda Clone.

```
class Stiva implements Cloneable {
    private int[] v; private int top;

    Stiva Clone() {
        try {
            Stiva ObNou = (Stiva) clone();
            // ObNou.v = ( int[] ) v.clone();
            return ObNou;
        }
        catch (CloneNotSupportedException e) { return null; }
    }

    Stiva(int nrmax) { v = new int[nrmax]; top = -1; }

    void push(int val) { v[++top]= val; }

    int pop() { return v[top--]; }

    void scrie() {
        for (int i=0; i<=top; i++) IO.write(v[i]+" ");
        IO.writeln();
    }
}
```

```

class Clonare {
    public static void main(String[] s) {
        Stiva S1 = new Stiva(10);
        S1.push(1); S1.push(3); S1.push(5); S1.push(7);
        Stiva S2 = S1.Clone();
        S2.pop(); S2.pop(); S2.push(13);
        S1.scrie(); S2.scrie();
    }
}

```

La ieșire va apărea:

```

1 3 13 7
1 3 13

```

care nu este rezultatul dorit. Explicația constă în faptul că la clonare, la copierea câmpurilor în noul obiect S2, a fost copiată *adresa* tabloului *v* (tipul tablou este un tip referință), deci se lucrează cu aceeași stivă; câmpul *top* este însă distinct pentru cele două obiecte.

Pentru a lucra cu stive distincte, *v* va trebui clonat explicit. Aceasta se realizează transformând comentariul din *Clone* în instrucțiune efectivă. Cu această modificare, programul va produce acum la ieșire rezultatele dorite:

```

1 3 5 7
1 3 13

```

6.8. Clase interne

Până acum am lucrat sub restricția că o clasă nu poate conține o altă clasă. Dar Java permite să definim clase nu numai ai pachetelor, dar și ca membri ai unei clase, în interiorul unei metode sau a unui bloc. Mai mult, așa cum vom vedea mai jos, pot apărea clase fără nume.

Clasele care sunt membrii unui pachet sunt considerate *clase de nivel superior*.

Clasele declarate într-o clasă se împart în două categorii:

- *clase de nivel superior*: este vorba de membrii statici ai unei clase de nivel superior; aceste clase au deci același statut cu clasele membre ale unui pachet;

- *clase interne*: clasele care nu sunt de nivel superior.

Într-o clasă de nivel superior putem plasa și o interfață.

Așa cum am menționat, într-un bloc sau corpul unei metode putem defini clase la fel cum putem defini variabile locale; aceste clase sunt clase interne.

O clasă internă poate conține la rândul ei o clasă internă.

Instanțierea unei clase interne este asociată instanțierii curente a clasei care o conține; drept urmare clasa internă poate accesa direct câmpurile clasei ce o cuprinde.

Cele de mai sus nu mai rămân valabile pentru clasele de nivel superior: o clasă care este membru static al unei clase nu poate folosi direct câmpurile clasei ce o cuprinde (dar putem realiza acest lucru în mod indirect, prin crearea unui obiect ce instanțiază clasa al cărui membru este).

O clasă internă nu poate fi folosită direct decât în clasa în care a fost declarată și în descendenții (în arborele de clase ai) acesteia. Domeniul ei de vizibilitate este cel al unui câmp, respectiv al unei variabile locale.

Avantajul utilizării claselor locale constă în primul rând în faptul că permite includerea lor exact acolo unde trebuie să apară din punct de vedere logic, dar și în ușurința folosirii lor (reamintim că din interiorul unei clase interne putem să ne referim direct la câmpurile clasei ce o conține).

Exemplul 10. Parcurgerea pe niveluri a arborilor.

Parcurgerea pe niveluri a arborilor constă în ordine în:

- vizitarea rădăcinii (vârfului de pe nivelul 0);
 - vizitarea vârfurilor de pe nivelul 1 (descendenții direcți ai vârfurilor de pe nivelul 1);
 - vizitarea vârfurilor de pe nivelul 2 (descendenții direcți ai vârfurilor de pe nivelul 2);
- etc.

Vizitarea vârfurilor se reduce la tipărirea numerelor lor de ordine.

Pentru reprezentarea arborilor am ales ca pentru fiecare vârf să specificăm fiul și fratele lui: un obiect de tipul *varf* are câmpurile *fiu* și *frate*.

Algoritmul folosește o coadă *C* în care inițial apare doar rădăcina *rad* a arborelui:

```
C ← ∅; C ← rad
```

```
while C ≠ ∅
```

```
    i ← C; vizitează i
```

```
    for toți j fii ai lui i
```

```
        C ← j
```

```
    endfor
```

```
endwhile
```

Pentru implementarea cozii am folosit o listă înălțuită, în care primul element este fals (deci coada propriu-zisă începe de la al doilea element al listei).

Programul corespunzător este următorul:

```

class varf {
    int v; varf fiu, frate; static varf rad;

    varf() { }
    varf(int val) { v=val; }
}

```



```

void creare() {
    int i;
    i = (int) IO.read(); rad = new varf(i); creare(rad);
}

void creare(varf x) { //atașează un fiu și un frate lui x
    int i; double d; varf y;
    if (x==null) return;
    IO.write("Fiul lui " + x.v + " : "); d = IO.read();
    if (! Double.isNaN(d) ) {
        i = (int) d; x.fiu = new varf(i); creare(x.fiu);
    }
    IO.write("Fratele lui " + x.v + " : "); d = IO.read();
    if (! Double.isNaN(d) ) {
        i = (int) d; x.frate = new varf(i); creare(x.frate);
    }
}

class elem {
    varf inf; elem leg;
    elem (varf v) { inf = v; }
}

String niveluri() {
    elem p,u,x; varf y; String s = "";
    p = new elem(null); u = new elem(rad); p.leg = u;
    while ( p != u ) {
        x = p.leg; s += x.inf.v + " "; p.leg = x.leg;
        if (p.leg == null) u = p; // !!!
        y = x.inf.fiu;
        while ( y != null ) {
            x = new elem(y); u.leg = x; u = x; y = y.frate;
        }
    }
    return s;
}

class Niveluri {
    public static void main(String[] arg) {
        varf Ob = new varf();
        Ob.creare();
        IO.writeln( "Niveluri : " + '\t' + Ob.niveluri() );
    }
}

```

Se observă că în clasa `varf` apare clasa internă `elem`; elementele listei înălțuite ce implementează coada sunt obiecte de tipul `elem`. Includerea clasei `elem` în clasa `varf` apare ca naturală, deoarece `elem` este folosită pentru prelucrări efectuate asupra instanțelor clasei `varf`.

Am menționat mai sus că există și clase fără nume (anonime). O clasă anonimă apare într-o expresie de forma:

```
new SuperA(...) {...}
```

prin care clasa anonimă extinde o clasă sau implementează o interfață `SuperA` și prin care este creat un obiect al clasei anonime.

Observație. Crearea de clase anonime permite *extinderea implicită a unei clase și implementarea implicită a unei interfețe* (fără a folosi `extends` sau `implements`). Drept urmare codul devine mai scurt și mecanismul este folosit "la locul potrivit". Această tehnică este larg folosită de exemplu în lucrul cu interfețe grafice.

Exemplul 11. Să considerăm următorul program:

```

abstract class A {
    A(int i) { IO.writeln(i); }
    void met() { };
}

class A_C {
    static A ObA;
    public static void main(String[] s) {
        ObA = new A(3) {
            void met() { IO.writeln("O.K."); }
        };
        ObA.met();
    }
}

```

Clasa abstractă `A` are un constructor și o metodă care nu prevede nici o acțiune. Pare ciudat faptul că deși metoda nu este abstractă, clasa a fost declarată ca fiind abstractă; acest "artificiu" este de fapt o invitație la a extinde clasa și a preciza acțiuni specifice pentru metode.

În clasa `A_C` apare metoda principală, în care obiectul `ObA` (având tipul declarat `A`) este creat ca având ca tip real o clasă anonimă care extinde clasa abstractă `A`. Clasa anonimă este o clasă internă. După invocarea constructorului este invocată metoda `met`; este vorba bineînțeles de metoda `met` redefinită în clasa anonimă. Drept urmare la ieșire va apărea:

```

3
O.K.

```

Exemplul 12. Spre deosebire de exemplul precedent, aici am ales ca o clasă anonimă să implementeze o interfață:

```

interface I {
    void met();
}

```

```

class I_C {
    static I ObI;
    public static void main(String[] s) {
        ObI = new I() {
            public void met() { IO.println("O.K."); }
        };
        ObI.met();
    }
}

```

Din nou este creată o clasă (internă) anonimă și un obiect având ca tip real această clasă; obiectul invocă metoda `met` redefinită în clasa anonimă.

Executarea programului produce la ieșire:

O.K.

O interfață nu poate avea constructori expliți. Există însă constructorul implicit cu acțiune nulă; acest constructor a fost folosit în exemplul de mai sus.

Este interesant de subliniat încă un aspect. Clasa `I_C` are modificatorul de acces implicit, dar în interiorul ei apare o metodă cu modificatorul (mai permisiv) **public**. Acesta se datorează faptului că metoda redefinește o metodă (publică) dintr-o interfață.

Clasele interne se dovedesc un instrument foarte util și ușor de mânuit și se va face apel la ele în continuare. Clasele anonime apar inițial ca fiind mai puțin "naturale", dar avantajele (menționate mai sus) ale acestei tehnici fac ca ea să fie foarte utilă și deci folosită, așa cum vom face în lucrul cu interfețe grafice.

7 EXCEPȚII. CONVERSII

7.1. Excepții

• Noțiunea de excepție

Un aspect important de care trebuie ținut cont ori de câte ori scriem un program este de a asigura robustețea sa. Prin aceasta înțelegem identificarea situațiilor nedorite (de exemplu introducerea unor date eronate, eșecul la încercarea de a avea acces la Internet etc.) și specificarea acțiunile corespunzătoare.

Există însă și reversul aspectului descris mai sus. Introducerea de verificări prin instrucțiuni de control face programul greu de citit și afectează buna sa structurare.

Ceea ce dorim este să avem un mecanism care:

- să permită ca fiecare condiție legată de robustețea programului să fie verificată exact acolo unde poate să apară;
- atunci când condiția apare (suntem în situația că s-a întâmplat ceva nedorit), controlul să fie trecut automat unei secțiuni de cod aflată în program "într-un loc potrivit" (care poate fi "departe" de locul unde condiția a fost detectată), cod care realizează acțiunea ce trebuie întreprinsă. Acest cod va fi numit *handler* (mânuiitor) *de excepție*.

Limbajul Java rezolvă problema de mai sus printr-un mecanism numit *excepție*. Excepția semnalează faptul că a intervenit o anumită condiție specială (nedorită); spunem că segmentul de cod care a detectat condiția *lansează o excepție* de tipul corespunzător. Lansarea unei excepții atrage după sine *captarea* ei de către un handler de excepție; în lipsa acestuia, programul se va încheia cu un mesaj de eroare.

Mai precis, trebuie să realizăm următoarele:

- să definim tipurile de excepție care să permită lansarea excepțiilor dorite;
- să lansăm o excepție la "locul potrivit" în program;
- să prevedem la "locul potrivit" un handler de excepție, care să capteze excepția și să întreprindă acțiunile corespunzătoare.

Sunt importante următoarele precizări:

- precizarea tipului de excepție prin clauza **throws** este importantă pentru că programatorul (ca și compilatorul) trebuie să știe ce excepție poate fi lansată și care este tipul ei, la fel cum trebuie să cunoască tipul valorii întoarse de o metodă;
- dacă din interiorul metodei dorim să prevedem lansarea mai multor excepții, atunci toate tipurile lor trebuie precizate în clauza **throws**, despărțite între ele prin virgulă;
- o metodă poate lansa numai excepțiile care sunt precizate în antetul său (mai precis în clauza **throws**); lansarea poate avea loc din interiorul metodei sau din interiorul altei metode pe care ea o invocă;
- nu este permis să lansăm excepții din interiorul inițializărilor statice; în schimb acest lucru este permis din interiorul constructorilor;
- la redefinirea unei metode într-o subclasă, nu putem preciza în **throws** excepții suplimentare.

• Construcția **try**

Mecanismul incipient descris mai sus permite crearea unei excepții și invocarea unui constructor al său. Dar o excepție lansată în corpul unei metode trebuie să aibă drept consecință executarea unor acțiuni cuprinse într-un handler de excepție prezent tot în corpul metodei. Tipul de excepție respectiv este însă o clasă, care poate avea și metode. În handlerul de excepție este posibilă invocarea unor metode ale clasei a cărei instanțiere este obiectul în discuție. Toate aceste aspecte sunt rezolvate de construcția **try**.

Construcția **try** are forma:

```
try bloc
catch(E_1 e_1) bloc1
catch(E_2 e_2) bloc2
...
finally bloc_final
unde:
```

- **E_1**, **E_2**, ... sunt tipuri de excepție (clase ce extind **Exception**);
- **e_1**, **e_2**, ... sunt identificatori;
- clauza **finally** este opțională;
- blocurile reprezintă tocmai handlerele de excepție.

Dacă în blocul **bloc** asociat lui **try** nu este lansată nici o excepție, atunci se trece de ultimul **catch**.

Să presupunem acum că din **bloc** este lansată o excepție; fie **E** tipul său. Se încearcă pe rând, în ordinea în care apar clauzele **catch**, asocierea ei cu una dintre ele. Dacă **E** coincide cu **E_1** sau este derivată din (extinde pe) **E_1**, atunci se execută blocul **bloc_1**, după care se trece de ultima clauză **catch**. În caz contrar se încearcă pe rând asocierea excepției cu **E_2**, **E_3** etc. Dacă nu s-a reușit o

asociere, se merge "la un nivel superior" într-o construcție **try** care o cuprinde pe cea în discuție etc. Dacă nici în acest mod nu s-a identificat o asociere, se trece în codul care a invocat metoda curentă etc. În final se va reuși o asociere, deoarece altfel ar apare o eroare chiar în faza de compilare.

Tot o eroare la compilare va apare și dacă de exemplu **E_2** este derivată din **E_1**; explicația constă în faptul că lansarea unei excepții de tipul **E_2** nu va putea conduce la captarea ei de către clauza identificată prin **E_2** (excepția va fi captată de clauza **catch** precedentă).

Conform celor de mai sus, este posibil ca o construcție **try** să nu aibă atașată nici o clauză **catch**.

Să mai observăm că la lansarea unor excepții din **bloc** (vezi forma generală a construcției **try**), instrucțiunile care îi urmează în acest bloc nu vor mai fi executate. Mai este clar și că o construcție **try** poate fi utilizată pentru captarea mai multor excepții.

Așa cum am anunțat anterior, o construcție **try** poate avea și o clauză **finally**. Aceasta permite executarea unei secvențe de instrucțiuni indiferent dacă din blocul **bloc** este lansată sau nu o excepție. Odată inclusă, clauza **finally** nu poate fi în nici un fel ocolită și blocul **bloc_final** este totdeauna executat. Un caz limită este de exemplu următorul:

```
try { ... return 0; ... }
finally { return 1; }
```

pentru care valoarea întoarsă este totdeauna 1.

De obicei clauza **finally** este folosită pentru a elibera anumite resurse, ca de exemplu resurse grafice sau fișiere (prin închiderea lor).

• Tratarea excepțiilor

Descriem în continuare modul tipic de utilizare a excepțiilor. Pentru simplificare vom considera un singur tip de excepție cu un unic constructor și o unică metodă.

La construcția următoarei clase:

```
class Clasa {
    Clasa(..) {...}
    .. Met(..) throws Exceptie {
        ...
        if(..) throw new Exceptie(..)
        ..
    }
}
```

programatorul a identificat în metoda **Met** o condiție deosebită care ar putea interveni și de aceea a conceput și clasa **Exceptie** pentru a putea prevedea acțiunile ce trebuie întreprinse la îndeplinirea condiției:

```
class Exceptie extends Exception {
    Exceptie(...) {...}
    ... MetExc(...) {...}
}
```

În mod normal, aceste activități sunt întreprinse o singură dată, în vederea unor aplicații ce urmează a fi dezvoltate. Pe de altă parte, clasa *Exceptie* poate fi folosită și de altă clasă decât *Clasa*; de exemplu atât pentru lucrul cu stive, cât și pentru cel cu cozi, prelucrările prevăzute pentru o încercare de adăugare a unui element în stiva/coada plină pot fi aceeași. De aceea de multe ori este natural ca cele două clase să se afle în unități de compilare separate.

Uterior, programatorul scrie un program care cuprinde o clasă *C*, din care este invocată metoda *Met* a clasei *Clasa*, ceea ce poate conduce la lansarea excepției respective. Este clar că aici este locul cel mai potrivit pentru a specifica acțiunea ce trebuie întreprinsă la lansarea excepției. Pentru aceasta este folosită construcția *try*, care captează obiectul excepție și prin intermediul lui invocă metoda *MetExc* a clasei *Exceptie*:

```
class C {
    Clasa Ob = new Clasa(...);
    ...
    try {
        ...
        Ob.Met(...);
        ...
    }
    catch (Exceptie e){
        ... e.MetExc(...); ...
    }
    ....
}
```

S-a prevăzut invocarea metodei *Met* a clasei *Clasa* din interiorul construcției *try*. În cazul în care apare condiția în discuție, este creat un obiect de tipul *Exceptie* prin invocarea constructorului său. Apoi controlul este trecut clauzei *catch* care captează (primește în parametrul său) obiectul creat, ceea ce permite invocarea metodei *MetExc* din *Exceptie*.

Acum discuția de la începutul capitolului capătă consistență: o excepție de tipul *Exceptie* este lansată din clasa *Clasa* și este captată în clasa *C*.

• Un exemplu

Vom aplica cele de mai sus la rezolvarea următoarei probleme:

Un vector *a* de lungime *n* conține *n* caractere. Ele trebuie mutate într-o ordine oarecare în vectorul *b*, prin intermediul unui vector auxiliar *stack* de lungime *lung*, care implementează o stivă. Operațiile permise sunt:

- mutarea unui element din *a* în vectorul *stack*: elementul curent (cel cu numărul de ordine cel mai mic) din *a* este mutat după elementele deja existente în stivă;

- extragerea (inclusiv ștergerea) unui element din vârful stivei și înscriserea sa în *b*, după ultimul element înscris în *b*.

Dificultatea problemei constă în tratarea cazului în care se încearcă extragerea unui element din stiva vidă și a cazului în care se încearcă introducerea unui element în stiva plină.

Prezentăm în continuare programul, urmat de explicațiile necesare.

Unitatea de compilare Plina.java:

```
class Plina extends Exception {
    void print(String s) { IO.write(s); }
}
```

Unitatea de compilare Vida.java:

```
class Vida extends Exception {
    void print(String s) { IO.write(s); }
}
```

Unitatea de compilare S.java:

```
class S {
    int lung, ns; char[] stack;
    S(int lung) { this.lung=lung; stack = new char[lung]; }
    void pune(char c) throws Plina {
        if (ns==lung) throw new Plina();
        else { IO.write(" P"+c); stack[ns]=c; ns++; }
    }
    char ia() throws Vida {
        if (ns==0) throw new Vida();
        else {
            IO.write(" I"+stack[--ns]); return stack[ns];
        }
    }
}
```

Unitatea de compilare Stiva.java:

```
import java.util.*;
class Stiva {
    public static void main(String[] sir) {
        int n=10, na, nb, i; char[] a, b; char c;
        a = new char[n]; b = new char[n];
        Random R = new Random();
        for (c='a', i=0; i<n; i++, c++) a[i]=c;
        na=0; nb=0; S Ob = new S(3);
        while (nb!=n) {
            try {
                if (R.nextInt()<0 && na<n) { Ob.pune(a[na]); na++; }
                else { b[nb] = Ob.ia(); nb++; }
            }
        }
    }
}
```

```

    catch (Vida e) {e.print("\t ~I");}
    catch (Plina e) {e.print("\t ~P");}
}
IO.writeln(); for (i=0; i<n; i++) IO.write(b[i]+" ");
IO.writeln();
}
}

```

Clasa principală este Stiva. Metoda sa main folosește, în afară de vectorii a și b, următoarele:

- variabila na desemnează poziția din a de pe care va fi mutat un element în stivă;
- variabila nb desemnează poziția din b pe care va fi înscris următorul element;

- obiectul R de tipul Random (clasă din pachetul java.util) este folosit pentru apelarea metodei nextInt fără argumente, ce întoarce în mod aleator un număr întreg;

- obiectul Ob de tipul S, unde clasa S implementează operațiile asupra stivei: metoda pune are un parametru de tip **char** pe care îl adaugă stivei, iar metoda ia fără parametri întoarce elementul din vârful stivei.

Metoda main din Stiva repetă, atâta timp cât în b nu au fost înscrise n caractere, următoarele acțiuni:

- este generat aleator un nou număr întreg;
- dacă valoarea sa este negativă și mai există elemente în a, este invocată metoda pune din clasa S cu intenția de a muta un element din a în stivă; invocarea este urmată de mărirea lui na cu o unitate. În caz contrar este invocată metoda ia cu intenția de a obține un element din stivă și de a-l muta în vectorul b;
- în final sunt listate elementele lui b.

Observăm că nu sunt tratate aici cazurile de excepție (adăugare la stivă plină sau extragere din stiva vidă), aceste verificări căzând în sarcina clasei S. Utilizarea clasei Random permite un nedeterminism în succesiunea de operații asupra stivei, ceea ce va conduce la vectori b diferiți de la o execuție la alta.

În clasa S este folosit vectorul stack de lungime lung pentru implementarea stivei. În ns este memorat numărul de elemente din stivă. În antetul metodei pune este declarat tipul de excepție plina, căruia îi corespunde clasa plina ce extinde clasa Exception. Deosebim două situații:

- dacă stiva este plină (ns=lung), atunci prin **throw** este creată o excepție (un obiect de tip Plina). Controlul este trecut acelei clauze **catch** din Stiva care este identificată prin tipul Plina al excepției. Obiectul creat este transmis în e. Prin invocarea e.print(...) este apelată metoda print a clasei Plina care tipărește mesajul primit la invocare. Să observăm că în acest mod se evită incrementarea lui na;

- dacă stiva nu este plină, atunci este înscris un nou element în stivă. Modul în care acționează metoda ia este analog.

Este ușor de observat că s-a folosit exact schema descrisă anterior, cu clasele Plina și Vida în rolul clasei Exceptie, cu S în postura clasei Clasa și cu Stiva în locul clasei C.

• Clasa Exception

În pachetul java.lang apare următoarea ierarhie de clase:

```

Object
  Throwable
    Error
    Exception

```

unde indentarea corespunde extinderii: de exemplu clasa Error extinde clasa Throwable.

Throwable este superclasa tuturor erorilor și excepțiilor din Java. Numai obiecte ce instanțiază această clasă sau subclase ale ei pot fi lansate prin **throw**. Pe de altă parte, în clauza **catch** poate apărea ca tip numai această clasă sau subclase ale ei. Toate aceste clase au un constructor fără parametri, precum și un constructor cu un parametru de tip String, prin care se poate specifica un mesaj de eroare.

Spre deosebire de clasa Exception, clasa Error este concepută pentru condiții anormale (erori majore), pentru care nu este rezonabil să prevedem acțiuni prin mecanismele descrise mai sus.

Prezentăm clasele din pachetul java.lang, derivate din (care extind) clasa Exception:

```

ClassNotFoundException
CloneNotSupportedException
IllegalAccessException
InstantiationException
InterruptedException
NoSuchFieldException
NoSuchMethodException
RuntimeException
  ArithmeticException
  ArrayStoreException
  ClassCastException
  IllegalArgumentException
  IllegalThreadStateException
  NumberFormatException
  IllegalMonitorStateException
  IllegalStateException
  IndexOutOfBoundsException
  ArrayIndexOutOfBoundsException
  StringIndexOutOfBoundsException

```

```
NegativeArraySizeException
NullPointerException
SecurityException
UnsupportedOperationException
```

7.2. Conversii

Java acordă o atenție deosebită tipurilor. Fiecare expresie are un tip ce poate fi dedus din structura expresiei și din tipul entităților ce intră în alcătuirea sa: literal, variabile și invocări de metode. Sunt însă permise, în anumite condiții bine precizate, conversii de la un tip la un alt tip; acest aspect este subiectul acestui subcapitol.

Evident, nu orice conversie este admisă. De exemplu expresia ce apare în instrucțiunea `if` poate avea numai tipul `boolean`, în caz contrar semnalându-se o eroare chiar la compilare. Mai precizăm că o conversie (admisă) poate conduce la o acțiune corespunzătoare în faza de executare, așa cum vom vedea mai jos; aceste acțiuni pot conduce în anumite cazuri la lansarea unei excepții.

Conversiile pot apărea în următoarele contexte:

- la atribuire, când tipul unei expresii trebuie convertit la tipul variabilei ce primește valoarea expresiei;
- la invocarea unei metode sau a unui constructor, când are loc transferul argumente → parametri;
- la conversii ce implică tipul `String`: orice tip poate fi convertit la `String`;
- la evaluarea unei expresii numerice: operanzii trebuie aduși la un tip comun, astfel încât expresia să poată fi evaluată;
- la o conversie explicită.

Ne propunem să prezentăm numai principalele aspecte legate de conversii.

• Conversii implicite

În această categorie intră conversiile efectuate automat, fără vreo precizare explicită în program.

Pentru tipurile primitive, următoarele conversii sunt implicite:

```
byte → short, int, long, float, double;
short → int, long, float, double;
char → int, long, float, double;      int → long, float, double;
long → float, double;                 float → double.
```

Unele dintre aceste conversii pot conduce la o pierdere a preciziei. De exemplu conversia de la `long` la `float` poate afecta cifrele cele mai puțin semnificative ale valorii inițiale. Într-adevăr, executarea secvenței de cod:

```
int i = 1234567890; float f = i; IO.write(i + "\t" + f);
```

produce la ieșire:

```
1234567890 1.234567894E9
```

Pentru tipurile referință, conversiile implicite sunt cele de la tipul `B` la tipul `A`, unde:

- clasa `B` este o subclasă a clasei `A`;
- interfața `B` este o subinterfață a interfeței `A`;
- clasa `B` implementează interfața `A`;
- `B` este `null`, iar `A` este orice clasă, interfață sau tablou.

Discuția anterioară asupra noțiunilor de tip declarat și tip real conduce la concluzia că o conversie de la o superclasă la o subclasă nu poate fi totdeauna acceptată. Astfel, dacă `B` este subclasă a lui `A`, o secvență de tipul:

```
A a; B b; a = new A(...); b = a;
```

nu este acceptabilă, deoarece de exemplu clasa `B` poate defini noi câmpuri.

Conversiile legate de tipul `String` se aplică numai operanzilor operatorului `+`, când cel puțin unul dintre acești operanzi este de tipul `String`; în acest caz operanzii de un tip diferit sunt convertiți (într-un mod specific tipului lor) la `String`.

Conversia implicită poate fi folosită și la transferul argumente → parametri. Un literal întreg nu pot fi însă transmis unui parametru de tip întreg al cărui domeniu de valori este mai restrâns. Astfel, invocarea metodei:

```
void met(short i) { }
```

```
prin met(1);
```

conduce la o eroare de compilare.

• Conversii explicite

Cunoaștem deja că o conversie explicită la un tip `T` se face prin prefixare cu `(T)`, ca de exemplu în:

```
T ObT = (T) Ob;
```

Evident, nu este permisă orice conversie (chiar explicită) de la un tip la un alt tip.

Pentru tipurile primitive sunt admise, în plus față de cele implicite, următoarele conversii explicite:

```
byte → char;                short → byte, char;
char → byte, short           int → byte, short, char;
long → byte, short, char, int, long
float → byte, short, char, int, long;
double → byte, short, char, int, long, float.
```

La fiecare astfel de conversie are loc o prelucrare specifică, ce poate conduce la o pierdere de precizie, dar și la o modificare a ordinului de mărime. Dintre aceste prelucrări menționăm următoarele:

- la conversia de la un tip întreg la un tip întreg cu domeniu de valori mai restrâns se taie biții din stânga (deci nu se mai păstrează neapărat semnul);
- la conversia de la un tip întreg la tipul **char** se păstrează numai cei 16 biți din dreapta;
- la conversia de la un tip în virgulă mobilă la un tip întreg, partea fracționară se pierde, făcându-se rotunjire spre 0;
- la conversia de la **double** la **float** se poate pierde din precizie, dar se poate obține și 0 sau "infinit" (vezi capitolul 2).

Exemplu. Secvența de cod următoare:

```
int i = '\ufff', j = 97 + 65536; short s = -130;
IO.writeln( (short) i + "\t" + (byte) s + "\t" + (char) j );
IO.writeln( (int) -2.8 );
double d = 5e-300; IO.writeln(d + "\t" + (float) d);
d = 1.8e300; IO.writeln(d + "\t" + (float) d);
```

produce la ieșire:

```
-1      126      a
-2
5.0E-300    0.0
1.8E300     Infinity
```

Pentru tipurile referință intră în discuție pentru a fi permise, în plus față de cele implicite, doar cele de la un tip A la un subtip B al său. Dacă A este o interfață și B o clasă, B nu poate fi declarată cu modificatorul **final** decât dacă implementează interfața A.

Condițiile de mai sus sunt necesare, dar nu suficiente.

În general, o conversie de la o superclasă la o subclasă (numită *downcasting*) este permisă numai dacă este făcută explicit și numai dacă referința la superclasă este de fapt o referință la subclasă. Iată un cadru tipic în care se realizează acest lucru (în continuare A este superclasă a lui B):

```
A a; B b1, b2;
b1 = new B(...); ...
a = b1; ...
b2 = (B) a;
```

Pentru a verifica dacă o conversie este permisă, putem folosi operatorul **instanceof**. Pentru tipurile A, B și a un obiect de tipul A, modul tipic de utilizare este:

```
if (a instanceof B) { ... }
```

În cazul în care conversia de la tipul A la tipul B nu poate intra în discuție (vezi mai sus), această instrucțiune provoacă o eroare de compilare. În caz contrar, expresia din **if** are valoarea **true** sau **false**, după cum conversia este permisă sau nu.

Încercarea de a efectua o conversie nepermisă conduce la lansarea excepției **ClassCastException**.



**FACILITĂȚI
STANDARD**

Prezentăm în acest capitol câteva dintre multiplele facilități oferite de Java pentru aplicații standard, numite și programe utilitare: lucrul cu funcții matematice, lucrul cu tablouri și șiruri de caractere, organizări standard ale datelor, sortări etc., dar și aspecte mai deosebite (lucrul cu clase). Vor fi prezentate clase, interfețe și metode ale lor care pun la dispoziție astfel de facilități.

8.1. Funcții matematice

Java pune la dispoziția utilizatorilor o largă gamă de funcții matematice, prin clasa **Math** din pachetul **java.lang**:

```
public final class Math extends Object
```

Clasa **Math** furnizează constantele **E** și **PI** de tip **double** (corespunzătoare lui e și π), precum și metode publice statice, unele menționate în continuare.

Metodele **abs** pentru determinarea valorii absolute au forma:

```
tip abs(tip t)
```

unde **tip** este **int**, **long**, **float** sau **double**.

Următoarele metode au un parametru de tip **double** și întorc o valoare de același tip:

- **sin**, **cos** și **tan** : pentru calculul funcțiilor trigonometrice respective;
- **asin**, **acos** și **atan** : pentru calculul funcțiilor trigonometrice inverse;
- **exp** și **log** : pentru calculul exponențialei și logaritmului în baza e ;
- **sqrt** : pentru calculul rădăcinii pătrate.

Metoda:

```
double pow(double a, double b)
```

întoarce valoarea a^b .

Metoda **random** fără parametri întoarce o valoare aleatoare de tip **double** din intervalul $[0, 1)$.

Metoda:

```
int round(float f)
întoarce valoarea întreagă cea mai apropiată de f.
```

8.2. Clasa Random

Clasa Random apare în pachetul java.util:

```
public class Random extends Object
și are un constructor fără parametri.
```

Metodele publice:

```
boolean nextBoolean()
int nextInt()
long nextLong()
double nextDouble()
float nextFloat()
întorc următoarea valoare aleatoare de tipul respectiv.
```

Metoda:

```
int nextInt(int n)
întoarce următoarea valoare întreagă aleatoare din intervalul [0, n).
```

Metoda:

```
void nextBytes(byte[] b)
întoarce, în tabloul furnizat ca argument, valori aleatoare de tip byte.
```

8.3. Clasele înfășurătoare pentru tipurile primitive

Java asociază tipurilor primitive așa-numitele *clase înfășurătoare*, conform următoarei ierarhii:

```
Object
  Boolean
  Character
  Number (clasă abstractă)
    Byte
    Short
    Integer
    Long
    Float
    Double
```

Clasele înfășurătoare, cuprinse în pachetul java.lang și declarate cu modificatorul final, pun la dispoziție metode publice și statice utile, ca de exemplu cele pentru transformarea unui șir de caractere într-un întreg. Ele mai permit crearea de obiecte pentru păstrarea valorii unui tip primitiv; aceste obiecte

pot fi folosite în contextele în care este așteptat un obiect și nu un tip primitiv. De asemenea clasele înfășurătoare constituie locul potrivit pentru anumite constante (ca de exemplu MIN_VALUE, POSITIVE_INFINITY) și anumite metode (de exemplu isNaN)

Toate clasele înfășurătoare în afară de Boolean pun la dispoziție constantele MIN_VALUE și MAX_VALUE, reprezentând cea mai mică și cea mai mare valoare de tipul respectiv; în cazul claselor Double și Float, MIN_VALUE este cea mai mică valoare *pozitivă*.

În clasele Double și Float mai apar și constantele NaN, NEGATIVE_INFINITY și POSITIVE_INFINITY menționate în capitolul 2.

Fie *tip* unul dintre tipurile primitive boolean, char, byte, short, int, long, float și double și fie *Tip* clasa înfășurătoare corespunzătoare. Toate clasele *Tip* au modificatorii public și final și au constructorii:

```
Tip(tip t)
Tip(String s)
```

prin care este creat un obiect pe baza valorii trimise ca argument, fie prin utilizarea tipului primitiv respectiv, fie prin transmiterea unui șir de caractere ce este transformat într-o valoare de acel tip; transmiterea unui șir de caractere ce nu este o reprezentare validă pentru tipul respectiv lansează excepția:

NumberFormatException.

De asemenea clasele *Tip* au metodele publice:

```
String toString() : întoarce reprezentarea ca șir de caractere a obiectului;
tip tipValue() : întoarce valoarea de tip primitiv asociată obiectului;
boolean equals(Object ob) : întoarce true dacă ob nu este null și
reprezintă aceeași valoare de tipul tip ca și obiectul ce a invocat metoda;
int compareTo(Tip t) : invocată de un obiect Ob de tipul Tip, metoda
întoarce un rezultat negativ, zero sau pozitiv, după cum valoarea (primitivă) a
lui Ob este mai mică egală sau mai mare decât valoarea lui t.
```

Clasele *Tip* numerice (cele ce extind Number) mai includ metoda:

```
static tip parseTip(String s)
ce întoarce valoarea de tip primitiv tip a șirului s; pentru tipul Integer,
metoda are numele parseInt. Dacă s nu este o reprezentare validă a unui literal
de tipul tip, va fi lansată excepția: NumberFormatException..
```

În clasele Double și Float apar și metodele:

```
boolean isNaN(double d) și boolean isNaN(float d)
despre care am vorbit în capitolul 2.
```

Clasa Character mai conține următoarele metode utile:

```
boolean isLowerCase(char c) : verifică dacă c este "literă mică"
boolean isUpperCase(char c) : verifică dacă c este "literă mare"
boolean isSpace(char c) : verifică dacă c este un spațiu alb;
```


char toLowerCase(char c) : întoarce "litera mică" ce corespunde lui c;
char toUpperCase(char c) : întoarce "litera mare" corespunzătoare lui c.

Un obiect de tipul unei clase înfășurătoare mai poate fi obținut și prin invocarea metodei:

static Tip valueOf(String s)

din clasa Tip, ce reprezintă o alternativă la constructorul cu un parametru.

8.4. Aritmetica numerelor mari

Java permite efectuarea de calcule asupra numerelor oricât de mari. Instrumentele necesare se află în clasele din următoarea ierarhie din pachetul java.math:

```
Object
  Number
    BigInteger
    BigDecimal
```

Clasele BigInteger și BigDecimal pun la dispoziție metode pentru o mare varietate de operații aritmetice asupra numerelor întregi, respectiv zecimale, de orice mărime.

Vom restrânge prezentarea la clasa BigInteger.

Atât clasa BigInteger, cât și constructorii și metodele sale sunt publice.

Ne mărginim a prezenta numai constructorul:

BigInteger(String s)

care creează un obiect a cărei valoare este valoarea întreagă reprezentată de s și care poate lansa excepția NumberFormatException.

Metodele acestei clase permit operațiile uzuale cu întregi, adaptându-le la o lungime arbitrară a acestora. În cazul în care o metodă nu poate întoarce un rezultat de tipul anunțat, este lansată excepția ArithmeticException. Prezentăm în continuare câteva dintre aceste metode și rezultatul întors de invocarea lor de către obiectul a de tip BigInteger:

Metodă	Rezultat întors
BigInteger <u>add</u> (BigInteger b)	a+b
BigInteger <u>subtract</u> (BigInteger b)	a-b
BigInteger <u>multiply</u> (BigInteger b)	a*b
BigInteger <u>divide</u> (BigInteger b)	a/b
BigInteger <u>remainder</u> (BigInteger b)	a%b
BigInteger[] <u>divideAndRemainder</u> (BigInteger b)	a/b și a%b
BigInteger <u>abs</u> ()	a

BigInteger <u>negate</u> ()	-a
BigInteger <u>gcd</u> (BigInteger b)	cmmdc(a, b)
BigInteger <u>pow</u> (BigInteger b)	a ^b
int <u>signum</u> ()	sign(a)
BigInteger <u>mod</u> (BigInteger b)	a mod b
BigInteger <u>and</u> (BigInteger b)	a & b
BigInteger <u>or</u> (BigInteger b)	a b
BigInteger <u>not</u> (BigInteger b)	! b
BigInteger <u>xor</u> (BigInteger b)	a xor b
BigInteger <u>shiftLeft</u> (int n)	a << n
BigInteger <u>shiftRight</u> (int n)	a >> n
int <u>compareTo</u> (BigInteger b)	-1, 0 sau 1 (după cum a<b, a=b sau a>b)
BigInteger <u>min</u> (BigInteger b)	min(a, b)
BigInteger <u>max</u> (BigInteger b)	max(a, b)
int <u>intValue</u> (BigInteger b)	(int) a
long <u>longValue</u> (BigInteger b)	(long) a
float <u>floatValue</u> (BigInteger b)	(float) a
double <u>doubleValue</u> (BigInteger b)	(double) a
String <u>toString</u> ()	reprezentarea lui a ca șir
boolean <u>equals</u> (BigInteger b)	a==b (ca valori numerice)

cu precizarea că spre deosebire de remainder, mod întoarce o valoare pozitivă.

O ultimă metodă pe care o prezentăm este:

boolean isProbablePrime(int p)

care întoarce true dacă probabilitatea ca a să fie număr prim este mai mare decât 1-1/2^p. Timpul necesar executării metodei este proporțional cu valoarea lui p.

8.5. Lucrul cu șiruri de caractere

Șirurile (de caractere) din Java sunt obiecte standard. Orice șir este un obiect al clasei String, ce apare în pachetul java.lang:

```
public final class String extends Object
```

Cunoaștem deja câțiva operatori pentru lucrul cu șiruri:

- operatorul +, folosit pentru concatenare;
- operatorul +=, ce creează un nou șir prin concatenarea șirului din stânga operatorului cu șirul din dreapta sa.

Un (obiect de tip) șir nu poate fi modificat (este "read-only") dar, prin utilizarea metodelor clasei String putem obține pe baza lui noi șiruri.

Clasa String conține constructorii:

```
public String() și public String(String s)
```

care permit să construim un nou șir inițializat cu șirul vid, respectiv cu șirul s.

Toate metodele clasei String au modificatorul de acces public. Trecem la prezentarea câtorva dintre ele.

Metodele esențiale ale clasei String sunt:

`int length()` : întoarce lungimea șirul curent (care a invocat metoda);
`char charAt(int i)` : întoarce caracterul de pe poziția `i` din șirul curent.

Să observăm diferențele față de lucrul cu tablouri. Dacă `s` este un șir, iar `a` un tablou, atunci:

- `s.length()` este lungimea șirului, iar `a.length` este lungimea tabloului;

- `s.charAt(i)` este caracterul de pe poziția `i` din șir, iar `a[i]` este elementul de pe poziția `i` din tablou.

Invocarea metodei `charAt` cu un indice în afara șirului lansează excepția: `IndexOutOfBoundsException`. Această excepție mai poate fi lansată și de alte metode prezentate în continuare (identificarea lor se face cu ușurință).

• Metode pentru aflarea unui indice din șir

Următoarele metode ale clasei String întorc un rezultat de tipul `int`, ce reprezintă o poziție cu o anumită proprietate din șirul `s` prin care au fost invocate (dacă o astfel de poziție nu există, este întoarsă valoarea `-1`).

Metodă	Întoarce indicele din șir de pe ...
<code>indexOf(char ch)</code>	prima poziție pe care apare <code>ch</code>
<code>indexOf(char ch, int p)</code>	prima poziție, mai mare sau egală cu <code>p</code> , pe care apare <code>ch</code>
<code>indexOf(String sub)</code>	prima poziție începând de la care apare șirul <code>sub</code>
<code>indexOf(String sub, int p)</code>	prima poziție, mai mare sau egală cu <code>p</code> , începând de la care apare șirul <code>sub</code>
<code>lastIndexOf(char ch)</code>	ultima poziție pe care apare <code>ch</code>
<code>lastIndexOf(char ch, int u)</code>	ultima poziție, cel mult egală cu <code>u</code> , pe care apare <code>ch</code>
<code>lastIndexOf(String sub)</code>	ultima poziție începând de la care apare șirul <code>sub</code>
<code>lastIndexOf(String sub, int u)</code>	ultima poziție, cel mult egală cu <code>u</code> , începând de la care apare șirul <code>sub</code>

Exemplul 1. Următorul program citește două șiruri și determină numărul de apariții al celui de al doilea în primul. Șirurile sunt introduse la executare din linia de comandă.

```
class nr_apar {
    static int nrapar(String s, String subs) {
```

```
int nr=0, indice, poz=0;
indice = s.indexOf(subs,poz);
while (indice>=0 && indice<s.length()) {
    nr++; poz++;
    indice = s.indexOf(subs,poz);
}
return nr;
}

public static void main(String[] s) {
    IO.writeln( "" + nrapar(s[0],s[1]) );
}
}
```

• Compararea șirurilor

Clasa String prevede metode pentru compararea conținutului șirului care invocă metoda cu conținutul unui șir transmis ca argument. Vom presupune că șirul care invocă metodele este `s`. Compararea se referă la șiruri în totalitatea lor sau la subsecvențe ale lor.

Compararea se face conform ordinii lexicografice: compararea a două caractere constă în compararea numerelor lor de ordine în setul de caractere Unicode.

```
boolean equals(String s1) :
    întoarce true dacă șirurile coincid;
int compareTo(String s1) :
    întoarce o valoare negativă (dacă s<s1), zero (dacă s=s1) sau o valoare pozitivă (dacă s>s1);
boolean regionMatches(int p, String s1, int p1, int lung) :
    compară secvențele de lungime lung ce încep în s de pe poziția p, respectiv în s1 de pe poziția p1; este întoarsă valoarea true dacă și numai dacă aceste secvențe coincid;
boolean startsWith(String pre) : întoarce true dacă șirul pre este prefix al lui s și false altfel;
boolean endsWith(String suf) : întoarce true dacă șirul suf este sufix al lui s și false altfel.
```

Să remarcăm că metoda `equals` nu poate fi suplinită de operatorul `==` deoarece prin `s==s1` se compară două referințe și nu conținutul lor.

O altă modalitate de a compara șiruri este furnizată de metoda:

```
String intern()
care întoarce un șir cu același conținut ca cel al șirului care invocă metoda, având în plus următoarea proprietate: pentru orice două șiruri cu același conținut,
```

valoarea întoarsă de metoda `intern` este aceeași. Drept urmare putem verifica dacă două șiruri au același conținut prin:

```
s1.intern() == s2.intern()
```

care este mai rapidă decât:

```
s1.equals(s2).
```

• Determinarea unui subsir cu o anumită proprietate

Metodele clasei `String` ce se încadrează aici sunt următoarele (vom presupune din nou că șirul care invocă metoda este notat prin `s`):

```
String substring(int p) :
```

este întors subsirul lui `s` ce începe de pe poziția `p`;

```
String substring(int p, int u) :
```

este întors subsirul lui `s` ce începe de pe poziția `p` și se termină pe poziția `u-1`;

```
String replace(char c1, char c2) :
```

este întors șirul obținut din `s` prin înlocuirea tuturor caracterelor `c1` prin caracterul `c2`;

```
String toUpperCase() :
```

fiecare caracter este înlocuit cu echivalentul său majusculă, dacă acesta există;

```
String toLowerCase() :
```

fiecare "literă mică" este înlocuită cu "literă mare", dacă această corespondență există;

```
String trim() :
```

întoarce șirul obținut din `s` prin eliminarea spațiilor albe de la începutul și sfârșitul său;

```
String concat(String s1) :
```

întoarce șirul `s+s1`.

• Șiruri și tablouri de caractere

Clasa `String` prevede metode pentru transformarea șirurilor în tablouri de caractere, precum și pentru transformarea inversă:

Metoda `toCharArray()` invocată de un șir `s` întoarce un tablou de caractere de aceeași lungime și cu aceleași elemente.

Metoda statică `copyValueOf` are două forme:

```
String copyValueOf(char[] a)
```

```
String copyValueOf(char[] a, int p, int lung)
```

și întoarce șirul corespunzător tabloului `a` de caractere, respectiv numai acea secvență din tablou ce începe pe poziția `p` și are lungimea `lung`.

Exemplul 2. Pentru a determina șirul obținut din șirul `s` prin considerarea elementelor sale în ordine inversă, putem scrie următoarea metodă:

```
public static String invers(String s) {
    char c;
    char[] a = s.toCharArray();
    for (int p=0,u=a.length-1; p<u; p++,u--) {
        c=a[p]; a[p]=a[u]; a[u]=c;
    }
    return String.copyValueOf(a);
}
```

• Conversii

În multe cazuri suntem puși în situația de a converti valori numerice în șiruri sau de a trebui să lucrăm cu o valoare numerică memorată într-un șir.

Șirul ce corespunde unei valori numerice sau booleene `x` se obține simplu prin:

```
String.valueOf(x)
```

Metoda `valueOf` a clasei `String` mai poate fi invocată și pentru un caracter: va fi întors șirul format din acel unic caracter.

Transformarea inversă (de la un șir ce conține un literal boolean sau numeric) la acel literal se poate realiza folosind metodele puse la dispoziție de clasele înfășurătoare.

O altă clasă utilă pentru lucrul cu șiruri este clasa `StringTokenizer`, care permite detectarea și extragerea de entități dintr-un șir. Această clasă va fi prezentată în subcapitolul despre organizări ale datelor.

8.6. Prelucrări asupra tablourilor

În acest subcapitol vom prezenta câteva facilități oferite de Java pentru unele prelucrări asupra tablourilor, cum ar fi sortarea elementelor tabloului sau căutarea binară a unei valori în tablou (în cazul în care elementele tabloului sunt deja sortate).

Aceste prelucrări au la bază o relație de ordine totală. Dacă pentru tipurile primitive ea este implicită, pentru tipurile referință (reamintim că elementele unui tablou pot fi de orice tip) ea trebuie definită explicit.

• Interfața `Comparator`

Interfața `Comparator`:

```
public interface Comparator
```

apare în pachetul `java.util` și în ea este anunțată metoda:

```
public int compare(Object Ob1, Object Ob2)
```

menită să compare cele două argumente conform unei relații de ordine totale, specificate în implementările metodei. Dacă notăm relația de ordine prin `<`, atunci rezultatul întors trebuie să fie negativ, egal cu zero sau pozitiv, după cum

$Ob1 < Ob2$, $Ob1 = Ob2$ sau $Ob1 > Ob2$. Cade în sarcina programatorului să definească în mod corect relația de ordine totală.

Odată definit un *comparator* (un obiect al unei clase ce implementează *Comparator*), el poate fi folosit în diferite contexte, de exemplu pentru sortarea elementelor unui tablou pe baza unei relații de ordine stabilită de programator.

• Clasa Arrays

Clasa Arrays:

public class Arrays extends Object
apare în pachetul java.util și pune la dispoziție mai multe metode (toate publice și statice) pentru prelucrări asupra tablourilor.

În cele ce urmează, prin *tip* este desemnat orice tip primitiv.

Metodele de sortare sunt definite astfel:

```
void sort(tip[] t)
void sort(Object[] O, Comparator c)
și realizează sortarea elementelor tabloului pe baza relației implicite de ordine
pentru tipurile primitive, respectiv pe baza comparatorului specificat ca argument.
Mai sunt utile și variantele:
```

```
void sort(tip[] t, int p, int u)
void sort(Object[] O, int p, int u, Comparator c)
prin care se poate sorta numai porțiunea din tablou cu indicii p..u-1. În acest caz
trebuie ca pentru tabloul tab primit ca argument să fie verificată relația
 $0 \leq p < u \leq \text{tab.length}$ , în caz contrar fiind lansată una dintre excepțiile:
IllegalArgumentException sau ArrayIndexOutOfBoundsException.
```

Metodele de verificare a egalității a două tablouri sau a două porțiuni ale lor determină dacă două tablouri sunt egale, adică dacă au același număr de elemente și elementele de pe aceleași poziții sunt egale.

```
boolean equals(tip[] t1, tip[] t2)
boolean equals(Object[] O1, Object[] O2, Comparator c)
boolean equals(tip[] t1, tip[] t2, int p, int u)
boolean equals(Object[] O1, Object[] O2, int p, int u,
                Comparator c)
```

cu aceleași precizări ca mai sus.

Metodele de căutare a unui element într-un tablou ordonat de elemente:

```
int binarySearch(Tip[] t, Tip x)
int binarySearch(Object[] O, Object x, Comparator c)
verifică dacă x este unul dintre elementele tabloului. În caz afirmativ, este întoarsă poziția pe care se află x. În caz contrar, este întoarsă valoarea (negativă!) -p-1, unde p este poziția pe care ar trebui inserat x. În prima formă, Tip este tipul unei clase înfășurătoare și nu este necesar un comparator explicit, dacă relația de ordine
```

cu care lucrăm este cea uzuală pentru tipul primitiv corespunzător. În a doua formă se specifică explicit un comparator.

Exemplul 3. Este creat un tablou cu trei componente de tipul elem, care definește doar două câmpuri întregi. Mai este folosit un comparator care specifică ordinea lexicografică pentru compararea obiectelor de tipul elem.

```
import java.util.*;
class elem {
    int x; int y;
    elem(int i, int j) { x=i; y=j; }
}

class comp implements Comparator {
    public int compare(Object ee, Object ff) {
        elem e = (elem)ee; elem f = (elem)ff;
        if (e.x==f.x && e.y==f.y) return 0;
        else if ( e.x<f.x || e.x==f.x && e.y<f.y ) return -1;
        else return 1;
    }
}

class Compar {
    public static void main(String[] s) {
        elem[] tab =
            {new elem(1,2), new elem(3,1), new elem(1,1)};
        comp c = new comp();
        Arrays.sort(tab,c);
        for (int i=0; i<tab.length; i++) {
            System.out.println(tab[i].x + " " + tab[i].y);
        }
    }
}
```

8.7. Organizări ale datelor

Pe lângă tablouri, Java pune la dispoziție o largă varietate de organizări ale datelor, însoțite de modalități de gestionare a lor: adăugare, ștergere, regăsire etc.

Prezentăm ierarhia de interfețe din pachetul java.util ce stă la baza diferitelor organizări ale datelor:

Collection
Set
List
Map
Enumeration

Interfața `Collection` este rădăcina ierarhiei de colecții din Java.

O *colecție* este un grup de elemente, nu neapărat distincte.

Interfața `Collection` nu are clase care o implementează direct. Ea anunță metode ca de exemplu `add`, `remove`, `contains`, `isEmpty`, `size` cu semnificație evidentă.

O *mulțime* (corespunzătoare interfeței `Set`) este o colecție care nu conține duplicate și este menită să abstractizeze noțiunea matematică de mulțime.

O *listă* (corespunzătoare interfeței `List`) este o colecție ordonată, eventual cu duplicate. Ea anunță în plus metode pentru extragerea unei subliste, pentru înlocuirea unui element de pe o anumită poziție cu un alt element etc.

O *mapare* (corespunzătoare interfeței `Map`) este o funcție (în sens matematic), ce asociază unei *chei* o anumită *valoare*. Funcția nu este neapărat injectivă, deci mai multor chei li se poate asocia aceeași valoare. Atât cheile, cât și valorile sunt obiecte. O mapare permite trei abordări distincte, în funcție de: mulțimea de chei, colecția de valori sau mulțimea de perechi (*cheie, valoare*).

O *enumerare* (corespunzătoare interfeței `Enumeration`) permite ca un obiect "de acest tip" să genereze câte un element prin invocarea metodei `nextElement`. Această interfață mai anunță metoda `hasMoreElements`. De exemplu clasa `Vector`, care va fi prezentată pe larg în continuare, include metoda `elements` care întoarce un obiect de tip `Enumeration`, ce poate fi folosit de exemplu pentru listarea elementelor unui vector `v`:

```
for (Enumeration e = v.elements(); e.hasMoreElements(); )
    System.out.println(e.nextElement() + " ");
```

Dintre clasele care implementează interfețele de mai sus, enumerăm următoarele:

```
class LinkedList implements List (pentru lucrul cu liste înlănțuite)
class HashTable implements Map (pentru stocarea și regăsirea informațiilor
    folosind tabele de dispersie)
precum și clasele Vector și StringTokenizer, ce vor fi prezentate ulterior.
```

O clasă folosită din pachetul `java.util` este următoarea:

```
class Collections extends Object
care pune la dispoziție diverse operații asupra colecțiilor (în special asupra listelor),
prin metodele sale statice. Această clasă va fi și ea prezentată în continuare.
```

• Clasa `Vector`:

```
public class Vector implements List
permite lucrul cu un tablou de elemente oarecare, dar cu multe facilități
suplimentare. Un vector (obiect de tip Vector) are dimensiune variabilă, spre
deosebire de un tablou obișnuit, care are lungime fixă. Într-un vector putem insera
pe orice poziție noi elemente, dimensiunea sa crescând automat. De asemenea
putem extrage elemente din vector.
```

La fiecare moment de timp, vectorul este caracterizat prin:

- *tabloul curent* asociat vectorului, în care sunt păstrate elementele vectorului;

- *capacitate*: mărimea tabloului curent;

- *lungime*: numărul de elemente efective din vector. Lungimea este totdeauna cel mult egală cu capacitatea, iar elementele efective apar pe primele poziții ale vectorului (începând cu poziția 0);

- *increment*: cantitatea cu care crește automat capacitatea vectorului atunci când, la o operație de inserare a unui nou element, lungimea este egală cu capacitatea. Gestionarea incrementului se poate face atât implicit, cât și explicit.

Clasa `Vector` are următorii constructori:

```
Vector()
```

```
Vector(int c)
```

```
Vector(int c, int i)
```

prin invocarea căruia ia naștere un vector vid (de lungime 0), pentru care putem specifica (opțional) o capacitate și un increment.

Câmpurile clasei sunt următoarele:

```
protected int elementCount;
```

```
protected Object[] elementData;
```

reprezentând lungimea vectorului, respectiv tabloul asociat vectorului.

Toate metodele clasei sunt publice.

În cele ce urmează vom numi *indice valid* un indice din intervalul $i..lungime-1$, unde i apare ca parametru în metodele ce urmează. La specificarea unui indice nevalid, este lansată excepția `ArrayIndexOutOfBoundsException`.

Metodele ce modifică vectorul sunt următoarele:

```
void setElementAt(Object Ob, int i)
```

adaugă obiectul `Ob` pe poziția i , peste cel existent; i trebuie să fie un indice valid;

```
void removeElementAt(int i)
```

elimină elementul de pe poziția i și deplasează la stânga elementele de pe pozițiile următoare; i trebuie să fie un indice valid;

```
void insertElementAt(Object Ob, int i)
```

deplasează la dreapta elementele de pe pozițiile $i..lungime$ și inserează obiectul `Ob` pe poziția i ; i trebuie să fie un indice valid;

```
void addElement(Object Ob)
```

adaugă obiectul `Ob` la sfârșitul vectorului;

```
boolean removeElement(Object Ob)
```

elimină, cu deplasarea la stânga a următoarelor elemente, prima apariție în vector a obiectului `Ob`. Dacă obiectul `Ob` nu apare în vector, efectul este nul.

Metoda întoarce rezultatul căutării;

```
void removeAllElements()
```

elimină toate elementele vectorului.

Metodele care examinează conținutul vectorului sunt următoarele:

```
int size()
    întoarce lungimea vectorului;
boolean isEmpty()
    verifică dacă vectorul este vid;
boolean contains(Object Ob)
    verifică dacă obiectul Ob apare în vector;
int indexOf(Object Ob)
    întoarce prima poziție pe care Ob apare în vector, respectiv -1 dacă Ob nu
    apare în vector;
int lastIndexOf(Object Ob)
    întoarce ultima poziție pe care Ob apare în vector, respectiv -1 dacă Ob nu
    apare în vector;
Object elementAt(int i)
    întoarce elementul din vector de pe poziția i; i trebuie să fie un indice valid;
Object[] toArray()
    întoarce un tablou cu elementele (efective ale) vectorului;
Object firstElement()
    întoarce primul element al vectorului;
Object lastElement()
    întoarce ultimul element al vectorului.

    Ultimele două metode lansează excepția NoSuchElementException
    dacă vectorul este vid.
```

Mai prezentăm și metodele:

```
void setSize(int lung)
    stabilește o nouă lungime pentru vector. Dacă lung > lungime, se adaugă
    obiecte nule. Dacă lung < lungime, sunt eliminate elementele de pe pozițiile
    lung..lungime;
boolean equals(Object Ob)
    verifică dacă vectorul care a invocat metoda este egal cu Ob (se verifică atât
    lungimea, cât și egalitatea elementelor corespunzătoare).
```

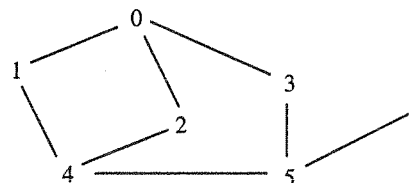
Exemplul 4. Parcurgerea pe niveluri a grafurilor neorientate.

În capitolul 2 am prezentat un program pentru parcurgerea DF a grafurilor neorientate, iar în capitolul 6 am prezentat un program pentru parcurgerea pe niveluri a arborilor. Prezentăm acum o implementare a parcurgerii pe niveluri a unui graf neorientat oarecare. Pentru simplificare, vom considera că graful este conex.

Parcurgerea pe niveluri a unui graf presupune alegerea unui vârf drept vârf inițial și vizitarea vârfurilor grafului în ordinea distanțelor lor față de vârfurile inițiale. Prin vizitarea unui vârf se înțelege efectuarea unor prelucrări, ce depind de problema concretă, asupra vârfului. În cele ce urmează, parcurgerea unui vârf va consta în scrierea numărului său de ordine.

Parcurgerea pe niveluri este utilă de exemplu în situațiile în care căutăm vârful cel mai apropiat de vârful inițial și care are o anumită proprietate (evident, în acest caz vizitarea vârfurilor include verificarea proprietății respective).

Să considerăm de exemplu următorul graf:



pentru care listele vecinilor vârfurilor sunt următoarele:

$L_0 = \{1, 2, 3\}$; $L_1 = \{0, 4\}$; $L_2 = \{0, 4\}$; $L_3 = \{0, 5\}$;
 $L_4 = \{1, 2, 5\}$; $L_5 = \{3, 4, 6\}$; $L_6 = \{5\}$.

Dacă alegem vârful 0 drept vârf inițial, parcurgerea pe niveluri trebuie să listeze vârfurile în ordinea: 0 1 2 3 4 5 6.

Pentru a evita vizitarea repetată a unui același vârf, vom atașa fiecărui vârf o variabilă v_i , ce are valoarea true sau false după cum vârfurile au fost sau nu vizitate. Algoritmul folosește o coadă C:

```
v_i ← false, pentru orice vârf i
C ← ∅; C ← 0 {vârfurile inițiale}; v_0 ← true
while C ≠ ∅
    i ← C; vizitează i
    for toți j ∈ L_i
        if ! v_j then C ← v_j; v_j ← true
    endif
endfor
endwhile
```

Programul are aceeași structură ca cel din capitolul 2, menționat mai sus. Numele clasei df a fost schimbat în nivel. În tabloul temp, al cărui număr curent de elemente este ntemp, sunt citite succesiv listele vecinilor vârfurilor. Metoda parcurg implementează algoritmul de mai sus.

```
import java.util.*;
class nivel {
    int[][] mat; int n;

    nivel() {
        int[] temp; int ntemp; double d;
        IO.write("n= "); n = (int) IO.read();
        mat = new int[n][n]; temp = new int[n];
        for (int i=0; i<n; i++) {
            IO.write("Vecinii lui "+i+" : ");
```

CIPRIAN COMȘA
Nr. 406

```

    ntemp = 0; d = IO.read();
    while( ! Double.isNaN(d) )
        { temp[ntemp++] = (int) d; d = IO.read(); }
    mat[i] = new int[ntemp];
    for (int j=0; j<ntemp; j++) mat[i][j] = temp[j];
}
IO.writeln("*****");
}

void parcurg() {
    boolean[] v = new boolean[n]; Integer Int; int i,j,k;
    Vector coada = new Vector();
    coada.addElement(new Integer(0)); v[0] = true;
    while ( ! coada.isEmpty() ) {
        Int = (Integer) coada.firstElement();
        coada.removeElementAt(0);
        i = Int.intValue(); IO.write(i+"\t");
        for (k=0; k<mat[i].length; k++) {
            j = mat[i][k];
            if( ! v[j] ) {
                coada.addElement(new Integer(j)); v[j]=true;
            }
        }
    }
}

class Niveluri {
    public static void main(String[] s) {
        nivel Ob = new nivel(); Ob.parcurg();
    }
}

```

Pentru implementarea cozii a fost folosit vectorul `coada`. Conform disciplinei de coadă, extragerile din vector s-au făcut de pe poziția 0, iar adăugările s-au făcut la sfârșitul vectorului.

Să observăm că pentru a introduce în vector numerele atașate vârfurilor, a fost nevoie să apelăm la clasa `Integer`, deoarece vectorii pot avea ca elemente doar obiecte și nu tipuri primitive.

• Clasa `Stack`:

`public class Stack extends Vector`
adaugă clasei `Vector` facilitățile uzuale de lucru cu stivă.

Clasa `Stack` are un constructor fără parametri, care crează o stivă vidă, precum și următoarele metode publice:

Object `push`(Object Ob)
adaugă obiectul Ob în vârful stivei și îl întoarce ca rezultat;

boolean `empty`()
verifică dacă stiva este vidă;
int `search`(Object Ob)
dacă obiectul Ob nu apare în stivă, este întoarsă valoarea -1. Dacă Ob apare în stivă, este întoarsă poziția lui Ob relativă la vârful stivei (se consideră că vârful stivei apare pe poziția 1).
Object `pop`()
elimină obiectul din vârful stivei și îl întoarce ca rezultat;
Object `peek`(Object Ob)
întoarce obiectul din vârful stivei, fără a-l elimina din stivă.
Ultimele două metode lansează excepția `EmptyStackException` în cazul în care stiva este vidă.

• Clasa `StringTokenizer`:

`public class StringTokenizer extends Object`
implements Enumeration
furnizează metode pentru a extrage entități dintr-un șir. Entitățile sunt separate între ele prin unul sau mai multe caractere, cu rol de delimitatori. Delimitatorii impliciti sunt `'\t'`, `'\n'`, `'\r'` și `'\f'`; putem însă specifica explicit o mulțime de delimitatori. De asemenea putem alege între a considera sau nu un delimitator ca fiind o entitate.

Detectarea entităților se face secvențial de la începutul șirului spre sfârșitul său.

Observație. Despărțirea unui șir în entități se poate face și apelând la metodele clasei `StreamTokenizer` folosită în clasa `IO` prezentată în al doilea capitol. Această clasă detectează entități mai "fine" (de exemplu literali) și va fi prezentată în capitolul dedicat facilităților de intrare/ieșire.

Constructorii clasei `StringTokenizer` sunt următorii:

`public StringTokenizer(String s)`
`public StringTokenizer(String s, String delim)`
`public StringTokenizer(String s, String delim, boolean b)`
unde `s` este șirul ce urmează a fi separat în entități, `delim` este un șir de caractere ce cuprinde mulțimea de separatori aleasă de utilizator, iar `b` permite să alegem între a considera sau nu delimitatorii drept entități.

Principalele metode puse la dispoziție de clasa `StringTokenizer` sunt:

`public boolean hasMoreTokens()`
`public String nextToken()`
unde `hasMoreTokens` verifică dacă mai există entități în șir după poziția curentă, iar `nextToken` întoarce următoarea entitate; această a doua metodă lansează excepția `NoSuchElementException` dacă după poziția curentă nu mai urmează nici o entitate.

Modalitatea tipică de utilizare a acestei clase pentru un șir *s*, în ipoteza că suntem "mulțumiți" cu delimitatorii impliciți și nu dorim să-i considerăm ca entități, este următoarea:

```
StringTokenizer st = new StringTokenizer(s);
String e;
while (st.hasMoreTokens()) {
    e = st.nextToken(); prel(e);
}
```

unde metoda *prel* realizează o prelucrare a entității (subșirului) *e*, specifică aplicației curente.

Așa cum am menționat mai sus, clasa *StringTokenizer* este mai restrictivă decât *StreamTokenizer*, dar poate fi folosită cu succes și pentru detectarea literalilor, când cunoaștem precis succesiunea tipurilor lor în șir și apelăm la metodele claselor înfășurătoare.

Exemplul 5. Dacă șirul supus prelucrării conține numai literali întregi, aceștia pot fi obținuți astfel:

```
import java.util.*;
class Test {
    public static void main(String[] sir) {
        String s = " 23 \t 45\n", e="";
        StringTokenizer st = new StringTokenizer(s);
        while( st.hasMoreTokens() ) {
            e = st.nextToken(); IO.writeln(""+Integer.parseInt(e));
        }
    }
}
```

• Clasa Collections

Clasa *Collections* din pachetul *java.util*:
`public class Collections extends Object`
 pune la dispoziția utilizatorului metode publice și statice pentru operații asupra colecțiilor, dintre care menționăm următoarele:

```
void reverse(List lista):
    inversează ordinea elementelor din listă;
void shuffle(List lista):
    permută aleator ordinea elementelor din listă;
void copy(List lista1, List lista2):
    copiază lista1 în lista2;
void sort (List lista, Comparator c):
    sortează, prin metoda sortării prin interclasare și conform ordinii stabilite prin comparatorul c, lista primită ca argument;
```

`int binarySearch(List lista, Object x, Comparator c):`

folosește metoda căutării binare pentru a verifica dacă obiectul *x* apare în lista *lista*, ordonată conform comparatorului *c*. În caz afirmativ, este întoarsă poziția pe care se află *x*. În caz contrar, este întoarsă valoarea (negativă!) *-p-1*, unde *p* este poziția pe care ar trebui inserat *x*.

`Object min(Collection col, Comparator comp):`

întoarce cel mai mic element din listă (conform ordinii stabilite prin comparatorul *c*);

`Object max(Collection col, Comparator comp):`

întoarce cel mai mare element din listă (conform ordinii stabilite prin comparatorul *c*).

Pentru ultimele patru metode există și varianta de a nu specifica explicit un comparator. Această variantă nu este aplicabilă în general, deoarece la încercarea de a compara două obiecte nu e clară modalitatea de comparare și drept urmare va fi lansată excepția *ClassCastException*. Varianta poate fi totuși folosită de exemplu când elementele din listă au același tip și anume o clasă înfășurătoare.

Exemplul 6. Citim succesiv de la intrare câte două numere întregi *x* și *y*. Dacă *y* este diferit de 0, construim un obiect de tipul *elem* (vezi *Exemplul 3*), iar în caz contrar construim un obiect de tipul *Integer* pe baza valorii lui *x*. Citirea, și deci crearea de noi obiecte, se oprește imediat ce pentru *x* am citit valoarea 0. Dorim să sortăm obiectele folosind metoda sortării prin inserție binară. Pentru compararea a două obiecte folosim următoarele reguli:

- dacă ambele sunt de tipul *elem*, se folosește ordinea lexicografică, așa cum am procedat și în *Exemplul 3*;
- orice element de tip *Integer* este considerat mai mic decât orice obiect de tipul *elem*;
- pentru două elemente de tipul *Integer* se folosește ordinea naturală a valorilor lor primitive.

Pentru rezolvare, vom introduce succesiv obiectele create într-un vector *V* și anume prin inserție binară: prin invocarea metodei *binarySearch* a clasei *Collections* cu comparatorul *c* corespunzător, obținem poziția din *V* unde trebuie inserat noul obiect; apoi inserăm obiectul pe acea poziție. Să observăm că în loc de vector nu putem folosi un tablou, deoarece elementele nu au același tip; în plus tablourile au dimensiune fixă, deci nu putem efectua inserarea. Pentru detectarea tipului real al elementelor lui *V* apelăm la operatorul *instanceof*.

Programul corespunzător este următorul:

```
import java.util.*;
class elem {
    int x,y;
    elem(int x,int y) { this.x=x; this.y=y; }
    void print() { IO.writeln(x + " " + y); }
}
```



```

class comp implements Comparator {
    public int compare(Object ee, Object ff) {
        boolean b1 = ee instanceof elem,
               b2 = ff instanceof elem;
        if(b1 && b2) {
            elem e = (elem)ee; elem f = (elem)ff;
            if (e.x==f.x && e.y==f.y) return 0;
            else if ( e.x<f.x || e.x==f.x && e.y<f.y ) return -1;
            else return 1;
        }
        else if(b2) return -1;
        else if(b1) return 1;
        else return
            ((Integer)ee).compareTo((Integer)ff);
    }
}

class Sort {
    public static void main(String[] s) {
        int x,y,poz; Object Ob; comp c = new comp();
        Vector V = new Vector();
        x = (int) IO.read(); y = (int) IO.read();
        while ( x != 0 ) {
            if(y != 0) Ob = new elem(x,y);
            else Ob = new Integer(x);
            poz = Collections.binarySearch(V,Ob,c);
            if (poz<0) poz = -poz-1; V.insertElementAt(Ob,poz);
            x = (int) IO.read(); y = (int) IO.read();
        }
        IO.writeln("*****");
        for(int i=0; i<V.size(); i++) {
            Ob = V.elementAt(i);
            if(Ob instanceof elem) ((elem)Ob).print();
            else IO.writeln( ((Integer)Ob).intValue() + " ");
        }
    }
}

```

8.8. Lucrul cu clase. Clasele `Class` și `Method`

Așa cum știm, clasa reprezintă unitatea fundamentală de programare în Java, iar obiectele de tipul unei clase reprezintă instrumentul folosit pentru invocarea metodelor clasei. Până acum atenția principală a fost îndreptată asupra modului în care acționează obiectele, cu scopul de a îndeplini dezideratele programatorului.

Java permite însă să lucrăm și cu clase, nu numai cu instanțieri ale lor.

În centrul lucrului cu clase stă clasa `Class`, care apare în pachetul `java.lang` și este declarată prin:

```
public final class Class extends Object
```

Un *obiect clasă* (obiect de tip `Class`) este o reprezentare a unei clase dintr-o aplicație în curs de executare. În reprezentare sunt incluse informații despre numele clasei, modificatorii săi, constructorii, câmpurile și metodele sale etc.

Un obiect clasă poate fi obținut prin invocarea metodei `getClass` a clasei `Class`. De exemplu prin:

```
Class C_Ob = Ob.getClass();
```

obținem obiectul clasă `C_Ob` ce sintetizează informații despre clasa a cărei instanțiere este obiectul `Ob`. Este important de precizat că obiectul `C_Ob` sintetizează informații despre clasa ce constituie *tipul real* al obiectului `Ob`.

Informațiile despre clasa ce constituie tipul real al lui `Ob` se pot obține prin invocarea, prin intermediul acestui obiect, a metodelor clasei `Class`.

Clasa `Class` nu are constructori și nici câmpuri.

Dintre metodele, toate publice, ale clasei `Class` prezentăm deocamdată doar următoarele:

```
String getName()
```

întoarce numele clasei care este asociată obiectului curent (obiectului clasă ce a invocat metoda);

```
boolean isInstance(Object Ob)
```

este echivalentul dinamic al operatorului `instanceof`: întoarce `true` dacă și numai dacă obiectul `Ob` poate fi convertit la tipul obiectului curent;

```
String getSuperClass()
```

întoarce numele superclasei asociate obiectului curent;

```
Class[] getInterfaces()
```

întoarce un tablou cu interfețele implementate de clasă, în ordinea în care apar în clauza `implements`;

```
Class[] getClasses()
```

întoarce un tablou cu clasele și interfețele publice care sunt membri ai clasei reprezentate de obiectul curent;

```
int getModifiers()
```

întoarce modificatorii clasei, codificați ca un întreg (vezi documentația).

Exemplul 7. Următorul program:

```

interface I { }
class A implements I { }
class B extends A { }
class Clasa {
    static void print(Class C) {
        Class D = C.getSuperclass();
        IO.writeln(C.getName()+" "+D.getName());
    }
}

```

```

public static void main(String[] sir) {
    A Ob = new A();
    I ObI = new A(); Class CI = ObI.getClass(); print(CI);
    A ObA = new B(); Class C = ObA.getClass(); print(C);
    IO.println("" + C.isInstance(Ob));
    ObA = new A(); C = ObA.getClass(); print(C);
    IO.println("" + C.isInstance(Ob));
}
}

```

produce la ieșire:

```

A java.lang.Object
B A
false
A java.lang.Object
true

```

Clasa `Class` mai conține și metode care întorc și informații despre constructorii, câmpurile și metodele clasei reprezentate de obiectul curent care invocă aceste metode. Aceste informații sunt de tipurile claselor din următoarea ierarhie din pachetul `java.lang.reflect`:

Member	(interfață)
<u>Constructor</u>	
<u>Field</u>	
<u>Method</u>	

Vom restrânge prezentarea la clasa `Method`.

Clasa `Method` este declarată prin:

```
final class Method implements Member
```

Un obiect de tip `Method` constituie reprezentarea unei metode și furnizează, prin invocarea metodelor acestei clase, informații despre metoda asociată obiectului. Clasa `Method` nu are constructori, dar un obiect de acest tip poate fi creat prin invocarea metodelor `getMethod` și `getMethods` ale clasei `Class`:

```
Method[] getMethods()
```

întoarce un tablou conținând obiectele de tip `Method` asociate metodelor ce sunt membri publici ai clasei reprezentate de obiectul curent;

```
Method getMethod(String nume, Class[] tipuri)
```

întoarce obiectul de tip `Method` asociat metodei cu numele `nume` și semnatura `tipuri` ce este membru public al clasei reprezentate de obiectul curent. Dacă metoda căutată nu există, este lansată excepția `NoSuchMethodException`.

Următoarele metode ale clasei `Method` sunt toate publice și furnizează informații despre metoda asociată obiectului curent:

```

String getName()
    întoarce numele metodei;
int getModifiers()
    întoarce modificatorii metodei, codificați ca un întreg; pentru decodificare
    trebuie folosită clasa Modifier (vezi documentația);
Class getReturnType()
    întoarce tipul rezultatului produs de metodă;
Class[] getParameterTypes()
    întoarce semnatura metodei;
Class[] getExceptionTypes()
    întoarce un tablou cu tipurile excepțiilor declarate de metodă.

```

Prezentăm și o ultimă, dar foarte importantă metodă a clasei `Method`:

```
Object invoke(Object Ob, Object[] args)
```

care realizează invocarea metodei reprezentată de obiectul curent; `Ob` este obiectul care invocă metoda, iar argumentele sunt sintetizate în tabloul `args`. Pentru metodele statice, `Ob` este ignorat.

Metoda `invoke` poate lansa diferite excepții, pe care nu le vom enumera aici, de exemplu în situații ca: `Ob` nu este în măsură să invoce metoda, `args` nu coincide cu semnatura metodei, obiectele din `args` nu pot fi convertite la tipurile din semnatura metodei etc.

Exemplul 8. Exemplul își propune să furnizeze un mecanism prin care, cunoscând șirul de caractere ce reprezintă numele unei metode, să invoce acea metodă.

Prezentăm programul, în care (pentru simplificare) am folosit calea nerecomandată de a nu identifica toate excepțiile ce pot fi lansate.

Unitatea de compilare `InvocMet.java`:

```

import java.lang.reflect.Method;
public class InvocMet {
    public static void invoc(Object Ob, String nume,
        Object[] args) throws Exception {
        Class ObC = Ob.getClass();
        Method met=null;
        Class[] argsTypes = new Class[args.length];
        for (int i=0; i<args.length; i++)
            argsTypes[i]=args[i].getClass();
        met = ObC.getMethod(nume, argsTypes);
        met.invoke(Ob, args);
    }
}

```

Unitatea de compilare `Metoda.java`:

```

class C {
    public void met1() { IO.println("1"); }
}

```

```

    public void met2(Integer i)
    { IO.writeln("2 "+i.toString()); }
}

public class Metoda {
    public static void main(String[] sir) throws Exception {
        C Ob = new C();
        Object[] arg = new Object[0];
        InvocMet.invoc(Ob, "met"+1, arg);
        Integer Int = new Integer(55);
        arg = new Object[1]; arg[0] = Int;
        InvocMet.invoc(Ob, "met"+2, arg);
    }
}

```

Metoda invoc din clasa InvocMet prevede următoarele acțiuni:

- primește un obiect Ob, numele unei metode și un tablou args de obiecte;
- creează obiectul clasă ObC ce reprezintă tipul lui Ob;
- pe baza tabloului args, construiește în tabloul argstypes o semnătură și atribuie elementelor acestui tablou valorile din args;
- obține obiectul met de tipul Method corespunzător metodei ce trebuie invocată, pe baza numelui său și a semnăturii construite;
- prin "met.invoke(Ob, args); " realizează invocarea dorită.

8.9. Clasa System

Clasa System apare în pachetul java.lang:
 public final class System extends Object
 și cuprinde câmpuri și metode publice și statice utile.

Câmpurile (constantele) clasei sunt următoarele:

```

final InputStream in
final PrintStream out
final PrintStream err

```

și corespund fluxurilor de intrare, ieșire și de eroare (în care vor fi tipărite mesajele de eroare). În mod standard, in corespunde tastaturii, iar out și err corespund ecranului.

Amănunte despre InputStream și PrintStream pot fi găsite în capitolul referitor la facilitățile de intrare/ieșire.

Dintre metodele clasei menționăm următoarele:

```

void setIn(InputStream in)
    fluxul de intrare devine in;
void setOut(PrintStream out)
    fluxul de ieșire al datelor devine out;

```

```

void setErr(PrintStream out)
    fluxul de ieșire al mesajelor de eroare devine out;
long currentTimeMillis()
    întoarce timpul curent, relativ la 1 ianuarie 1970, în milisecunde;
void exit(int s)
    termină aplicația curentă.

```

Alte metode importante ale clasei sunt cele ce vizează proprietățile sistemului și gestionarul de securitate. Ne mărginim deocamdată la a semnală existența lor, dar vom reveni asupra acestor aspecte în ultimele capitole ale cărții.

CIPRIAN COMȘA
 Nr. -133-

9 FIRE DE EXECUTARE. PROGRAMARE PARALELĂ ȘI CONCURENȚĂ

Programarea obișnuită presupune existența unui procesor (dispozitiv de executare a instrucțiunilor); instrucțiunile unicului program sunt executate secvențial de către unicul procesor existent.

În viața reală asistăm în permanență la mai multe acțiuni ce se desfășoară în același timp. Unele dintre ele sunt independente, altele au un grad mai mic sau mai mare de dependență: încep în același timp, se așteaptă unele pe altele, sunt serializate etc.

Analog, calculatoarele moderne realizează, atât prin resurse software cât și hardware, desfășurarea concomitentă a mai multor acțiuni. Posibilitatea de a lansa și gestiona mai multe *fire de executare* (proces) este principala noțiune nou introdusă de sistemul de operare Windows.

Acțiunile ce se desfășoară concomitent pot fi foarte variate: programe "tradiționale", secvențe de animație, sunet etc. În acest capitol vom presupune că acțiunile sunt secvențe de instrucțiuni.

Conceptual, se presupune prezența pe sistemul gazdă a mai multor procesoare.

Pentru prelucrarea separată a fiecărei acțiuni, va fi lansat un fir de executare, care va prelua sarcina de a executa acțiunea respectivă; în cazul ideal, acela în care există un procesor liber, această sarcină este materializată de un astfel de procesor. În cazul existenței unui singur procesor, sau (mai general) atunci când numărul de acțiuni ce trebuie executate concomitent depășește numărul de procesoare libere, executarea concurentă este simulată: unitatea centrală alocă pentru fiecare fir de executare cuante de timp.

Elementele legate de programarea firelor de executare fac parte integrantă din limbajul Java, în acest mod compilatorul Java putând "vedea" aceste fire. De aceea este justificată afirmația că *Java este un mediu de executare*.

Chiar dacă aplicația nu apelează la fire de de executare, în fapt există două astfel de fire: cel curent și cel folosit de colectorul de reziduuri.

9.1. Crearea firelor de executare

• Clasa Thread

O primă modalitate de a crea și lansa în executare fire (de executare) este de a folosi clasa Thread din pachetul java.lang:

```
public class Thread extends Object implements Runnable
```

unde despre interfața Runnable vom vorbi mai jos.

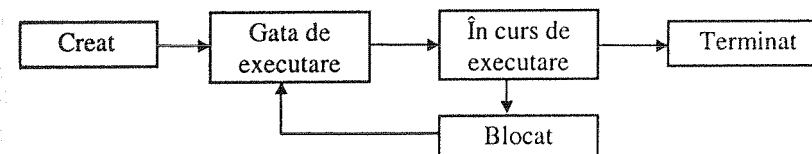
Pentru a crea un fir de executare, trebuie început prin a crea un obiect de tipul Thread:

```
Thread fir = new Thread();
```

După ce obiectul este creat, el poate fi configurat și apoi se poate trece la executarea sa. Configurarea cuprinde elemente ca de exemplu precizarea unui nume, a unei priorități etc. Când firul este gata de executare, se invocă metoda sa start fără argumente. Ca urmare firul de executare începe să execute metoda sa run.

Când metoda run se încheie, firul de executare se termină.

Prezentăm în continuare schema generală a stărilor în care se poate afla un proces (fir de executare).



Un fir (de executare) este *gata de executare* dacă îndeplinește toate condițiile pentru a se trece la executarea sa, dar nu i s-a alocat încă un procesor; când un procesor liber preia firul, acesta trece în starea *în curs de executare*. *Blocarea* unui fir de executare poate fi realizată pe baza unei condiții (de exemplu un semafor); ca urmare firul trece în starea *blocat*. La deblocarea sa, realizată prin mecanisme dintre care unele vor fi prezentate în continuare, el revine în starea gata de executare, așteptând ca un procesor să devină liber și să reia executarea sa.

Metoda run din clasa Thread nu prevede vreo acțiune. De aceea trebuie ca programatorul să extindă clasa Thread și să rescrie (suprapună) metoda run, precizând acțiunea dorită.

Se impun câteva precizări privind firele de executare.

Firul principal de executare folosește ca punct de plecare clasa principală (cea care conține metoda main) și "știe" că trebuie să execute instrucțiunile din metoda main.

Un fir de executare este lansat de un obiect ce instanțiază o clasă ce extinde clasa Thread. Fie Tip numele clasei și Ob numele obiectului. Lansarea se face prin apelul Ob.start().

Firul folosește (ca și firul principal) programul curent, cu deosebirea că acum "clasa principală" este considerată a fi instanțierea clasei Tip reprezentată prin obiectul Ob. În plus, firul de executare știe că trebuie să execute instrucțiunile din metoda "principală" run a clasei Tip.

Să presupunem că în clasa Tip mai apare și o metodă met. Trebuie clarificată diferența între invocările Ob.start() și Ob.met(...). Invocarea Ob.met(...) are următorul efect: obiectul Ob este "pus" să execute metoda met și nu se trece mai departe decât după ce această activitate a fost încheiată. În schimb, invocarea Ob.start() realizează lansarea unui fir de executare pe baza obiectului Ob; deci obiectul Ob este folosit ca model de plecare pentru noul fir de executare și nu este "pus" să execute ceva. În continuare noul fir de executare își desfășoară activitatea concurrent cu firul care l-a lansat. Obiectul Ob nu este "ocupat" cu vreo activitate, deci în continuare el poate executa acțiuni. De exemplu obiectul poate fi pus să execute o metodă sau să modifice un câmp al clasei a cărei instanțiere este. De asemenea el poate influența activitatea firului de executare lansat pe baza lui; această posibilitate este folosită frecvent pentru a controla terminarea firelor de executare.

Exemplul 1. Să considerăm următorul program:

```
class Tip extends Thread {
    int k; boolean continua=true;

    public void run() {
        while(continua) IO.write(" " + k++);
        IO.writeln("Fir");
    }

    public void terminare() { continua=false; }
}

class P1 {
    public static void main(String[] w)
        Tip Ob = new Tip(); Ob.start();
        while ( IO.readch() != '\n' );
        Ob.terminare(); IO.writeln("main");
    }
}
```

Pe baza obiectului Ob este lansat un fir de executare. Activitatea acestuia constă în a tipări, atâta timp cât variabila booleană continua are valoarea true, șirul numerelor naturale. Firul de executare principal prevede citirea repetată de caractere, până când este introdus <Enter>; în acel moment obiectul Ob execută metoda terminare, al cărei efect este modificarea valorii lui continua în false, ceea ce va determina oprirea firului de executare lansat pe baza obiectului

Ob. Firul de executare principal se termină și el. Să remarcăm că nu este previzibil dacă mesajul "Fir" va fi tipărit înainte sau după mesajul "main".

• Interfața Runnable

Interfața Runnable apare în pachetul java.lang și este declarată prin:

```
public interface Runnable
și anunță doar metoda:
public void run()
```

Utilizarea interfeței Runnable constituie o alternativă la extinderea clasei Thread. Avantajul constă în primul rând în însuși faptul că este o interfață: o clasă oarecare poate implementa Runnable și extinde o altă clasă (pe când o clasă ce extinde Thread nu mai poate extinde vreo altă clasă).

Modul tipic de utilizare este următorul:

```
class Execut implements Runnable {
    . . .
    public void run() { . . . }
    . . .
}

class Clasa {
    . . .
    Execut Ob = new Execut(...);
    Thread Fir = new Thread(Ob);    // *
    Fir.start();                    // **
    . . .
}
```

prin care este creat (*) și apoi lansat (**) firul de executare Fir. Observăm că a fost folosit constructorul cu un argument, al cărui tip este o clasă ce implementează interfața Runnable.

Exemplul 2. Pentru ilustrarea celor de mai sus, reluăm *Exemplul 1*, utilizând interfața Runnable în loc de clasa Thread.

```
class Execut implements Runnable {
    int k; boolean continua=true;
    public void run() {
        while (continua) IO.write(" " + k++);
        IO.writeln("Fir");
    }
    public void terminare() { continua = false; }
}

class RunFir {
    public static void main(String[] arg) {
        Execut Ob = new Execut();
        Thread fir = new Thread(Ob);
        fir.start();
    }
}
```

```

while ( IO.readch() != '\n');
Ob.terminare(); IO.writeln("main");
}
}

```

cu observația că metoda `terminare` a fost invocată prin intermediul obiectului `Ob` (este greșit să folosim `fir` pentru această invocare). Apare deci clară distincția între un fir de executare și obiectul pe baza căruia a fost lansat.

9.2. Un prim mod de sincronizare

Vom prezenta în acest paragraf un prim mod de sincronizare, cu largă aplicabilitate în transcrierea în Java a algoritmilor paraleli.

Să ne situăm în cazul în care un fir de executare lansează unul sau mai multe fire de executare care, prin natura problemei, trebuie să aștepte terminarea activităților acestora înainte de a întreprinde o nouă acțiune (în lipsa unei astfel de măsuri, firul principal își continuă activitatea în paralel cu firele pe care le-a lansat).

Pentru rezolvarea problemei poate fi folosită metoda `join` a clasei `Thread`. Ea are următoarele două forme:

```

public final void join() throws InterruptedException
public final void join(long mili) throws InterruptedException

```

Prima formă face ca firul de executare care a lansat la rândul său un fir de executare să aștepte terminarea acestuia un timp nelimitat, înainte de a trece la o următoarea acțiune prevăzută în program. A doua formă limitează așteptarea la un număr specificat de milisecunde.

Exemplul 3. Să considerăm următorul program:

```

class Tip extends Thread {
    static int k=1;
    public void run() {
        for (int i=0; i<500; i++) IO.write(k+" ");
    }
}

class P2 {
    public static void main (String[] sir)
        throws InterruptedException {
        Tip Ob = new Tip(); Ob.start();
        Thread.sleep(50);
        Ob.k = 2; Ob.join();
        IO.writeln("****");
    }
}

```

După ce lansează un nou fir de executare pe baza obiectului `Ob`, firul principal își întrerupe activitatea 50 milisecunde, apoi modifică în 2 valoarea câmpului `k` al clasei `Tip` și așteaptă ca acesta să își încheie activitatea, înainte de a tipări trei asteriscuri și de a își termina la rândul său executarea. Drept urmare la ieșire va apare un șir de 1, urmat de un șir de 2.

Firul principal de executare nu este cu nimic deosebit de cele pe care le lansează; în particular, aceasta înseamnă că firul principal, după ce lansează un fir, își poate încheia activitatea (se poate termina), întreaga aplicație încheindu-se atunci când firul nou lansat se termină. Afirmatia de mai sus este valabilă pentru orice fir (nu neapărat cel principal) ce lansează alte fire.

9.3. Despre programarea paralelă

Numeroase aplicații științifice necesită în zilele noastre calculatoare cu putere de calcul ce depășește cu mult un miliard de instrucțiuni pe secundă.

Performanța unui calculator se poate exprima fie prin numărul de operații în virgulă mobilă pe secundă (Mflops), fie prin numărul de instrucțiuni pe care este capabil să le execute pe secundă (Mips). Încercarea de creștere a acestor performanțe, prin proiectarea unor circuite cât mai sofisticate, s-a izbit de anumite limitări fizice: disiparea unei cantități mari de energie într-un volum redus, probleme legate de accesul la memorie etc., ca și de costul mult prea ridicat al unor astfel de echipamente.

O soluție alternativă, radical diferită, a fost aceea a utilizării unor arhitecturi paralele. Ideea de bază de la care s-a pornit a fost aceea a unui număr mare de procesoare având o putere și un cost relativ scăzute, care cooperează la rezolvarea unei probleme ce necesită un calculator cu o putere mare de calcul.

Există astăzi un număr mare de calculatoare paralele și o diversitate de arhitecturi, unele fiind încă într-un stadiu experimental sau de prototip. Aceste arhitecturi se disting mai ales prin numărul și complexitatea procesoarelor utilizate: spectrul lor variază de la un număr scăzut de procesoare de putere ridicată, cum ar fi CRAY 2, la mii de procesoare simple cum ar fi Connection Machine.

În momentul în care se pune problema proiectării unui sistem paralel trebuie ținut cont de mai multe aspecte: nivelul la care se realizează paralelismul, modul de comunicare între procesoare, topologia lor (modalitatea de interconectare). Fiecare din aceste aspecte poate constitui un criteriu de clasificare.

Precizăm în continuare modelul de calcul paralel cu care vom lucra.

Un prim aspect este cel legat de *modul în care procesoarele comunică între ele*. În continuare vom folosi modelul **P-RAM** (Parallel Random Access Machine), care presupune că toate procesoarele au acces la o memorie comună. În

acest mod nu se impun restricții ce țin de resursele fizice disponibile (hardware). Se presupune că fiecare procesor comunică direct în ambele sensuri cu memoria comună; dacă procesorul P_i dorește să transmită o valoare procesorului P_j , atunci procesorul P_i va scrie valoarea într-o locație a memoriei comune, iar P_j va citi valoarea din acea locație de memorie.

P-RAM este un model cu *memorie distribuită*: un număr de procesoare lucrează sincron și comunică prin intermediul unei memorii comune dinamice (*random access machine*). Fiecare procesor poate fi accesat în mod aleator (*uniform-cost random access machine*) și poate efectua operații și instrucțiuni uzuale.

Un al doilea aspect este legat de *activitatea procesoarelor*. Vom alege modelul **SIMD** (Single Instruction, Multiple Data Stream) care presupune că toate procesoarele efectuează aceleași operații, dar pe seturi de date diferite. Menționăm doar că un model destul de folosit este și modelul **MIMD** (Multiple Instruction, Multiple Data Stream).

Un al treilea aspect se referă la drepturile pe care le au mai multe procesoare *la scrierea și citirea concomitentă* (paralelă) în/dintr-o locație de memorie. Mai precis, trebuie specificat dacă mai multe procesoare pot citi sau nu concomitent din aceeași locație de memorie, respectiv dacă mai multe procesoare pot scrie concomitent în aceeași locație de memorie.

Vom presupune că folosim modelul **CREW** (Concurrent Read, Exclusive Write), care permite citire concomitentă din aceeași locație de memorie, dar interzice scrierea concomitentă (concurrentă) în aceeași locație de memorie.

Forma generală a unei instrucțiuni paralele este:

```
for tofi  $x \in X$  in parallel
  instrucțiune( $x$ )
enfor
```

unde X este o mulțime finită.

Semnificația acestei instrucțiuni este următoarea:

- fiecărui element x din mulțimea X i se atribuie câte un procesor;
- fiecare dintre aceste procesoare execută în paralel cu celelalte instrucțiunea menționată, bineînțeles pentru elementul $x \in X$ respectiv.

Executarea instrucțiunii paralele se termină când toate procesoarele și-au terminat propriile calcule. Această sincronizare este cea tipică programării paralele. Ea poate fi rezolvată folosind elementele din paragraful precedent. În paragraful următor vom exemplifica modul de implementare al instrucțiunii paralele pe o metodă generală de programare din calculul paralel.

Ideea conform căreia un algoritm ce rulează pe p procesoare va determina soluția de p ori mai repede este greșită, în primul rând pentru că există probleme

care sunt greu sau chiar imposibil de paralelizat, iar în interiorul unui proces vor exista instrucțiuni care se vor executa secvențial.

Timpul de calcul paralel este perioada care s-a scurs de la inițierea primului proces și momentul când toate procesele au fost terminate. El depinde nu numai de complexitatea operațiilor, ci și de complexitatea comunicării, de sincronizare, de limitele schimbului de date etc. Performanțele unui program sunt desigur influențate și de numărul de procesoare utilizate.

9.4. Implementarea în Java a metodei arborelui binar

Este vorba de o metodă generală, cu largă aplicabilitate, din programarea paralelă.

Numele ei provine de la faptul că este utilizat un *arbore binar complet*, adică un arbore binar cu $2^k - 1$ vârfuri în care:

- vârfurile sunt situate pe nivelurile $0, 1, \dots, k-1$;
- vârfurile situate pe nivelurile $0, 1, \dots, k-2$ au exact doi descendenți, iar cele de pe nivelul $k-1$ sunt vârfuri terminale (frunze);
- vârfurile de pe un nivel i oarecare sunt numerotate cu $2^i \dots 2^{i+1} - 1$; descendenții vârfului i sunt vârfurile $2i$ și $2i+1$.

Fie vectorul $a = (a_0, \dots, a_{n-1})$ și o operație asociativă $\otimes$. Se urmărește calculul valorii $a_0 \otimes a_1 \otimes \dots \otimes a_{n-1}$.

Vom presupune că n este o putere a lui 2 ($n = 2^m$). În caz contrar vom completa vectorul la dreapta la o lungime egală cu o putere a lui 2, cu elementul neutru al operației $\otimes$:

- 0 dacă $\otimes$ este operatorul de adunare;
- 1 dacă $\otimes$ este operatorul de înmulțire;
- $+\infty$ dacă $\otimes$ este operatorul de minim;
- $-\infty$ dacă $\otimes$ este operatorul de maxim;
- 0 dacă $\otimes$ este operatorul de disjuncție logică;
- 1 dacă $\otimes$ este operatorul de conjuncție logică.

Vom construi un arbore binar complet și vom atașa valori vârfurilor sale. În rădăcină va fi calculată valoarea finală cerută de problemă. Fiecărui nod intern i se atașează o valoare parțială și anume "suma" corespunzătoare (sub)arborelui de rădăcină i ; pe fiecare nivel (adâncime în arbore) calculele se execută în paralel. Metoda este de tip *bottom-up*, adică vom parcurge arborele pe niveluri, plecând de la frunze și mergând spre rădăcină.

Într-o primă etapă dublăm dimensiunea vectorului și mutăm elementele vectorului pe pozițiile $a_n, \dots, a_{2n-1}$:

```

for i=0,n-1 in parallel do
    an+i ← ai                                (1)
endfor

```

În a doua etapă parcurgem arborele de jos în sus, atribuind valori vârfurilor interne. Valoarea fiecărui vârf se calculează pe baza valorilor descendenților săi, folosind operatorul $\otimes$.

```

for k=m-1,0,-1 do
    for i=2k,2k+1-1 in parallel            (2)
        ai ← a2i  $\otimes$  a2i+1
    endfor
endfor

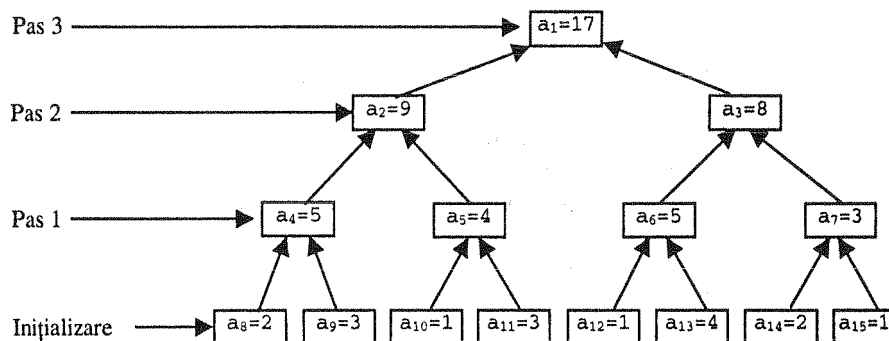
```

Să observăm că valoarea a_1 atașată unui vârf i este "suma" valorilor atașate frunzelor din subarborele de rădăcină i . Prin urmare rezultatul se obține în a_1 .

Se folosesc n procesoare. Timpul este $O(m) = O(\log n)$, deoarece ciclul **for** interior necesită timp constant și se execută de m ori.

Exemplul 4. Ilustrăm cele de mai sus pentru calculul sumei a n numere.

Considerăm vectorul $a = (3, 7, 8, 3, 9, 2, 3, 1)$ cu $n = 8 = 2^3$ și $m = 3$. Operatorul $\otimes$ va fi în acest caz operatorul de sumare. Figura de mai jos ilustrează fiecare pas al algoritmului.



Programul care urmează transcrie în limbajul Java metoda arborelui binar.

Clasa "principală" `ArbBin` conține câmpurile a (vectorul cărui i se aplică metoda), n (numărul elementelor vectorului) și m cu $n = 2^m$. Metoda principală începe cu citirea elementelor vectorului, folosind metodele clasei `IO`.

Pentru efectuarea calculului (1), adică pentru deplasarea elementelor vectorului pe pozițiile $n \dots 2n-1$, este folosită clasa `Tip1` ce extinde clasa `Thread`. Constructorul acesteia furnizează valoarea i , iar valoarea lui n este transmisă folosind câmpul static al clasei. Metoda `run` prevede executarea calculului (1).

Sunt create n obiecte de tipul `Tip1`, pe baza cărora sunt lansate n fire de executare.

Să observăm că pentru a folosi modelul P-RAM (cu memorie comună) vectorul a a fost declarat cu `static` (variabilă de clasă), astfel încât să poată fi referit din afara clasei, prefixat cu numele clasei `ArbBin` în care a fost declarat și creat. De asemenea remarcăm utilizarea metodei `join` pentru așteptarea terminării firelor de executare lansate, caracteristică programării paralele.

Pentru efectuarea prelucrării (2) se procedează în mod similar, folosind de această dată clasa `Tip2`, ce extinde și ea clasa `Thread`.

```

class Tip1 extends Thread {
    static int n; int i;
    public Tip1(int i) { this.i = i; }
    public void run() { ArbBin.a[n+i] = ArbBin.a[i]; }
}

class Tip2 extends Thread {
    int i;
    public Tip2(int i) { this.i = i; }
    public void run() {
        ArbBin.a[i] = ArbBin.a[2*i] + ArbBin.a[2*i+1];
    }
}

class ArbBin {
    static int[] a = new int[32]; static int n;
    public static void main(String[] sir) {
        int m=0, i;
        IO.write("m= "); m = (int) IO.read();
        int n=1; for (i=0; i<m; i++) n *= 2;
        IO.write("Numerele : ");
        for (i=0; i<n; i++) a[i] = (int) IO.read();
        // -----
        Tip1[] Ob1 = new Tip1[n]; Tip1.n = n;
        for (i=0; i<n; i++) Ob1[i] = new Tip1(i);
        for (i=0; i<n; i++) Ob1[i].start();
        try { for (i=0; i<n; i++) Ob1[i].join(); }
        catch (InterruptedException e) { }
        // -----
        Tip2[] Ob2 = new Tip2[n];
        int p = n/2, u = n;
        for (int k=m-1; k>=0; k--) {
            for (i=p; i<u; i++) Ob2[i] = new Tip2(i);
            for (i=p; i<u; i++) Ob2[i].start();
            try { for (i=p; i<u; i++) Ob2[i].join(); }
            catch (InterruptedException e) { }
            for (i=p; i<u; i++) IO.write(a[i] + " "); IO.writeln();
            p = p/2; u = u/2;
        }
        IO.writeln("Suma = " + a[1]);
    }
}

```


Încheiem acest subcapitol cu câteva precizări asupra clasei publice Thread din pachetul java.lang și a unor metode (tot publice) ale sale:

```
class Thread extends Object implements Runnable
    Thread()
    Thread(String s)
        constructor ce atașează numele s firului de executare creat; prin invocarea
        constructorului Thread fără parametri, firul generat va primi automat un
        nume de forma "Thread-n", unde n este un întreg
    Thread(Runnable r)
    Thread(Runnable r, String s)
    final void join() throws InterruptedException
    final void join(long millis) throws InterruptedException
    void start()
    void run()
    static void sleep(long millis)
```

9.5. Despre programarea concurrentă

Am văzut că *programarea paralelă* pune accentul pe acțiunile ce trebuie executate de către fiecare procesor. Scopul programării paralele este de a elabora algoritmi care să speculeze existența mai multor procesoare pentru a micșora timpul de executare. Problemele sunt cele obișnuite din programarea secvențială, dar metodele de elaborare a programelor paralele vor fi evident diferite de cele din programarea secvențială.

Aproape întotdeauna, executarea concurrentă a mai multor procese (fire de executare) cere o anumită *cooperare* între ele, deci sunt necesare anumite mecanisme care să asigure comunicarea și sincronizarea între procese. Specificarea cooperării între procese în termenii mecanismelor disponibile, deci scrierea unor programe concurente, constituie obiectul *programării concurente*. Este nevoie de un alt mod de gândire, de alte "unelte", de altă înțelegere a noțiunii de program corect.

Esența programării concurente nu depinde de numărul procesoarelor fizice disponibile. Putem considera în permanență că dispunem de suficiente procesoare fizice pentru a atașa câte unul fiecăruia dintre procesele concurente. De asemenea putem considera că avem la dispoziție un singur procesor, care "alocă" proceselor intervale de timp aleatoare. Mai general, putem considera că este folosit *modelul aleator*, care constă din repetarea ciclică a următoarelor acțiuni de către fiecare procesor fizic:

- 1) alege *aleator* unul dintre procesele cărora nu îi este asociat un procesor fizic;
- 2) execută, un interval de timp *aleator*, instrucțiuni ale sale.

Nu se va face nici o presupunere asupra vitezei de lucru a procesoarelor. Fiecare procesor trebuie gândit ca având un ceas logic propriu. Cu alte cuvinte, orice supoziții asupra raportului vitezelor a două procesoare, ca și asupra ritmității executării instrucțiunilor în cod mașină din cadrul aceluiași proces, sunt neavenite (nu ne ocupăm aici de sistemele concurente în timp real).

În conformitate cu cele de mai sus, trebuie subliniat că programarea concurrentă rezervă o serie de surprize informaticianului obișnuit cu programarea secvențială. Prezentăm în continuare câteva dintre ele.

- în programarea secvențială, următoarea secvență de instrucțiuni:

```
i=1; i=2;
```

este echivalentă cu a doua instrucțiune și prezența ei într-un program trezește îndoiele asupra profesionalismului autorului programului. În programarea concurrentă secvența de mai sus nu are nimic redundant. Într-adevăr, în intervalul de timp dintre executarea celor două instrucțiuni este posibil ca celelalte procese să execute diferite instrucțiuni care să folosească efectiv faptul că pe parcursul unui interval de timp (asupra lungimii căruia nu putem face nici o ipoteză) valoarea lui *i* este egală cu 1.

- efectul instrucțiunii:

```
i=1; if (i==1) j=j+i;
```

nu constă neapărat în incrementarea cu o unitate a lui *j*. Pe de o parte condiția din *if* nu este neapărat verificată, deoarece (între momentul efectuării atribuirii și momentul efectuării comparației) valoarea lui *i* poate fi eventual modificată de un alt proces. Pe de altă parte, chiar presupunând că valoarea lui *i* este 1 în momentul comparației, ea se poate schimba înainte de efectuarea adunării. În schimb afirmația "valoarea lui *i* a fost egală cu 1 la un moment de timp anterior" este adevărată și poate fi utilă (de exemplu în a demonstra că procesul are o anumită evoluție).

Este ușor de observat că aceste comportări "anormale" din punctul de vedere al programării secvențiale își au cauza în utilizarea de către mai multe procese a unei memorii comune.

Programarea concurrentă se mai deosebește de programarea secvențială și prin faptul că programele pot avea *comportare nedeterministă*: "la executări diferite pentru același set de date de intrare, rezultatele nu vor fi neapărat aceleași".

Astfel, dacă unul dintre procese execută instrucțiunile:

```
i=1; i=i+3;
```

iar un altul execută instrucțiunile:

```
i=5; i=i-2;
```

atunci printre valorile ce pot rezulta în final pentru *i* se numără de exemplu 3, 4, 6.

De aceea trebuie studiate toate scenariile posibile pentru a avea siguranța că vom obține rezultatul dorit.

Programarea concurrentă este menită să rezolve interacțiunile dintre procese. Atenția principală este îndreptată deci asupra modalităților în care pot fi realizate interacțiunile dorite și mai puțin asupra a ceea ce realizează programele secvențiale ce constituie fiecare proces în parte. Natura problemelor este diferită

de cea întâlnită în programarea secvențială. Sunt tratate în special *probleme de simulare* a unor situații concurente reale și nu probleme care urmăresc obținerea unui rezultat numeric.

Iată două exemple tipice de probleme întâlnite în programarea concurentă:

1) **Problema rezervării biletelor:** Fiecare terminal al unei rețele de calculatoare este plasat într-un punct de vânzare a biletelor pentru un anumit spectacol. Se cere să se găsească o modalitate de a evita vânzarea a două bilete pentru același loc.

2) **Problema Producător - Consumator:** Producătorul fabrică articole de un anumit tip și le plasează pe o bandă. Consumatorul ia câte un obiect de pe bandă și îl "consumă". Se cere să se simuleze aceste activități, evitându-se încercările producătorului de a plasa un articol pe bandă când aceasta este plină, precum și încercările consumatorului de a lua un obiect de pe bandă când aceasta este goală (dificultatea constă deci în realizarea unei sincronizări).

Vom numi *secțiune critică* o secvență de instrucțiuni a căror executare trebuie să se supună următoarei reguli: în momentul în care un proces P începe executarea primei instrucțiuni din secvență, nici un alt proces nu poate să execute instrucțiuni din secvență înainte de terminarea executării ultimei instrucțiuni din secvență de către procesul P . În aceste condiții spunem că are loc o *excludere reciprocă* (cel mult un proces se poate afla într-o secțiune critică). Importanța și rolul secțiunilor critice vor fi puse în evidență pe un nou exemplu.

Problema grădinilor ornamentale (atribuirii multiple). Intrarea în grădinile ornamentale ale unui oraș oriental se face prin mai multe porți. Se pune problema finerii evidenței numărului total de persoane care au intrat în grădini.

O încercare de rezolvare este următoarea:

- este introdusă variabila globală `total`, a cărei valoare curentă reprezintă numărul persoanelor ce au intrat în grădini;
- fiecărei porți i se asociază în mod natural un proces, ce prevede ca în mod repetat valoarea variabilei `total` să fie mărită cu o unitate, corespunzător intrării unei persoane pe poarta respectivă.

Încercarea de mai sus nu este corectă: în final `total` poate avea o valoare diferită de cea așteptată; mai mult, la executări diferite ale programului ce transpune această încercare de rezolvare, se pot obține rezultate diferite. Explicația constă în primul rând în faptul că instrucțiunea `total=total+1` nu este nedivizibilă (nu este o acțiune "atomică"), iar în al doilea rând în modul de executare al programelor concurente. Într-adevăr, instrucțiunea `total=total+1` este în realitate o secvență formată din trei instrucțiuni în cod mașină:

```
LOAD total, r      { încarcă total în registrul r al CPU }
INC r              { incrementează valoarea lui r cu 1 }
STORE r, total     { salvează valoarea registrului în total }
```

Pentru exemplificare, să considerăm cazul a două porți (procese) $P1$ și $P2$. Fie $r1$ și $r2$ registrele unității centrale folosite de cele două procese. Atunci în cazul în care pe fiecare poartă intră trei persoane, este posibilă (printre multe altele) următoarea succesiune de acțiuni (următorul scenariu):

Acțiune	total	r1	r2
	0		
P1 încarcă		0	
P1 incrementează		1	
P2 încarcă			0
P2 incrementează			1
P2 salvează	1		
P2 încarcă			1
P2 incrementează			2
P2 salvează	2		
P1 salvează	1		

urmată de o succesiune analoagă în care se interschimbă $P1$ cu $P2$. Rezultatul final va fi 2, mai mic chiar decât numărul persoanelor ce intră prin fiecare poartă!

Să observăm că dacă instrucțiunea `total=total+1` (mai corect spus cele trei instrucțiuni mașină în care se descompune) ar forma o secțiune critică, rezultatul ar fi totdeauna cel corect.

9.6. Rezolvarea excluderii reciproce în Java

Vom transcrie mai întâi în Java încercarea de rezolvare de mai sus. Lansăm n fire de executare care prevăd fiecare mărirea de un număr dat de ori a unei variabile comune `total`. Pentru a simula modul în care se realizează efectiv incrementarea lui `total` cu o unitate (este utilizat un registru), este folosită variabila suplimentară `temp`. În plus, pentru a accentua caracterul nedeterminist, este folosită metoda `delay` cu zero argumente ce amână cu (cel puțin) un număr de milisecunde generat aleator, executarea următoarei instrucțiuni; în acest scop este folosită metoda statică `random` din pachetul `java.lang.Math`.

Exemplul 5.

```
class Tip extends Thread {
    static int total; int i;
    Tip(int i) { this.i=i; }
```

```

void delay() {
    try { sleep( (int) (100 * Math.random()) ); }
    catch (InterruptedException e) { }
}

public void run() {
    int temp;
    for (int j=1; j<=10; j++) {
        IO.write(" " + i); delay(); temp=temp;
        delay(); temp++; delay(); total=temp;
    }
}

class Grad_Rau {
    public static void main(String[] sir) {
        Tip[] Ob = new Tip[5];
        for (int i=0; i<5; i++) Ob[i] = new Tip(i);
        for (int i=0; i<5; i++) Ob[i].start();
        try { for (int i=4; i>=0; i--) Ob[i].join(); }
        catch (InterruptedException e) { }
        IO.println(); IO.println("Total = " + Tip.total);
    }
}

```

Executarea programului de mai sus face ca la ieșire să apară numai în mod excepțional valoarea 50 preconizată. Mai mult, valoarea tipărită la ieșire poate diferi de la o executare la alta. Explicația constă, așa cum am remarcat și mai sus, în următoarele:

- am folosit variabila comună (ce poate fi accesată simultan de mai multe fire de executare) total;
- operațiile prevăzute de metoda run a clasei Tip nu sunt atomice.

Pentru a realiza *excluderea reciprocă* asupra operației de incrementare a variabilei comune total, vom folosi facilitatea din limbajul Java de a declara o metodă cu modificatorul synchronized.

Regula este următoarea: *dacă mai multe fire de executare apelează metode declarate cu modificatorul synchronized prin intermediul aceluiași obiect* Ob având ca tip clasa C ce conține metodele respective, atunci la fiecare moment de timp cel mult un fir de executare este în curs de a executa o astfel de metodă. Dacă un fir de executare este în curs de a executa o metodă sincronizată, el o va executa până la capăt; orice alt fir de executare ce invocă o astfel de metodă va fi blocat. Când un fir termină de executat o metodă sincronizată, atunci unul dintre firele de executare blocate va fi deblocat și va începe efectiv invocarea metodei dorite.

Este folosită următoarea terminologie:

- obiectul Ob se numește *monitor*;
- spunem că monitorul este *liber* dacă nu este în curs de executare nici o metodă sincronizată a sa; în caz contrar, adică dacă (exact) o metodă sincronizată a sa este în curs de executare, spunem că monitorul este *ocupat*;

- dacă un fir de executare invocă, prin intermediul monitorului Ob, o metodă sincronizată a clasei C și monitorul este ocupat, firul va fi blocat în *mulțimea (de fire blocate) de așteptare asociată monitorului*.

- dacă un fir este în curs de a executa, prin intermediul obiectului Ob, o metodă sincronizată a clasei C, spunem că el *deține controlul (exclusiv) asupra monitorului*;

- când firul de mai sus termină de executat metoda sincronizată, *el pierde controlul asupra monitorului*. Dacă mulțimea asociată monitorului este nevidă, se alege "la întâmplare" (în funcție de implementarea concretă) un fir din această mulțime și acesta trece la executarea metodei pe care acesta a încercat (fără succes) să o invoce. Acum monitorul este din nou ocupat și noul fir este cel care deține controlul asupra monitorului.

Atragem atenția că definiția de mai sus a monitorului este valabilă doar pentru Java. Alte definiții întâlnite în literatura de specialitate presupun existența unor anumite operații asupra monitoarelor, impun o disciplină de coadă pentru mulțimea de așteptare asociată monitorului etc.

Mai menționăm faptul că Java permite nu numai sincronizarea unei întregi metode, dar și sincronizarea doar a unui bloc (delimitând astfel mai strict porțiunea "critică" din cod, în scopul creșterii vitezei programului). Nu ne vom ocupa de acest aspect, mai ales că mecanismele de bază nu diferă.

Exemplul 6. În programul care urmează este prezentată o soluție corectă a problemei grădinilor ornamentale. Firele de executare invocă metoda numara, declarată cu synchronized, a clasei Contor prin intermediul aceluiasi obiect (monitor) C de tipul clasei Contor.

```

class Contor {
    static int total;
    synchronized void numara() {
        int temp;
        temp=temp; temp++; total=temp;
    }
}

class Tip extends Thread {
    int i; static Contor C;
    Tip(int i) { this.i=i; }

    public void run() {
        for (int j=0; j<10; j++) {
            C.numara(); IO.write(" " + i);
        }
    }
}

class Grad_Bun {
    public static void main(String[] w) {
        Tip.C = new Contor();
    }
}

```

```

    Tip[] Ob = new Tip[5];
    for (int i=0; i<5; i++) Ob[i] = new Tip(i);
    for (int i=0; i<5; i++) Ob[i].start();
    try { for (int i=4; i>=0; i--) Ob[i].join(); }
    catch (InterruptedException e) { }
    IO.writeln(); IO.writeln("Total = " + C.total);
}
}

```

9.7. Primitive de sincronizare.

Problema Producător - Consumator

Principalele primitive (metode) de sincronizare puse la dispoziție de Java sunt următoarele metode publice ale clasei rădăcină `Object`:

```

final void wait()
final void wait(long t)
final void notify()
final void notifyAll()

```

Înainte de a trece la prezentarea lor, specificăm că ele:

- pot lansa excepția `IllegalMonitorStateException` dacă firul curent nu deține controlul asupra monitorului reprezentat de obiectul curent;
- trebuie să fie invocate din interiorul unei metode sincronizate sau a unui bloc sincronizat.

• Metodele `wait`

Fie `F` firul de executare din care este invocată una dintre aceste metode și fie `Ob` obiectul (monitorul) curent. Executarea metodei suspendă (blochează) firul `F`, adăugându-l mulțimii atașate monitorului `Ob`; în același timp monitorul este "eliberat", fiind gata să ofere controlul asupra sa unuia dintre (eventualele) fire din mulțimea sa de așteptare.

Firul `F` rămâne în mulțimea de așteptare a monitorului până când este îndeplinită una dintre următoarele condiții:

- un alt fir de executare invocă metoda `notify` prin intermediul obiectului `Ob` și firul `F` este cel ales pentru a prelua controlul asupra monitorului;
- un alt fir de executare invocă metoda `notifyAll` prin intermediul obiectului `Ob`;
- numai pentru metoda `wait` cu un parametru: a expirat timpul de așteptare `t` măsurat în milisecunde și specificat ca argument.

În continuare firul `F` așteaptă ca monitorul să devină liber (invocarea metodelor `notify` și `notifyAll` nu eliberează monitorul), după care el revine în starea dinaintea invocării lui `wait`; cu alte cuvinte, doar în acest moment se încheie executarea metodei `wait`.

• Metodele `notify` și `notifyAll`

Metoda `notify` "trezește" unul dintre eventualele fire din mulțimea de așteptare asociată monitorului care a invocat metoda. Acesta devine candidat la a prelua controlul asupra monitorului conform mecanismului descris mai sus.

Metoda `notifyAll` diferă de `notify` prin aceea că "trezește" toate eventualele fire din mulțimea de așteptare asociată monitorului care a invocat metoda.

• Problema Producător - Consumator

Reamintim enunțul problemei: *Un producător și un consumator își desfășoară în același timp activitatea, folosind în comun o bandă de lungime lung. Producătorul produce câte un obiect și îl plasează la un capăt al benzii. Consumatorul ia câte un obiect de la celălalt capăt al benzii și îl consumă.*

Dificultatea problemei constă în faptul că producătorul și consumatorul pun/iau obiecte pe/de pe bandă în ritmuri imprevizibile, ceea ce poate conduce la următoarele situații limită:

- producătorul încearcă să pună un obiect în banda plină;
- consumatorul încearcă să ia un obiect de pe banda vidă.

Exemplul 7. Prezentăm programul, urmat de explicații.

```

import java.util.*;
class Time extends Random {
    int delay() { return (int) (100.0f * nextFloat()); }
}

class Banda {
    int lung=3, n, p, u=lung-1;
    char[] coada = new char[lung];
    static Time ObT = new Time();

    synchronized char ia() {
        char ch;
        while (n == 0) {
            try { wait(); }
            catch (InterruptedException e) {}
        }
        ch = coada[p]; p = (p+1)%lung; n--;
        IO.write(" I" + ch); notify(); return ch;
    }

    synchronized void pune(char ch) {
        while (n == lung) {
            try { wait(); }
            catch (InterruptedException e) {}
        }
    }
}

```

```

        u = (u+1)%lung; coada[u] = ch; n++;
        IO.write(" P"+ch); notify();
    }
}

class Prod implements Runnable {
    Banda b;
    Prod(Banda bb) { b = bb; }

    public void run() {
        char ch;
        for (ch='a'; ch<='z'; ch++) {
            b.pune(ch);
            try { Thread.sleep(Banda.OBT.delay()); }
            catch (InterruptedException e) {}
        }
    }
}

class Cons implements Runnable {
    Banda b;
    Cons(Banda bb) { b = bb; }

    public void run() {
        char ch;
        do {
            ch = b.ia();
            try { Thread.sleep(Banda.OBT.delay()); }
            catch (InterruptedException e) {}
        } while (ch != 'z');
    }
}

class PC {
    public static void main(String[] w) {
        Banda b = new Banda();
        Prod P = new Prod(b); Cons C = new Cons(b);
        Thread FirP = new Thread(P); FirP.start();
        Thread FirC = new Thread(C); FirC.start();
        try { FirP.join(); FirC.join(); }
        catch (InterruptedException e) {}
    }
}

```

Clasa Time furnizează un alt mod de accentuare a nedeterminismului: metoda delay întoarce un număr întreg cuprins între 0 și 100. Această modalitate va fi folosită pentru a întârzia un timp aleator operațiile de punere și luare pe/de pe bandă. În metoda delay a fost folosită funcția nextFloat a clasei Random din pachetul java.util, funcție ce întoarce o valoare de tip float din intervalul [0,1).

Clasa Banda ține evidența cozii corespunzătoare benzii: vectorul coada are lungimea lung, primul său element este pe poziția p, ultimul pe poziția u, iar numărul de elemente de pe bandă este n. Metoda ia realizează extragerea unui element de pe bandă, iar metoda pune corespunde introducerii unui nou element pe bandă; elementele sunt caractere. Metodele ia și pune sunt declarate cu modificatorul synchronized.

Obiectul P de tipul Prod, clasă ce implementează interfața Runnable, lansează un fir de executare corespunzător producătorului; sunt produse literele mici ale alfabetului și se încearcă plasarea lor pe bandă prin invocarea metodei pune.

Obiectul C de tipul Cons, clasă ce implementează și ea interfața Runnable, lansează un fir de executare corespunzător consumatorului; prin invocarea repetată a metodei ia se încearcă preluarea unui caracter de pe bandă, până când s-a obținut caracterul 'z'.

Cele două fire de executare folosesc același obiect b de tipul Banda pentru invocarea metodelor ia și pune.

Sincronizarea este asigurată prin folosirea metodelor wait și notify.

Observație. Clișeu standard de folosire a metodei wait este următorul:

```
while ( !condiție ) wait();
```

și a fost folosit ca atare în programul de mai sus deși, prin natura problemei concrete, puteam înlocui while cu if. Motivul este că, în general, deblocarea nu garantează îndeplinirea condiției.

9.8. Lucrul cu semafoare în Java

• Noțiunea de semafor

În anul 1968, Dijkstra introduce noțiunea de *semafor* ca modalitate de rezolvare a problemelor programării concurente.

Semaforul este un tip de date, caracterizat deci prin valori posibile și operații permise.

O variabilă de tip semafor poate lua ca valori numai numere naturale.

Operațiile permise în lucrul cu semafoare sunt prezentate în continuare, unde s este o variabilă de tip semafor.

1) inițializarea valorii lui s cu un număr natural;

2) wait(s) : if s>0 then s ← s-1
 else procesul curent este blocat

3) signal(s) : if există procese blocate
 then este deblocat unul dintre ele
 else s ← s+1

În definiția semafoarelor este inclus faptul că operațiile de mai sus, deși includ mai multe acțiuni, sunt *atomice* (mai spunem că este vorba de *primitive*). Drept urmare pentru orice variabilă *s* de tip semafor, operațiile asupra ei (efectuate de diferite procese ce se execută concurrent) se execută sub excludere reciprocă.

Cele trei operații de mai sus sunt *singurele* permise în lucrul cu semafoare; prin urmare o variabilă de tip semafor nu poate fi folosită în calcule aritmetice, comparații etc.

În continuare vom folosi, prin abuz de limbaj, termenul de semafor și pentru o variabilă de tip semafor.

Prezentăm acum un exemplu, menit să ofere o imagine plastică a operațiilor asupra unui semafor oarecare *s*.

Considerăm o parcare pentru mașini, prevăzută cu o intrare și o ieșire. Fie *s* numărul curent de locuri libere din parcare. Parcarea este gestionată de un paznic și orice (încercare de) intrare și ieșire în/din parcare se face sub controlul său; aceasta asigură atomicitatea operațiilor de intrare/ieșire. Există două moduri în care paznicul discută cu șoferii, corespunzătoare operațiilor *wait* și *signal*:

- *wait(s)* : Un șofer anunță paznicul că dorește să intre în parcare. Dacă există locuri libere, mașina va ocupa unul dintre ele și ca urmare valoarea lui *s* se micșorează cu o unitate. În caz contrar, mașina este blocată la intrare.

- *signal(s)* : Un șofer anunță paznicul că vrea să iasă din parcare. Dacă există unul sau mai mulți șoferi blocați la intrare, unul dintre ei este deblocat și atunci concomitent cu ieșirea mașinii considerate, o altă mașină intră în parcare; drept urmare valoarea lui *s* rămâne neschimbată. În caz contrar, ieșirea mașinii considerate face ca numărul de locuri libere din parcare să crească cu o unitate.

Revenind la problema grădinilor ornamentale, să observăm că asigurarea excluderii reciproce asupra instrucțiunii neatomice *total=total+1* poate fi rezolvată simplu cu ajutorul semafoarelor astfel:

- se introduce un semafor *s*, inițializat cu 1;

- instrucțiunea amintită este plasată între "parantezele" *wait* și *signal*:

```
wait(s);
    total = total+1;
signal(s);
```

Nu vom da o demonstrație matematică a acestei rezolvări, ci ne vom mărgini la a face analogia cu o parcare cu un singur loc, în care mașina care intră în parcare "execută" instrucțiunea menționată.

Evident, acest mecanism simplu se poate extinde imediat la orice secvență de instrucțiuni ce se referă la o memorie comună și care trebuie executată prin excludere reciprocă.

• Implementarea semafoarelor în Java. Aplicații

Semafoarele pot fi implementate în Java folosind următoarea clasă:

```
class Semafor {
    private int value = 0;

    public Semafor(int initial) { value = initial; }

    public synchronized void W() {
        value--;
        if (value < 0)
            try { wait(); }
            catch (InterruptedException e) { }
    }

    public synchronized void S() {
        value++;
        if (value <= 0) notify();
    }
}
```

Observații:

- constructorul clasei realizează, odată cu crearea unui obiect, și inițializarea valorii *value* a semaforului;

- metodele *W* și *S* corespund primitivelor *wait* și *signal* descrise mai sus, ce operează asupra semafoarelor;

- fie *sem* un semafor, deci un obiect de tipul *Semafor*. Atunci operațiile *wait(sem)* și *signal(sem)* din modelul general propus de Dijkstra vor trebui înlocuite respectiv prin *sem.W()* și *sem.S()*.

Nu vom da o demonstrație detaliată a funcționării corecte a celor două metode ale clasei *Semafor*, ci ne vom mărgini la următoarele două precizări:

1) Există procese blocate dacă și numai dacă valoarea lui *value* este negativă. În acest caz, numărul proceselor blocate este *-value*;

2) prin *notify()* firul curent își continuă executarea. Firul trezit (dacă există) devine gata de executare, dar datorită modificatorului *synchronized* nu poate fi preluat de către un procesor decât când firul ce a executat *notify()* părăsește metoda sincronizată.

Exemplul 8. Rezolvarea cu semafoare a problemei grădinilor ornamentale.

Prezentăm direct programul în Java:

```
class Tip extends Thread {
    static int total; int i;
    static Semafor ExRec = new Semafor(1);
    Tip(int ii) { i = ii; }

    public void run() {
        int temp;
        for (int j=0; j<10; j++) {
            try { sleep( (int) (100 * Math.random()) ); }
            catch (InterruptedException e) { }
        }
    }
}
```

```

        ExRec.W();
        IO.write(i+"\t");
        temp = total; temp++; total = temp;
        ExRec.S();
    }
}

class Grad_Sem {
    public static void main(String[] string) {
        Tip[] Ob = new Tip[5];
        for (int i=0; i<5; i++) Ob[i] = new Tip(i);
        for (int i=0; i<5; i++) Ob[i].start();
        try { for (int i=0; i<5; i++) Ob[i].join(); }
        catch (InterruptedException e) { }
        IO.writeln("\nTotal= " + Tip.total);
    }
}

```

Exemplul 9. Problema filozofilor chinezi.

La o masă rotundă stau 5 filozofi chinezi. Principala lor activitate este cea de a gândi, dar evident din când în când trebuie să mănânce, folosind pentru aceasta câte două bețișoare. Știind că între oricare doi filozofi se află un bețișor și că un filozof poate mânca doar dacă a ridicat de pe masă atât bețișorul din stânga sa, cât și bețișorul din dreapta sa, se cere să se simuleze activitățile filozofilor așezați la masă.

Se observă că singura restricție este aceea ca la nici un moment de timp să nu mănânce doi filozofi vecini (ar trebui ca ambii să folosească un același bețișor).

Vom presupune că la stânga filozofului i se află bețișorul i , iar la dreapta sa bețișorul $i+1$ (la dreapta filozofului $nf=5$ se va afla bețișorul cu numărul 1).

Fiecărui filozof îi asociem un proces (fir de executare), iar fiecărui bețișor i îi asociem un semafor b_i ; încercarea de ridicare a unui bețișor corespunde interogării semaforului asociat bețișorului.

O primă variantă de rezolvare a problemei propune ca încercarea de ridicare a unui bețișor să fie concretizată printr-o operație wait asupra semaforului corespunzător, iar repunerea bețișorului pe masă să fie materializată printr-o operație signal asupra semaforului corespunzător. Aceasta revine la următoarele acțiuni pentru filozoful cu numărul i :

```

repeat
{ filozoful  $i$  gândește }
wait( $b[i]$ );
wait( $b[(i \bmod nf)+1]$ );
{ filozoful  $i$  mănâncă }
signal( $b[i]$ );
signal( $b[(i \bmod nf)+1]$ );
until false

```

Această variantă nu este acceptabilă, deoarece poate conduce la blocare globală (deadlock), în situația în care toți filozofii ridică simultan bețișorul aflat la stânga lor: în continuare nici un filozof nu va putea ridica bețișorul din dreapta sa și întreg sistemul se blochează.

O modalitate de a evita blocarea globală este aceea de a considera că problema se rezolvă ca între adevărați filozofi: în permanență va exista cel puțin un filozof altruist, care renunță *momentan* să mănânce (ceea ce este echivalent cu a considera că scaunul său este liber); de aici va rezulta că cel puțin un filozof va avea posibilitatea de a ridica bețișoarele aflate în stânga și dreapta sa, deci va putea să mănânce.

Pentru a modela această soluție, vom utiliza un semafor ScaunLiber, care va memora în permanență numărul de filozofi care renunță *momentan* să mănânce. Inițializarea sa se poate face cu orice valoare întreagă din intervalul $[1, nf-1]$, dar cu cât alegem o valoare mai mare, cu atât gradul de concurență va fi mai ridicat.

Să observăm că nu interesează care sunt filozofii altruști, ci doar care este numărul lor. De asemenea este evident că "parantezele" wait(ScaunLiber) și signal(ScaunLiber) trebuie plasate înainte de încercarea de ridicare a primului bețișor și respectiv după semnalarea repunerii pe masă a ultimului dintre cele două bețișoare pe care filozoful le-a folosit pentru a mânca. Într-adevăr, cât timp filozoful gândește, nu are importanță dacă există sau nu filozofi altruști.

Prezentăm în continuare programul corespunzător în Java. Vom introduce în program și posibilitatea de a urmări evoluția sa. Pentru aceasta vom insera scrierea unui mesaj care să indice începutul acțiunii de a mânca (litera M urmată de numărul de ordine al filozofului), precum și scrierea unui mesaj care să indice începutul acțiunii de a gândi (litera G urmată de numărul de ordine al filozofului).

```

class Comun {
    static Semafor[] b;    static Semafor ScaunLiber;
    static int n;
    Comun(int n) {
        this.n = n;
        ScaunLiber = new Semafor(n-1);
        b = new Semafor[n];
        for (int i=0; i<n; i++) b[i] = new Semafor(1);
    }
}

class Filozof extends Thread {
    int i;
    Filozof(int ii) { i = ii; }

    public void run() {
        for (int j=0; j<10; j++) {
            try { sleep( (int) (100 * Math.random()) ); }
            catch(InterruptedException e) { }
            Comun.ScaunLiber.W();

```

```

        Comun.b[i].W(); Comun.b[(i+1)%Comun.n].W();
        IO.write("  M"+i);
        try { sleep( (int) (100 * Math.random()) ); }
        catch(InterruptedException e) { }
        IO.write("  G"+i);
        Comun.b[i].S(); Comun.b[(i+1)%Comun.n].S();
        Comun.ScaunLiber.S();
    }
}

class FilSem {
    static int i;
    public static void main(String[] str) {
        IO.write("n = "); int n = (int) IO.read();
        new Comun(n);
        Filozof[] F = new Filozof[n];
        for (i=0; i<n; i++) F[i] = new Filozof(i);
        for (i=0; i<n; i++) F[i].start();
    }
}

```

9.9. Alte facilități oferite de clasa Thread

Toate câmpurile și metodele prezentate în continuare sunt publice.

```

final String getName() : întoarce numele firului curent;
final void setName() : modifică numele firului curent;
static void yield()
    firul curent este temporar întrerupt, pentru a da și altor fire șansa de a trece din
    starea "gata de executare" în starea "în curs de executare";
Thread currentThread() : întoarce firul curent;
final boolean isAlive()
    întoarce true dacă firul curent a pornit, dar încă nu s-a terminat.

```

O altă facilitate constă în posibilitatea de a asocia *priorități* firelor de executare. Sunt folosite următoarele constante declarate cu static final int: MIN_PRIORITY NORM_PRIORITY MAX_PRIORITY a căror valoare depinde de sistemul gazdă. Prioritățile ce pot fi atașate trebuie să fie cuprinse între MIN_PRIORITY și MAX_PRIORITY inclusiv; prioritatea asociată implicit este NORM_PRIORITY.

De asemenea sunt prevăzute metodele publice:

```

final void setPriority(int p) : stabilește prioritatea firului curent;
final int getPriority() : întoarce prioritatea firului curent.

```

În *principiu*, în momentul în care un fir de executare F cu prioritate mai mare decât cel curent intră în starea "gata de executare", firul curent este întrerupt și firul F intră în starea "în curs de executare". Revenind la metoda `yield`, invocarea ei face

ca firul curent să fie temporar întrerupt, iar controlul să fie cedat altui fir de aceeași prioritate. Rularea, de un număr "suficient" de ori, a următorului exemplu ne convinge că lucrurile se petrec astfel numai în principiu.

Exemplul 10.

```

class Tip extends Thread {
    int i;
    Tip(int i) { this.i = i; }
    public void run() {
        for (int j=0; j<40; j++) { IO.write(i+" "); yield(); }
    }
}

class yields {
    public static void main(String[] sir) {
        Tip[] S = new Tip[4], T = new Tip[4];
        for (int i=0; i<4; i++) {
            S[i] = new Tip(i); S[i].setPriority(8);
            T[i] = new Tip(i+5); T[i].setPriority(4);
        }
        for (int i=0; i<4; i++) T[i].start();
        for (int i=0; i<4; i++) S[i].start();
    }
}

```

În afară de firele de executare create studiate până acum (numite și *fire utilizator*), mai există și o altă categorie de fire, numite *demoni* (fire demon).

Metodele din clasa Thread pentru lucrul cu demoni sunt următoarele:

```

final void setDaemon(boolean b)
    marchează firul curent ca fiind un fir demon sau un fir utilizator. Această
    metodă trebuie invocată înainte de a porni, prin invocarea metodei start, firul
    respectiv;
final boolean isDaemon()
    verifică dacă firul curent este sau nu un demon.

```

Firele demon sunt de obicei create pentru a aduce diferite servicii celorlalte fire din aplicație; un exemplu îl constituie collectorul de reziduuri. Firele demon au prioritate mai mică decât orice fir utilizator. Discuția de mai sus asupra cedării *în principiu* a dreptului la executare firului cu prioritate mai mare arată că totuși și firele demon au o șansă de a întreprinde activități.

Precizare. O aplicație care lansează fire de executare se încheie fie dacă a fost invocată metoda `exit` a clasei `System`, fie dacă toate firele de executare care nu sunt demoni s-au terminat.

Cu titlu informativ, menționăm că există și posibilitatea de a forma din mai multe fire de executare un *grup de fire*, în acest mod luând naștere o structură arborescentă (orice fir poate crea alte fire, pe care le poate grupa sau nu). Pentru gruparea firelor este prevăzută clasa `ThreadGroup` din pachetul `java.lang`.

10 FACILITĂȚI DE INTRARE/IEȘIRE

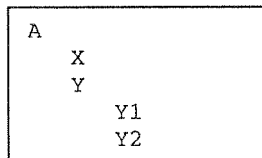
10.1. Schema generală. Fluxuri de intrare/ieșire

Începem prin a preciza că toate clasele ce oferă facilități de intrare/ieșire se găsesc în pachetul `java.io`.

Numărul acestor clase și al metodelor lor este foarte mare, ceea ce poate lăsa la început impresia că totul este stufos și complicat. În realitate, dacă este bine înțeleasă schema generală folosită în lucrul cu fluxurile de intrare/ieșire, lucrurile se simplifică în mod considerabil.

Tehnica principală folosită este *suprapunerea de obiecte*, care aplicată la fluxuri de intrare/ieșire se numește *suprapunere de fluxuri*.

Exemplul 1. Să considerăm următoarea ierarhie de clase:



Clasa A este o clasă abstractă, iar metoda `met` este anunțată în A și redefinită în subclasele sale. Clasele Y, Y1 Y2 au un câmp de tipul A, ce primește valoare prin constructorii claselor. În clasele Y1 și Y2 apar în plus metodele `met1`, respectiv `met2`.

```

abstract class A {
    abstract void met();
}

class X extends A {
    void met() { IO.println("X"); }
}

class Y extends A {
    A Ob;

```

```

Y() { }
Y(A x) { Ob = x; }
void met() { IO.write("Y "); Ob.met(); }
}

class Y1 extends Y {
    Y1(A x) { Ob = x; }
    void met() { IO.write("Y1 "); Ob.met(); }
    void met1() { IO.println("met1"); }
}

class Y2 extends Y {
    Y2(A x) { Ob = x; }
    void met() { IO.write("Y2 "); Ob.met(); }
    void met2() { IO.println("met2"); }
}

class Schema {
    public static void main(String[] sir) {
        X ObX; Y ObY; Y1 ObY1; Y2 ObY2;
        ObX = new X();
        ObY = new Y(ObX); ObY.met();
        ObY1 = new Y1( new Y2(ObX) ); ObY1.met(); ObY1.met1();
        ObY2 = new Y2( new Y1(ObX) ); ObY2.met(); ObY2.met2();
        Y2 obY2 = new Y2( new Y1( new X() ) );
        obY2.met(); obY2.met2();
    }
}

```

Să consideră acum următoarea clasă, care conține doar metoda principală:

```

class Schema {
    public static void main(String[] sir) {
        X ObX; Y ObY; Y1 ObY1; Y2 ObY2;
        ObX = new X();
        ObY = new Y(ObX); ObY.met();
        ObY1 = new Y1( new Y2(ObX) ); ObY1.met(); ObY1.met1();
        ObY2 = new Y2( new Y1(ObX) ); ObY2.met(); ObY2.met2();
        Y2 obY2 = new Y2( new Y1( new X() ) );
        obY2.met(); obY2.met2();
    }
}

```

Este mai întâi creat obiectul `ObX` de tipul `X`. Apoi este creat obiectul `ObY` transmitând constructorului clasei `Y` obiectul `ObX`; spunem că `ObY` *suprapune* obiectul `ObX`. Invocarea `ObY.met()` va produce la ieșire:

Y X
deoarece este invocată metoda `met` din Y (corespunzător tipului real al lui `ObY`).

Apoi este creat obiectul `ObY1`, care suprapune un obiect de tipul `Y2`, care la rândul său suprapune obiectul `ObX`. În clasa `Y1`, care extinde clasa `Y`, este redefinită metoda `met` și este adăugată metoda `met1`. Invocarea, prin intermediul obiectului `ObY1`, a metodelor `met` și `met1` produce la ieșire:

Y1 Y2 X
met1

În continuare este creat obiectul `ObY2`, care suprapune un obiect de tipul `Y1`, care la rândul său suprapune obiectul `ObX`. Invocarea, prin intermediul obiectului `ObY2`, a metodelor `met` și `met2` produce la ieșire:

Y2 Y1 X
met2

În sfârșit, ceea ce s-a realizat pentru obiectul ObY2 mai este realizat și pentru obiectul obY2, rescriind suprapunerea în alt mod. Bineînțeles, la ieșire va fi produs același lucru.

Observație. În literatura despre Java este consacrată sintagma "suprapunerea obiectelor *adaugă noi facilități (funcționalități)*". Adăugarea de noi facilități înseamnă de obicei adaptarea metodelor clasei suprapuse la contextul clasei care o suprapune. În exemplul considerat, "noua funcționalitate" aleasă constă în punerea în evidență a drumului de clase care se suprapun.

Această sintagmă nu trebuie însă înțeleasă greșit, de exemplu în sensul că obiectul care "suprapune" are acces la metodele obiectului "suprapus". Astfel, pentru exemplul de mai sus, faptul că obiectul ObY1 de tipul Y1 suprapune un obiect de tipul Y2 nu permite invocarea ObY1.met2(); obiectul ObY1 poate invoca numai metodele clasei Y1, inclusiv cele rezultate prin moștenire.

Tehnica descrisă mai sus este folosită în continuare pentru fluxurile de intrare/ieșire, care sunt obiecte de tipul claselor din java.io.

Facilitățile de intrare/ieșire din Java au la bază noțiunea de flux. Un *flux* este o succesiune de elemente (octeți sau caractere), citite și scrise secvențial.

Pentru un flux de intrare sursa datelor poate fi un fișier, dar și un șir sau tablou de octeți, respectiv caractere. Pentru un flux de ieșire, datele transmise sunt stocate într-un fișier sau într-un tablou de octeți, respectiv caractere. Vom vedea că mai este posibilă (și chiar recomandată) utilizarea zonelor tampon. De asemenea este posibil ca un flux de ieșire să "comunice" cu un flux de intrare, în sensul că datele scrise în fluxul de ieșire vor constitui sursa pentru fluxul de intrare.

Mai trebuie precizat (deși nu constituie o noutate față de alte limbaje și este deja cunoscut de la citirea de la dispozitivul standard de intrare cu ajutorul clasei IO), faptul că dacă la citire nu sunt încă date disponibile în flux și nu s-a detectat sfârșitul fluxului, atunci firul de executare care realizează citirea va fi blocat până când vor exista date disponibile. Analog, în cazul a două fluxuri comunicante ce folosesc o zonă tampon, firul de executare care are sarcina să scrie va fi blocat în situația în care zona tampon este plină.

O primă clasificare a fluxurilor are în vedere elementele de bază care sunt transmise: caractere sau octeți. Tendința actuală este spre fluxuri ce lucrează la nivel de caracter (doi octeți), dar sunt oferite și facilități pentru fluxuri la nivel de octet deoarece la acest nivel se lucrează deocamdată atunci când folosim fișiere sau transmitem date la distanță. Drept urmare, o primă și incompletă clasificare a ierarhiei de clase (din pachetul java.io) este următoarea:

Object	
InputStream	(clasă abstractă pentru citire la nivel de octet)
OutputStream	(clasă abstractă pentru scriere la nivel de octet)
Reader	(clasă abstractă pentru citire la nivel de caracter)
Writer	(clasă abstractă pentru scriere la nivel de caracter)

Nu vom urmări în acest capitol o prezentare exhaustivă a facilităților de intrare/ieșire din Java. Recomandăm cititorului să consulte în permanență documentația ce însoțește versiunea de Java cu care lucrează.

Toate clasele, interfețele și metodele din pachetul java.io care vor fi prezentate în continuare au modificatorul public, iar în plus aproape toate metodele conțin clauza throws IOException. De aceea nu vom mai specifica explicit acest modificator și nici această clauză.

10.2. Fluxuri ce lucrează la nivel de octet

10.2.1. O structură simplificată de clase

Prezentăm pentru început doar o parte a structurii de clase folosite pentru operațiile de intrare/ieșire:

```
Object
  DataInput (interfață)
  InputStream
    FileInputStream
    FilterInputStream
      DataInputStream implements DataInput
  DataOutput (interfață)
  OutputStream
    FileOutputStream
    FilterOutputStream
      DataOutputStream implements DataOutput
```

cu următoarele observații:

- clasele InputStream și OutputStream sunt abstracte;
- DataInput și DataOutput sunt interfețe;
- toate clasele neabstracte, afară de FileInputStream și de FileOutputStream, au un constructor cu un parametru de tipul InputStream, respectiv de tipul OutputStream.

Exemplul 2. Într-o primă etapă vom citi de la intrarea standard un număr natural n și apoi n numere reale; vom crea în directorul curent un fișier cu numele out.dat în care vom scrie datele citite. Într-o a doua etapă vom citi din fișierul out.dat valoarea n și cele n numere și le vom tipări la ieșirea standard.

Prima etapă este realizată de următorul program:

```
import java.io.*;
class Unu {
    public static void main(String[] sir) throws Exception {
        int n;
```

```

        DataOutputStream g = new DataOutputStream(
            new FileOutputStream("out.dat") );
        IO.write("n= "); n = (int) IO.read(); g.writeInt(n);
        IO.writeln("Introduceți " + n + " numere reale");
        for (int i=0 ; i<n ; i++) g.writeDouble( IO.read() );
        g.close();
    }
}

```

unde:

- prin `new FileOutputStream("out.dat")` este creat fișierul `out.dat`;
- obiectul `g` de tipul `DataOutputStream` folosește metodele `writeInt` și `writeDouble` (anunțate în interfața `DataOutput` și implementate în clasa `DataOutputStream`) pentru a scrie în fișierul `out.dat`;
- pentru închiderea fișierului este invocată metoda `close` a clasei `FilterInputStream` (moștenită din `DataOutputStream`).

A doua etapă este realizată de următorul program:

```

import java.io.*;
class Doi {
    public static void main(String[] sir) throws Exception {
        double d; int n;
        DataInputStream g = new DataInputStream(
            new FileInputStream("out.dat") );

        n = g.readInt();
        for (int i=0 ; i<n; i++)
            IO.write( g.readDouble() + "\t" );
        IO.writeln(); g.close();
    }
}

```

unde:

- prin `new FileInputStream("out.dat")` este deschis fișierul `out.dat`;
- obiectul `g` de tipul `DataInputStream` folosește metodele `readInt` și `readDouble` (anunțate în interfața `DataInput` și implementate în clasa `DataInputStream`) pentru a citi din fișierul `out.dat`;
- pentru închiderea fișierului este invocată metoda `close` a clasei `FilterInputStream` (moștenită din `DataInputStream`).

Se observă că s-a folosit tehnica de suprapunere a fluxurilor prezentată în primul paragraf. Pentru simplificare am folosit varianta nerecomandată de a alege, prin `throws Exception`, acțiunea standard în cazul apariției tuturor excepțiilor.

Observație. Metodele de scriere în fluxul de ieșire scriu datele respective într-un format recunoscut de metodele de citire "complementare" (de exemplu metodele `writeInt` și `readInt` sunt complementare).

Trecem în continuare la descrierea claselor și interfețelor menționate mai sus, prezentând numai principalele câmpuri și metode ale lor.

• Interfața `DataOutput`

Această interfață prezintă contractele unor metode ce urmează a fi realizate în clasele ce o implementează. Este vorba de metode menite a scrie, într-un flux de ieșire neprecizat, date de tipuri primitive, precum și șiruri de caractere.

```

interface DataOutput {
    void write(int b)
    void write(byte[] b) throws NullPointerException
    void write(byte[] b, int init, int lung)
        throws NullPointerException, IndexOutOfBoundsException
    void writeBytes(String s) throws NullPointerException
    void writeChars(String s) throws NullPointerException
    void writeUTF(String s)
    void writeBoolean(boolean v)
    void writeByte(int v)
    void writeShort(int v)
    void writeChar(int v)
    void writeInt(int v)
    void writeLong(long v)
    void writeFloat(float v)
    void writeDouble(double v)
}

```

Metoda `write` cu un argument de tip `int` intenționează să scrie valoarea lui `b` în fluxul de ieșire. Este scris un octet și anume cel mai puțin semnificativ din reprezentarea lui `b`.

Metoda `write` cu un argument de tip tablou de octeți intenționează să scrie toți octeții lui `b` în fluxul de ieșire.

Metoda `write` cu trei argumente intenționează să scrie din `b`, începând de pe poziția `init` a sa, `lung` octeți în fluxul de ieșire.

Pentru toate aceste metode `write`, în cazul în care `b=null` este lansată excepția `NullPointerException`.

Metoda `writeBytes` intenționează să scrie `s.length` octeți în fluxul de ieșire. Fiecare octet este cel mai puțin semnificativ din reprezentarea caracterului corespunzător.

Metoda `writeChars` intenționează să scrie `s.length` caractere în fluxul de intrare. Sunt scriși câte doi octeți pentru fiecare caracter.

Ambele aceste metode lansează excepția `NullPointerException` dacă `s=null`.

Metoda `writeUTF` este gândită pentru a scrie un șir de caractere folosind o codificare specială (numită UTF), care este independentă de platformă.

Toate celelalte metode intenționează să scrie un număr de octeți egal cu lungimea pe care sunt reprezentate tipurile respective din sufixul lui `write`. De exemplu metoda `writeBoolean` scrie într-un octet valoarea `(byte)1` sau `(byte)0`, după cum `v` este `true` sau `false`. Metodele `writeByte` și `writeShort` scriu un octet, respectiv doi octeți; din nou este vorba de octeții cei mai puțin semnificativi din reprezentarea lui `v`.

• Interfața `DataInput`

Interfața `DataInput` reprezintă "complementara" lui `DataOutput`. Sunt citiți octeți din fluxul de intrare și din ei sunt reconstituite date de tipuri primitive, precum și șiruri de caractere.

```
interface DataInput {
    void readFully(byte[] b) throws NullPointerException
    void readFully(byte[] b, int init, int lung)
        throws NullPointerException, IndexOutOfBoundsException
    int skipBytes(int n)
    boolean readBoolean()
    byte readByte()
    int readUnsignedByte()
    short readShort()
    int readUnsignedShort()
    char readChar()
    int readInt()
    long readLong()
    float readFloat()
    double readDouble()
    String readLine()
    String readUTF()
}
```

Observație. Dacă s-a ajuns la sfârșitul fluxului de intrare înainte de a se fi citit numărul dorit de octeți, va fi lansată excepția `EOFException`, care este asemănătoare excepției `IOException`.

Metoda `readFully` cu un parametru intenționează să citească în `b` din fluxul de intrare atâția octeți cât este lungimea lui `b`.

Metoda `readFully` cu trei parametri intenționează să citească în `b`, începând de pe poziția `init` a sa, `lung` octeți din fluxul de intrare. În cazul în care `init+lung>b.length`, este lansată excepția `IndexOutOfBoundsException`.

Pentru ambele metode `readFully`, dacă `b=null` atunci este lansată excepția `NullPointerException`.

Metoda `readUTF` este gândită pentru a citi un șir de caractere scris folosind o codificarea UTF (vezi interfața `DataOutput`).

Metoda `skipBytes` încearcă să avanseze `n` octeți în fluxul de intrare. Dacă se ajunge prematur la sfârșitul fluxului de intrare, avansarea se oprește aici. Metoda întoarce numărul real de octeți peste care s-a avansat.

Metoda `readLine` citește octeți până se ajunge la un octet corespunzător unui terminator de linie (`'\r'`, `'\n'` sau `'r'` urmat de `'\n'`). Octeții citiți sunt transformați în caractere, care compun șirul de caractere întors de metodă. Înainte de a întoarce rezultatul, metoda trece de terminatorul de linie respectiv.

Toate celelalte metode intenționează să citească un număr de octeți egal cu lungimea pe care sunt reprezentate tipurile respective; *se presupune că datele citite au fost scrise în fișier cu metodele complementare anunțate în interfața `DataOutput`*. De exemplu metoda `readBoolean` citește un octet și întoarce `true` dacă și numai dacă octetul citit este zero; se presupune că octetul respectiv a fost scris în fișier cu ajutorul metodei `writeBoolean`.

Un caz special îl constituie metodele `readUnsignedByte` și `readUnsignedShort`. Ele presupun că metodele complementare au scris date din intervalul `0..255`, respectiv din intervalul `0..65535`.

• Clasa abstractă `OutputStream`

```
abstract class OutputStream {
    abstract void write(int b)
    void write(byte[] b) throws NullPointerException
    void write(byte[] b, int init, int lung)
        throws NullPointerException, IndexOutOfBoundsException
    void flush()
    void close()
}
```

Acțiunea metodelor `write` este cea descrisă la prezentarea interfeței `DataOutput`. Acțiunea metodelor `flush` și `close` este nulă, fluxul de intrare nefiind precizat. Intenția (materializată în metode ce le redefinesc) este următoarea: în cazul unui flux ce folosește o zonă tampon, `flush` va trimite octeții către destinație, chiar dacă zona tampon nu este încă umplută complet; metoda `close` va închide fluxul de ieșire.

• Clasa abstractă `InputStream`

```
abstract class InputStream {
    abstract int read()
    int read(byte[] b) throws NullPointerException
    int read(byte[] b, int init, int lung)
        throws NullPointerException, IndexOutOfBoundsException
    long skip(long n)
    int available()
    void close()
}
```

Metodele `read` își amână acțiunea până când există informație disponibilă în fluxul de intrare sau s-a detectat sfârșitul de fișier.

Metoda `read` fără nici un parametru intenționează să citească un octet din fluxul de intrare; rezultatul întors este un întreg din intervalul 0..255. Dacă s-a detectat sfârșitul de fișier, rezultatul întors este -1.

Metoda `read` cu un parametru intenționează să citească în tabloul `b` un număr de octeți egal cu `b.length`; citirea se face octet cu octet, ca la metoda `read` fără parametri. Dacă în fluxul de intrare mai sunt doar `k` octeți până la sfârșitul de fișier, cu `k < b.length`, ei sunt citați în primele `k` elemente ale lui `b`. Rezultatul întors de metodă este numărul real de octeți citați. Dacă de la început fluxul de intrare este la sfârșitul său, rezultatul întors este -1. Excepția `NullPointerException` apare dacă `b=null`.

Metoda `read` cu trei parametri intenționează să citească în `b`, începând de pe poziția `init` a sa, lung octeți din fluxul de intrare. În cazul în care `init+lung > b.length`, este lansată excepția `IndexOutOfBoundsException`. Excepția `NullPointerException` apare dacă `b=null`. Dacă în fluxul de intrare mai sunt doar `k` octeți până la sfârșitul de fișier, cu `k < lung`, ei sunt citați în pozițiile `init..init+k-1` ale lui `b`. Rezultatul întors de metodă este numărul real de octeți citați. Dacă de la început fluxul de intrare este la sfârșitul său, rezultatul întors este -1.

Metoda `skip` încearcă să avanseze cu `n` poziții (octeți) în fluxul de intrare, nedepășind bineînțeles sfârșitul fluxului. Rezultatul întors este numărul de poziții peste care s-a trecut efectiv.

Acțiunea metodei `available`, materializată în metode ce o redefinesc, este de a întorace numărul de octeți ce pot fi citați la momentul curent din fluxul de intrare.

• Clasele `FilterInputStream` și `FilterOutputStream`

Clasa `FilterOutputStream` redefinesc toate metodele interfeței `OutputStream` cu versiuni care sunt aplicate fluxului de ieșire `out`; `out` este un câmp al clasei, care este inițializat în urma invocării constructorului. Subclasele lui `FilterOutputStream` redefinesc metodele acestei clase pentru diferite modalități specifice de scriere și includ noi metode.

```
class FilterOutputStream extends OutputStream {
    protected OutputStream out;
    FilterOutputStream(OutputStream out)
        redefiniri ale metodelor clasei OutputStream
}
```

Clasa `FilterInputStream` redefinesc toate metodele interfeței `InputStream` cu versiuni care sunt aplicate fluxului de intrare `in`; `in` este un câmp al clasei, care este inițializat în urma invocării constructorului. Subclasele lui `FilterInputStream` redefinesc metodele acestei clase pentru diferite modalități specifice de citire și includ noi metode.

```
class FilterInputStream extends InputStream {
    protected InputStream in;
    FilterInputStream(InputStream in)
        redefiniri ale metodelor clasei InputStream
}
```

• Clasele `FileOutputStream` și `FileInputStream`

Metodele clasei `FileOutputStream` implementează, respectiv redefinesc metodele cu aceleași nume și semnătură din `OutputStream`, aplicându-le pentru fluxul de ieșire primit ca argument de constructor.

```
class FileOutputStream extends OutputStream {
    FileOutputStream(String nume)
        throws SecurityException, FileNotFoundException
    FileOutputStream(String nume, boolean adaug)
        throws SecurityException, FileNotFoundException
    redefiniri ale metodelor read, flush și close din OutputStream
    protected void finalize()
}
```

Constructorul cu un parametru al clasei deschide fluxul de ieșire constituit de fișierul precizat prin șirul de caractere `nume`. Dacă fișierul nu poate fi deschis, atunci este lansată excepția `FileNotFoundException`.

Constructorul cu doi parametri al clasei permite, în cazul în care `adaug` este `true`, scrierea în flux în continuarea informației existente.

Metoda `finalize` realizează invocarea metodei `close`, atunci când nu mai există referiri la fluxul de ieșire (de exemplu fluxul de ieșire poate fi folosit de mai multe fire de executare).

Metodele clasei `FileInputStream` implementează, respectiv redefinesc metodele cu aceleași nume și semnătură din `InputStream`, aplicându-le pentru fluxul de intrare primit ca argument de constructor.

```
class FileInputStream extends InputStream {
    FileInputStream(String nume)
        throws SecurityException, FileNotFoundException
    redefiniri ale metodelor write, skip, available și close
    din OutputStream
    protected void finalize()
}
```

Constructorul clasei deschide fluxul de intrare constituit de fișierul precizat prin șirul de caractere nume. Dacă fișierul nu poate fi deschis, atunci este lansată excepția `FileNotFoundException`.

Metoda `finalize` realizează invocarea metodei `close`, atunci când nu mai există referiri la fluxul de intrare.

• Clasele `DataOutputStream` și `DataInputStream`

Clasele `DataOutputStream` și `DataInputStream` oferă ca facilități suplimentare posibilitatea ca fluxurile să nu mai fie privite strict la nivel de octet, ci ca succesiuni de date primitive sau șiruri de caractere. Pentru aceasta, datele vor fi scrise în fluxul de ieșire într-un format independent de modul de reprezentare al datelor în sistemul pe care se lucrează. Este momentul să amintim că unele sisteme folosesc reprezentarea *Big Endian* (în care octetul cel mai semnificativ apare la cea mai mică adresă de memorie), iar altele folosesc reprezentarea *Little Endian* (în care octetul cel mai semnificativ apare la cea mai mare adresă de memorie).

Constructorul clasei `DataOutputStream` creează un nou flux de ieșire ce suprapune pe cel primit ca argument.

Metodele clasei (cu excepția lui `size`) redefinesc, respectiv implementează, metodele clasei `FilterOutputStream` și interfeței `DataOutput` pentru noul flux de ieșire.

```
class DataOutputStream extends FilterOutputStream
    implements DataOutput {
    DataOutputStream(OutputStream out)
    protected int written;
    implementarea metodelor write din interfața DataOutput
    void flush()
    final int size()
}
```

Câmpul `written` ține evidența numărului de octeți scriși în fluxul de ieșire până la momentul curent. Metoda `size` întoarce valoarea curentă a lui `written`.

Constructorul clasei `DataInputStream` creează un nou flux de intrare ce suprapune pe cel primit ca argument.

Metodele acestei clase redefinesc, respectiv implementează, metodele clasei `FilterInputStream` și interfeței `DataInput` pentru noul flux de intrare.

```
class DataInputStream extends FilterInputStream
    implements DataInput {
    DataInputStream(InputStream in)
    implementarea metodelor read și skipBytes din interfața DataOutput
}
```

10.2.2. O structură extinsă de clase

Prezentăm în continuare o structură mai completă de clase și interfețe folosite pentru intrări/ieșiri la nivel de octet. Precizăm din nou că nu ne propunem o prezentare exhaustivă.

Object	
<code>DataInput</code>	(interfață)
<code>InputStream</code>	(clasă abstractă)
<code>FileInputStream</code>	
<code>FilterInputStream</code>	
<code> DataInputStream</code>	
<code> BufferedInputStream</code>	
<code> PushBackInputStream</code>	
<code>PipedInputStream</code>	
<code>DataOutput</code>	(interfață)
<code>OutputStream</code>	(clasă abstractă)
<code>FileOutputStream</code>	
<code>FilterOutputStream</code>	
<code> DataOutputStream</code>	
<code> BufferedOutputStream</code>	
<code>PrintStream</code>	
<code>PipedOutputStream</code>	

• Clasa `InputStream` (metode pentru lucrul cu zone tampon)

Prezentăm în continuare alte trei metode ale clasei `InputStream`, gândite pentru utilizarea zonelor tampon (buffere), ce vor fi redefinite în clasele `BufferedInputStream` și `BufferedOutputStream`.

Idea principală este folosirea unei zone tampon (un tablou de octeți) ca intermediar pentru citirea/scrierea datelor în fluxul efectiv. Prin utilizarea zonei tampon se evită accesarea fișierului pentru fiecare citire/scriere, în acest mod micșorându-se timpul de lucru.

În plus, pentru fluxurile de intrare, există facilitatea de a marca o anumită poziție din buffer, astfel încât la invocarea unei anumite metode să se reia citirea din zona tampon de la poziția marcată. Metodele sunt următoarele:

`void mark(int marklimit)` : marchează poziția curentă din zona tampon și stabilește o limită pentru numărul de octeți ce pot fi recitiți

`void reset()` : repoziționează fluxul de intrare la poziția specificată la ultima invocare a metodei `mark`

`boolean markSupported()` : verifică dacă fluxul de intrare permite utilizarea metodelor `mark` și `reset`.

Deoarece acțiunea metodelor `mark` și `reset` din clasa `InputStream` este nulă și metodele vor fi redefinite în clasele `BufferedInputStream` și `BufferedOutputStream`, vom reveni cu precizări la prezentarea acestor clase.

• Clasa `BufferedInputStream`

Funcționalitățile pe care le adaugă această clasă unui flux de intrare constau în posibilitatea de a folosi o zonă tampon pentru flux. Pe măsură ce se citesc octeți sau se sare peste un număr de octeți, zona tampon este actualizată prin includerea de noi octeți din fluxul de intrare.

Metodele clasei redefinesc metodele cu același nume și aceeași semnătură din `InputStream` și `FilterInputStream`, adaptându-le la lucrul cu zona tampon.

```
class BufferedInputStream extends FilterInputStream {
    protected byte[] buffer;
    protected int count=0, pos=0, markpos=-1, marklimit=0;
    BufferedInputStream(InputStream in)
    BufferedInputStream(InputStream in, int dim)
    int read()
    int read(byte[] b, int init, int lung)
        throws NullPointerException, IndexOutOfBoundsException
    void mark(int limit)
    void reset()
    boolean markSupported()
    long skip(long n)
    int available()
}
```

Constructorii clasei creează zona tampon buffer ca un tablou de octeți, a cărui lungime este omisă (caz în care lungimea zonei tampon este aleasă automat) sau precizată efectiv prin parametrul `dim`.

Octeții valizi din buffer ocupă primele `count` poziții din buffer. Valoarea lui `pos` este totdeauna în intervalul `0..count`. Dacă `pos < count`, următorul octet ce va fi citit este cel de pe poziția `pos`; dacă `pos = count`, la următoarea invocare a unei metode `read` sau `skip` se așteaptă ca fluxul în să primească date. Actualizarea celor două câmpuri se face automat.

La invocarea metodei `mark` este marcată poziția curentă `pos` și este stabilit numărul maxim de octeți `marklimit` ce pot fi citați înainte ca marcarea să fie invalidată, ceea ce se realizează prin actualizarea lui `marklimit` și `markpos` respectiv cu `limit` și `pos`.

La invocarea metodei `reset` se consideră ultima marcarea (dacă a existat vreuna). Dacă ea nu a fost invalidată (prin citirea a mai mult de `marklimit` octeți), în continuare citirea se reia din zona tampon de la poziția marcată. Dacă nu a avut loc nici o marcarea sau dacă ultima marcarea a fost invalidată (situații identificate prin `markpos = -1`), este lansată excepția `IOException`.

Totdeauna `markpos` este în intervalul `-1..pos`. Dacă `markpos != -1`, toți octeții de pe pozițiile `markpos..pos-1` vor rămâne în buffer (chiar dacă eventual vor fi translați în urma unor citiri), atâta timp cât `pos - markpos ≤ marklimit`.

Metoda `read` cu trei argumente adaugă o nouă facilitate față de metoda din `InputStream` pe care o redefinesc. Mai precis, se încearcă în mod repetat citirea de octeți până când se ajunge în una dintre următoarele situații: s-a citit numărul dorit de octeți, s-a ajuns la sfârșit de fișier sau `available` arată că în fluxul de intrare nu mai există la momentul curent octeți disponibili.

Metoda `available` întoarce numărul de octeți disponibili (atât în zona tampon, cât și în fluxul de intrare) pentru citire; este deci întors rezultatul invocării metodei `available` pentru fluxul de intrare în, la care se adaugă `count-pos`.

• Clasa `BufferedOutputStream`

Această clasă oferă facilități analoge cu cele din clasa `BufferedInputStream`, dar bineînțeles pentru un flux de ieșire.

Metodele clasei redefinesc metodele cu același nume și aceeași semnătură din `FilterOutputStream`, adaptându-le la lucrul cu zona tampon.

```
class BufferedOutputStream extends FilterOutputStream {
    protected byte[] buffer;
    protected int count;
    BufferedOutputStream(OutputStream out)
    BufferedOutputStream(OutputStream out, int dim)
    void write(int b)
    void write(byte[] b, int init, int lung)
        throws NullPointerException, IndexOutOfBoundsException
    void flush()
}
```

Constructorii clasei creează zona tampon buffer ca un tablou de octeți, a cărui lungime este omisă (caz în care lungimea zonei tampon este aleasă automat) sau precizată efectiv prin parametrul `dim`.

Octeții efectivi ce au fost scriși în zona tampon ocupă primele `count` poziții din buffer.

Exemplul 3. Adaptăm *Exemplul 1* la lucrul cu zone tampon, utilizând un buffer pentru fluxul de intrare din clasa `Doi`. Pentru aceasta este suficient să înlocuim declararea fluxului `g` prin:

```
DataInputStream g = new DataInputStream(
    new BufferedInputStream(
        new FileInputStream("out.dat") ) );
```

• Clasa `PushbackInputStream`

Această clasă oferă facilitatea suplimentară ca după ce a fost citit un octet, el să fie "pus înapoi" în fluxul de intrare; în continuare el va fi primul citit. Este

folosită o zonă tampon, în care vor fi repuși octeți, analog ca pentru clasa `BufferedInputStream`, dar fără posibilitățile de marcare și revenire explicite.

```
class PushbackInputStream extends FilterInputStream {
    protected int pos = -1;
    protected byte[] buffer;
    PushbackInputStream(InputStream in)
    PushbackInputStream(InputStream in, int dim)
    int read()
    int read(byte[] b, int init, int lung)
        throws NullPointerException, IndexOutOfBoundsException
    void unread(int b)
    void unread(byte[] b)
    void unread(byte[] b, int init, int lung)
    int available()
    long skip(long n)
    boolean markSupported()
    int close()
}
```

Valoarea curentă a câmpului `pos` este -1 dacă zona tampon nu conține octeți valizi (repuși); în caz contrar ea indică primul octet ce va fi citit.

Metodele `read` realizează citirea începând cu octeții valizi din zona tampon.

Metodele `unread` pun octeți înapoi la începutul zonei tampon, în fața celor deja existenți. Dacă operația implică depășirea lungimii zonei tampon, este lansată excepția `IOException`.

Metoda `available` redefineste pe cea din `FilterInputStream`, invocând-o pe aceasta și adăugând la rezultat numărul de octeți din `buffer`.

Metoda `skip` trece mai întâi peste octeții din zona tampon.

Metoda `markSupported` întoarce valoarea `false`, întrucât metodele `mark` și `reset` nu pot fi invocate.

Exemplul 4. Se presupune că fluxul de intrare conține nume de străzi despărțite între ele prin șirul de caractere "Str. ". Se cere listarea tuturor numelor de străzi, inclusiv șirul de caractere menționat.

Va fi folosit un tablou de octeți de aceeași lungime cu zona tampon din fluxul având ca tip clasa `PushbackInputStream`. În momentul în care se detectează șirul de caractere despărțitor, este scris numele noii străzi, iar octeții corespunzători următorului nume de stradă sunt repuși în flux.

```
import java.io.*;
public class PushBack {
    public static void main (String[] sir) throws Exception {
        String prefix = "Str. ";
        byte[] temp = new byte[10];
        String s_temp = "";
        int poz, nr, dif;
```

```
        PushbackInputStream p = new PushbackInputStream (
            new FileInputStream("push.dat"), 10);
        nume = "";
        while ( p.available() > 0 ) {
            nr = p.read(temp);
            s_temp = new String(temp);
            s_temp = s_temp.substring(0,nr);
            nume += s_temp;
            poz = nume.indexOf(prefix);
            if (poz >= 0) {
                dif = nume.length()-poz-5;
                IO.println("Strada noua: Str. " +
                    nume.substring(0,poz));
                s_temp = s_temp.substring(s_temp.length()-dif);
                nume = "";
                if (dif>0) p.unread(temp,nr-dif,dif);
            }
        }
        IO.println("Strada noua: Str. " + s_temp);
    }
}
```

• Clasele `PipedOutputStream` și `PipedInputStream`

Funcționalitatea nouă oferită de aceste clase constă în crearea a câte un obiect `pos`, respectiv `pis` de fiecare din aceste tipuri, obiecte ce sunt "conectate" în următorul sens: ceea ce este scris prin intermediul obiectului `pos` este citit prin intermediul obiectului `pis`. Fiecare dintre obiecte (fluxuri) cunoaște identitatea celuilalt. Este recomandat să folosim fire de executare separate pentru utilizarea celor două obiecte, deoarece încercarea de a realiza acest tip de transmisie din cadrul aceluiași fir de executare poate conduce la blocare totală. Este folosită o zonă tampon cu disciplină de coadă. Orice obiect de unul dintre tipurile `PipedInputStream` și `PipedOutputStream` trebuie conectat la exact un obiect de celălalt tip, în caz contrar fiind lansată o excepție.

```
PipedInputStream extends InputStream {
    protected static final int PIPE_SIZE; // PIPE_SIZE
    protected byte[] buffer;
    protected int in, out;
    PipedInputStream(PipedOutputStream pos)
    PipedInputStream()
    void connect(PipedOutputStream pos)
    protected void receive(int b)
    implementarea metodelor read, available și close din InputStream.
}
```

Constructorul cu un parametru creează un obiect de tipul `PipedInputStream` și îl conectează la obiectul `pos` primit ca parametru.

Constructorul fără parametri creează un obiect, dar nu realizează conectarea; aceasta trebuie realizată ulterior prin invocarea metodei connect.

Câmpul PIPE_SIZE reprezintă mărimea cozii de intrare, ale cărei elemente apar în buffer. Câmpurile in și out indică pozițiile în care va fi primit următorul octet, respectiv poziția din care va fi citit primul octet din acest flux de intrare. Coada vidă este identificată prin in<0, iar coada plină este identificată prin in==out.

Prin invocarea metodei receive este primit un octet de intrare.

```
class PipedOutputStream extends InputStream {
    PipedOutputStream(PipedOutputStream pos)
    PipedOutputStream()
    void connect(PipedInputStream pis)
    implementarea metodelor write, flush și close din clasa OutputStream
}
```

Constructorul cu un parametru creează un obiect de tipul PipedOutputStream și îl conectează la obiectul pis primit ca parametru. Constructorul fără parametri creează un obiect, dar nu realizează conectarea; aceasta trebuie realizată ulterior prin invocarea metodei connect.

Observație. Dacă fluxul de intrare pis și fluxul de ieșire pos sunt ambele neconectate, conectarea lor se poate realiza și prin oricare dintre invocările pis.connect(pos) și pos.connect(pis).

Exemplul 4. În acest exemplu este reluată problema *Producător - Consumator*; sunt transmise valorile din intervalul 10..99.

```
import java.io.*;
class PipeFiles extends PipedInputStream {
    PipedInputStream pis; PipedOutputStream pos;
    PipeFiles() {
        try {
            IO.println(" + PIPE_SIZE);
            pis = new PipedInputStream();
            pos = new PipedOutputStream();
            pis.connect(pos);
        }
        catch (IOException e) { }
    }
}

class Prod extends Thread {
    PipedOutputStream pos;
    Prod(PipeFiles f) { pos = f.pos; }
    public void run() {
        byte b;
        try {
```

```
        for (b=10; b<100; b++) {
            sleep( (int) (200*Math.random()) );
            IO.write(" P" + b); pos.write(b);
        }
    }
    catch (IOException e) { }
    catch (InterruptedException e) { }
}

class Cons extends Thread {
    PipedInputStream pis;
    Cons(PipeFiles f) { pis = f.pis; }
    public void run() {
        int b = 77;
        try {
            do { b=pis.read(); IO.write(" I" + b); }
            while (b<99);
        }
        catch (IOException e) { }
    }
}

class Piped {
    public static void main(String[] sir) {
        PipeFiles f = new PipeFiles();
        Prod P = new Prod(f); Cons C = new Cons(f);
        C.start(); P.start();
    }
}
```

Observație. Pentru a putea tipări câmpul PIPE_SIZE (declarat cu protected) al clasei PipedInputStream, clasa PipeFiles a fost declarată ca extinzând-o pe aceasta din urmă.

• Clasa PrintStream

Această clasă oferă facilitatea de a scrie într-un flux de ieșire diferite date într-un format "prienos", adică asemănător reprezentării lor matematice uzuale. În plus, este oferită (opțional) posibilitatea de a goli automat zona tampon după ce a fost scris un tablou de octeți sau un octet egal cu '\n'. Să remarcăm că metodele acestei clase nu lansează niciodată excepția IOException.

```
class PrintStream extends FilterOutputStream {
    PrintStream(OutputStream out)
    PrintStream(OutputStream out, boolean adaug)
    redefiniri ale metodelor write, flush și close
    din clasa FilterOutputStream.
```

```

void print(String s)
void print(char[] s) throws NullPointerException
void print(boolean b)
void print(char c)
void print(int i)
void print(long l)
void print(float f)
void print(double d)

```

cu observația că pentru fiecare metodă print există și "perechea" ei println, care mai adaugă caracterul '\n' în fluxul de ieșire. În plus există și metoda println fără parametri, care scrie doar caracterul de sfârșit de linie '\n'.

Constructorul cu doi parametri oferă, prin al doilea parametru, posibilitatea de a goli automat zona tampon, așa cum am menționat mai sus.

Metodele print și println scriu argumentul lor într-un format "prietenos" și anume identic cu cel produs de invocările String.valueOf(x).

Mai menționăm că fluxul standard de ieșire out din clasa System:
 public static final PrintStream out;
 este deja deschis și gata să accepte date de ieșire.

10.3. Fluxuri ce lucrează la nivel de caracter

Fluxurile la nivel de caracter operează asupra fișierelor, șirurilor de caractere și tablourilor de caractere.

În cazul fișierelor, sunt totuși scriși sau citiți octeți, ceea ce ridică problema codificării caracterelor ca octeți. Fiecare instanță a mașinii virtuale are o codificare implicită lipsă, ce depinde în mod standard de sistemul de operare folosit. Poate fi însă specificată și o codificare explicită.

După cum vom vedea, multe dintre clasele care vor fi prezentate reprezintă un analog al claselor cunoscute de la fluxurile ce lucrează la nivel de octet. Multe metode au același nume și o acțiune analoagă cu cele deja cunoscute și de aceea nu le vom mai prezenta în detaliu.

De asemenea este de semnalat posibilitatea de a furniza unui flux un obiect pe baza căruia să se execute sub excludere reciprocă diferitele operații de citire/scriere asupra acelui flux.

10.3.1. O structură simplificată de clase

O primă structură de clase pe care o prezentăm are în vedere în primul rând fișierele:

Object Writer (clasă abstractă) OutputStreamWriter FileWriter Reader (clasă abstractă) InputStreamReader FileReader

• Clasa Writer

```

abstract class Writer extends Object {
    protected Object lock;
    protected Writer()
    protected Writer(Object lock)
    void write(int c)
    abstract void write(char[] buffer, int init, int lung)
    void write(String sir)
    void write(String sir, int init, int lung)
    abstract void flush()
    public abstract void close()
}

```

Începem prin a observa că în această clasă abstractă metodele abstracte sunt flush(), close() și write(char[], int, int).

Câmpul lock oferă posibilitatea de a realiza excluderea reciprocă asupra secțiunilor critice și pe baza altui obiect decât this.

Acțiunea metodei flush constă în a goli imediat eventuala zonă tampon cu care se lucrează.

• Clasa Reader

```

abstract class Reader extends Object {
    protected Object lock;
    protected Reader()
    protected Reader(Object lock)
    int read()
    int read(char[] cbuf)
    abstract int read(char[] cbuf, int init, int lung)
    long skip(long n)
    boolean ready()
    boolean markSupported()
    void mark(int limit)
    void reset()
    abstract void close()
}

```

Metoda `read` fără parametri întoarce caracterul citit sub forma unui întreg din intervalul 0..65535. Celelalte metode `read` întorc numărul de caractere citite. Dacă s-a ajuns la sfârșit de fișier, este întoarsă valoarea -1.

Metoda `ready` întoarce valoarea `true` dacă următoarea eventuală invocare `read()` nu va duce la blocare.

Metoda `mark` marchează poziția curentă din flux și stabilește, prin intermediul argumentului `limit`, numărul maxim de caractere ce vor fi citite înainte ca marcarea să fie invalidată.

Metoda `skip` încearcă să avanseze cu `n` poziții (caractere) în fluxul de intrare, nedepășind bineînțeles sfârșitul fluxului. Rezultatul întors este numărul de poziții peste care s-a trecut efectiv.

• Clasa `OutputStreamWriter`

```
class OutputStreamWriter extends Writer {
    OutputStreamWriter(OutputStream out)
        throws UnsupportedOperationException
    OutputStreamWriter(OutputStream out, String codif)
        throws UnsupportedOperationException
    String getEncoding()
    void write(int c)
    void write(String str, int init, int lung)
    void flush()
    void close()
}
```

Metoda `getEncoding` întoarce numele codificării utilizate de flux, sub forma unui șir de caractere; reamintim că este vorba de codificarea caracterelor ca octeți. Să observăm că la constructorul cu doi parametri, al doilea reprezintă numele unei codificări.

• Clasa `FileWriter`

```
class FileWriter extends OutputStreamWriter {
    FileWriter(String nume)
    FileWriter(String nume, boolean adaug)
}
```

Ca și în cazul clasei `FileOutputStream`, constructorul deschide fișierul precizat prin nume și, numai dacă este folosit cel cu doi parametri și al doilea primește valoarea `true`, asigură scrierea în fișier în continuarea informației existente.

Următoarele două clase nu mai necesită explicații:

• Clasa `InputStreamReader`

```
class InputStreamReader extends Reader {
    InputStreamReader(InputStream in)
    InputStreamReader(InputStream in, String codif)
        throws UnsupportedOperationException
    String getEncoding()
    int read()
    int read(char[] cbuf, int init, int lung)
    boolean ready()
    void close()
}
```

• Clasa `FileReader`

```
class FileReader extends InputStreamReader {
    FileReader(String nume) throws FileNotFoundException
}
```

Exemplul 6. Două fire de executare scriu caractere în același flux de ieșire. Pentru realizarea excluderii reciproce se folosește sincronizarea pe baza unui aceleiași obiect, care este transmis câmpului `lock` al clasei `Writer`.

```
import java.io.*;
class Fir extends Thread {
    Writer f; char ch;
    Fir(Writer fis, char c) { f = fis; ch = c; };
    public void run() {
        synchronized(f) {
            for (int i=0; i<=99; i++) {
                try {
                    sleep( (int) (100*Math.random()) ); f.write(ch);
                }
                catch (InterruptedException e) { }
                catch (IOException e) { }
            }
        }
    }
}

class Fisier extends FileWriter {
    Fisier(Object Ob, String nume) throws IOException {
        super(nume); lock = Ob;
    }
}

class Char2 {
    public static void main(String[] sir) throws Exception {
        Object Ob = new Object();
```

```

Fisier f = new Fisier(Ob, "Char2.dat");
IO.writeln( f.getEncoding() );
Fir F1 = new Fir(f, '1'); Fir F2 = new Fir(f, '2');
F1.start(); F2.start();
F1.join(); F2.join();
f.close();
}
}

```

10.3.2. O structură extinsă de clase

Structura extinsă de clase prezentată în acest paragraf, ce realizează prelucrarea fluxurilor la nivel de caracter, este următoarea:

Object	
Writer	(clasă abstractă)
BufferedWriter	
CharArrayWriter	
OutputStreamWriter	
FileWriter	
PipedWriter	
PrintWriter	
Reader	(clasă abstractă)
BufferedReader	
LineNumberReader	
CharArrayReader	
InputStreamReader	
FileReader	
PipedReader	

• Clasele `BufferedWriter` și `BufferedReader`

Aceste clase permit utilizarea unei zone tampon pentru creșterea eficienței operațiilor de scriere/citire efectuate asupra fluxurilor de date.

Clasa `BufferedWriter` are o metodă `newLine` care realizează terminarea liniei curente din fluxul de ieșire, folosind modul în care este definită pe platforma pe care lucrăm noțiunea de separator de linie (nu toate platformele folosesc în acest scop `'\n'`).

Clasa `BufferedReader` prevede o metodă `readLine` ce întoarce o linie de text.

```

class BufferedWriter extends Writer {
    BufferedWriter(Writer out)
    BufferedWriter(Writer out, int dim)
    void write(int c)
    void write(char[] buffer, int init, int lung)
    void write(String s, int init, int lung)

```

```

void newLine()
void close()
}

```

Constructorul cu un parametru stabilește o lungime implicită pentru zona tampon. Prin invocarea constructorului cu doi parametri putem stabili o lungime explicită pentru zona tampon.

Metoda `newLine` înscrie în fluxul de ieșire separatorul de linie propriu platformei pe care lucrăm.

```

class BufferedReader extends Reader {
    BufferedReader(Reader in, int dim)
    BufferedReader(Reader in)
    int read()
    int read(char[] cbuf, int init, int lung)
    String readLine()
    long skip(long n)
    boolean ready()
    boolean markSupported()
    void mark(int limit)
    void reset()
    void close()
}

```

Constructorul cu un parametru stabilește o lungime implicită pentru zona tampon. Prin invocarea constructorului cu doi parametri putem stabili o lungime explicită pentru zona tampon.

Metoda `readLine` citește din fluxul de intrare o *linie de text*. Linia de text se încheie cu primul terminator de linie; ea este întoarsă, dar fără terminatorul de linie, ca rezultat.

Celelalte metode lucrează analog cu cele omonime din clasa `BufferedInputStream`.

• Clasele `CharArrayWriter` și `CharArrayReader`

Clasa `CharArrayWriter` creează un tablou de caractere care este folosit ca buffer pentru datele ce sunt înscrise în el. Lungimea zonei tampon crește automat pe măsură ce în ea sunt scrise caractere. Zona tampon poate fi returnată, folosind metodele `toCharArray`, `toString` și `writeTo`, sub forma unui tablou de caractere, a unui șir de caractere, respectiv a unui flux (de ieșire) de caractere.

```

class CharArrayWriter extends Writer {
    protected char[] buf;
    protected int count;
    CharArrayWriter()
    CharArrayWriter(int dim)
    void write(int c)

```

```

void write(char[] c, int init, int lung)
void write(String str, int init, int lung)
void writeTo(Writer out)
void reset()
char[] toCharArray()
int size()
String toString()
void flush()
void close()
)

```

Câmpul buf reprezintă zona tampon unde sunt scrise datele, iar câmpul count ține evidența numărului de caractere scrise în buffer.

Constructorul fără parametri inițializează zona tampon la o dimensiune implicită. Constructorul cu doi parametri stabilește o dimensiune inițială pentru zona tampon.

Metoda size întoarce dimensiunea curentă a zonei tampon. Metoda reset permite reinițializarea zonei tampon; de exemplu după scrierea unor date în buffer, conținutul acestuia poate fi salvat într-un șir de caractere, zona tampon este reinițializată și după scrierea altor date, datele nou scrise pot fi salvate într-un fișier.

Clasa CharArrayReader folosește ca flux de intrare un tablou de caractere specificat la invocarea constructorului.

```

class CharArrayReader extends Reader {
    protected char[] buf;
    protected int pos;
    protected int markedPos;
    protected int count;
    CharArrayReader(char[] buf)
    CharArrayReader(char[] buf, int init, int lung)
    int read()
    int read(char[] b, int init, int lung)
    long skip(long n)
    boolean ready()
    boolean markSupported()
    void mark(int limit)
    void reset()
    void close()
}

```

Constructorii permit ca un tablou de caractere sau o porțiune a acestuia să fie considerat în continuare ca flux de intrare la nivel de caracter.

Metodele au o acțiune analoagă cu cele omonime din clasa BufferedInputStream. Evident, citirea unui caracter nu implică eliminarea sa din buffer.

Exemplul 7. Următorul program, care nu necesită informații suplimentare:

```

import java.io.*;
class CharArray {
    public static void main(String[] sir) throws IOException {
        CharArrayWriter f; CharArrayReader g;
        char[] buffer; int c;
        f = new CharArrayWriter();
        f.write('a'); f.write('b'); IO.println(f.toString());
        buffer = f.toCharArray();
        g = new CharArrayReader(buffer); c = g.read();
        while (c != -1) {
            IO.write( (char) c + " "); c = g.read();
        }
        IO.println();
        f.write('c'); IO.println(f.toString());
        buffer = f.toCharArray();
        g = new CharArrayReader(buffer); c = g.read();
        while (c != -1) {
            IO.write( (char) c + " "); c = g.read();
        }
        IO.println();
    }
}

```

produce la ieșire:

```

ab
ab
abc
abc

```

• Clasele PipedWriter și PipedReader

Aceste două clase constituie echivalentul pentru lucrul la nivel de caracter al facilităților oferite de clasele PipedOutputStream și PipedInputStream pentru lucrul la nivel de octet.

```

class PipedWriter extends Writer {
    PipedWriter()
    PipedWriter(PipedReader pr)
    void connect(PipedReader pr)
    void write(int c)
    void write(char[] cbuf, int init, int lung)
    void flush()
    void close()
}

```

```

class PipedReader extends Reader {
    PipedReader()
    PipedReader(PipedWriter pw)
}

```

```

void connect(PipedWriter pw) Exception
int read()
int read(char[] cbuf, int init, int lung)
boolean ready()
void close()
}

```

Metoda `ready` întoarce valoarea `true` în cazul în care zona tampon circulară este nevidă, respectiv `false` în caz contrar.

• Clasa `PrintWriter`

Această clasă oferă facilitatea de a scrie date în fluxul de ieșire, într-un mod analog cu ceea ce realizează clasa `PrintStream`, adică datele sunt scrise într-un mod formatat, specific metodelor `toString`. Nu este lansată niciodată excepția `IOException`; pentru a verifica dacă a apărut o eroare, poate fi folosită metoda `checkError`.

Spre deosebire de `PrintStream`, zona tampon va fi golită nu atunci când este scris un caracter de linie nouă, ci după ce a fost invocată o metodă `println`; o astfel de metodă include în fluxul de ieșire caracterul de linie nouă specific platformei pe care lucrăm.

```

class PrintWriter extends Writer {
    protected Writer out;
    PrintWriter(Writer out)
    PrintWriter(Writer out, boolean adaug)
    PrintWriter(OutputStream out)
    PrintWriter(OutputStream out, boolean adaug)
    void flush()
    void close()
    boolean checkError()
    void setError()
    void write(int c)
    void write(char[] buf, int init, int lung)
    void write(char[] buf)
    void write(String s, int init, int lung)
    void write(String s)
    void print(boolean b)          void println(boolean b)
    void print(char c)             void println(char c)
    void print(int i)              void print(int i)
    void print(long l)             void println(long l)
    void print(float f)            void println(float f)
    void print(double d)           void println(double d)
    void print(char[] s)           void println(char[] s)
    void print(String s)           void println(String s)
    void print(Object obj)         void println(Object obj)
    void println()
}

```

Metoda `checkError` golește zona tampon și verifică dacă a intervenit o eroare. Metoda `setError` indică faptul că a apărut o eroare.

10.4. Analizorul lexical `StreamTokenizer`

Clasa `StreamTokenizer` din pachetul `java.io` extinde direct clasa `Object` din pachetul `java.lang`. Ea permite, prin intermediul câmpurilor și metodelor sale, să acționăm asupra unui flux de intrare în scopul detectării entităților sale; prin urmare suntem în prezența unui *analizor lexical*.

Entitățile care pot fi identificate în mod standard sunt:

- identificatorii;
- șirurile de caractere;
- sfârșitul de fișier (EOF).

Opțional, mai pot fi identificate și următoarele entități:

- sfârșitul de linie (EOL);
- comentariile de sfârșit de linie (care încep cu `"//"` și continuă până la sfârșitul liniei);

- comentariile generale (care încep cu `"/*"` și se termină cu prima succesiune `"*/"`),

cu observația că eventuala identificare a comentariilor este însoțită de ignorarea lor (deci nu de furnizarea conținutului lor).

Fiecare octet citit din fluxul de intrare este privit ca un caracter din intervalul delimitat de `'\u0000'` și de `'\u00FF'` și este considerat ca având nici unul, unul sau mai multe dintre următoarele atribute: spațiu alb al analizorului, caracter alfabetic, caracter numeric, identificator de șiruri (apostrof sau ghilimele) și caracter comentariu.

```

class StreamTokenizer extends Object {
    int ttype
    static final int TT_EOF        // TT_EOF
    static final int TT_EOL        // TT_EOL
    static final int TT_NUMBER     // TT_NUMBER
    static final int TT_WORD       // TT_WORD
    String sval
    double nval
    StreamTokenizer(InputStream is) // depreciată
    StreamTokenizer(Reader r)
    int nextToken()
    void eolIsSignificant(boolean flag)
    void slashStarComments(boolean flag)
    void slashSlashComments(boolean flag)
    void ordinaryChar(int ch)
    void commentChar(int ch)
    void quoteChar(int ch)
}

```

```

void ordinaryChars(int low, int hi)
void whitespaceChars(int low, int hi)
void wordChars(int low, int hi)
void resetSyntax()
int lineno()
}

```

Constructorul clasei are un parametru de tipul `InputStreamReader` și realizează următoarele inițializări (definiri implicite):

- caracterele de la 'a' la 'z', de la 'A' la 'Z' și de la 0xA0 la 0xff sunt considerate a fi caractere alfabetice;
- caracterele de la 0x00 la 0x20 sunt considerate a fi spațiile albe ale analizorului; să observăm că sunt incluse aici spațiile albe uzuale;
- caracterul '/' este caracter comentariu;
- apostroful și ghilimelele sunt considerate identificatori de șiruri;
- sfârșiturile de linie nu sunt recunoscute ca atare (sunt tratate drept spații albe);
- comentariile nu sunt recunoscute.

Clasa `StreamTokenizer` conține următoarele câmpuri constante:
`TT_EOF` ; `TT_EOL` ; `TT_NUMBER` ; `TT_WORD`
 corespunzătoare în ordine entităților: sfârșit de fișier, sfârșit de linie, valoare numerică și identificator.

Tipul entității identificate va fi regăsit în câmpul `ttype`. Valorile pe care le poate lua `ttype` sunt:

- dacă entitatea detectată este un unic caracter, valoarea lui `ttype` este valoarea caracterului, convertită la un întreg;
- '\'', dacă a fost detectat un șir de caractere cuprins între ghilimele;
- una dintre cele 4 constante de mai sus.

Câmpul `sval` memorează un identificator sau un șir de caractere, în situația în care a fost detectată o astfel de entitate; în caz contrar, valoarea lui `sval` este `null`.

Câmpul `nval` are semnificație dacă a fost recunoscută o valoare numerică de tip `double`, caz în care memorează acea valoare. Precizăm că este vorba numai de numere zecimale cu sau fără punct; analizorul nu recunoaște $31.4e-1$ ca număr în virgulă mobilă și nici `0xee` ca număr hexazecimal.

Dacă se dorește identificarea sfârșiturilor de linie, a comentariilor de sfârșit de linie și a celor cu asterisc, trebuie invocate în prealabil următoarele metode cu specificarea lui `true` ca argument:

```

EolIsSignificant    slashStarComments    slashSlashComments

```

Metodele `ordinaryChar`, `commentChar` și `quoteChar` au un parametru de tipul `int`. Apelarea lor pentru un caracter `ch` face ca `ch` să devină respectiv un caracter obișnuit (fără vreun atribut), un caracter comentariu sau un caracter pentru identificarea șirurilor de caractere.

Mai menționăm și metodele `ordinaryChars`, `whitespaceChars` și `wordChars` ce au două argumente (fie ele `p` și `u`) de tipul `int`. Prin apelarea lor, caracterele din secvența `p..u` capătă respectiv atributele caracter obișnuit (fără atribute), spațiu alb al analizorului, atributul alfabetic.

Invocarea metodei `resetSyntax` face ca toate caracterele să fie considerate caractere obișnuite. Metoda `lineno` fără parametri întoarce numărul liniei curente.

Dar metoda centrală a clasei `StreamTokenizer` este metoda fără parametri `nextToken`, a cărei invocare are drept efect detectarea următoarei entități din șirul de caractere de la intrare. Acțiunile întreprinse sunt descrise în continuare.

Mai întâi sunt ignorate spațiile albe ale analizorului, cu următoarea excepție: dacă este întâlnit sfârșitul de linie și acesta este recunoscut de analizor, atunci entitatea detectată va fi `TT_EOL`.

Dacă este întâlnit un caracter numeric, se încearcă detectarea unei valori numerice. Dacă respectivul caracter este '-' dar următorul este nenerumeric, atunci entitatea detectată va fi acest caracter. În caz contrar, este detectată o valoare numerică, avansând în șirul de intrare până la întâlnirea unuia dintre următoarele caractere: al doilea '-', al doilea '.', primul caracter nenerumeric sau sfârșitul de fișier; drept urmare entitatea detectată este `TT_NUMBER`, iar valoarea ei se regăsește în câmpul `nval`.

Dacă este întâlnit un caracter alfabetic, va fi detectat identificatorul delimitat la dreapta de primul dintre următoarele tipuri de caractere: nealfabetic, nenerumeric sau sfârșit de fișier.

Dacă este întâlnit un caracter cu atributul comentariu, atunci toate caracterele până la sfârșitul de linie sau de fișier sunt ignorate, cu următoarea excepție: dacă este întâlnit sfârșitul de linie și acesta este recunoscut de analizor, atunci entitatea detectată va fi `TT_EOL`.

Dacă este întâlnit un caracter identificator de șir de caractere, entitatea detectată va fi `TT_WORD`, iar valoarea ei poate fi regăsită în `sval`. Valoarea constă din toate caracterele până la primul dintre următoarele: același caracter identificator de șir, sfârșit de linie sau sfârșit de fișier.

Dacă este întâlnită secvența `//` și analizorul recunoaște comentariile de sfârșit de linie, atunci sunt ignorate toate caracterele până la primul sfârșit de linie sau fișier.

Dacă este întâlnită secvența `/*` și analizorul recunoaște comentariile "cu asterisc", atunci sunt ignorate toate caracterele până la prima apariție a secvenței `*/` sau până la primul sfârșit de fișier.

Dacă nu suntem în nici una dintre cazurile enumerate, atunci suntem în prezența unui caracter obișnuit, entitatea întoarsă fiind `TT_WORD`.

Exemplul 8. Considerăm următorul program:

```
import java.io.*;
class Token {
    public static void main (String[] str) throws IOException {
        StreamTokenizer st =
            new StreamTokenizer(
                new InputStreamReader(System.in) );
        st.slashStarComments(true);
        st.slashSlashComments(true); st.eolIsSignificant(true);
        st.quoteChar('$'); st.ordinaryChars('A', 'A');
        for ( int entitate = st.nextToken();
            entitate != StreamTokenizer.TT_EOF;
            entitate = st.nextToken() ) {
            if (st.ttype == st.TT_NUMBER)
                IO.println("Numar : " + st.nval);
            else if (st.ttype == st.TT_WORD)
                IO.println("Identif : " + st.sval);
            else if (st.ttype == st.TT_EOL ) IO.println("EOL");
            else if ( (st.ttype == '$') || (st.ttype == '\\') )
                IO.println("Sir : " + st.sval);
            else if (st.ttype > 0)
                IO.println("Caracter : " + (char) st.ttype);
        }
    }
}
```

La executare sunt detectate următoarele entități în fluxul de intrare: identificatorii care nu conțin caracterul 'A', valorile numerice, șirurile de caractere (inclusiv cele cuprinse între două caractere '\$' succesive), comentariile (generale și de sfârșit de linie), sfârșitul de linie și cel de fișier.

10.5. Fișiere cu acces direct. Clasa `RandomAccessFile`

Clasa `RandomAccessFile` oferă posibilitatea de a lucra cu fișiere cu acces direct.

Accesul este la nivel de octet. Sunt permise atât scrieri, cât și citiri din fișier. Fișierul, nou sau existent, poate fi deschis fie pentru citire, fie pentru citire și scriere.

Accesul direct este asigurat prin gestionarea unui *indicator* care identifică poziția începând de la care se face o citire sau scriere; după o astfel de operație, indicatorul avansează cu numărul de octeți citați sau scriși. Este asigurată atât posibilitatea de a afla poziția indicatorului (reprezentând numărul de octeți din fișier aflați la stânga indicatorului), cât și posibilitatea de a da indicatorului o valoare, poziționându-l astfel la locul de unde dorim să aibă loc următoarea citire sau scriere.

Mai este oferită posibilitatea de a stabili o lungime pentru fișier. Această lungime nu este definitivă: o operație de scriere care face ca indicatorul să depășească sfârșitul curent al fișierului are ca urmare extinderea acestuia; în

schimb o poziționare dincolo de sfârșitul curent al fișierului nu schimbă lungimea fișierului.

Încercarea de a citi dincolo de sfârșitul fișierului conduce la lansarea excepției `EOFException`.

Clasa `RandomAccessFile` lasă programatorului o largă libertate de acțiune prin posibilitățile de a poziționa indicatorul și a schimba lungimea fișierului. Aceasta poate conduce însă la situații nedorite cum ar fi citirea unei informații nedefinite, așa cum vom preciza la descrierea metodelor; cade deci în sarcina programatorului să evite asemenea situații.

```
class RandomAccessFile extends Object
    implements DataOutput, DataInput
    RandomAccessFile(String nume, String mod)
        throws FileNotFoundException
    long getFilePointer()
    void seek(long poz)
    long length()
    void setLength(long lungime)
    implementările metodelor read și write din DataInput și DataOutput
    void close()
}
```

Constructorul creează un flux cu acces direct constituit dintr-un fișier al cărui nume este specificat în `nume`. Dacă fișierul nu există, el este creat. Fișierul poate fi folosit numai pentru citire (dacă `mod` este "r") sau atât pentru citire cât și pentru scriere (dacă `mod` este "rw").

Metoda `getFilePointer` întoarce poziția curentă a indicatorului, iar metoda `seek` poziționează indicatorul la poziția `poz`. Așa cum am menționat, poziționarea indicatorului dincolo de lungimea curentă a fișierului nu conduce la mărirea acestei lungimi curente, ci va fi realizată la o eventuală scriere începând de pe această poziție; în plus în această situație pot apărea porțiuni nedefinite ale fișierului.

Metoda `length` întoarce lungimea curentă a fișierului.

Metoda `setLength` stabilește lungime ca fiind noua lungime curentă a fișierului. Fie `lc` lungimea curentă a fișierului. Dacă `lungime < lc` atunci fișierul este trunchiat, iar dacă `lungime > lc` atunci fișierul este extins (drept urmare apare o porțiune nedefinită în fișier).

Metoda `close` închide fluxul; în continuare nu mai pot fi efectuate operații de citire/scriere și nici nu mai putem redeschide fluxul (dar putem redeschide fișierul printr-un nou flux).

Exemplul 9. În fișierul cu acces direct `raf.dat` sunt înscrise numerele întregi 0, 1, ..., 9 intercalate cu caracterele 'a', 'b', ..., 'g'. Apoi de 100 de ori este selectată aleator una dintre aceste informații și produsă la ieșire.


```

import java.io.*;
class RAF {
    public static void main(String[] sir) throws IOException {
        RandomAccessFile raf =
            new RandomAccessFile("raf.dat", "rw");
        int i, poz, nr; char ch;
        for (i=0, ch='a'; i<10; i++, ch++) {
            raf.writeInt(i); raf.writeChar(ch);
        }
        nr = 0;
        do {
            poz = (int) (30 * Math.random()); poz *= 2;
            raf.seek(poz); nr++;
            if (poz%6 == 0) IO.write(raf.readInt() + "\t");
            else if (poz%6 == 4) IO.write(raf.readChar() + "\t");
            else nr--;
        }
        while (nr<100);
        raf.close();
    }
}

```

10.6. Clasa File

Clasa File, care va fi descrisă doar pe scurt aici, permite specificarea numelor de fișiere și directoare într-un mod independent de sistemul de operare de pe mașina gazdă, precum și includerea în program a unor operații asupra fișierelor și directoarelor (creare, ștergere, renumire etc.).

Pentru separarea informațiilor ce identifică o cale, sistemele Windows și UNIX folosesc respectiv caracterele '\\' și '/'; într-o înșiruire de căi, cele două sisteme folosesc respectiv caracterele ';' și ':'. Pentru a permite independența de platformă, clasa File introduce pentru cele două tipuri de separatori constantele:

```

static String separator;
static String pathSeparator;

```

care primesc automat pe fiecare sistem reprezentarea corespunzătoare.

Unul dintre constructorii clasei File are un parametru de tipul String. La invocare putem să-i transmitem un șir de caractere ce reprezintă de exemplu numele unui fișier cu calea respectivă fie în reprezentarea specifică sistemului gazdă, fie în reprezentarea abstractă ce include constantele menționate.

Mai precizăm că un obiect de tip File ce specifică un fișier poate fi folosit ca argument pentru crearea de fluxuri de tipurile FileInputStream și FileOutputStream.

Exemplul 10. Revenim la *Exemplul 1*. Dorim să-l modificăm astfel încât programul Unu să creeze un subdirector dir în directorul curent și fișierul out.dat să fie plasat în acest subdirector. În plus cele două programe trebuie să fie independente de mașina gazdă.

Pentru aceasta, crearea fluxului g în clasa Unu poate fi realizată de exemplu astfel:

```

String s = File.separator;
File f = new File("dir"); IO.writeln( "" + f.mkdir() );
DataOutputStream g = new DataOutputStream(
    new FileOutputStream("dir"+s+"out.dat") );
unde prin invocarea metodei mkdir este efectiv creat directorul dir; metoda
întoarce true sau false, după cum crearea a reușit sau nu.

```

În clasa Doi, deschiderea fluxului este realizată prin:

```

File f = new File("dir"+File.separator+"out.dat");
DataInputStream g = new DataInputStream(
    new FileInputStream(f) );

```

11

INTERFEȚE
GRAFICE

Facilitatea de a crea interfețe grafice constituie unul dintre imperativele limbajelor de programare moderne. Adăugarea unei interfețe grafice la o aplicație poate contribui decisiv la succesul acesteia. În plus, programarea interfețelor grafice este foarte atractivă pentru majoritatea categoriilor de utilizatori.

Interfețele grafice, așa cum sugerează și numele lor, servesc în principal la interacțiunea dintre utilizator și obiectele grafice de pe ecran. Într-un program "clasic", la executarea unei instrucțiuni de citire se așteaptă ca să fie introduse datele necesare. Analog, un obiect grafic "așteaptă" să fie întreprinsă o acțiune asupra sa, pentru a răspunde cu executarea unui anumit cod. În ambele situații se așteaptă declanșarea unui eveniment (introducerea de date de intrare, respectiv acțiuni asupra unui obiect grafic) pentru ca aplicația să reacționeze într-un mod specificat de programator.

Limbajul Java pune la dispoziție o mare varietate de facilități de elaborare și prelucrare a interfețelor grafice. Aceste facilități sunt cuprinse în bibliotecile de clase AWT (*Abstract Windowing Toolkit*).

Nu ne propunem o prezentare nici măcar succintă a *tuturor* instrumentelor puse la dispoziție de Java. Vom insista numai asupra principiilor ce stau la baza lucrului cu interfețe grafice și vor fi prezentate numai modelele care, din punctul de vedere al autorului, sunt cele mai importante.

11.1. Un prim exemplu

Există trei aspecte care trebuie luate în considerare:

- crearea și afișarea pe ecran a unei suprafețe grafice pe care urmează a fi plasate diferite obiecte grafice; în plus pentru suprafața grafică trebuie stabilită o configurație inițială (un șablon), constând din dimensiunile ei, unele obiecte grafice plasate de la început pe ea etc.;

- o modalitate de poziționarea a obiectelor pe suprafața grafică;

- un mecanism prin care programul să răspundă la diferite acțiuni executate asupra obiectelor grafice (ca de exemplu un clic pe unul dintre acestea), conform dorințelor programatorului.

Vom ilustra cele de mai sus pe un exemplu, urmând ca apoi să facem o analiză mai amănunțită a modului de lucru cu interfețe grafice.

Exemplul urmărește înscrierea pe suprafața grafică a unui câmp care să conțină un număr și a unui câmp în care să apară valoarea cu care acesta trebuie incrementat, astfel încât la cerere numărul să fie actualizat conform incrementului.

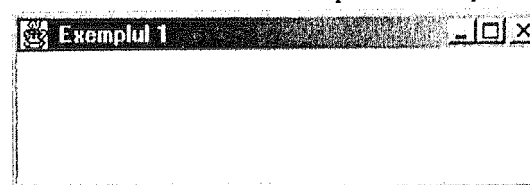
Urmărind cele trei aspecte menționate mai sus, aplicația va fi prezentată în trei pași.

Exemplul 1. Considerăm programul următor:

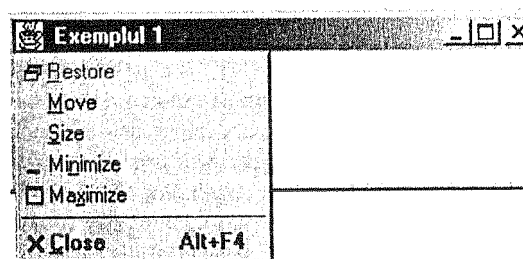
```
import java.awt.*;
public class Ex1 {
    public static void main(String[] args) {
        F f = new F(" Exemplul 1 ");
        f.setSize(300,100); f.setVisible(true);
    }
}

class F extends Frame {
    F(String s) { setTitle(s); }
}
```

Executarea lui face ca pe ecran să apară:



cu un aspect desigur familiar cititorilor. De exemplu printr-un clic pe icon-ul Java (stânga sus), va apărea:



Prezentăm în continuare pe scurt explicațiile necesare.

Pachetul `java.awt` include (evident printre altele) și următoarea ierarhie de clase:

```

Component (clasă abstractă)
  Container
    Window
      Frame
  
```

Clasa abstractă `Component`, aflată în rădăcina arborelui claselor din AWT, definește interfața general valabilă pentru toate componentele grafice. Obiectele de tip `Component` se numesc *componente* (grafice).

Un *container* (obiect de tipul `Container`) este o componentă care poate include alte componente AWT; includerea se face conform unei structuri de listă.

O *fereastră AWT* (obiect de tipul `Window`) este o fereastră fără margini și fără bară de meniuri.

Un *cadru* (obiect de tipul `Frame`) este o fereastră AWT cu margini, titlu și bară de meniuri.

În exemplul de mai sus, la crearea cadrului s-a specificat și un titlu. Dimensiunile cadrului, în număr de pixeli, au fost precizate prin invocarea metodei `setSize`. Crearea suprafeței grafice nu implică automat afișarea ei pe ecran; de aceea a fost necesară invocarea metodei `setVisible` cu argumentul `true`.

Este de notat că nu putem închide suprafața grafică prin clic pe unul dintre icon-uri, ci doar în mod forțat.

Exemplul 2. Continuăm exemplul precedent prin includerea de componente în container:

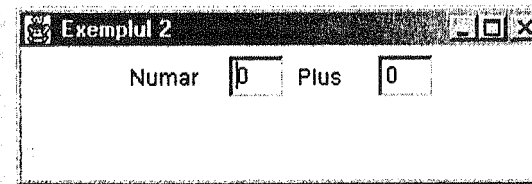
```

import java.awt.*;
public class Ex2 {
    public static void main(String[] args) {
        F f = new F(" Exemplul 2 ");
        f.setSize(300,100); f.setVisible(true);
    }
}

class F extends Frame {
    Label L1,L2; TextField T1,T2;

    F(String s) {
        setTitle(s); setLayout(new FlowLayout());
        L1 = new Label("Numar"); add(L1);
        T1 = new TextField("0"); add(T1);
        L2 = new Label("Plus"); add(L2);
        T2 = new TextField("0"); add(T2);
    }
}
  
```

La executare va apărea pe ecran:



Au fost incluse următoarele componente:

- două *etichete* (*label*), pe care au fost scrise textele "Numar" și "Plus";
- două *câmpuri de text* (*textfield*) al căror conținut a fost inițializat cu "0".

O componentă de tip `TextField` permite editarea unei singure linii de text. O *etichetă* (componentă de tip `Label`) permite afișarea unei linii de text (neditabilă) în container și este folosită de obicei pentru a pune lângă o componentă un text explicativ, așa cum am făcut mai sus "atașând" câmpurilor de text pentru număr și increment etichetele "Numar" și "Plus".

Adăugarea în container a componentelor de mai sus s-a făcut cu ajutorul metodei `add` din clasa `Container`.

Mai intervine și modul de poziționare a componentelor în container. Pentru aceasta trebuie folosit un *gestionar de poziționare* (*layout manager*).

În pachetul `java.awt` apare interfața `LayoutManager`. Ea este implementată de mai multe clase din același pachet. Fiecărui container i se asociază implicit un astfel de gestionar; de exemplu clasei `Frame` i se asociază implicit gestionarul `BorderLayout` (o clasă din `java.awt` ce implementează interfața `LayoutManager`), asociere care nu a avut nici o implicație în *Exemplul 1*, deoarece nu au fost adăugate componente în container.

Pentru *Exemplul 2* am ales gestionarul de poziționare `FlowLayout`. Acest gestionar plasează componentele în container conform regulii "de la stânga la dreapta și de sus în jos"; drept urmare este relevantă ordinea în care componentele sunt adăugate (prin `add`) în container. Precizarea gestionarului de poziționare se face prin invocarea metodei `setLayout` din clasa `Container`.

Observație. Modificarea dimensiunilor suprafeței grafice nu alterează regula de poziționare stabilită prin gestionarul ales.

Trecem la ultima etapă a rezolvării problemei enunțate. Apar mai multe elemente noi.

După cum se poate verifica pe exemplul precedent, putem edita informația dintr-un câmp de text, ceea ce nu este însă suficient: trebuie să putem citi informația nou introdusă în câmp și să putem scrie o informație într-un astfel de câmp. Pe de altă parte, programul trebuie să fie înștiințat de modificarea câmpului, pentru a trece la citirea valorii înscrise în câmp și la prelucrarea ei.

Pentru aceasta vom mai introduce o componentă și anume un *buton* (instanță a clasei `Button` ce extinde `Component`). Un buton are proprietatea că acționarea sa conduce la apariția unui așa-numit *eveniment*. Mai trebuie ca

programul să fie capabil să sesizeze apariția unui astfel de eveniment și să treacă la executarea acțiunilor corespunzătoare. Vom aduce precizări după prezentarea programului în noua sa formă.

Exemplul 3. Variabilele *c1* și *c2* vor memora (ca întregi) șirurile de caractere din câmpurile de text corespunzătoare numărului și respectiv incrementului.

```
import java.awt.*;
import java.awt.event.*;

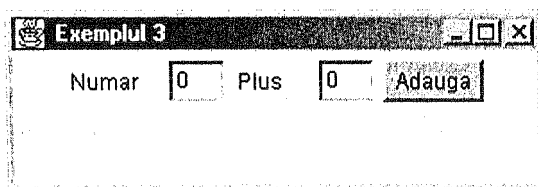
public class Ex3 {
    public static void main(String[] args) {
        F f = new F(" Exemplul 3 ");
        f.setSize(300,100); f.setVisible(true);
    }

    class F extends Frame implements ActionListener {
        Label L1,L2; TextField T1,T2; Button B;
        int c1,c2;

        F(String s) {
            setTitle(s); setLayout(new FlowLayout());
            L1 = new Label("Numar"); add(L1);
            T1 = new TextField("0"); add(T1);
            L2 = new Label("Plus "); add(L2);
            T2 = new TextField("0"); add(T2);
            B = new Button("Adauga"); add(B);
            B.addActionListener(this);
        }

        public void actionPerformed(ActionEvent e) {
            c2 = Integer.parseInt( T2.getText() );
            c1 += c2; T1.setText( Integer.toString(c1) );
        }
    }
}
```

Suprafața grafică are acum următoarea formă:



Cititorul poate verifica faptul că acum lucrurile se desfășoară în modul dorit: dacă introducem un întreg în câmpul corespunzător incrementului și apăsăm pe buton, câmpul de text corespunzător numărului este incrementat cu acest întreg.

Se observă că pe lângă crearea și afișarea butonului împreună cu o etichetă ce îl identifică, este invocată metoda `addActionListener`, iar în clasa *F* a mai fost adăugată metoda `actionPerformed`.

Prin invocarea metodei `addActionListener`, obiectul curent a devenit (a fost înregistrat ca) *gestionar (listener) al evenimentelor* ce pot fi declanșate de acționarea butonului. Aceasta este posibil numai dacă gestionarul este un obiect de tipul unei clase ce implementează interfața `ActionListener`, specificând o acțiune concretă pentru unica metodă (și anume `actionPerformed`) a acesteia.

La acționarea butonului se execută "automat" metoda `actionPerformed`. Metoda prevede următoarele: citirea în *c2* a întregului din câmpul de text corespunzător incrementului, adăugarea acestei valori lui *c1* și înscrierea valorii lui *c1* în câmpul de text corespunzător numărului.

Prezentăm în continuare câteva ierarhii de clase pentru a putea localiza tipurile și metodele folosite în exemplul de mai sus.

O primă ierarhie este următoarea:

Object	(din pachetul java.lang)
EventObject	(din pachetul java.util)
AWTEvent	(clasă abstractă din pachetul java.awt)
ActionEvent	(din pachetul java.awt.event)

Se observă că nu am folosit în nici un mod parametrul *e* de tip `ActionEvent` din metoda `actionPerformed`. Acest parametru este folosit în general la identificarea evenimentului ce a condus la invocarea metodei; în exemplul nostru acest lucru nu este necesar, deoarece există un unic eveniment ce trebuie gestionat: acționarea butonului. Vom reveni cu precizări în subcapitolul următor.

O a doua ierarhie este:

EventListener	(interfață din pachetul java.util)
ActionListener	(interfață din pachetul java.awt.event)

În interfața `ActionListener` este anunțată metoda:

```
public void actionPerformed(ActionEvent e)
```

care deci trebuie implementată în clasa care o folosește. De aceea clasa *F* din **Exemplul 3** implementează interfața `ActionListener`, specificând o acțiune concretă pentru această metodă.

11.2. Schema generală de lucru cu AWT

Plecând de la exemplul de mai sus, prezentăm principalele aspecte legate de lucrul cu interfețe grafice.

• Modele de suprafață grafică (containere)

Principalele modele de suprafețe grafice sunt cele asociate claselor din următoarea ierarhie din pachetul `java.awt`:

Component (clasă abstractă)
Container
Window
Frame
Dialog
ScrollPane
Panel
Applet

O *fereastră AWT* (obiect de tip `Window`) este o suprafață grafică dreptunghiulară fără margini și fără bară de meniuri.

După cum am văzut, un *cadru* extinde o fereastră prin adăugarea de margini, a unui titlu și a unei bare de meniuri.

O *fereastră de dialog* (obiect de tip `Dialog`) extinde o fereastră AWT prin adăugarea de margini și a unui titlu; ea este folosită în principal pentru a prelua informații de la utilizator.

O *fereastră de defilare* (obiect de tipul `ScrollPane`) implementează un cadru de defilare pe orizontală și verticală pentru o componentă inclusă în container.

Un *panou* (obiect de tip `Panel`) este folosit pentru a grupa mai multe componente.

Despre *applet-uri* (obiecte de tipul `Applet`) vom vorbi în ultimul capitol al acestei cărți.

Precizăm că numai componentele de tip `ScrollPane`, `Panel` și `Applet` pot fi adăugate în alte containere, *deci o fereastră nu poate fi adăugată unui container*. Posibilitatea ca un container să fie inclus în alt container face ca noțiunile de *container fiu* și *container tată* să aibă o semnificație evidentă.

• Componente elementare

Componentele elementare (numite și controale grafice) permit interacțiunea cu utilizatorul. Ele corespund claselor ce apar subliniate în următoarea ierarhie din pachetul `java.awt`:

Component	
Button	buton
Canvas	suprafață de desenare
Checkbox	căsuță de opțiuni
Choice	listă de opțiuni cu alegere unică
Label	etichetă
List	listă de opțiuni cu alegere multiplă
Scrollbar	bară de defilare
TextComponent	
TextArea	arie de text
TextField	câmp de text

Componentele elementare pot fi adăugate containerelor. Fie `C` un container și `c` o componentă elementară. Adăugarea lui `c` în `C` se realizează prin: `C.add(c)`; adică prin invocarea metodei `add` declarată în clasa `Container` prin: `Component add(Component c)` metodă ce realizează adăugarea componentei la sfârșitul containerului.

• Gestionari de poziționare

Pentru poziționarea componentelor pe o suprafață grafică avem posibilitatea de a folosi mai mulți gestionari de poziționare, care oferă diferite variante de adăugare a componentelor pe suprafața grafică. Câțiva dintre acești gestionari corespund claselor din următoarea ierarhie din pachetul `java.awt`:

LayoutManager	(interfață)
FlowLayout	
GridLayout	
LayoutManager2	(interfață)
CardLayout	
BorderLayout	
GridBagLayout	

• Evenimente

În lucrul cu interfețe grafice, o acțiune asupra unei componente grafice declanșează un *eveniment*. Un eveniment este un obiect al unei clase `tipEvent` din următoarea ierarhie de clase:

Object	(din pachetul <code>java.lang</code>)
EventObject	(din pachetul <code>java.util</code>)
AWTEvent	(din pachetul <code>java.awt</code>)
tipEvent	(din pachetul <code>java.awt.event</code>)

unde *tip* este tipul evenimentului. Principalele tipuri de evenimente sunt:

<i>tip</i>	<i>Apare de exemplu la:</i>
Action	acțiuni asupra unei componente elementare (buton etc.)
Adjustment	acțiuni asupra unei componente de tip Scrollbar
Component	deplasări, redimensionări, ascunderi (componenta devine invizibilă)
Container	adăugare, ștergere
Focus	obținerea/pierderea controlului asupra tastaturii
Item	selecția sau deselecția de componente (liste, căsuțe de opțiuni etc.)
Key	apăsarea sau eliberarea unei taste
Mouse	clic, apăsare, eliberare, ieșire/intrare în componentă
MouseMotion	mișcare, târare (drag)
Text	modificarea textului din componentă
Window	închidere, minimizare/maximizare

Fiecare dintre clasele de mai sus are desigur constructorii, câmpurile și metodele proprii. Prezentarea componentelor elementare se va face într-unul dintre subcapitolele următoare.

• Înregistrarea unui obiect ca gestionar al unui eveniment

Înregistrarea este realizată prin invocarea metodelor `addTipListener`:

```
public void addTipListener(TipListener a)
```

unde *a* este de tipul unei clase ce implementează interfața `TipListener` sau extinde clasa abstractă `TipAdapter` (vezi mai jos).

Astfel, în *Exemplul 3*, am inclus invocarea:

```
B.addActionListener(this)
```

prin care obiectul (aici cadrul) curent este înregistrat ca gestionar la acționări asupra butonului.

Următorul tabel localizează metodele `addTipListener`:

<i>tip</i>	<i>Clasele din java.awt în care apare metoda</i>
Action	List, Button, TextField
Adjustment	Scrollbar
Component	Component
Container	Container
Focus	Component
Item	Checkbox, List, Choice
Key	Component
Mouse, MouseMotion	Component
Text	TextComponent
Window	Window

Un gestionar de evenimente poate fi asociat mai multor evenimente.

De asemenea, unei componente simple îi putem asocia mai mulți gestionari de evenimente; în acest caz, declanșarea evenimentului este comunicată *tuturor* gestionarilor.

Mai menționăm că pentru fiecare metodă `addTipListener` există și o metodă "reciprocă" `removeTipListener`, prin care un gestionar de evenimente încetează de a mai fi asociat unei componente simple.

• Interfețe și clase abstracte asociate evenimentelor

Fiecărui tip *tip* de eveniment îi este asociat o interfață `TipListener` în pachetul `java.awt.event`. Cu excepția tipurilor Action, Adjustment, Item și Text, aceste interfețe sunt implementate în același pachet de clasele abstracte `TipAdapter` corespunzătoare; metodele din aceste clase nu sunt abstracte, dar prevăd acțiunea nulă.

<code>EventListener</code>	(interfață din pachetul <code>java.util</code>)
<code>TipListener</code>	(interfață din pachetul <code>java.awt.event</code>)
<code>TipAdapter</code>	(clasă abstractă din pachetul <code>java.awt.event</code>)

Metodele din aceste interfețe și clase sunt declarate prin:

```
public void nume_metoda(TipEvent e)
```

și corespund câte unui "subeveniment" al evenimentului identificat prin *tip*. De exemplu un eveniment de tipul `MouseMotion`, care corespunde generic acționării (cu mouse-ul) asupra unei componente, are ca subevenimente târârea, apăsarea, eliberarea mouse-ului, ca și intrarea și ieșirea din componentă. Pentru fiecare subeveniment este prevăzută câte o metodă în interfața `MouseMotionListener` și în clasa `MouseMotionAdapter`.

• Gestionarea evenimentelor

Dacă dorim ca un obiect *Ob* de tipul unei clase *C* să devină (să fie înregistrat ca) gestionar pentru un eveniment *tip* și să prevedem o acțiune la declanșarea evenimentului, trebuie ca *C* să implementeze interfața `TipListener` sau să extindă clasa `TipAdapter`, specificând acțiunea dorită.

Cum alegem între a implementa interfețe și a extinde clase abstracte?

Știm că o clasă poate extinde o singură clasă, dar implementa oricâte interfețe. Din acest punct de vedere strategia de a implementa interfețe este mai avantajoasă. Apare însă dezavantajul că trebuie implementate *toate* metodele anunțate de interfață.

Prin extinderea claselor abstracte avem posibilitatea de a redefini numai metodele corespunzătoare unor evenimente pentru care dorim să întreprindem o acțiune efectivă. Însă în acest caz nu mai putem extinde și alte clase.

Mai avem și posibilitatea de a implementa/extinde *implicit* interfețe/clase prin definirea de clase interne, mecanism care la început pare greoi.

Concluzia este că programatorul trebuie să-și aleagă varianta care i se pare mai potrivită și/sau cu care îi este mai ușor să lucreze.

Să revenim la *Exemplul 3*. Pentru acționări asupra butonului am ales să implementăm interfața `ActionListener` (ce prevede doar metoda `actionPerformed`).

Vom completa programul din acest exemplu deoarece are totuși un inconvenient: nu permite terminarea aplicației decât în mod forțat. Aceasta se datorează faptului că și închiderea unei ferestre (în cazul nostru a suprafeței grafice) declanșează și ea un eveniment, pentru care trebuie precizată o acțiune, altfel această acțiune fiind cea nulă. O rezolvare a acestui inconvenient este plasarea în constructorul clasei `F` a instrucțiunii:

```
addWindowListener(
    new WindowAdapter() {
        public void windowClosing(WindowEvent e)
            { System.exit(0); }
    }
); // end addWindowListener
```

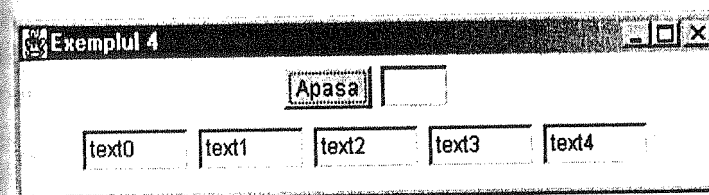
cu următoarele observații:

- de această dată am ales să extindem clasa `WindowAdapter`;
- am redefinit numai metoda `windowClosing`, deoarece ne interesează doar subevenimentul de închidere al ferestrei;
- pentru celelalte subevenimente (ca de exemplu activarea sau dezactivarea ferestrei) este moștenită acțiunea nulă din metodele clasei `WindowAdapter`;
- extinderea clasei s-a făcut în mod implicit, apelând la clase interne.

Mai punem în evidență și un alt aspect important legat de gestionarea evenimentelor. În exemplul precedent gestionarul evenimentelor apare în interiorul aplicației (interfața `ActionListener` a fost extinsă chiar de clasa `F` care a creat cadrul). În general însă este indicat ca *implementarea (respectiv extinderea) să fie separată de aplicație*. Această disciplină de lucru devine chiar imperativă dacă dorim ca unui eveniment să-i asociem mai mulți gestionari.

Înainte de a prezenta un exemplu menit să ilustreze câteva dintre elementele prezentate mai sus, *invităm pe cititor să remarce aspectele comune ale tratării excepțiilor și gestionării evenimentelor*.

Exemplul 4. Dorim să creăm o interfață grafică cu următorul aspect (look):



în care apar un buton, un câmp de text, precum și alte 5 câmpuri de text. Dorim ca:

- cele 5 câmpuri de text să fie grupate; vom folosi modelul `Panel`;
- la acționări succesive ale butonului, caracterele din câmpurile de text să treacă "de la literă mică la literă mare" și invers; în plus, la fiecare apăsare a butonului dorim ca la ieșirea standard să fie produs mesajul "Buton apasat";
- la introducerea unui șir de caractere (pentru simplificare) nevid în primul câmp de text și apăsarea tastei `<Enter>`, caracterele din celelalte 5 câmpuri de text să treacă "de la literă mică la literă mare" sau invers, după cum șirul de caractere începe cu o literă mare sau una mică.

Programul este următorul:

```
import java.awt.*;
import java.awt.event.*;
class Ex4 {
    public static void main(String[] s) {
        F f = new F("Exemplul 4");
        P p = new P(5); f.add(p);
        HModif HM = new HModif(f.b, f.t, p);
        f.b.addActionListener(HM); f.t.addActionListener(HM);
        f.setSize(400, 100); f.setVisible(true);
    }
}

class F extends Frame implements ActionListener {
    Button b; TextField t;
    F(String titlu) {
        setTitle(titlu); setLayout(new FlowLayout());
        b = new Button("Apasa"); add(b);
        b.addActionListener(this);
        t = new TextField(" "); add(t);
        addWindowListener(
            new WindowAdapter() {
                public void windowClosing(WindowEvent e)
                    { System.exit(0); }
            }
        );
    }

    public void actionPerformed(ActionEvent e) {
        System.out.println("Buton apasat");
    }
}
```

```

class P extends Panel {
    TextField[] tf;
    P(int nr) {
        tf = new TextField[nr];
        for (int i=0; i<nr; i++) {
            tf[i] = new TextField("text"+i); add(tf[i]);
        }
    }
}

class HModif implements ActionListener {
    Button b; TextField t; P p;
    String s,s0; boolean bool; char c;
    HModif(Button b, TextField t, P p) {
        this.b = b; this.t = t; this.p = p;
    }
    void modif(boolean bool) {
        for (int i=0; i<p.tf.length; i++) {
            s = p.tf[i].getText();
            if (bool) s = s.toLowerCase();
            else s = s.toUpperCase();
            p.tf[i].setText(s);
        }
    }
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() instanceof Button)
            if ( e.getActionCommand().equals("Apasa") ) {
                modif(bool); bool = !bool;
            }
        else ;
        else {
            s = t.getText(); t.setText(""); s0 = s.substring(0,1);
            boolean bool = s0.equals(s0.toLowerCase());
            modif(bool);
        }
    }
}

```

A fost creat un cadru *f* în care s-au adăugat un buton *b*, un câmp de text *t* și un panou *p* ce grupează 5 câmpuri de text.

Butonului *i* s-au asociat doi gestionari: obiectul *f* de tip *F* și obiectul *HM* de tipul *HModif*. Constructorului clasei *HModif* i se transmit ca argumente butonul, câmpul de text și panoul (o variantă este să folosim un singur argument și anume întreg cadrul *f*).

Câmpului de text *t* i s-a asociat de asemenea gestionarul *HM*.

Suntem deci în prezența unui exemplu în care unei componente grafice *i* se asociază mai mulți gestionari, iar un gestionar este asociat mai multor componente grafice.

La acționarea butonului, cei doi gestionari asociați întreprind acțiunile din metodele *actionPerformed* corespunzătoare: este tipărit mesajul "Buton apasat", respectiv (prin invocarea metodei *modif*) se efectuează modificările anunțate asupra câmpurilor de text din panou. Se observă deci că, așa cum am anunțat anterior, la apariția unui eveniment intră în acțiune toți gestionarii asociați.

La introducerea unui text (nevid!) în câmpul de text *t* și apăsarea tastei <Enter>, gestionarul *HM* asociat modifică, tot prin intermediul metodei *modif*, cele 5 câmpuri de text din panoul *p*.

Întrucât gestionarul *HM* este asociat atât butonului *b*, cât și câmpului de text *t*, metoda *actionPerformed* din clasa *HModif* este invocată în ambele cazuri. Pentru a distinge care dintre cele două evenimente a fost declanșat, folosim obiectul *e* de tip *ActionEvent* transmis metodei ca argument.

Pentru identificarea evenimentului care a condus la invocarea metodei *actionPerformed* am folosit metoda *getSource* din clasa *EventObject*, care întoarce obiectul creat prin declanșarea evenimentului:

```
public Object getSource()
```

Am mai ținut cont că în general pot exista mai multe surse de același tip (în cazul nostru butoane) care să conducă la apariția evenimentului; este posibil ca pentru butoane diferite să fie specificate acțiuni diferite. De aceea, prin invocarea metodei *getActionCommand* din clasa *ActionEvent*:

```
public String getActionCommand()
```

care întoarce șirul de caractere asociat acțiunii (în cazul nostru eticheta butonului acționat), ne-am asigurat că este într-adevăr vorba de butonul a cărui etichetă este "Apasa".

11.3. Containere. Fonturi și culori

În acest subcapitol vom prezenta diferite tipuri de componente, dintre cele menționate anterior.

Vom începe însă prin a prezenta modul în care putem alege fonturi pentru editarea textelor, precum și culori pentru fundal și prim plan.

• Fonturi

Pentru stabilirea fontului folosit în textele ce pot fi editate pe o suprafață grafică, Java pune la dispoziție clasa *Font* din pachetul *java.awt*:

```
public class Font extends Object
```

Ne rezumăm la modalitatea cea-mai simplă de lucru cu această clasă.

Prezentăm constructorul:

`Font(String nume, int stil, int dim)`

- nume este numele fontului (de exemplu "Times New Roman", "Helvetica" etc.);
- stil este stilul ales pentru font; vom folosi pentru stil doar câmpurile întregi și statice ale clasei `Font`: `PLAIN`, `BOLD` și `ITALIC`;
- dim este mărimea fontului.

Modul tipic de alegere a unui font este invocarea metodei `setFont` a clasei `Component`:

public void `setFont(Font f)`

prin:

`setFont( new Font(nume, stil, dim) );`

• Culori

Facilitățile de alegere a unei culori pentru fundal și editare de texte sunt cuprinse în clasa `Color` din pachetul `java.awt`:

public class `Color` extends `Object`

care pune la dispoziție, pe lângă multe alte facilități, următoarele constante publice de tip `Color`:

`white`, `lightGray`, `gray`, `darkGray`, `black`, `red`, `pink`, `orange`, `yellow`, `green`, `magenta`, `cyan` și `blue`

precum și metodele:

public `Color brighter()`

public `Color darker()`

care creează o culoare mai deschisă, respectiv mai închisă, decât cea care a invocat metoda.

• Clasa Component

Dintre metodele, toate publice, ale clasei `Component` din `java.awt`:

public abstract class `Component`

menționăm următoarele:

void `setName(String nume)`

atribuie componentei numele `nume`;

String `getName()`

întoarce numele componentei;

void `setBackground(Color c)`

stabilește culoarea fundalului componentei;

void `setForeground(Color c)`

stabilește culoarea prim planului componentei;

void `setFont(Font f)`

stabilește fontul pentru componentă;

void `requestFocus()`

componenta curentă primește focus-ul (controlul asupra tastaturii), cu condiția ca ea să fie vizibilă; vezi și subcapitolul 11.6;

void `setLocation(int x, int y)`

stabilește poziția colțului din stânga sus al componentei.

Următoarele metode:

void `addKeyListener(KeyListener c)`

void `removeKeyListener(KeyListener c)`

unde `tip` este `Component`, `Focus`, `Key`, `Mouse` sau `MouseMotion` (vezi subcapitolul 1);

void `setVisible(boolean b)`

au fost prezentate în subcapitolul 2.

• Clasa Container

Clasa `Container` din pachetul `java.awt`:

public class `Container` extends `Component`

are constructorul:

`Container()`

Menționăm următoarele metode publice ale clasei:

`Component add(Component comp)`

adaugă componenta `comp` containerului;

void `remove(Component comp)`

înlătură componenta `comp` din container;

void `setLayout(LayoutManager g)`

asociază containerului gestionarul de poziționare `g`;

void `addContainerListener(ContainerListener c)`

asociază containerului gestionarul de evenimente `c`;

void `removeContainerListener(ContainerListener c)`

obiectul `c` încetează de a mai fi gestionar de evenimente pentru container.

• Clasa Frame

Clasa `Frame` apare de asemenea în pachetul `java.awt`:

public class `Frame` extends `Window`

Ne mărginim la a menționa constructorii:

`Frame()`

`Frame(String titlu)`

și metoda:

void `setTitle(String titlu)`

cu semnificații evidente.

• Clasa Panel

Clasa Panel din pachetul java.awt:

```
public class Panel extends Container
```

prevede constructorii:

```
Panel()
```

```
Panel(LayoutManager lm)
```

unde al doilea constructor asociază panoului un gestionar de poziționare.

11.4. Componente elementare

În acest subcapitol vom prezenta componentele elementare, precizând pentru fiecare constructor și metode, precum și tipul evenimentelor pe care le pot declanșa acțiunile asupra lor; pentru fiecare astfel de tip de eveniment va trebui specificată acțiunile dorite de utilizator, prin mecanismele prezentate în subcapitolul 2. Pentru simplificare, în exemple nu vom specifica acțiuni pentru toate tipurile de evenimente (de pildă ferestrele vor putea fi închise doar forțat), ci numai pentru cele relevante pentru exemplul considerat.

• Etichete. Clasa Label

Clasa:

```
public class Label extends Component
```

are constructorii:

```
Label()
```

construiește o etichetă cu text vid;

```
Label(String text)
```

construiește o etichetă cu textul specificat, aliniat la stânga

și constantele publice întregii RIGHT, CENTER și LEFT, prin care se poate specifica plasarea etichetei.

Dintre metodele clasei Label menționăm:

```
public void setText(String text)
```

```
public String getText()
```

```
public void setAlignment(int i)
```

prin care este înscris un text pe etichetă, este întors textul asociat etichetei, respectiv se specifică alinierea textului în etichetă (precizând pentru argument una dintre constantele clasei). Alinierea implicită este la stânga.

• Butoane. Clasa Button

Clasa:

```
public class Button extends Component
```

oferă facilități de lucru cu butoane. Un clic pe un buton declanșează un eveniment de tip `ActionEvent`.

Prezentăm în continuare constructorii și câteva metode ale clasei `Button`, toate publice:

```
Button()
```

construiește un buton fără etichetă;

```
Button(String label)
```

construiește un buton pe care este înscrisă eticheta `label`;

```
String getLabel()
```

întoarce eticheta înscrisă pe buton;

```
void setLabel(String label)
```

înscrie pe buton eticheta `label`.

Acționarea unui buton (prin clic cu mouse-ul) declanșează un eveniment de tipul `Action`.

• Câmpuri și arii de text de text. Clasele `TextField` și `TextArea`

Începem prin a prezenta clasa:

```
public class TextComponent extends Component
```

care este superclasa claselor `TextField` și `TextArea`.

Principalele metode publice ale clasei sunt:

```
void setText(String text)
```

setează textul afișat de componentă;

```
String getText()
```

întoarce textul afișat de componentă;

```
void setEditable(boolean b)
```

stabilește dacă textul atașat componentei este sau nu editabil;

```
void select(int p, int u)
```

selectează subtextul ce începe de pe poziția `p` și se termină pe poziția `u-1`.

Trebuie ca $p \leq u$; dacă $p = u$, atunci întreg textul este deselectat;

```
void selectAll()
```

selectează întreg textul;

```
String getSelectedText()
```

întoarce textul selectat;

```
int getSelectionStart()
```

întoarce poziția de început a zonei selectate;

```
int getSelectionEnd()
```

întoarce poziția de sfârșit a zonei selectate;

Mai specificăm că un subtext poate fi selectat și cu ajutorul mouse-ului.

Modificarea textului din componentă declanșează un eveniment de tipul

`Text`.

Clasa:

```
public class TextField extends TextComponent
```

oferă facilități de lucru cu *câmpurile de text*, ce permit editarea unei unice linii de text.

Prezentăm în continuare constructorii și câteva metode publice ale clasei:

```
TextField()
    construiește un câmp de text;
TextField(String text)
    construiește un câmp de text inițializat cu text;
TextField(int col)
    construiește un câmp de text cu col coloane;
TextField(String text, int col)
    construiește un câmp de text cu col coloane, inițializat cu text;
void setText(String text)
    înscrie textul text în câmp;
void addActionListener(ActionListener l)
void removeActionListener(ActionListener l)
```

La acționarea unei taste în câmpul de text, va fi declanșat un eveniment de tipul Key. La apăsarea tastei <Enter> este declanșat un eveniment de tipul Action.

Clasa:

```
public class TextArea extends TextComponent
oferă facilități de lucru cu arii de text, care generalizează câmpurile de text prin
acceptarea mai multor linii de text. De fapt este vorba de un unic text, în care
tastarea lui <Enter> pe o poziție are ca efect trecerea la o linie nouă, conform
noului caracter introdus. Putem atașa ariei o bară de defilare orizontală, una
verticală sau ambele aceste bare; utilitatea lor apare atunci când textul de pe o linie
este mai lung decât numărul de coloane stabilit pentru arie, respectiv când numărul
efectiv de linii este mai mare decât numărul de linii stabilit pentru aria de text.
```

Clasa TextArea are câmpurile constante, publice și întregi:

```
SCROLLBARS_NONE
SCROLLBARS_VERTICAL_ONLY
SCROLLBARS_HORIZONTAL_ONLY
SCROLLBARS_BOTH
prin care putem stabili sistemul de bare de defilare ales.
```

Constructorii clasei sunt:

```
public TextArea()
    construiește o arie de text cu ambele bare de defilare;
public TextArea(String text)
    construiește o arie de text cu ambele bare de defilare, în care este înscris textul
    text;
public TextArea(int m, int n)
    construiește o arie de text cu ambele bare de defilare, având m linii și n
    coloane;
```

```
public TextArea(String text, int m, int n)
    construiește o arie de text cu ambele bare de defilare, având m linii și n
    coloane și în care este înscris textul text;
public TextArea(String text, int m, int n, int defilare)
    în plus față de constructorul precedent, este specificat sistemul de bare de
    defilare ales; defilare trebuie să fie una dintre cele 4 constante ale clasei.
```

Dintre metodele, toate publice, ale clasei TextArea, prezentăm următoarele:

```
void setRows(int m)
    stabilește la m numărul de linii ale ariei de text;
void setColumns(int n)
    stabilește la n numărul de coloane ale ariei de text;
void insert(String str, int poz)
    inserează textul str la poziția poz din aria de text;
void append(String str)
    adaugă textul str ariei de text;
void replaceRange(String str, int p, int u)
    înlocuiește textul dintre pozițiile p și u cu textul str.
```

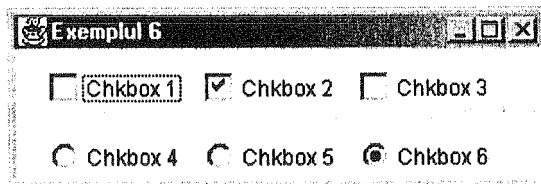
Pozițiile de mai sus se referă la întreg textul; așa cum am precizat, este vorba de un unic text, în care apariția unui terminator de linie determină în aria de text deplasarea pe următoarea linie.

Acționarea unei taste în câmpul de text conduce la declanșarea unui eveniment de tipul Key.

Exemplul 5. Vom afișa un câmp de text, o arie de text și un buton. La apăsarea tastei <Enter> în câmpul de text, conținutul său este adăugat ariei de text la sfârșitul acesteia. La apăsarea butonului, zona selectată din aria de text este copiată în câmpul de text și la ieșirea standard, după care este eliminată din aria de text.

```
import java.awt.*;
import java.awt.event.*;
class T_Area {
    public static void main(String[] s) {
        F f = new F("Exemplul 5");
        f.setSize(200,200); f.setVisible(true);
    }
}

class F extends Frame implements ActionListener {
    TextField tf; TextArea ta; Button b;
    F(String titlu) {
        setTitle(titlu); setLayout(new FlowLayout());
        tf = new TextField(20); add(tf);
        tf.addActionListener(this);
```



• Liste de opțiuni cu alegere unică. Clasa `Choice`

Clasa:

`public class Choice` extends `Component` permite crearea unei liste ce poate fi derulată (unui meniu "pop-up") de elemente (item-uri). Elementele unei astfel de liste sunt șiruri de caractere. La fiecare moment de timp, doar unul dintre elementele din listă este selectat (de unde și numele de listă de opțiuni cu alegere unică); elementul selectat apare în titlul listei. Selectarea se face prin clic cu mouse-ul pe elementul vizat.

Constructorul:

`public Choice()` creează un meniu pop-up pentru care prima informație ce va fi introdusă este automat cea selectată.

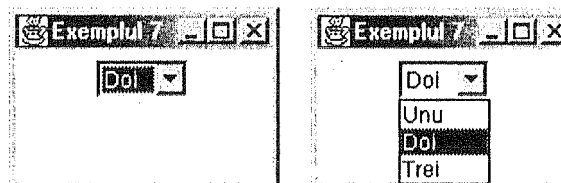
Clasa pune la dispoziție mai multe metode publice, printre care:

```
int getItemCount()
    întoarce numărul de elemente din listă;
String getItem(int index)
    întoarce elementul din listă cu numărul de ordine index;
void addItem(String item)
    adaugă un nou element listei;
void insert(String item, int index)
    inserează un nou element pe poziția precizată;
void remove(String item)
    șterge din listă prima apariție a elementului item;
void remove(int poz)
    șterge din listă elementul de pe poziția poz;
void removeAll()
    șterge toate elementele listei;
String getSelectedItem()
    întoarce elementul selectat al listei;
int getSelectedIndex()
    întoarce poziția elementului selectat.
```

O acțiune asupra unei liste de opțiuni cu alegere unică declanșează un eveniment de tipul `Item`.

Exemplul 7. Acest exemplu urmărește doar prezentarea modului în care apare o listă de opțiuni cu alegere unică.

```
import java.awt.*;
public class ChoiceDemo extends Frame {
    public static void main(String[] s) {
        Frame f = new ChoiceDemo(); f.setSize(150,100);
        f.setTitle("Exemplul 7"); f.setLayout(new FlowLayout());
        Choice ch = new Choice();
        ch.addItem("Unu"); ch.addItem("Doi"); ch.addItem("Trei");
        f.add(ch); f.setVisible(true);
    }
}
```



• Liste de opțiuni cu alegere multiplă. Clasa `List`

O listă de opțiuni cu alegere multiplă permite selectarea a oricâte elemente (tot șiruri de caractere, ca și în cazul listelor cu alegere unică) din listă. Selectarea/deselectarea se face în mod standard prin clic cu mouse-ul pe elementul respectiv.

Clasa `List` este declarată prin:

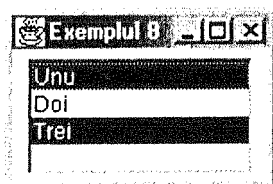
```
public class List extends Component
    Prezentăm în continuare doar unul dintre constructorii clasei și anume:
    public List(int linii, boolean b)
    unde linii reprezintă numărul inițial de elemente (linii) din listă. La invocare
    vom folosi totdeauna true pentru al doilea argument.
    Dintre metodele (publice) ale clasei prezentăm următoarele:
    String getItem(int poz)
        întoarce elementul de pe linia poz;
    String[] getItems()
        întoarce toate elementele listei;
    add(String item)
        adaugă un element listei, la sfârșitul acesteia;
    void add(String item, int poz)
        adaugă un element listei, pe poziția specificată;
    void replaceItem(String newValue, int poz)
        înlocuiește elementul de pe poziția poz cu cel specificat;
    void remove(String item)
        șterge din listă primul element cu informația specificată;
    void remove(int poz)
        șterge elementul din listă de pe poziția poz;
    void removeAll()
        șterge toate elementele listei;
```

```
String[] getSelectedItems()
    întoarce toate elementele selectate din listă;
int getRows()
    întoarce numărul curent de elemente din listă.
```

O acțiune asupra unei liste cu alegere multiplă declanșează un eveniment de tipurile `Item` și `Action`.

Exemplul 8. Exemplul își propune doar prezentarea modului în care apare o listă de opțiuni cu alegere multiplă.

```
import java.awt.*;
public class ListDemo extends Frame {
    public static void main(String[] s) {
        Frame f = new ListDemo(); f.setSize(150,100);
        f.setTitle("Exemplul 8"); f.setLayout(new FlowLayout());
        List lista = new List(0,true);
        lista.add("Unu"); lista.add("Doi"); lista.add("Trei");
        f.add(lista); f.setVisible(true);
    }
}
```



• Bare de defilare. Clasa `Scrollbar`

O *bară de defilare* (obiect de tipul `Scrollbar`) apare pe ecran ca o bară orientată pe orizontală sau verticală, ce conține un cursor care se deplasează de-a lungul barei (conform orientării acesteia) ca urmare a mutării cursorului cu ajutorul mouse-ului sau a acționării icon-urilor de la capetele barei. Barei îi este asociat (de obicei prin constructor) un domeniu de valori întregi consecutive delimitate de valoarea minimă `p` și de valoarea maximă `u`. Cursorul are o lungime `lung`.

Poziția curentă `poz` din bara de defilare este poziția extremității din stânga (respectiv de sus) a cursorului. Ei îi corespunde valoarea selectată din domeniul de valori asociat barei; această valoare poate fi obținută prin invocarea metodei `getValue`. Lungimea cursorului limitează numărul valorilor ce pot fi selectate la intervalul `p..u-lung`. Valoarea selectată poate fi folosită în diferite contexte ca de exemplu: extragerea unui subșir dintr-un șir plecând de la o poziție dată, stabilirea mărimii unui font, alegerea unei culori dintr-o paletă dată, setarea volumului sunetului, afișarea unei porțiuni dintr-o imagine etc.

Clasa `Scrollbar` este declarată prin:

```
public class Scrollbar extends Component
```

Câmpurile clasei sunt constantele publice întregi `HORIZONTAL` și `VERTICAL`.

Principalul constructor al clasei este:

```
public Scrollbar(int o, int val, int lung, int p, int u)
    care construiește o bară de defilare cu următoarele caracteristici:

```

- orientarea barei este dată de `o`, ce trebuie să fie unul dintre câmpurile constante ale clasei;
- `val` este valoarea inițială a poziției curente;
- `lung` este lungimea cursorului;
- `p` și `u` reprezintă valorile minimă și maximă din domeniul asociat barei.

Prezentăm următoarele metode publice ale clasei:

```
public int getValue()
```

întoarce valoarea curentă a barei de defilare;

```
public void setValue(int val)
```

setează valoarea selectată a barei de defilare. Dacă `val < p`, atunci `val` devine valoarea curentă a lui `p`; analog, dacă `val > u`, atunci `val` devine valoarea curentă a lui `u`.

Acțiunile asupra unei bare de defilare declanșează un eveniment de tipul `Adjustment`. Gestionarul de evenimente corespunzător trebuie să implementeze, respectiv extindă, metoda `adjustmentValueChanged` fără parametri a clasei `AdjustmentListener`.

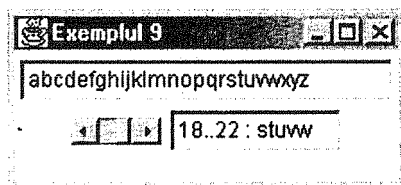
Exemplul 9. Este creat un câmp de text în care apar cele 26 de litere ale alfabetului englez, o bară de defilare pentru care domeniul de valori este `0..26` și un al doilea câmp de text în care vor apărea, ca rezultat al deplasării cursorului, cinci litere consecutive.

```
import java.awt.*;
import java.awt.event.*;
class ScrollbarDemo extends Frame {
    public static void main(String[] sir) {
        Scrollbar sb; final TextField text, subtext;
        Frame f = new ScrollbarDemo();
        f.setLayout(new FlowLayout());
        f.setSize(220,100); f.setTitle("Exemplul 9");
        sb = new Scrollbar(Scrollbar.HORIZONTAL, 0, 5, 0, 26);
        text = new TextField("abcdefghijklmnopqrstuvwxyz");
        subtext = new TextField(" ");
        sb.addAdjustmentListener(
            new AdjustmentListener() {
                public void adjustmentValueChanged(AdjustmentEvent e)
                {
                    int p = e.getValue();
```

```

String selectie = text.getText().substring(p,p+5);
subtext.setText(p+".."+(p+4)+" : "+selectie);
}
}
);
f.add(text); f.add(sb); f.add(subtext);
f.setVisible(true);
}
}

```



• Ferestre de dialog. Clasa Dialog

Să presupunem că la o acțiune asupra unei ferestre *F* este declanșat un eveniment care cere un răspuns din partea utilizatorului, de exemplu introducerea unui text. Problema poate fi rezolvată folosind cunoștințele anterioare, de exemplu specificând un handler care să citească de la tastatură sau dintr-un câmp de text al ferestrei informația dorită.

O altă posibilitate oferită de AWT este cea de a deschide o *fereastră de dialog* *D* prin intermediul căreia să furnizăm răspunsul. Fereastra de dialog va fi creată pe baza ferestrei *F*, care devine *proprietarul* lui *D*. O fereastră de dialog poate avea ca proprietar un cadru sau o altă fereastră de dialog. În cele mai multe cazuri, vom face ca fereastra de dialog să fie vizibilă doar atâta timp "cât este necesar", adică atâta timp cât prin acțiuni asupra componentelor sale furnizăm răspunsul dorit.

La declanșarea evenimentului, avem posibilitatea de a putea acționa numai asupra ferestrei de dialog sau și asupra proprietarului său. Cele două tipuri de dialog se numesc *modal*, respectiv *nemodal*.

Clasa *Dialog* este declarată prin:

```
public class Dialog extends Window
```

și are constructorii:

```
public Dialog(Frame p, String titlu, boolean modal)
```

```
public Dialog(Dialog p, String titlu, boolean modal)
```

unde *p* este proprietarul ferestrei de dialog, *titlu* este titlul ferestrei, iar prin *modal* stabilim dacă dialogul este sau nu *modal*.

Acțiunile asupra unei ferestre de dialog declanșează evenimente de tipul *Window*.

Exemplul 10. Reluăm *Exemplul 3*, în care într-un cadru apar două câmpuri de text *t1* și *t2*, precum și un buton *b*. Apăsarea butonului determină incrementarea valorii din *t1* cu valoarea introdusă în *t2*. De această dată dorim să interzicem incrementări negative. În acest scop, pentru un increment negativ acționarea butonului va transfera controlul unei ferestre de dialog în care va apărea un mesaj corespunzător și va trebui să acționăm un buton pentru a reveni la cadru (proprietarul ferestrei de dialog). Am ales tipul de dialog *modal*.

```

import java.awt.*;
import java.awt.event.*;

class DialogDemo {
    public static void main(String[] sir) {
        F f = new F(); D d = new D(f);
        Handler H = new Handler(f,d);
        f.b.addActionListener(H);
        d.db.addActionListener(H);
        f.setVisible(true);
    }
}

class F extends Frame {
    TextField t1,t2; Button b; int c1,c2;
    F() {
        setTitle("DialogDemo"); setLayout(new FlowLayout());
        setSize(200,100);
        t1 = new TextField("0"); add(t1);
        t2 = new TextField("0"); add(t2);
        b = new Button("Aduna"); add(b);
    }
}

class D extends Dialog {
    TextField dt; Button db;
    D(Frame f) {
        super(f,"Eroare",true); // (*)
        setLocation(201,0);
        setSize(150,90); setLayout(new FlowLayout());
        dt = new TextField("Increment negativ");
        db = new Button("O.K.");
        add(dt); add(db);
    }
}

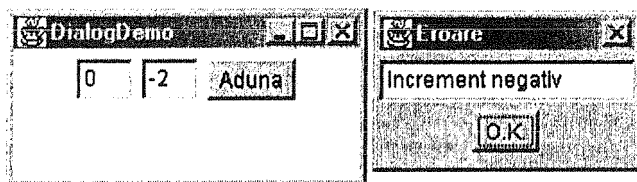
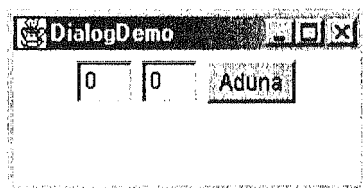
class Handler implements ActionListener {
    F f; D d;
    Handler(F ff,D dd) { f = ff; d = dd; }
    public void actionPerformed(ActionEvent e) {
        if (e.getActionCommand().equals("O.K."))
            { d.setVisible(false); }
    }
}

```

```

else {
    f.c2 = Integer.parseInt( f.t2.getText() );
    if (f.c2>=0) {
        f.c1 += f.c2; f.t1.setText( Integer.toString(f.c1) );
    }
    else d.setVisible(true);
}
}
}

```



Dacă în linia din program (*) am alege tipul nemodal de dialog, atunci la introducerea unui increment negativ s-ar putea acționa asupra ambelor ferestre.

• Suprafețe de desenare. Clasele Canvas și Graphics

Suprafețele grafice oferă facilitatea de control la nivel de pixel. Pe o astfel de suprafață se pot desena, folosind metode ale clasei Graphics, segmente de dreaptă, dreptunghiuri, ovale etc.; la desenare poate fi aleasă o culoare. De asemenea contururile închise pot fi "umplute" cu o culoare. Pe o suprafață de desenare mai pot fi plasate și imagini, pot fi create propriile butoane etc.

Clasa Canvas este deosebit de complexă, nu atât direct prin metodele sale, ci mai ales prin posibilitatea de a folosi alte clase ca de exemplu cele pentru grafică bi- și tridimensională. Vom reduce însă prezentarea la câteva facilități mai des folosite.

Modul standard de creare a unei suprafețe grafice constă în extinderea clasei Canvas, redefinind metoda paint, ce afișează componenta. Modificările ulterioare devin vizibile prin invocarea metodei repaint.

O suprafață grafică reprezintă un dreptunghi de pixeli, cu colțul din stânga sus având coordonatele (0, 0). Este folosit un *context de desenare*; nu vom intra în amănunte, ci vom folosi contextul de desenare standard disponibil.

Pentru clasa Canvas precizăm doar următoarele elemente:

```

public class Canvas extends Component
{
    public Canvas()
    {
        public paint(Graphics g)
    }
}

```

cu mențiunea că invocarea metodei paint pune la dispoziție contextul de desenare standard g. Faptul că este extinsă clasa Component permite captarea de evenimente specifice acestei clase, ca de exemplu Key, Mouse, MotionEvent.

Prezentăm în continuare câteva elemente legate de clasa Graphics. Atât clasa, cât și metodele sale, au modificatorii public și abstract. Menționăm că un context de desenare are atașate câteva informații curente, printre care culoarea curentă de desenare și fontul curent. Drept urmare, de exemplu trasarea unei linii și "umplerea" unui contur se fac folosind culoarea curentă.

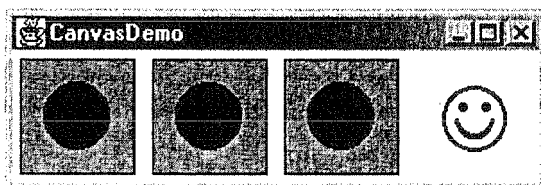
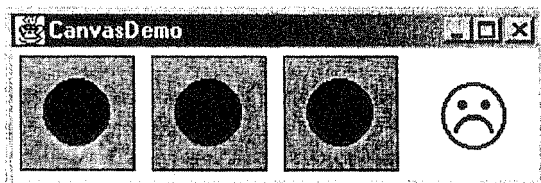
```

class Graphics extends Object
{
    Color getColor()
    void setColor(Color c)
        este obținută, respectiv setată, culoarea curentă de desenare;
    Font getFont()
    void setFont(Font f)
        este obținut, respectiv setat, fontul curent;
    void drawLine(int x1, int y1, int x2, int y2)
        este trasat un segment între punctele de coordonate (x1,y1) și (x2,y2);
    void drawPolyline(int[] x, int[] y, int n)
        este trasată o linie poligonală ce trece, în ordine, prin n puncte ale căror
        coordonate sunt specificate în tablourile x și y;
    void drawRect(int x, int y, int w, int h)
        este trasat un dreptunghi: colțul din stânga sus are coordonatele (x,y), iar
        lățimea și înălțimea sa sunt w, respectiv h;
    void fillRect(int x, int y, int w, int h)
        un dreptunghi specificat ca mai sus este "umplut" cu culoarea curentă;
    void drawOval(int x, int y, int w, int h)
        este trasată o elipsă tangentă la laturile dreptunghiului precizat;
    void fillOval(int x, int y, int w, int h)
        este umplută o elipsă tangentă la laturile dreptunghiului precizat.
}

```

În exemplul ce urmează sunt folosite unele dintre elementele de mai sus, dar și unele noi, care vor fi explicate după prezentarea programului.

Exemplul 11. Exemplul prezintă un joc rudimentar. Într-un panou sunt plasate patru suprafețe de desenare, simulând butoane. Primele trei conțin câte un oval colorat într-una dintre 5 culori specificate; un clic pe un astfel de "buton" îi schimbă culoarea, cu excepția situației în care toate cele trei butoane sunt colorate identic, caz în care efectul este nul. Al patrulea buton conține inițial o imagine .gif formată din caracterul "sad face"; un clic pe acest buton are efect doar dacă primele trei butoane au același conținut, caz în care este încărcată o imagine .gif formată din caracterul "happy face" (vezi figurile de mai jos).



```
import java.awt.*; import java.awt.event.*; import java.util.*;

class CanvasDemo {
    public static void main(String[] sir) {
        Frame F = new Frame("CanvasDemo");
        Panou P = new Panou(); P.adauga(); F.add(P);
        F.setSize(300,100); F.setVisible(true);
    }
}

class Panou extends Panel {
    Buton[] b = new Buton[4];
    void adauga() {
        setLayout(new GridLayout());
        for (int i=0; i<4; i++) {
            b[i] = new Buton(i);
            b[i].addMouseListener(
                new MouseAdapter() {
                    public void mouseClicked(MouseEvent e)
                    { ((Buton) e.getSource()).modif(); }
                }
            );
            add(b[i]);
        }
    }
}

class Buton extends Canvas {
    int i;
    static boolean gata; static int[] tip = {0,1,2};
    static Random R = new Random();
    static Color[] culoare = { Color.black, Color.red,
                                Color.blue, Color.green, Color.cyan };

    static Image
        no = Toolkit.getDefaultToolkit().getImage("no.gif"),
        yes = Toolkit.getDefaultToolkit().getImage("yes.gif");
```

```
Buton(int i) { this.i = i; }

public void paint(Graphics g) {
    Image imagine;
    int w = getSize().width, h = getSize().height;
    if(i==3) {
        if(gata) imagine = yes;
        else imagine = no;
        g.drawImage(imagine,w/4-1,h/4-1,this); return;
    }
    g.setColor(Color.magenta);
    g.drawRect(4,4,w-10,h-10);
    g.setColor(Color.lightGray);
    g.fillRect(5,5,w-11,h-11);
    g.setColor(culoare[tip[i]]);
    g.fillOval(w/4-1,h/4-1,w/2,h/2);
}

void modif() {
    boolean b = tip[0]==tip[1] && tip[1]==tip[2];
    if(b)
        if(i==3 && !gata) { gata = true; repaint(); return; }
        else return;
    else {
        if(i==3) return;
        tip[i] = (tip[i] + R.nextInt(4) + 1) % 5;
    }
    repaint();
}
}
```

Observații:

- prin `getSize().width` și `getSize().height` am obținut dimensiunile componente curente;

- o *imagine* este un obiect de tipul `Image`. Pentru importarea unei imagini am folosit invocarea:

`Toolkit.getDefaultToolkit().getImage(ume_fisier)`

unde fișierul *ume_fisier* poate fi numai de tipurile `.gif`, `.jpeg` sau `.png`.

11.5. Gestionari de poziționare

În acest subcapitol vom prezenta cei mai folosiți gestionari de poziționare (machetare): `FlowLayout`, `GridLayout` și `GridBagLayout`.

• Gestionarul de poziționare `FlowLayout`

`FlowLayout` este cel mai simplu gestionar de poziționare. Componentele sunt așezate în container în ordinea în care sunt introduse (prin invocarea metodei


```
int gridx
int gridy
int gridwidth
int gridheight
```

unde prin (gridx,gridy) specificăm poziția colțului din stânga sus al ariei alese pentru componentă, iar prin gridwidth și gridheight precizăm dimensiunile ariei.

```
int anchor
```

câmp prin care precizăm poziția celulei inițiale a ariei în interiorul acesteia. Valorile posibile sunt constantele publice CENTER, NORTH, NORTHEAST, EAST, SOUTHEAST, SOUTH, SOUTHWEST, WEST, NORTHWEST ale clasei GridBagConstraints. Valoarea implicită este CENTER.

Exemplul 12. Să considerăm următorul program:

```
import java.awt.*;
class F extends Frame {
    private GridBagLayout gbl;
    private GridBagConstraints gbc;

    private void Buton(int x,int y,int w,int h,String nume) {
        gbc.gridx = x; gbc.gridy = y;
        gbc.gridwidth = w; gbc.gridheight = h;
        Button buton = new Button(nume);
        gbl.setConstraints(buton,gbc);
        add(buton);
    }

    F(String s) {
        setTitle(s);
        gbl = new GridBagLayout();
        gbc = new GridBagConstraints();
        setFont( new Font("Helvetica",Font.BOLD,16) );
        setLayout(gbl);
        // gbc.fill = GridBagConstraints.BOTH;      {1}
        // gbc.weightx = 1; gbc.weighty = 1;        {2}
        Buton(0,0,1,2,"1");
        Buton(1,0,1,2,"2");
        Buton(2,0,1,1,"3");
        Buton(2,1,1,1,"4");
        Buton(0,2,1,2,"5");
        Buton(1,2,1,1,"6");
        Buton(2,2,1,1,"7");
        Buton(1,3,2,1,"8");
    }
}

class GBL {
    public static void main(String s[]) {
```

```
F f = new F("GridBagLayout");
f.setSize(200,240); f.setVisible(true);
}
```

Programul va afișa pe ecran un cadru având în centru structura de celule inițiale din figura 1 de mai jos.

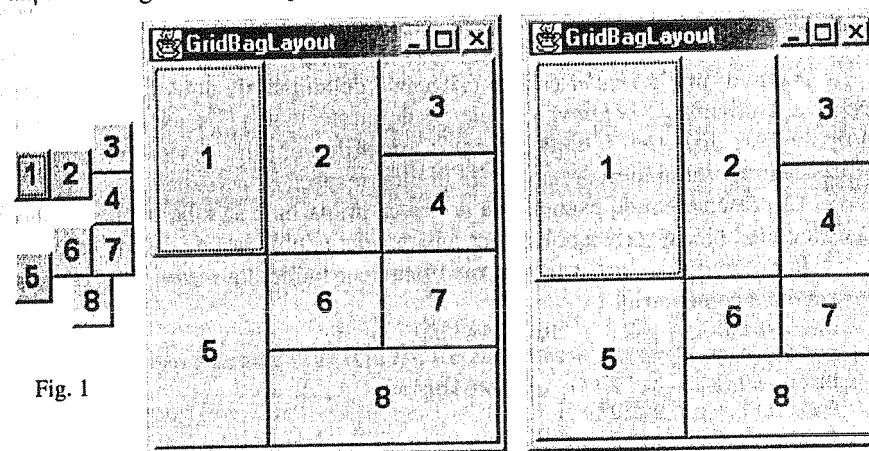


Fig. 1

Fig. 2

Fig. 3

Următoarea facilitate pe care o prezentăm este cea de redimensionare a unei componente, prin care celula inițială este expandată în cadrul ariei asociate componentei. În acest scop, sunt folosite următoarele câmpuri publice ale clasei GridBagConstraints:

```
int fill
```

celula inițială este expandată conform lui fill, care poate lua valorile care urmează;

```
static final int NONE
```

celula de bază nu este expandată; este valoarea implicită a lui fill;

```
static final int BOTH
```

celula de bază este expandată atât pe orizontală, cât și pe verticală, componenta ocupând acum întreaga arie asociată;

```
static final int HORIZONTAL
```

celula de bază este expandată pe orizontală în aria asociată componentei;

```
static final int VERTICAL
```

celula de bază este expandată pe orizontală în aria asociată componentei.

O altă facilitate este oferită de următoarele câmpuri ale clasei GridBagConstraints:

```
public double weightx
public double weighty
```

prin care putem asocia ponderi (dimensiunilor) liniilor și coloanelor rețelei. Valoarea lor implicită este 0. Pentru moment ne mărginim la a preciza că prin atribuirea de valori nenule acestor câmpuri, rețeaua este extinsă pe orizontală, respectiv pe verticală, la întreg containerul.

Ca aplicație la cele de mai sus, dacă ridicăm statutul de comentariu liniilor marcate cu {1} și {2}, fiecare componentă este expandată la întreaga arie asociată, iar rețeaua este expandată la întreg cadrul, obținându-se imaginea din figura 2.

Numim *linie a rețelei* o fâșie orizontală delimitată de două linii complete succesive. Noțiunea de *coloană a rețelei* se definește analog. De exemplu rețeaua din figura 2 are două linii și două coloane. Câmpurile `weightx` și `weighty` permit modificarea raportului între dimensiunile liniilor, respectiv a coloanelor.

Să presupunem de exemplu că dorim ca prima linie să aibă pondere dublă față de a doua linie, iar prima coloană să aibă pondere dublă față de a doua coloană.

În programul de mai sus vom modifica doar liniile din metoda `Buton` care urmează după comentariul {1}:

```
gbc.weightx = 2;      gbc.weighty = 1;
Buton(0,0,1,2,"1");
Buton(0,2,1,2,"5");  gbc.weightx = 1;
Buton(1,0,1,2,"2");
Buton(2,0,1,1,"3");
Buton(2,1,1,1,"4");  gbc.weighty = 0.5;
Buton(1,2,1,1,"6");
Buton(2,2,1,1,"7");
Buton(1,3,2,1,"8");
```

ca rezultat obținându-se imaginea din figura 3.

11.6. Evenimente

În acest subcapitol vom prezenta principalele tipuri de evenimente. Reamintim că dacă *tip* este unul dintre `Component`, `Container`, `Focus`, `Key`, `Mouse` sau `MouseMotion`, putem alege între a extinde clasele abstracte și publice `tipAdapter` și a extinde interfețele publice `tipListener`.

• Clasa `ComponentAdapter` și interfața `ComponentListener`

```
void componentResized(ComponentEvent e)
    este invocată la modificarea mărimii componenteii;
void componentMoved(ComponentEvent e)
    este invocată la modificarea poziției componenteii;
void componentShown(ComponentEvent e)
    este invocată atunci când componenta devine vizibilă;
void componentHidden(ComponentEvent e)
    este invocată atunci când componenta devine invizibilă.
```

• Clasa `ContainerAdapter` și interfața `ContainerListener`

```
void componentAdded(ContainerEvent e)
    este invocată atunci când o nouă componentă este adăugată containerului;
void componentMoved(ContainerEvent e)
    este invocată atunci când o componentă fost înlăturată din container.
```

• Clasa `FocusAdapter` și interfața `FocusListener`

```
void focusGained(FocusEvent e)
    este invocată atunci când o componentă a devenit ținta intrării de la tastatură;
void focusLost(FocusEvent e)
    este invocată atunci când o componentă a pierdut controlul asupra tastaturii.
```

O componentă primește focus-ul (controlul de la tastatură) prin invocarea metodei `requestFocus` din `Component` sau prin clic de mouse pe componentă.

• Clasa `KeyAdapter` și interfața `KeyListener`

```
void keyTyped(KeyEvent e)
    este invocată atunci când o tastă a fost apăsată și apoi eliberată;
void keyPressed(KeyEvent e)
    este invocată atunci când o tastă a fost apăsată;
void keyReleased(KeyEvent e)
    este invocată atunci când o tastă (anterior apăsată) a fost eliberată.
```

• Clasa `MouseAdapter` și interfața `MouseListener`

```
void mouseClicked(MouseEvent e)
    este invocată la un clic de mouse pe o componentă;
void mousePressed(MouseEvent e)
    este invocată la apăsarea unui buton al mouse-ului pe o componentă;
void mouseReleased(MouseEvent e)
    este invocată la eliberarea unui buton (anterior apăsat) al mouse-ului pe o componentă;
void mouseEntered(MouseEvent e)
    este invocată atunci când cursorul mouse-ului intră într-o componentă.
    Precizăm că nu se face distincție între butoanele mouse-ului.
```

• Clasa `MouseMotionAdapter` și interfața `MouseMotionListener`

```
void mouseMoved(MouseEvent e)
    este invocată atunci când cursorul mouse-ului a fost mutat pe o componentă,
    fără ca vreun buton să fie apăsat;
```

`void mouseDragged(MouseEvent e)`
 este invocată atunci când un buton al mouse-ului a fost apăsat pe o componentă și apoi a fost deplasat; acest eveniment va continua a fi furnizat componentei până când butonul mouse-ului a fost eliberat, indiferent dacă eliberarea se face această componentă sau pe o altă componentă.

- **Clasa `WindowAdapter` și interfața `WindowListener`**

`void windowOpened(WindowEvent e)`
 este invocată la deschiderea unei ferestre AWT;
`void windowClosed(WindowEvent e)`
 este invocată la închiderea unei ferestre AWT;
`void windowActivated(WindowEvent e)`
 este invocată la activarea unei ferestre AWT;
`void windowDeactivated(WindowEvent e)`
 este invocată la dezactivarea unei ferestre AWT.

- **Interfața `ActionListener`**

`void actionPerformed(ActionEvent e)`
 este invocată la întreprinderea unei acțiuni.

- **Interfața `TextListener`**

`void textValueChanged(TextEvent e)`
 este invocată la modificarea conținutului unui text.

- **Interfața `ItemListener`**

`void itemStateChanged(ItemEvent e)`
 este invocată în urma unei acțiuni de selectare sau deselectare.

- **Interfața `AdjustmentListener`**

`void adjustmentValueChanged(AdjustmentEvent e)`
 este invocată în urma unei acțiuni asupra unei componente de tip `Scrollbar`.

12 SERIALIZAREA OBIECTELOR

Capitolul de față constituie pe de o parte o completare la cel referitor la facilitățile de intrare/ieșire, iar pe de altă parte furnizează câteva elemente strict necesare transmiterii datelor la distanță, de care ne vom ocupa în capitolul următor.

Facilitățile legate de serializare/deserializare apar în pachetul `java.io`. Din nou vom omite în prezentarea lor modificatorul public (care este valabil pentru toate clasele și metodele) și excepția `IOException` (ce apare în clauza `throws` a tuturor metodelor ce vor fi prezentate).

12.1. Necesitatea serializării și deserializării obiectelor

În multe situații apare necesitatea ca unul sau mai multe obiecte să poată fi memorate pentru a fi folosite și după încheierea aplicației care le-a creat. De exemplu într-o aplicație poate fi creată o listă sau un arbore pe baza unor date citite de la intrare, structuri care pot avea semnificație și în afara aplicației, în sensul că se dorește utilizarea lor la un moment de timp ulterior. Pentru aceasta starea lor trebuie salvată într-un mod care să permită refacerea lor ulterioară. În acest scop Java oferă mecanismele de *serializare* și *deserializare*, care realizează tocmai salvarea, respectiv refacerea obiectelor. Una dintre aplicațiile evidente ale acestor mecanisme constă în serializarea obiectelor pe un sistem, urmată de transmiterea lor la distanță și deserializarea lor pe o nouă mașină gazdă pentru a fi folosite în continuare.

Serializarea constă în memorarea, într-un anumit format, a stării obiectelor într-un tablou de octeți sau într-un fișier. Acestea au un caracter serial: pot accepta numai secvențe de octeți. De aici derivă denumirile de serializare/deserializare. Mai este evident că vor fi folosite operații de intrare/ieșire la nivel de octet.

Facilitatea ca obiectele să poată "supraviețui" și după terminarea executării programului care le-a creat se mai numește *persistența datelor*.

După cum vom vedea, Java oferă modalități standard pentru serializare și deserializare, dar și posibilitatea ca programatorul să își conceapă propriile instrumente pentru realizarea acestor mecanisme, ceea ce oferă o mare flexibilitate în controlarea lor.

În cele ce urmează ne vom îndrepta atenția în special asupra fișierelor ca fluxuri pentru salvarea/restaurarea obiectelor.

Întrucât vom vorbi de transmiterea la distanță abia în următoarele capitole, exemplele care urmează vor presupune că serializarea se va face în cadrul unui director, iar deserializarea se va realiza din alt director. Ele vor fi numite ad-hoc *director sursă* și *director destinație* și nu sunt neapărat distincte.

Mai avertizăm cititorul că nici aici expunerea nu va fi exhaustivă: vor fi prezentate numai principalele facilități legate de serializarea obiectelor.

12.2. Un prim exemplu

Deși nu au fost prezentate încă elementele necesare, am ales să prezentăm câteva dintre ele pe un prim exemplu.

Exemplul 1. Directorul sursă conține clasa Angajat:

```
import java.io.*;
public class Angajat implements Serializable{
    String nume; int varsta; int salariu;
    static String firma;

    public Angajat(String n, int v, int s) {
        nume=n; varsta=v; salariu=s;
    }

    public void print() {
        System.out.println("Firma:\t\t" + firma + "\n" +
            "Nume:\t\t" + nume + "\n" +
            "Varsta:\t\t" + varsta + "\n") +
            "Salariul:\t" + salariu + "\n");
    }
}
```

și clasa Serial:

```
import java.io.*;
public class Serial {
    public static void main(String[] sir) throws Exception {
        Angajat.firma = "SRL Serial";
        Angajat Unu = new Angajat("Vasile Vasile", 25, 1485);
        Angajat Doi = new Angajat("Ion Ion", 24, 420);
        FileOutputStream fos = new FileOutputStream("Serial");
        ObjectOutputStream oos = new ObjectOutputStream(fos);
        // oos.writeUTF(Angajat.firma);
        oos.writeObject(Unu); oos.writeObject(Doi);
        oos.close(); fos.close();
    }
}
```

Facem deocamdată abstracție de conținutul comentariilor de sfârșit de linie.

Metoda principală creează două instanțe ale clasei Angajat. Cele două obiecte corespund la doi angajați fictivi; informațiile despre ei sunt primite de constructorul clasei.

Este creat un flux oos de tipul ObjectOutputStream pentru serializarea obiectelor, care prin metoda cunoscută de suprapunere a fluxurilor se referă la fișierul cu numele Serial din directorul curent. Clasa ObjectOutputStream extinde clasa abstractă OutputStream. Prin invocarea metodei writeObject se realizează serializarea (într-un mod standard) a obiectelor create și scrierea rezultatului serializării în fluxul de ieșire.

Clasa Angajat implementează interfața Serializable, care va fi prezentată ulterior. Prin abuz de limbaj vom numi *clasă serializabilă* o clasă ce implementează această interfață. Să reținem că *numai instanțele unei clase serializabile pot fi serializate*.

Mai observăm că un flux de serializare poate conține mai multe obiecte.

Presupunem în continuare că în directorul destinație apar (de exemplu au fost copiate) fișierele Angajat și Serial, precum și următoarea clasă, al cărei nume este tot Serial:

```
import java.io.*;
public class Serial {
    public static void main(String args[]) throws Exception {
        FileInputStream fis = new FileInputStream("Serial");
        ObjectInputStream ois = new ObjectInputStream(fis);
        // Angajat.firma = ois.readUTF();
        Angajat Unu = (Angajat) ois.readObject();
        Angajat Doi = (Angajat) ois.readObject();
        Unu.print(); Doi.print();
        ois.close(); fis.close();
    }
}
```

Este creat un flux ois de tipul ObjectInputStream pentru deserializarea obiectelor, care prin metoda cunoscută de suprapunere a fluxurilor se referă la fișierul cu numele Serial. Clasa ObjectInputStream extinde clasa abstractă InputStream. Prin invocarea metodei readObject se realizează deserializarea (într-un mod standard) a obiectelor din fluxul de intrare, după care aceste obiecte invocă metoda print.

Metoda readObject întoarce un obiect de tipul Object, ceea ce face necesară conversia explicită la tipul Angajat.

Să observăm că la ieșire, în dreptul numelui firmei apare null. Aceasta se datorează faptului că în mecanismul de serializare nu sunt incluse și câmpurile statice. Explicația constă în aceea că serializarea se referă la obiecte, iar câmpurile statice sunt variabile de clasă.

Să trecem la situația în care comentariile de sfârșit de linie din clasele de mai sus devin instrucțiuni efective. În acest mod includem și câmpul static `firma` în procesul de serializare. Este folosită scrierea și citirea lui explicită în/din fișier cu ajutorul metodelor complementare `writeUTF` și `readUTF`, a căror acțiune este deja cunoscută. Drept urmare, la ieșire va apărea și numele firmei.

Este momentul să prezentăm efectul modificadorului `transient` (`transient` este cuvânt cheie). Un câmp declarat cu acest modificador nu va fi inclus în procesul de serializare. Aceasta corespunde de exemplu situației în care la deserializare el nu mai este folosit. Dacă pentru un anumit obiect dorim totuși să transmitem și acest câmp, va trebui să o facem explicit, la fel ca pentru câmpurile statice.

12.3. Ce se transmite la serializare?

Răspundem parțial la această întrebare, menționând că nu sunt transmise:

- câmpurile statice și cele cu modificadorul `transient`;
- corpurile metodelor: este transmisă numai semnatura lor.

Exemplul 2. Presupunem că este serializat un obiect al următoarei clase:

```
import java.io.*;
public class UnuC implements Serializable {
    int x,y;
    UnuC(int a, int b) {
        x = a; y = b;
    }
    void print() { System.out.println("Suma =\t" + (x+y)); }
}
```

Dacă la deserializare în directorul destinație în metoda `print` a clasei `UnuC` înlocuim `(x+y)` prin `(x*y)`, iar obiectul de tipul `UnuC` obținut prin deserializare invocă această metodă, la ieșire va apărea produsul (și nu suma) celor două câmpuri.

Formulăm în continuare următoarea întrebare, aparent stupidă: *Câte obiecte sunt serializate atunci când serializăm un obiect?*

Aici intervine noțiunea de graf asociat unui obiect. *Graful asociat unui obiect* constă din obiectul respectiv, dar și din obiectele referite direct sau indirect de el.

La serializarea unui obiect este serializat întregul graf asociat obiectului respectiv. Această facilitate este foarte utilă și este ilustrată în continuare.

Exemplul 3. Serializarea unei liste circulare.

Directorul sursă conține clasa `element`:

```
import java.io.*;
class element implements Serializable {
    char c; element leg;
    static element p,u;

    element() { }
    element(char ch) { c = ch; }

    void creare() {
        char ch = IO.readch(); p = new element(ch); u = p;
        ch = IO.readch();
        while (ch != '$') { u = u.adaug(ch); ch = IO.readch(); }
        u.leg = p;
    }

    element adaug(char ch) {
        element x = new element(ch); leg = x; return x;
    }

    String parcurg(element x) {
        if (x==null) return "";
        else if (x.leg == p) return x.c + "";
        else return x.c + parcurg(x.leg);
    }
}
```

precum și clasa `Lista1`:

```
import java.io.*;
public class Lista1 {
    public static void main(String[] sir) throws Exception {

        element Ob = new element(); Ob.creare();
        Ob = element.p; System.out.println(Ob.parcurg(Ob));

        FileOutputStream fos = new FileOutputStream("lista");
        ObjectOutputStream oos = new ObjectOutputStream(fos);
        oos.writeObject(Ob);
        oos.close(); fos.close();
    }
}
```

Metoda principală a clasei `Lista1` construiește o listă înălțuită circulară formată din caracterele citite de la intrare până la întâlnirea caracterului '\$'. Capul listei apare în câmpul static `p` al clasei `elem`. La serializarea lui `p` sunt de fapt serializate toate elementele (obiectele) din listă, adică întregul graf asociat acestui obiect.

Într-adevăr, dacă în directorul destinație includem clasa `element` și următoarea clasă `Lista2`:

```
import java.io.*;
public class Lista2 {
    public static void main(String[] sir) throws Exception {

        FileInputStream fis = new FileInputStream("lista");
        ObjectInputStream ois = new ObjectInputStream(fis);
        element p = (element) ois.readObject();

        element.p = p; System.out.println( p.parcurs(p) );
        ois.close(); fis.close();
    }
}
```

atunci la executarea metodei principale vor fi listate toate elementele listei.

Exemplul de mai sus arată că serializarea "nu se încurcă" dacă un obiect din graful asociat se autoreferă. Să menționăm că în acest caz mecanismul de serializare "memorează" o singură dată un astfel de obiect.

Vom continua discuția începută în acest capitol după prezentarea mai amănunțită a unor clase și metode puse la dispoziție de Java pentru realizarea mecanismelor de serializare/deserializare.

12.4. Interfețe și clase folosite la serializare/deserializare

• Interfața `Serializable`

Interfața este declarată prin:

```
public interface Serializable
```

Interfața nu are nici metode, nici câmpuri. Rolul său este de a autoriza serializarea și deserializarea instanțelor sale.

Clasele ce extind această interfață sunt automat serializabile. Menționăm că la serializarea grafului asociat unui obiect, dacă un obiect din acest graf nu este serializabil, atunci va fi lansată excepția `NotSerializableException`.

• Clasa `ObjectOutputStream`

```
public class ObjectOutputStream extends OutputStream
    implements DataOutput, ObjectOutput, ObjectOutputStreamConstants {
    public ObjectOutputStream(OutputStream os)
        throws IOException
    public final void writeObject(Object Ob) throws IOException
    public void defaultWriteObject() throws IOException
    implementarea metodelor write din DataOutput, flush și close
    din OutputStream
}
```

Nu vom descrie aici interfețele implementate.

Un flux de ieșire ce instanțiază această clasă permite scrierea de date primitive și de grafuri asociate unor obiecte într-un mod standard, care permite reconstituirea lor folosind metodele clasei `ObjectInputStream`.

Metoda `defaultWriteObject` va fi descrisă mai jos.

• Clasa `ObjectInputStream`

```
public class ObjectInputStream extends InputStream
    implements DataInput, ObjectInput, ObjectStreamConstants {
    public ObjectInputStream(InputStream is) throws IOException
    public final Object readObject()
        throws ClassNotFoundException, IOException
    public void defaultReadObject()
        throws ClassNotFoundException, IOException
    implementarea metodelor read din DataInput, available și close
    din InputStream
}
```

Nu vom descrie aici nici interfața `ObjectInput`.

La deserializare sunt alocate noi adrese, prevenindu-se astfel scrierea peste obiecte existente. Drept urmare, serializarea și deserializarea pot fi realizate în cadrul aceluiași program, fără "pericolul" ca obiectul serializat și cel obținut prin deserializare să aibă aceeași adresă.

Metodele `defaultWriteObject` și `defaultReadObject` sunt complementare și sunt folosite dacă dorim să facem o serializare/deserializare specială (un exemplu va fi prezentat la sfârșitul acestui subcapitol). Pentru aceasta trebuie procedat astfel:

- clasa serializabilă respectivă trebuie să conțină următoarele metode, cu exact următoarele semnături:

```
private void writeObject(ObjectOutputStream oos)
private void readObject(ObjectInputStream oos)
```

- metoda `writeObject` astfel implementată trebuie să conțină o invocare a metodei `defaultWriteObject` prin intermediul obiectului `oos` (realizându-se astfel serializarea standard), urmată de alte invocări de metode prin `oos`, reprezentând serializările speciale;

- metoda `readObject` astfel implementată trebuie să conțină o invocare a metodei `defaultReadObject` prin intermediul obiectului `ois`, urmată de alte invocări de metode prin `ois`, reprezentând deserializările speciale. Informațiile astfel citite sunt incluse automat în rezultatul întors de invocarea metodei `readObject`.

La invocarea metodelor `writeObject` și `readObject` pentru un obiect se verifică dacă obiectul ce trebuie serializat/deserializat este o instanță a unei clase ce conține aceste metode. În caz afirmativ, se întreprind acțiunile de mai sus, iar în caz contrar se întreprinde acțiunea standard.

12.5. Serializarea claselor ce extind alte clase și implementează interfețe

Evident, este vorba de subclase ce extind explicit alte clase.

Începem prin a preciza că o subclasă a unei clase serializabile este automat serializabilă.

La serializarea unei clase are loc și serializarea superclaselor sale serializabile; drept urmare câmpurile și semnatura metodelor din superclase sunt incluse în acest proces.

Exemplul 4. În directorul sursă apar:

```
- interfața Interf:
interface Interf { int i = 333; }

- clasa Super cu clasa internă Tip:
import java.io.*;
public class Super implements Serializable, Interf {
    class Tip implements Serializable {
        boolean b;
        Tip(boolean bb) { b = bb; }
    }
    int a=2; Tip T = new Tip(true);
    Super() { }
    void met() { IO.println("Super " + a + " " + T.b); }
}

- clasa Sub:
import java.io.*;
public class Sub extends Super {
    int a;
    Sub(int aa) { a = aa; }
    void met() {
        IO.println("Sub " + a + " " + i); super.met();
    }
}

- clasa Trei:
import java.io.*;
public class Trei {
    public static void main(String[] sir) throws IOException {
        Super Ob = new Sub(1); Ob.met();
        FileOutputStream fos = new FileOutputStream("Trei");
        ObjectOutputStream oos = new ObjectOutputStream(fos);
        oos.writeObject(Ob);
        oos.close(); fos.close();
    }
}
```

Iar în directorul destinație apar clasele Super și Sub, interfața Interf (în care constanta i are acum valoarea 444), fișierul Trei rezultat prin executarea metodei principale din directorul sursă, precum și următoarea clasă Trei:

```
import java.io.*;
public class Trei {
    public static void main(String[] sir) throws Exception {
        Super Ob;
        FileInputStream fis = new FileInputStream("Trei");
        ObjectInputStream ois = new ObjectInputStream(fis);
        Ob = (Sub) ois.readObject(); Ob.met();
        ois.close(); fis.close();
    }
}
```

La executare se obține la ieșire:

```
Sub 1 444
Super 2 true
```

Observații:

- clasa internă Tip trebuie să extindă interfața Serializable; în caz contrar la executarea aplicației din directorul sursă ar fi semnalată o eroare (ar fi lansată excepția `NotSerializableException`);
- moștenirea în clasa Sub a câmpului T din superclasa Super face ca acest câmp (obiect) să fie inclus și el în serializare;
- modificarea constantei din interfața Interf în directorul destinație are ca urmare tipărirea valorii modificate;
- serializarea ține cont de tipul declarat și tipul real al obiectelor, după cum se poate vedea din rezultatele obținute la ieșire; o dovadă în plus este că dacă în directorul destinație am elimina metoda met din clasa Sub, ar fi semnalată o eroare la executare.

12.6. Cazul claselor ale căror superclase sunt neserializabile

Să presupunem că dorim serializarea unui obiect de tipul unei clase serializabile care extinde însă o clasă neserializabilă; dorim ca și câmpurile superclasei să fie incluse în serializare, pentru ca apoi, după deserializare, să le putem folosi.

O condiție necesară este ca superclasa să conțină constructorul fără parametri, pentru a realiza inițializarea câmpurilor sale.

După cum vom vedea pe exemplul ce urmează, acest caz poate fi în general tratat fără probleme.

Exemplul 5. Să considerăm clasa neserializabilă Super:

```
class Super {
    protected String nume; protected int an;
    public Super() { }
    Super(String nume, int an) {
        this.nume = nume; this.an = an;
    }
}
```

și subclasa sa serializabilă Sub:

```
import java.io.*;
class Sub extends Super implements Serializable {
    int salariu; boolean casatorit;
    Sub(String nu, int an, int sa, boolean ca) {
        super(nu, an); salariu = sa; casatorit = ca;
    }
    private void writeObject(ObjectOutputStream oos)
        throws Exception {
        oos.defaultWriteObject();
        oos.writeObject(nume); oos.writeInt(an);
    }
    private void readObject(ObjectInputStream ois)
        throws Exception {
        ois.defaultReadObject();
        nume = (String) ois.readObject(); an = ois.readInt();
    }
    public String toString() {
        return("\t" + nume + "\n\t" + an + "\n\t" +
            salariu + "\n\t" + casatorit + "\n");
    }
}
```

în care semnificația câmpurilor este evidentă.

Relativ la subclasa Super vom nota doar că ea include constructorul fără argumente.

Relativ la subclasa Sub se impun următoarele observații:

- metodele `writeObject` și `readObject` includ și salvarea, respectiv refacerea câmpurilor superclasei; de aceea pentru serializarea/deserializarea informațiilor referitoare la subclasă s-au invocat metodele `defaultWriteObject` și `defaultReadObject`; pentru câmpurile superclasei s-au utilizat serializarea/deserializarea explicite;

- câmpul `nume` al superclasei a fost scris/citit ca obiect, ceea ce la citire a necesitat o conversie explicită;

- includerea metodei `toString` constituie o redefinire a metodei `toString` a clasei `Object`; ea este invocată implicit atunci când într-o operație de scriere apare numele unui obiect de tipul clasei `Sub`;

- modificatorul `protected` al câmpurilor clasei `Super` nu ridică probleme, aceste câmpuri fiind direct accesibile din subclasa `Sub`.

Următoarea clasă realizează serializarea/deserializarea unei instanțe a clasei `Sub`:

```
import java.io.*;
public class NonSerialSuper {
    public static void main(String[] sir) throws Exception {
        Sub original = new Sub("Ionescu", 1975, 3333333, false);
        Sub copie = null;

        FileOutputStream fos = new FileOutputStream("temp");
        ObjectOutputStream oos = new ObjectOutputStream(fos);
        oos.writeObject(original);
        fos.close(); oos.close();

        FileInputStream fis = new FileInputStream("temp");
        ObjectInputStream ois = new ObjectInputStream(fis);
        copie = (Sub) ois.readObject();
        fis.close(); ois.close();

        System.out.println("Original:\n" + original);
        System.out.println("Copie:\n" + copie);
    }
}
```

Observație. Lucrurile se complică dacă unul sau mai multe dintre câmpurile clasei `Super` au modificatorul `private`. În acest caz astfel de câmpuri pot fi incluse în serializare numai dacă în clasă apar metode care furnizează valorile acestor câmpuri, respectiv metode care pot seta valoarea lor.

Încheiem discuția despre serializarea și deserializarea obiectelor cu mențiunea că aceste operații pot fi controlate în întregime de programator prin implementarea metodelor `readExternal` și `writeExternal` ale interfeței `Externalizable` din pachetul `java.io`.

13 SISTEME DISTRIBUITE. SOCKET-URI

Începem cu precizarea că noțiunile prezentate în continuare, privind rețelele și sistemele distribuite, se reduc la minimum posibil pentru prezentarea facilităților de lucru în rețea folosite de Java. Pentru o discuție mult mai completă, sugerăm cititorului să consulte [9].

13.1. Sisteme distribuite

O *rețea* este formată dintr-o mulțime de calculatoare și periferice (imprimante, plote, scannere, modemuri etc.) conectate între ele.

Un *sistem distribuit* este o colecție de calculatoare și/sau procesoare autonome (având o unitate de comandă proprie), interconectate (capabile de a schimba informații între ele). Elementele colecției poartă numele de *noduri*. Un astfel de sistem poate fi programat pentru a rezolva probleme de concurență și de paralelism.

Sistemele distribuite au apărut ca o necesitate pentru:

- schimbul de informații între noduri;
- partajarea unor resurse (printere, memorie backup, unități de disc etc.);
- siguranța în funcționare: dacă un nod "cade", întregul sistem trebuie (dacă este posibil) să fie capabil să asigure în continuare funcționarea, prin redirijarea (*rutarea*) mesajelor prin nodurile funcționale;
- posibilitatea specializării unor module.

• Tipuri de rețele

O primă clasificare a rețelilor este constituită de împărțirea lor în rețele LAN (*Local Area Network*) și rețele WAN (*Wide Area Network*).

Rețelele LAN sunt folosite de obicei pentru a partaja resurse (fișiere și periferice) și de a facilita schimbul de informații între nodurile componente. Conectarea tipică este realizată prin cablu, cu o limită dată pentru un drum între două noduri (de exemplu 10 km).

La fiecare moment de timp, într-o astfel de rețea este transmis un singur mesaj.

Numele rețelei desemnează produsul și nu rețeaua (așa cum se întâmplă la rețelele WAN). De exemplu Ethernet este un produs al companiei XEROX.

Problemele algoritmice tipice care se pun pentru astfel de rețele sunt următoarele:

- 1) *sincronizarea*: așteptarea îndeplinirii unei condiții;
- 2) *difuzarea totală (broadcasting)*: transmiterea unui mesaj către toate celelalte noduri;
- 3) *selectarea unui proces* pentru îndeplinirea anumitor acțiuni;
- 4) *detectarea terminării*: un nod ce întreprinde o acțiune, în care antrenează și alte noduri, trebuie să fie capabil să detecteze momentul încheierii acțiunii;
- 5) *excluderea reciprocă*: utilizarea unor resurse sub excludere reciprocă (de exemplu modificarea unui fișier sau scrierea la imprimantă);
- 6) *detectarea și prevenirea blocării totale (deadlock)*;
- 7) *gestionarea distribuită a fișierelor*.

Rețelele WAN permit comunicarea pe distanțe mai mari, prin legături directe între noduri, realizate prin linie telefonică, fibră optică, legătură via satelit etc. Ele sunt grafuri orientate.

Problemele algoritmice care se pun pentru astfel de rețele sunt următoarele:

- 1) *siguranța schimbului de informații* pe fiecare arc al grafului. Informația eronată trebuie recunoscută ca atare și corectată. Menționăm că este posibil ca mesajele să sosească într-un nod în altă ordine decât cea în care au fost transmise, pot fi duplicate etc. Toate aceste aspecte trebuie luate în considerare și tratate corespunzător;
- 2) *selectarea de căi de comunicare (rutare)* între nodurile care schimbă informații, deoarece un graf complet este prea costisitor și în general nici nu este realizabil. Modul de rutare trebuie actualizat în cazul în care un nod al rețelei cade (devine nefuncțional);
- 3) *evitarea congestionării unor căi de comunicație*, realizată tot prin (re)rutare;
- 4) *evitarea blocării totale*;
- 5) *securizarea datelor și a transmisiilor*.

Deosebirile esențiale între cele două tipuri de rețele se referă la:

- 1) *securizare*: în rețele LAN se presupune că transmiterea este corectă, pe când în rețelele WAN trebuie presupus că în timpul transmiterii mesajelor se poate întâmpla "ceva rău";
- 2) *timpul de comunicare*: în rețelele WAN timpul de comunicare este mult mai mare (de 10^k ori mai mare, $k \geq 1$) decât în rețelele LAN; această timp este la rândul său mult mai mare decât cel cerut de prelucrările locale;

3) *omogeneitatea*: în rețelele WAN trebuie presupus că sunt utilizate simultan mai multe protocoale (vezi paragraful următor), ceea ce pune problema recunoașterii și conversiei între ele;

4) *încrederea reciprocă*: nu este presupusă în rețelele WAN și drept urmare trebuie luate măsuri de securitate adecvate.

• Protocoale

Comunicarea între componentele unei rețele (respectiv unui sistem distribuit) se face prin mulțimi de reguli, numite generic *protocoale*.

Un protocol cuprinde atât formatul mesajului ce este efectiv transmis, cât și modul în care trebuie răspuns la mesajul respectiv.

Menționăm că existența unor protocoale unanim recunoscute permite ca într-o aplicație distribuită componentele software să fie dezvoltate independent și implementate în diverse limbaje de programare pe sisteme ce pot folosi, de exemplu, reprezentări diferite ale datelor.

Protocolul folosit pe Internet este *IP (Internet Protocol)*. Mesajele circulă în *pachete*, ce pot fi:

- *pachete Internet*, pentru care protocolul este *TCP (Transmission Control Protocol)*;

- *datagrame*, pentru care protocolul este *UDP (User Datagram Protocol)*.

În continuare ne vom îndrepta atenția numai asupra pachetelor Internet. Datagramele, deși mai ușor de utilizat, nu asigură recepționarea pachetelor în ordinea în care au fost transmise, impun restricții asupra volumului datelor transmise etc.

Internetul folosește *nume (adrese) simbolice* pentru rețele și pentru mașinile gazdă; ele se mai numesc *nume de domeniu*. Lor li se asociază în mod biunivoc *adrese (numere) IP*, ce sunt folosite efectiv la comunicarea între nodurile rețelei. Asocierea cade în sarcina unui *sistem de nume de domeniu (DNS = Domain Name System)*.

O adresă numerică IP are forma:

a.b.c.d

unde a, b, c, d sunt numere naturale din mulțimea {0, 1, ..., 255}.

• Modelul Client/Server

După cum știm, comunicarea între nodurile unei rețele constă în transmiterea (recepționarea) de pachete către (de la) gazde ale aceleiași rețele sau ale unei alte rețele.

Modelul utilizat pe scară largă în sistemele distribuite (și care va fi cel folosit în continuare) este *sistemul Client/Server*. În acest model există:

- o mulțime de *procese Server*, fiecare jucând rolul de gestionar de resurse pentru o colecție de resurse de un anumit tip;

- o mulțime de *procese Client*; fiecare dintre ele execută activități care necesită acces la resurse hard/soft făcute disponibile (partajate) de servere.

Un gestionar de resurse poate avea și el nevoie de resurse gestionate de un alt proces Server. Drept urmare, unele procese pot fi atât de tip Client, cât și de tip Server. Este respectată însă regula conform căreia doar serverele gestionează resurse.

Schițăm în continuare modul în care are loc "dialogul" între clienți și servere.

Serverele sunt cele care își încep primele activitatea. În mod tipic, un server oferă succesiv clienților posibilitatea de a se conecta la el (spunem că *acceptă conexiuni* de la clienți). La început clientul își manifestă dorința de a se conecta și, dacă serverul este gata să accepte conexiunea, aceasta se realizează efectiv; este vorba deci de o acțiune de la client către server. Apoi transmisia informațiilor devine bidirecțională, fluxul de informații putând circula acum în ambele sensuri.

Teoretic, activitatea unui server se desfășoară la infinit. În unele dintre exemplele care urmează, terminarea acțiunii unui server are loc numai din considerente didactice.

Pentru conectare la server, clientul trebuie să cunoască adresa serverului (fie cea numerică, fie cea simbolică), precum și numele *portului* pus la dispoziție de server. Serverul poate oferi mai multe servicii, pentru fiecare existând un număr de port. Serviciile standard și porturile corespunzătoare sunt următoarele:

7 : <i>ping</i> (ecou)	21 : <i>ftp</i> (transfer de fișiere)	
23 : <i>telnet</i>	25 : <i>email</i>	
79 : <i>finger</i>	80 : <i>Web</i>	110 : <i>POP3</i> (Post Office Protocol)

Porturile din intervalul 0..1023 sunt în general rezervate pentru servicii speciale (de genul celor de mai sus) și pentru clienți privilegiați.

Așa cum am precizat mai sus, clientul este cel care inițiază conexiunea prin rețea, specificând adresa serverului și portul prin care dorește să comunice. Serverul trebuie să precizeze numai portul; apoi el trebuie să execute o comandă prin care să anunțe că este gata să accepte conexiuni pe portul respectiv; drept urmare el rămâne în așteptare până când un client dorește să se conecteze și conexiunea este stabilită cu succes. Un server poate accepta conexiuni de la mai mulți clienți; evident, pentru ca mai mulți clienți să comunice "simultan" cu un server, acesta trebuie să creeze câte un fir de executare pentru fiecare dintre ei.

• Clasa *InetAddress*

Această clasă, aflată în pachetul *java.net*, furnizează informații asupra adreselor (simbolică și numerică) unui calculator gazdă (stație de lucru).

Clasa nu are constructori publici.

Principalele sale metode sunt următoarele:

```
public static InetAddress getLocalHost()
    returnează numele calculatorului gazdă (pe care se află în curs de executare
    aplicația). Acest nume, convertit la String, are forma:
    adresă_simbolică/adresă_IP;
public static InetAddress getByName(String s)
    dându-se ca argument adresa simbolică sau numerică a calculatorului gazdă,
    metoda întoarce un obiect de tipul InetAddress; metoda poate lansa
    excepția UnknownHostException;
public String getHostName()
    întoarce adresa simbolică a calculatorului gazdă;
public byte[] getAddress()
    întoarce un tablou de 4 octeți, ce compun adresa numerică.
```

Exemplul 1. Următorul program are ca scop determinarea adreselor simbolică și numerică ale calculatorului gazdă, precum și obținerea uneia dintre aceste adrese cunoscând-o pe cealaltă:

```
import java.net.*;
class Adrese {
    public static void main(String[] sss) throws Exception {
        InetAddress gazda = null;
        gazda = InetAddress.getLocalHost();
        IO.println(gazda.toString());

        String simbolic = gazda.getHostName();
        IO.write(simbolic+"/");
        byte[] ip ; ip = gazda.getAddress();
        String numeric = "";
        for (int i=0; i<3; i++) numeric += (ip[i] & 0xff) + ".";
        numeric += (ip[3] & 0xff) + ""; IO.println(numeric);

        String adr = "Granada"; // adresa: simbolică --> numerică
        gazda = InetAddress.getByName(adr);
        ip = gazda.getAddress(); numeric = "";
        for (int i=0; i<3; i++) numeric += (ip[i] & 0xff) + ".";
        numeric += (ip[3] & 0xff) + ""; IO.println(numeric);
    }
}
```

Presupunând că adresele simbolică și numerică ale stației de lucru sunt "Granada", respectiv "193.226.51.46", executarea programului de mai sus va produce la ieșire:

```
Granada/193.226.51.46
Granada/193.226.51.46
193.226.51.46
```

Singurele explicații necesare sunt legate de lucrul cu octeți pentru memorarea celor patru componente ale adresei numerice. Problema apare din faptul că o componentă a adresei numerice ia valori în intervalul 0..255, iar o variabilă de tip byte ia valori în intervalul -128..127.

Să considerăm de exemplu că o variabilă *v* de tip byte primește valoarea 129. Reprezentarea ei internă este 10000001 (după 127 "urmează" -128, apoi -127 etc.), deci de fapt este vorba de valoarea -127. Pentru a obține valoarea 129, trebuie să facem o extindere cu zerouri la stânga, pentru ca primul bit să devină zero (corespunzător unei valori pozitive). Pentru aceasta considerăm literalul întreg în formă hexazecimală 0xff ai cărui primi 24 de biți sunt 0, iar ultimii 8 biți sunt 1; prin utilizarea operatorului & ce acționează pe biți, rezultatul operației *v* & 0xff are în reprezentarea sa hexazecimală 24 de biți egali cu 0, urmați de 10000001, deci e vorba de numărul 129.

Observație. Se pune în mod natural întrebarea: "Are sens să studiem facilitățile oferite de Java pentru lucru în rețea dacă avem la dispoziție un singur calculator?". Răspunsul este afirmativ, deoarece putem de exemplu deschide mai multe ferestre de comandă și să le folosim (lucrând de exemplu în directoare diferite) ca și când am lucra pe calculatoare diferite legate în rețea. Drept adresă IP putem folosi numele calculatorului, numele standard "localhost" sau "127.0.0.1". Este chiar indicat să verificăm în astfel programele pe care le scriem. Desigur, este vorba doar de o primă verificare, pentru că lucrul în rețea implică și alte aspecte, care nu pot fi surprinse în acest mod de lucru.

13.2. Comunicare prin socket-uri

• Modul de lucru cu socket-uri

Socket-urile sunt folosite pentru transmiterea de date folosind protocolul TCP/IP. Ele sunt obiecte ce trebuie create la ambele capete ale conexiunii. Socket-urile client sunt obiecte de tipul clasei Socket, iar socket-urile server sunt obiecte de tipul clasei ServerSocket; ambele clase fac parte din pachetul java.net și vor fi prezentate în continuare. Socket-urilor li se pot atașa un flux de intrare și unul de ieșire, prin care pot recepționa, respectiv transmite, date.

Prezentăm în continuare schema generală a lucrului cu socket-uri, apelând la facilitățile de intrare/ieșire la nivel de octet.

Un mod tipic de creare a unui *socket client* este următorul:

```
try {
    Socket cs = null; cs = new Socket("adresa", nrport);
    DataInputStream is =
        new DataInputStream(cs.getInputStream());
    DataOutputStream os =
        new DataOutputStream(cs.getOutputStream());
}
```

```
catch(UnknownHostException e) { ... }
catch(IOException e) { ... }
```

unde adresa este adresa IP a serverului, iar nrport este numărul portului ales pentru comunicare. Socket-ului îi sunt atașate fluxul os ce va fi folosit pentru a transmite date serverului, precum și fluxul is ce va fi folosit pentru recepționarea datelor transmise de server. Fluxurile pot fi înzestrate și cu facilități suplimentare ca de exemplu lucrul cu zone tampon, dar pentru simplificare vom folosi schema de mai sus.

Un mod tipic de creare a unui *socket server* este următorul:

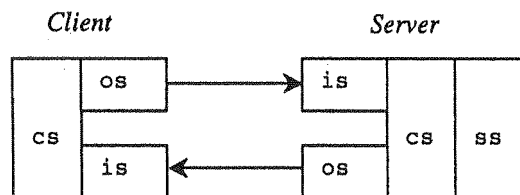
```
ServerSocket ss = null; Socket cs = null;
try {
    ss = new ServerSocket(nrport);
    // IO.println("Serverul a pornit");
}
catch(IOException e) { ... }
```

```
try {
    cs = ss.accept();
    DataInputStream is =
        new DataInputStream(cs.getInputStream());
    DataOutputStream os =
        new DataOutputStream(cs.getOutputStream());
}
catch(UnknownHostException e) { ... }
catch(IOException e) { ... }
```

Observăm că pentru server nu este necesară precizarea unei adrese IP, ci numai a unui port, care trebuie să coincidă cu cel folosit de client.

După crearea socket-ului server *ss*, acesta invocă metoda *accept*. Acțiunea acestei metode constă în a aștepta ca un client să încerce să se lege la server; în momentul în care acest lucru se întâmplă și legătura s-a stabilit, metoda creează și întoarce un socket *cs* de tip client.

În continuare lucrurile decurg la fel ca pentru client: socket-ului client *cs* îi atașăm un flux de intrare și unul de ieșire. Modul în care are loc transmiterea de date (informații) între client și server este cel din figura de mai jos.



Mai menționăm că în final toate socket-urile, precum și fluxurile atașate lor, trebuie închise explicit prin invocarea metodei *close* fără parametri.

• Clasele *Socket* și *ServerSocket*

Clasa *Socket* din pachetul *java.net*:

`public class Socket extends Object`
oferă constructori publici și metode publice pentru lucrul cu socket-urile client. Dintre ele menționăm următoarele:

```
Socket(String gazda, int port)
    throws IOException, UnknownHostException
    creează un socket și îl conectează la portul port al serverului cu numele
    simbolic gazda;
Socket(InetAddress adresa, int port) throws IOException
    creează un socket și îl conectează la portul port al serverului cu adresa IP
    adresa;
InputStream getInputStream() throws IOException
    întoarce un flux de intrare pentru socket-ul curent;
OutputStream getOutputStream() throws IOException
    întoarce un flux de ieșire pentru socket-ul curent;
void close() throws IOException
    închide (distruge) socket-ul curent.
```

Clasa *ServerSocket* din pachetul *java.net*:

`public class ServerSocket extends Object`
oferă un constructor public, precum și metode publice pentru lucrul cu socket-urile server. Dintre ele menționăm următoarele:

```
ServerSocket(int port) throws IOException
    creează un socket de tip server la portul port;
Socket accept() throws IOException
    așteaptă realizarea unei conexiuni la socket-ul server curent. În momentul în
    care conexiunea este realizată, metoda creează și întoarce un socket de tip
    client;
void close() throws IOException
    închide (distruge) socket-ul curent.
```

• Clasa *URL*

Clasa *URL* este declarată în pachetul *java.net* prin:

`public final class URL extends Object implements Serializable`

Un obiect de tip *URL*, numit și *locatie URL*, este o referință la o resursă Web. Resursa poate fi un fișier, un director, dar și un obiect mai complex ca de exemplu o cerere către o bază de date sau un motor de căutare. Ca sintaxă, o astfel de adresă cuprinde 4 părți:

`protocol:// adresa :port /cale`

unde *protocol* desemnează serviciul exportat de server sau căutat de client (de exemplu http sau ftp), *adresa* este adresa simbolică sau numerică a mașinii gazdă referite, *port* portul folosit pentru comunicare, iar *cale* identifică resursa.

Un exemplu este următorul:

`http://www.ncsa.uiuc.edu:8080/demoweb/url-primer.html`

Sunt folosite și locații URL *parțiale*, ce specifică doar unele părți din forma completă a locației; acest lucru este posibil în situațiile în care celelalte părți sunt implicate sau se deduc din context. De exemplu poate lipsi numele mașinii gazdă (dacă facem referire la mașina locală) sau portul (dacă este folosit un port implicit).

Constructorul "complet" are forma:

```
public URL(String protocol, String gazda,
           int port, String fisier)
```

în care parametrii au semnificații evidente.

Dintre metodele, toate publice, ale clasei URL menționăm următoarele:

```
String getPath()
    întoarce calea din locația URL;
int getPort()
    întoarce numărul portului din locația URL;
String getProtocol()
    întoarce protocolul din locația URL;
String getHost()
    întoarce adresa mașinii gazdă din locația URL;
String getFile()
    întoarce numele fișierului din locația URL;
String toString()
    întoarce reprezentarea sub formă de șir de caractere a locației URL.
```

13.3. Aplicații

Exemplul 2. Dorim să aflăm dacă un server având o adresă dată oferă sau nu servicii Web.

Va fi creat doar un socket de tip client, căruia nu este necesar să-i atașăm fluxuri de intrare și de ieșire. Adresa este furnizată în comanda java.

```
// Executarea se face prin: java WebCheck adresa
import java.io.*;
import java.net.*;

public class WebCheck {
    public static void main(String[] adresa) {
        Socket web = null;
        IO.write("Serverul " + adresa[0]);
```

```
try {
    web = new Socket(adresa[0],80);
    IO.writeln(" ofera servicii Web"); web.close();
}
catch(IOException e) {
    IO.writeln(" nu ofera servicii Web");
}
}
```

Exemplul 3. Ne propunem să realizăm un chat (o conversație) între două calculatoare.

Pentru simplificare, considerăm că fiecare mesaj trimis de la un calculator la celălalt se reduce la o linie de text.

Serverul este primul care trebuie pornit. Apoi clientul este cel care începe discuția, transmitând un mesaj și așteptând să primească mesajul de răspuns, activități care se repetă până când clientul transmite mesajul "STOP". Serverul repetă și el succesiunea de activități recepție + transmisie până când primește mesajul "STOP".

Clientul va folosi unitatea de compilare Client.java, iar serverul va folosi unitatea de compilare Server.java, prezentate în continuare.

```
// Unitatea de compilare Client.java
import java.net.*; import java.io.*;
class Client {
    public static void main (String[] sir) throws IOException {
        Socket cs = null;
        DataInputStream is = null;
        DataOutputStream os = null;
        try {
            cs = new Socket("localhost",5678);
            is = new DataInputStream(cs.getInputStream());
            os = new DataOutputStream(cs.getOutputStream());
        }
        catch(UnknownHostException e) {
            IO.writeln("Host inexistent");
        }
        DataInputStream stdin = new DataInputStream(System.in);
        String linie;
        for( ; ; ) {
            linie = stdin.readLine()+"\n";
            os.writeBytes(linie);
            IO.writeln("Transmisie : \t" + linie);
            if (linie.equals("STOP\n")) break;
            linie = is.readLine();
            IO.writeln("Receptie : \t" + linie);
        }
    }
}
```

```

        IO.println("GATA");
        cs.close(); is.close(); os.close();
    }
}

// Unitatea de compilare Server.java
import java.net.*; import java.io.*;
class Server {
    public static void main (String[] sir) throws IOException {
        ServerSocket ss = null; Socket cs = null;
        DataInputStream is = null;
        DataOutputStream os = null;
        ss = new ServerSocket(5678);
        IO.println("Serverul a pornit !");
        cs = ss.accept();
        is = new DataInputStream(cs.getInputStream());
        os = new DataOutputStream(cs.getOutputStream());

        DataInputStream stdin = new DataInputStream(System.in);
        String linie;
        for( ; ; ) {
            linie = is.readLine();
            IO.println("Receptie :\t" + linie);
            if (linie.equals("STOP")) break;
            linie = stdin.readLine()+"\n"; os.writeBytes(linie);
        }
        cs.close(); is.close(); os.close();
    }
}

```

Observație. Metoda `readLine` a clasei `DataInputStream` este considerată "depreciată" (*deprecated*), deoarece în unele situații nu asigură o conversie corectă octet → caracter; de aceea la compilare apare un mesaj de avertizare corespunzător. Este indicat să apelăm la transmiterea datelor la nivel de caracter dar, pentru simplificarea expunerii, nu vom proceda în acest mod.

Exemplul 4. Dorim să descărcăm un fișier aflat pe o pagină Web.

Adresa serverului, numele fișierului căutat și portul (80) vor fi specificate în această ordine în comanda `java`. Un exemplu este următorul:

```
java HTTPClient www.nus.edu.sg NUSinfo/UG/ug.html 80
```

Practic, vom transmite serverului, sub forma unui șir de caractere, mesajul:

```
GET nume_fișier
```

ce reprezintă o comandă din limbajul *HTTP* (*HyperText Transmission Protocol*) și pe care un server de Web știe să o interpreteze corespunzător.

Nu vom intra în detalii legate de HTTP. Despre comanda `GET` este însă necesar să spunem că ea trebuie urmată de o linie vidă.

Fișierul recepționat va fi inclus sub același nume în directorul curent.

```

// Unitatea de compilare HTTPClient.java
import java.io.*;
import java.net.*;
class HTTPClient {
    public static void main(String[] sir) throws IOException {
        Socket cs = null;
        int port = Integer.parseInt(sir[2]);
        DataInputStream is = null;
        DataOutputStream os = null;
        try {
            cs = new Socket(sir[0],port);
            is = new DataInputStream(cs.getInputStream());
            os = new DataOutputStream(cs.getOutputStream());
        }
        catch(UnknownHostException e)
        { IO.println("Host inexistent"); }

        String mesaj = "GET " + sir[1] + "\n\n";
        os.writeBytes(mesaj);

        DataOutputStream fis =
            new DataOutputStream(new FileOutputStream(sir[1]));
        int c;
        while( (c=is.read()) != -1) {
            IO.write("" + (char) c); fis.write(c);
        }
        fis.close(); is.close(); os.close(); cs.close();
        IO.println("\nGata!");
    }
}

```

Exemplul 5. Crearea propriului server de Web.

Dorim să creăm propriul nostru server Web, mai precis un server simplu, capabil doar să răspundă cererilor de descărcare a unui fișier efectuate prin comanda `GET` prezentată mai sus.

Vom folosi fire de executare pentru a satisface cereri de la mai mulți clienți. Serverul va aștepta în mod repetat câte o solicitare de la un client și, când aceasta sosește, va porni un nou fir de executare pentru clientul respectiv; metoda `run` va realiza descărcarea fișierului cerut.

Pentru o implementare corectă, trebuie spus că la transmiterea unei comenzi `GET`, după această comandă și până la linia vidă, vor mai fi transmise și alte comenzi, care nu vor fi descrise aici. Mai precis, vom proceda astfel: vom ignora liniile de text vide ce preced comanda `GET`, vom citi informațiile din această comandă și le vom prelucra corespunzător, iar apoi vom ignora liniile până la și inclusiv linia vidă.

```
// Unitatea de compilare HTTPServer.java
```

```

import java.io.*; import java.net.*;
import java.util.*; // pentru StringTokenizer
class HTTPServer {
    public static void main(String[] sir) throws IOException {
        ServerSocket ss = null; Socket cs = null;
        ss = new ServerSocket(8080);
        IO.println("\nServerul a pornit");
        while (true) {
            cs = ss.accept();
            IO.write("Client nou. Comanda: ");
            new Conexiune(cs);
        }
    }
}

class Conexiune extends Thread {
    Socket cs=null;
    DataOutputStream os = null; DataInputStream is = null;
    String linie;

    public Conexiune(Socket client) throws IOException {
        cs = client;
        is = new DataInputStream(cs.getInputStream());
        os = new DataOutputStream(cs.getOutputStream());
        this.start();
    }

    public void run() {
        String mesaj = null, fisier = null; int c;
        try {
            while ( (mesaj = is.readLine()) != null ) {
                if (mesaj.equals("")) break;
                IO.println(mesaj);
                StringTokenizer t = new StringTokenizer(mesaj);
                String token = t.nextToken();
                if (token.equals("GET")) fisier = t.nextToken();
            }
        } catch (IOException e) {
            { IO.println("Eroare receptie cerere Web"); }
        }
        try {
            FileInputStream f = new FileInputStream(fisier);
            while( (c=f.read()) != -1) os.write(c);
            os.write(-1); f.close();
            cs.close(); is.close(); os.close();
        } catch(IOException e) {
            { IO.println("Eroare fisier cerut"); }
        }
    }
}

```

Exemplul 6. Dorim să folosim un calculator aflat la distanță pentru a efectua anumite calcule asupra unor date din programul în curs de executare pe calculatorul local.

Întrucât scopul este de a pune în evidență mecanismul folosit și nu calculele în sine, vom reduce problema la a transmite la distanță două tablouri numerice de aceeași lungime și de a primi ca rezultat suma celor două tablouri.

Pentru transmiterea tablourilor, vom apela la mecanismul de serializare.

Clasa AdunaTablou, ce implementează interfața Serializable, conține:

- un câmp x de tip tablou de întregi, inițializat prin constructorul clasei;
- metoda out, care întoarce câmpul x.

Această clasă trebuie să se afle atât pe calculatorul local (pe post de client), cât și pe cel aflat la distanță (folosit ca server).

```

// Unitatea de compilare AdunaTablou.java
import java.io.*;
public class AdunaTablou implements Serializable {
    private int[] x;
    AdunaTablou(int[] mesaj) { x = mesaj; }
    public int[] out() { return x; }
}

```

Clientul creează două tablouri de întregi și un socket prin care se leagă la server. Fluxurile de intrare/ieșire sunt înzestrate, prin suprapunere, cu facilitățile oferite de clasele ObjectInputStream și ObjectOutputStream.

Cele două tablouri sunt serializate și transmise serverului. În fluxul de intrare va fi primit rezultatul: suma celor două tablouri. Acest rezultat este deserializat și tipărit la ieșirea standard:

```

// Unitatea de compilare AdunaClient.java
import java.io.*; import java.net.*;
public class AdunaClient {
    public static void main(String[] args) throws Exception {
        int[] a = {1,2,3,4}; int[] b = {4,3,2,1};
        int[] c = new int[10]; int i;
        Socket cs = new Socket("172.16.0.112",5555);
        IO.println("S-a legat");
        InputStream is = null; OutputStream os = null;
        is = cs.getInputStream(); os = cs.getOutputStream();
        ObjectOutputStream oos = new ObjectOutputStream(os);
        IO.println("S-a creat fisierul de iesire Object");
        ObjectInputStream ois = new ObjectInputStream(is);
        IO.println("S-a creat fisierul de intrare Object");

        AdunaTablou x = new AdunaTablou(a);
        AdunaTablou y = new AdunaTablou(b);
        AdunaTablou z = null;
    }
}

```

```

oos.writeObject(x); IO.write("Transmis: ");
for (i=0; i<a.length; i++) IO.write(a[i]+" ");
IO.writeln();
oos.writeObject(y); IO.write("Transmis: ");
for (i=0; i<b.length; i++) IO.write(b[i]+" ");
IO.writeln();

z = (AdunaTablou) ois.readObject(); c = z.out();
IO.write("Rezultat receptionat: ");
for (i=0; i<c.length; i++)
    IO.write(c[i]+" "); IO.writeln();

ois.close(); oos.close(); cs.close();
}
}

```

Complementar, serverul primește cele două obiecte, obține din ele tablourile respective, le adună, asociază rezultatului un obiect de tipul *AdunaTablou* și transmite acest obiect clientului:

```

// Unitatea de compilare AdunaServer.java
import java.io.*; import java.net.*;
public class AdunaServer {
    public static void main(String args[]) throws Exception {
        ServerSocket ss = null; Socket cs = null;
        InputStream is = null; OutputStream os = null;
        ss = new ServerSocket(5555);
        cs = ss.accept();
        IO.writeln("S-a legat");
        is = cs.getInputStream(); os = cs.getOutputStream();

        ObjectInputStream ois = new ObjectInputStream(is);
        ObjectOutputStream oos = new ObjectOutputStream(os);

        AdunaTablou x = null, y=null, z =null;;
        int[] a,b,c; int i;
        a = new int[10]; b = new int[10]; c = new int[10];
        x = (AdunaTablou) ois.readObject(); a = x.out();
        y = (AdunaTablou) ois.readObject(); b = y.out();
        IO.write("Receptionat: ");
        for (i=0; i<a.length; i++) IO.write(a[i]+" ");
        IO.writeln(); IO.write("Receptionat: ");
        for (i=0; i<b.length; i++) IO.write(b[i]+" ");
        IO.writeln();

        for (i=0; i<a.length; i++) c[i]=a[i]+b[i];
        IO.write("Rezultat transmis: ");
        for (i=0; i<c.length; i++) IO.write(c[i]+" ");
        z = new AdunaTablou(c);
        oos.writeObject(z); oos.flush();
        ois.close(); oos.close(); cs.close(); ss.close();
    }
}

```

13.4. Semnarea digitală a mesajelor

În drumul de la emițător la receptor, un mesaj poate fi interceptat de un "intrus", care poate să-l modifice, ceea ce poate conduce la situații cel puțin nedorite. De exemplu, un client poate trimite o cerere de plată unei bănci, cu specificarea contului în care suma să fie virată; intrusul care interceptează mesajul poate schimba numărul contului (înlocuindu-l cu unul propriu) și transmite mai departe mesajul către bancă.

Problema de care ne ocupăm este de a prezenta o modalitate prin care receptorul să fie sigur că mesajul primit este autentic, adică provine într-adevăr de la emițătorul real și nu de la un intrus. O soluție este ca emițătorul să *semneze* mesajul, adică să includă în el un tablou de octeți numit *semnătură digitală*.

Principalele condiții pe care trebuie să le îndeplinească o semnătură digitală sunt următoarele:

- să identifice elemente ca de exemplu numele emițătorului și momentul de timp al transmiterii;
- să fie relativ ușor de produs, dar și de recunoscut și verificat;
- să fie imposibil (sau măcar foarte dificil) de falsificat.

Pentru rezolvarea acestor probleme trebuie făcut apel la criptografie. Java permite utilizatorului să-și precizeze algoritmi proprii, dar implementează în mod standard algoritmul *DSA (Digital Signature Algorithm)*. În continuare vom reduce expunerea la utilizarea acestui algoritm.

Vom lucra, pentru simplificare, cu un unic emițător și un unic receptor. Ambii cunosc un tablou de octeți parola, ce constituie un număr de identificare (parolă) a clientului. În cazul general, receptorul dispune de un repertoriu al emițătorilor pe care îi cunoaște, repertoriu în care numele fiecărui emițător este însoțit de o parolă; din mesajul primit receptorul extrage numele emițătorului și deduce, pe baza repertoriului, parola corespunzătoare.

Prezentăm în continuare mecanismul standard de creare și utilizare a semnăturilor digitale.

Emițătorul întreprinde următoarele acțiuni:

- compune mesajul efectiv;
- construiește un *generator de chei*;
- construiește, cu ajutorul generatorului de chei, o *cheie privată* și o *cheie publică* (între aceste chei există o corespondență bine stabilită);
- construiește un *generator de semnături digitale*;
- prin intermediul generatorului de semnături și pe baza cheii private și a parolei, construiește tabloul de octeți ce constituie semnătura digitală;
- transmite mesajul efectiv, semnătura digitală și cheia publică.

Receptorul întreprinde următoarele acțiuni:

- primește mesajul efectiv, semnătura digitală și cheia publică;
- construiește un generator de semnături digitale;

- prin intermediul acestui generator verifică autenticitatea mesajului astfel: obține o semnătură digitală pe baza cheii publice primite și a parolei, apoi testează dacă rezultatul concordă cu semnătura digitală primită;
- dacă mesajul este considerat autentic, îl prelucrează.

Va fi folosit modelul Client-Server, cu emițătorul pe post de client și receptorul pe post de server.

În modelul prezentat mai sus vor interveni mai multe clase și interfețe publice din pachetul `java.security`. În mod standard, obiectele de tipul acestor clase sunt create nu prin invocarea unor constructori, ci a unor metode statice ce le întorc drept rezultat.

Clasa `KeyPair` este un depozitar al unei perechi formate din cheie publică și cea privată:

```
final class KeyPair extends Object implements Serializable
    PublicKey getPublicKey()
    PrivateKey getPrivate()
```

unde cele două metode întorc cheia publică, respectiv cea privată. `PublicKey` și `PrivateKey` sunt două interfețe fără constante și fără metode.

Clasa `KeyPairGenerator` constituie tipul generatorilor de chei.

```
abstract class KeyPairGenerator
    static KeyPairGenerator getInstance("DSA")
        creează și întoarce un generator de chei, folosind algoritmul DSA;
    void initialize(int lung)
        inițializează generatorul pentru chei de lungime lung; lungimea este măsurată
        în octeți, iar valorile ei standard sunt 512, 768 și 1024;
    KeyPair generateKeyPair()
        întoarce o pereche de chei.
```

Clasa `Signature` constituie tipul generatorilor de semnături. Semnătura digitală poate fi privită ca un câmp ascuns al acestei clase.

```
abstract class Signature
    static Signature getInstance("SHA/DSA")
        creează și întoarce un generator de semnături (pentru algoritmul DSA);
    final void initSign(PrivateKey privK)
        inițializează semnătura digitală pe baza cheii private privK;
    final void update(byte[] parola)
        actualizează semnătura digitală pe baza tabloului parola; actualizarea trebuie
        efectuată atât de emițător, cât și de receptor;
    final byte[] sign()
        întoarce șirul de octeți ce constituie semnătura digitală;
    void initVerify(PublicKey pubK)
        inițializează generatorul pentru verificare;
    final boolean verify(byte[] sig)
        verifică autenticitatea semnăturii sig.
```

Exemplul 7. Vom ilustra cele de mai sus pentru cazul în care mesajul efectiv este un fișier. Evident, vom apela la serializarea obiectelor.

Atât emițătorul, cât și receptorul, vor cunoaște clasa:

```
import java.io.*; import java.security.*;

public class SignedObject implements Serializable {
    String s;          // nume fisier
    byte[] b;          // continut fisier
    byte[] sig;        // semnatura digitala
    PublicKey pubK;    // cheia publica

    public SignedObject(String ss, byte[] bb,
                        byte[] ssid, PublicKey ppubK) {
        s = ss; b = bb; sig = ssid; pubK = ppubK;
    }
}
```

Emițătorul va executa programul:

```
// Executarea se face prin: java Emitator nume_fisier
import java.io.*; import java.net.*; import java.security.*;

public class Emitator {
    public static void main(String args[]) throws Exception {
        byte[] parola = {1,2,3};
        Socket cs = null;
        ObjectOutputStream os = null;
        cs = new Socket("localhost",4567);
        os=new ObjectOutputStream(cs.getOutputStream());

        IO.println("Generez cheile; s-ar putea sa dureze...");
        KeyPairGenerator kgen =
            KeyPairGenerator.getInstance("DSA");
        kgen.initialize(512);
        KeyPair kpair=kgen.generateKeyPair();
        PublicKey pubK = kpair.getPublic();
        PrivateKey privK = kpair.getPrivate();

        IO.println("Generez semnatura...");
        Signature sig = Signature.getInstance("SHA/DSA");
        sig.initSign(privK); sig.update(parola);
        FileInputStream fis = new FileInputStream(args[0]);
        byte[] b = new byte[fis.available()];
        fis.read(b);

        SignedObject SObj =
            new SignedObject(args[0],b,sig.sign(),pubK);
        os.writeObject(SObj);
        IO.println("S-au transmis: fisier + semnatura !");
        fis.close(); os.close(); cs.close();
    }
}
```

Receptorul va executa programul:

```
import java.io.*; import java.net.*;
import java.security.*;

public class Receptor {
    public static void main(String[] args) throws Exception {
        byte[] parola = {1,2,3};
        ServerSocket ss = null; Socket cs = null;
        ObjectInputStream ois = null;
        KeyPairGenerator kgen; KeyPair kpair;

        ss = new ServerSocket(4567);
        IO.println("Serverul a pornit...");
        cs = ss.accept();
        ois = new ObjectInputStream(cs.getInputStream());

        SignedObject SObj = (SignedObject) ois.readObject();
        IO.println("S-a receptionat obiectul!");

        Signature sig = Signature.getInstance("SHA/DSA");
        sig.initVerify(SObj.pubK); sig.update(parola);
        boolean valid = sig.verify(SObj.sig);
        if(valid) {
            IO.println("S-a primit fisierul : " + SObj.s);
            FileOutputStream fos = new FileOutputStream(SObj.s);
            fos.write(SObj.b); fos.close();
        }
        else IO.println("Semnatura nu este valida!");
        ois.close(); cs.close();
    }
}
```

14

INVOCAREA LA DISTANȚĂ A METODELOR (RMI)

14.1. Introducere

Să presupunem că de pe o mașină Java dorim să comandăm executarea unui cod pe o altă mașină. Putem presupune că acest cod constituie o metodă a unei clase; metoda urmează a fi executată de un obiect de tipul clasei, obiect aflat pe a doua mașină. Atunci pe prima mașină trebuie să avem informații despre acest obiect, numit *obiect (aflat) la distanță*, pentru a putea să invocăm prin intermediul său metoda respectivă.

Folosind modelul Client - Server, este clar că trebuie făcute următoarele presupuneri:

- clientul trebuie să cunoască o referință la obiectul la distanță;
- invocarea este făcută de client, dar realizată efectiv de server prin intermediul obiectului, care este "gata să primească" cererea;
- clientul și serverul trebuie să fie conectați, pentru a permite serverului să primească informații despre metoda ce trebuie invocată și argumentele ce trebuie folosite la invocare; având aceste informații, serverul poate comanda obiectului să invoce metoda pentru argumentele date (primite).

Ce trebuie să cunoască clientul despre obiectul aflat la distanță? Un minim îl constituie numele și signatura metodelor ce urmează a fi invocate (având deci încredere că obiectul de pe server respectă contractul stabilit la scrierea clasei a cărei instanțiere este) și câmpurile ce sunt adresate. În acest scop, structura cea mai adecvată este o interfață care să cuprindă aceste informații.

Ce trebuie să cuprindă clasa de pe server a cărei instanță este obiectul? Răspunsul este clar: clasa trebuie să fie o implementare (directă sau indirectă) a interfeței folosite de client.

Se observă că avem deja cunoștințele necesare unei invocări la distanță (și anume cu ajutorul socket-urilor) și putem prezenta un prim exemplu.

Exemplul 1. Clasa cu care lucrăm conține un câmp și două metode: pune (care incrementează valoarea câmpului cu valoarea primită ca parametru) și

curent (care întoarce valoarea curentă a câmpului). Putem gândi aceste acțiuni ca urmărind evidența unui cont, inițial egal cu zero. Clientul actualizează contul prin depuneri/retrageri și poate cere valoarea sumei din cont. Sfârșitul activității clientului are loc atunci când el actualizează contul cu valoarea 0. Atât clientul cât și serverul vor folosi interfața *Interf* ce anunță metodele *pune* și *curent*; implementările metodelor vor urmări acțiunile descrise mai sus.

Clientul și serverul vor fi conectați printr-un socket. Clientul trimite în mod repetat serverului fie mesajul "pune" urmat de suma cu care actualizează contul, fie mesajul "curent" (caz în care așteaptă să-i sosească o valoare). Serverul primește în mod repetat câte un mesaj de la client. Dacă acesta este "pune", atunci mai citește o valoare din fluxul de intrare și o adaugă contului curent; în caz contrar întoarce valoarea contului curent.

Pe mașina gazdă pentru client se află interfața *Interf* :

```
public interface Interf {
    void pune(int i);
    int curent();
}

și clasele ClientSocket și Client:

// unitatea de compilare ClientSocket.java
import java.io.*;
import java.net.*;
public class ClientSocket implements Interf {
    Socket cs; DataOutputStream os; DataInputStream is;
    ClientSocket() throws IOException {
        cs = new Socket("localhost", 6789);
        os = new DataOutputStream(cs.getOutputStream());
        is = new DataInputStream(cs.getInputStream());
    }
    public void pune(int i) {
        try { os.writeUTF("pune"); os.writeInt(i); }
        catch(IOException e) { }
    }
    public int curent() {
        int k = 9999;
        try { os.writeUTF("curent"); k = is.readInt(); }
        catch(IOException e) { }
        return k;
    }
    public void terminare() throws IOException {
        os.close(); is.close(); cs.close();
    }
}

// unitatea de compilare Client.java
public class Client {
    public static void main(String[] sir) throws Exception {
        int val; ClientSocket CS = new ClientSocket();
```

```
for ( ; ; ) {
    val = (int) IO.read(); if (val==0) break; CS.pune(val);
}
IO.writeln("Valoarea curenta = " + CS.curent());
CS.pune(0); CS.terminare();
}
}
```

Pe mașina gazdă pentru server se află clasele *Clasa* (care implementează implicit interfața *Interf*) și *Server*:

```
// unitatea de compilare Clasa.java
class Clasa {
    private int val;
    Clasa(int i) { val = i; }
    void pune(int i) { val += i; }
    int curent() { return val; }
}

// unitatea de compilare Server.java
import java.io.*; import java.net.*;
class Server {
    public static void main(String[] sir) throws Exception {
        ServerSocket ss = new ServerSocket(6789);
        Socket cs = ss.accept();
        Clasa Ob = new Clasa(0);
        DataOutputStream os =
            new DataOutputStream(cs.getOutputStream());
        DataInputStream is =
            new DataInputStream(cs.getInputStream());
        String metoda; int inc = 0;
        do {
            metoda = is.readUTF();
            if ( metoda.equals("pune") ) {
                inc = is.readInt();
                if (inc==0) continue;
                Ob.pune(inc);
            }
            else if ( metoda.equals("curent") )
                os.writeInt( Ob.curent() );
            else {
                IO.writeln("Metoda inexistentă!"); System.exit(0);
            }
        }
        while (inc != 0);
        os.close(); is.close(); cs.close(); ss.close();
    }
}
```

Observații:

- în clasa *Clasa* apare câmpul *val* declarat cu *private*, ce nu apare (bineînțeles) în *Interf*;
- s-a folosit mecanismul de comunicare prin intermediul socket-urilor;

- în metoda principală a clasei Server s-a prevăzut un mecanism de terminare a acțiunii sale (și anume atunci când se cere să se incrementeze cu 0 valoarea curentă val a obiectului Ob) numai pentru a verifica programul; în mod normal serverul își continuă acțiunea.

14.2. Principiile modelului RMI

Invocarea la distanță a metodelor (*RMI = Remote Method Invocation*) este un model larg folosit în programarea concurrentă. Java preia acest model și îl extinde la programarea distribuită. La bază stă tot utilizarea socket-urilor, dar acest lucru devine transparent pentru utilizator, datorită nivelului superior de abstractizare.

Modelul RMI urmărește ca invocarea la distanță a unei metode să apară în cod sub o formă asemănătoare invocării locale.

Ideea de bază este simplă:

- serverul creează un obiect și îl înscrie într-un registru;
- clientul accesează acel registru, regăsește obiectul și invocă prin intermediul său metode ale clasei a cărei instanță este obiectul.

Facilitățile Java pentru invocarea la distanță a metodelor sunt în principal materializate în clase din pachetul `java.rmi` și vor fi prezentate în continuare.

Pentru a ușura înțelegerea principiilor modelului RMI, vom presupune deocamdată că serverul și clientul se află pe aceeași mașină. Cazul în care serverul și clientul se află pe mașini diferite va fi tratat în subcapitolul 14.5.

Un registru de nume pune la dispoziție facilități de stocare și regăsire a referințelor la obiectele (aflate) la distanță. Un astfel de registru ia naștere de exemplu ca rezultat al executării comenzii `rmiregistry`, ce are una dintre următoarele forme:

```
>rmiregistry
>rmiregistry port
```

Registru de nume furnizează un serviciu simplu de nume, care pune în corespondență o referință a obiectului la distanță cu numele obiectului; drept urmare, clientul poate obține un obiect la distanță cunoscând doar numele său. Registru este pornit pe mașina server, dar comunică și cu clientul, deci și acesta trebuie să aibă acces la el. Ca orice instrument bazat pe TCP/IP, registru așteaptă cererile la un port. Portul implicit pentru lucrul cu RMI este 1099.

Accesarea registrului pentru înscrierea sau regăsirea unui obiect se face prin intermediul metodelor clasei `Naming` din pachetul `java.rmi`.

În cele ce urmează vom nota prin `I` interfața comună (aflată atât pe server, cât și pe client), prin `C` clasa de pe server care o implementează, prin `Ob` obiectul de tipul clasei `C` pus la dispoziție de server, iar prin `met` metoda clasei `C` care urmează a fi invocată de client prin intermediul obiectului `Ob`.

Interfața `I` trebuie să extindă interfața `Remote` din pachetul `java.rmi`, iar toate metodele sale trebuie să conțină clauza:

```
throws RemoteException
```

prin care se anunță că poate fi lansată excepția `RemoteException`.

În mod standard, pe server sunt întreprinse următoarele acțiuni:

- clasa `C` trebuie să extindă clasa `UnicastRemoteObject` din pachetul `java.rmi.server`; clasa `UnicastRemoteObject` furnizează elementele necesare (referitoare la invocări, parametri, rezultat întors) pentru referințe la obiecte, utilizând fluxuri TCP;

- este creat obiectul `Ob` ce urmează a fi folosit de la distanță;

- obiectul `Ob` este înscris în registrul de nume, de exemplu prin invocarea metodei statice `rebind` a clasei `Naming`. La invocare trebuie să precizăm atât numele sub care dorim să înscriem obiectul `Ob`, cât și acest obiect.

Clientul întreprinde următoarele acțiuni:

- obține o referință la obiectul la distanță; aceasta se poate realiza invocând metoda statică `lookup` a clasei `Naming`, cu precizarea numelui sub care a fost înscris obiectul `Ob` în registru;

- folosește referința pentru a invoca metoda `met` a clasei `C`.

Suntem acum în măsură să prezentăm un exemplu ce ilustrează elementele de mai sus. Avertizăm cititorul că pentru executarea aplicației mai sunt necesare câteva elemente, care nu apar în cod și care vor fi prezentate ulterior.

Exemplul 2. Reluăm exemplul anterior și descriem cum poate RMI să rezolve problema, fără a apela (explicit) la socket-uri.

Următoarea interfață trebuie să apară atât pe client, cât și pe server:

```
import java.rmi.*;
public interface InterRMI extends Remote {
    void pune(int i) throws RemoteException;
    int curent() throws RemoteException;
}
```

Serverul folosește clasa `ServerRMI`:

```
import java.rmi.*; import java.rmi.server.*;
public class ServerRMI extends UnicastRemoteObject
    implements InterRMI {
    int val;
    public ServerRMI(int i) throws RemoteException { val = i; }
    public void pune(int i) throws RemoteException { val += i; }
    public int curent() throws RemoteException { return val; }

    public static void main(String[] args) throws Exception {
        ServerRMI Ob = new ServerRMI(0);
        Naming.rebind("rmi:///Ob", Ob);
        System.out.println("Serverul a pornit");
    }
}
```

care implementează interfața `InterRMI` și creează serverul efectiv.

Clientul folosește clasa `ClientRMI`:

```
import java.rmi.*;
public class ClientRMI {
    public static void main(String[] sir) throws Exception {
        int val;
        InterRMI Ob = (InterRMI) Naming.lookup("rmi:///Ob");
        for ( ; ; ) {
            val = (int) IO.read();
            if (val==0) break;
            Ob.pune(val);
        }
        IO.writeln("Val. curenta este : " + Ob.curent());
    }
}
```

care prevede (ca și în exemplul anterior) citirea și transmiterea de valori nenule către server și apoi (după ce a fost transmisă serverului valoarea 0) afișarea valorii curente.

Observații:

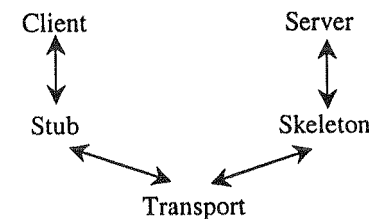
- dacă există mai mulți clienți, ei folosesc același câmp `val` al obiectului `Ob`, deci valoarea tipărită ca efect al invocării metodei `curent` va ține seamă și de invocările efectuate de alți clienți; cu alte cuvinte, clienții folosesc un cont comun;
- obiectul `Ob`, și implicit serverul, vor rămâne active și după ce clienții și-au încheiat activitatea. Acest lucru este normal, fiind vorba de un server, care nu știe când mai poate apărea un nou client;
- serverul nu se oprește după ce este executată ultima instrucțiune a metodei principale din `ServerRMI`, deoarece legătura stabilită prin invocarea metodei `Naming.rebind` rămâne activă. Oprirea serverului, care contrazice de fapt noțiunea de server, poate fi realizată prin invocarea metodei `Naming.unbind` sau în mod forțat (de exemplu prin `System.exit(0);`);
- aplicațiile sunt evident mai simple decât în exemplul precedent, datorită nivelului superior de abstractizare la care lucrăm.

Așa cum am anunțat anterior, pentru executarea aplicației mai trebuie făcute câteva precizări, care fac parte integrantă din mecanismul RMI.

Clasa `C` aflată pe server, care extinde interfața `Remote`, trebuie să fie supusă și unei compilări suplimentare, realizate de compilatorul `rmic`, aflat în același director cu `javac`. În urma compilării rezultă următoarele două clase:

```
C_Stub.class
C_Skeleton.class
```

numite în continuare *stub* și *skeleton*. Acestea servesc drept intermediari pentru transportul prin rețea, conform următoarei scheme:



`Stub`-ul preia de la client informațiile ce trebuie comunicate obiectului la distanță (numele metodei și lista de argumente) și le transmite prin rețea, într-un anumit format, `skeleton`-ului.

`Skeleton`-ul reconstituie informațiile primite prin rețea și le transmite serverului, care comandă obiectului să invoce metoda cu argumentele respective. Rezultatul este întors `skeleton`-ului și apoi transmis prin rețea. `Stub`-ul îl preia și îl comunică clientului.

Din cele de mai sus rezultă că `stub`-ul este creat pe server, dar este folosit de client. Întrucât lucrăm în ipoteza că serverul și clientul se află pe aceeași mașină, accesul clientului la `stub` poate fi asigurat prin includerea în comanda `java` a opțiunii `classpath` cu specificarea directorului în care serverul este activ.

Opțiunea `classpath` a fost prezentată în *Capitolul 4*. Menționăm în plus că fiecare fereastră de comenzi are propria sa cale pentru cod.

Prezentăm în continuare acțiunile ce trebuie întreprinse pentru executarea aplicațiilor de mai sus.

Fie `dirserver` și `dirclient` directoarele în care se află clasele pentru server și client. Din aceste directoare compilăm `ClientRMI.java`, respectiv `ServerRMI.java`. În plus, din directorul `dirserver` executăm:

```
dirserver>rmic ServerRMI
```

```
dirserver>java ServerRMI
```

```
dirclient>java -classpath dirserver;. ClientRMI
```

Încheiem acest subcapitol cu observația că dacă o metodă invocată la distanță are ca parametru un obiect (și nu un tip primitiv ca în cazul metodei `pune`), atunci clasa de tipul căreia este obiectul trebuie să fie serializabilă. Același lucru este valabil pentru situația în care rezultatul întors de metodă este un obiect.

14.3. Facilități Java pentru invocarea la distanță

• Interfața Remote

Această interfață apare în pachetul `java.rmi` și este declarată prin:

```
public interface Remote
```

Rolul său este de a identifica tipurile ale căror metode urmează a fi invocate de la distanță (de pe o mașină Java virtuală care nu este cea locală). Orice obiect aflat la distanță trebuie să implementeze o interfață `I` ce extinde direct sau indirect interfața `Remote`; numai metodele anunțate în `I` pot fi invocate de la distanță.

• Clasa UnicastRemoteObject

Această clasă, aflată în pachetul `java.rmi.server`, oferă facilități pentru referințe active biunivoce (unicast); prin intermediul constructorului pot fi create obiecte la distanță, ale căror metode să poată fi invocate folosind fluxuri TCP. Un obiect la distanță are ca tip o clasă ce extinde clasa `UnicastRemoteObject`.

```
public class UnicastRemoteObject
```

```
protected UnicastRemoteObject() throws RemoteException
```

```
public static RemoteStub exportObject(Remote Ob)
```

exportă obiectul la distanță, pentru a-l face disponibil să primească apeluri, folosind un port anonim (vezi subcapitolul 14.4).

• Clasa Naming

Clasa `Naming` din pachetul `java.rmi` oferă metode publice și statice pentru accesarea registrului de nume. Parametrul nume din metodele prezentate mai jos trebuie să fie o adresă URL de forma:

```
rmi://gazda:port/nume_efectiv
```

unde `rmi` este numele serviciului, iar `gazda` este o adresă IP numerică sau simbolică. Atât `gazda` cât și `port` pot lipsi, caz în care este sunt considerate a fi `localhost`, respectiv `1099`.

```
public final class Naming extends Object
```

```
void bind(String nume, Remote Ob)
```

înscrie în registru numele `nume`, asociindu-i obiectul `Ob`;

```
void rebind(String nume, Remote Ob)
```

înscrie în registru numele `nume`, asociindu-i obiectul `Ob` și anulând o eventuală asociere anterioară;

```
void unbind(String nume)
```

distruge asocierea precedentă din registru a lui `nume`;

```
Remote lookup(String nume)
```

întoarce o referință (un stub) la obiectul la distanță asociat în registru lui `nume`.

Metodele de mai sus pot lansa excepțiile `MalformedURLException`, `RemoteException`, `NotBoundException` și `AlreadyBoundException`.

O referință activă deținută de client are asociată un timp de arendare, a cărui valoare implicită este de 10 minute. Timpul de arendare este controlat de proprietatea sistemului cu numele `java.rmi.dgc.leaseValue`. Dacă înainte de expirarea acestui timp clientul nu reface legătura, referința este considerată "depășită" și este pusă la dispoziția collectorului de reziduuri.

Pe server, obiectul la distanță este pus și el la dispoziția collectorului de reziduuri dacă nici un client nu mai deține referințe către el și dacă legătura către el în registru dacă este suspendată (prin `unbind`).

Ilustrăm cele de mai sus pe un nou exemplu.

Exemplul 3. Dorim să modificăm *Exemplul 2* astfel încât fiecare client să aibă "serverul său propriu", de exemplu pentru ca valoarea curentă `val` din clasa `ServerRMI` să nu mai fie comună tuturor obiectelor client, adică fiecare client să aibă un cont separat.

Clientul va folosi interfețele `InterRMI` și `InterAdaug`, precum și clasa `ClientRMI`. Serverul va folosi interfețele `InterRMI` și `InterAdaug`, implementate respectiv de clasele `ServerRMI` și `Adaug`, precum și clasa `Server`.

Schema folosită este următoarea:

- metoda principală din `Server` creează un obiect de tipul `Adaug` (deci un server) cu numele `Ob`, cunoscut și de clienți;

- fiecare client se leagă întâi la acest server și prin invocarea metodei `adaug` a clasei `Adaug` primește un obiect de tipul `ServerRMI`; acesta va fi serverul său propriu, pe care îl va folosi la invocarea metodelor pune și curent.

Așa cum am menționat, atât serverul cât și clienții cunosc interfețele `InterRMI` și `InterAdaug`:

```
// unitatea de compilare InterRMI.java
import java.rmi.*;
public interface InterRMI extends Remote {
    void pune(int i) throws RemoteException;
    int curent() throws RemoteException;
}
```

```
// unitatea de compilare InterAdaug.java
import java.rmi.*;
public interface InterAdaug extends Remote {
    InterRMI adaug() throws Exception;
}
```

Clienții vor folosi clasa `ClientRMI`:

```
import java.rmi.*;
public class ClientRMI {
    public static void main(String[] args) throws Exception {
```

```

InterAdaug Ob = (InterAdaug) Naming.lookup("rmi:///Ob");
InterRMI ObNou = Ob.adaug();
int val;
for( ; ; ) {
    val = (int) IO.read(); if(val==0) break;
    ObNou.pune(val);
}
IO.writeln("Val. curenta este: " + ObNou.curent() );
}

Serverul va utiliza clasele Adaug, ServerRMI și Server:

// unitatea de compilare Adaug.java
import java.rmi.*; import java.rmi.server.*;
public class Adaug extends UnicastRemoteObject
    implements InterAdaug {
    static int contor;
    Adaug() throws RemoteException {}
    public InterRMI adaug() throws Exception {
        return new ServerRMI(0,contor++);
    }
}

// unitatea de compilare ServerRMI.java
import java.rmi.*; import java.rmi.server.*;
public class ServerRMI extends UnicastRemoteObject
    implements InterRMI {
    private int val, nr_ordine;
    public ServerRMI(int i,int nr) throws RemoteException {
        val = i; nr_ordine = nr;
    }
    public void pune(int i) {
        val += i; IO.writeln(nr_ordine + "\t: " + val);
    }
    public int curent() { return val; }
}

// unitatea de compilare Server.java
import java.rmi.*;
public class Server {
    public static void main(String[] args) throws Exception {
        Adaug Ob = new Adaug();
        Naming.rebind("rmi:///Ob",Ob);
        IO.writeln("Serverul s-a legat la registrul RMI");
    }
}

```

Vom crea doi clienți. Fie `dirserver`, `dirclient0` și `dirclient1` directoarele în care se află clasele pentru server și clienți.

Din directorul clienților executăm:
`javac ClientRMI.java`
 Din directorul `dirserver` executăm:

```

dirserver>javac ServerRMI.java
dirserver>rmic ServerRMI
dirserver>rmic Adaug

```

deoarece și clasa `Adaug` implementează interfețe la distanță.

Deschidem o fereastră în care, din `dirserver`, executăm:

```
dirserver>rmiregistry
```

Deschidem o a doua fereastră în care, din `dirserver`, executăm:

```
dirserver>java ServerRMI
```

Mai deschidem câte o fereastră pentru fiecare client în care executăm:

```
dirclient0>java -cp dirserver;. ClientRMI
```

```
dirclient1>java -cp dirserver;. ClientRMI
```

Constructorul `Adaug` fără parametri și a cărui acțiune este nulă trebuie totuși specificat explicit, pentru că poate eventual lansa excepția: `RemoteException`.

De această dată fiecare client "are serverul său", deci valorile tipărite la invocarea metodei `curent` se referă numai la incrementările făcute de el.

Trebuie avut în permanență grijă ca serverul și clienții să folosească numai ceea ce cunosc. Astfel, dacă rezultatul întors de metoda `adaug` din clasa `Adaug` ar fi precizat ca fiind de tipul `ServerRMI` și nu `InterRMI`, s-ar semna o eroare.

14.4. Apeluri inverse

Am văzut că Java, prin mecanismul RMI, oferă posibilitatea ca de pe o mașină client să invocăm o metodă `metS` care să fie executată de o mașină server. Pentru aceasta, pe mașina server a fost creat un obiect `Obs` de tipul clasei care include metoda respectivă și a fost întoarsă clientului o referință la acest obiect la distanță. Prin intermediul acestei referințe, de pe mașina client poate fi invocată metoda `metS`, care (bineînțeles) este executată pe mașina server.

În programarea concurentă, noțiunea de invocare la distanță cuprinde și un aspect suplimentar: posibilitatea ca mașina server să "întoarcă un răspuns", adică să determine executarea unei metode pe mașina client. Această facilități suplimentară poartă numele de *apel invers*. Exemplul clasic care ilustrează invocarea la distanță, inclusiv apelurile inverse, îl constituie transmiterea unui document prin fax: după recepționarea mesajului este întors un răspuns conținând scurte informații asupra transferul efectuat.

Java oferă și facilitățile de a folosi apeluri inverse. Principial, lucrurile decurg analog ca la invocarea la distanță a unei metode. Fie `metC` metoda care trebuie invocată pe mașina client din cadrul metodei `metS`. Este evident necesar ca pe mașina client să fie creat un obiect `ObC` de tipul clasei care include metoda `metC`, iar mașina server să primească o referință la acest obiect, pentru ca (pe baza acestei referințe) să poată invoca metoda `metC`, ce va fi executată pe mașina client.

Mai este de așteptat ca implementarea apelurilor inverse să fie mai simplă, deoarece legătura între cele două mașini este deja stabilită.

Într-adevăr, pentru realizarea apelurilor inverse, trebuie întreprinse doar următoarele două acțiuni:

- 1) crearea obiectului ObC și pregătirea lui pentru a fi exportat;
- 2) transmiterea obiectului ObC mașinii server.

Prima acțiune presupune că ObC este o instanță a unei clase ce extinde interfața Remote. Pregătirea sa pentru export este realizată prin invocarea următoarei metode publice și statice a clasei UnicastRemoteObject: RemoteStub `exportObject`(Remote ObC) throws RemoteException

A doua acțiune poate fi realizată simplu, comunicând serverului obiectul ObC prin includerea lui ca argument la invocarea unei metode prin intermediul referinței la obiectul Obs. Amintim că aceasta presupune că obiectul ObC este de tipul unei clase serializabile.

Exemplul 4. Revenim la *Exemplul 2*. Dorim ca o actualizare a contului (realizată printr-o invocare a metodei pune) să fie posibilă numai dacă valoarea contului rămâne pozitivă. Mai mult, clientul trebuie să tipărească unul dintre mesajele "O.K." sau "Insuficient", după cum actualizarea a fost sau nu efectuată.

Prezentăm modificările ce intervin față de *Exemplul 2*.

Este adăugată interfața Inter, atât pe client cât și pe server; ea anunță metoda raspuns cu un parametru boolean.

Clasa Client, prezentă pe client, implementează interfețele Inter și Serializable. Metoda raspuns tipărește unul dintre cele două mesaje de mai sus, în funcție de valoarea parametrului b.

Interfața InterRMI anunță în plus metoda register cu un parametru de tipul Inter.

În metoda principală a clasei ClientRMI s-au adăugat instrucțiunile: Client c = new Client(); UnicastRemoteObject.exportObject(c); Ob.register(c); care folosesc exact tiparul prezentat la descrierea apelurilor inverse.

În clasa ServerRMI intervin următoarele modificări:

- este adăugat câmpul ob de tipul Inter;
- este adăugată metoda register prin care câmpul ob primește o valoare;
- metoda pune din clasa ServerRMI setează variabila booleană b prin:

b = val+i >= 0;
și realizează apelul invers prin: ob.raspuns(b);

```
// Unitatea de compilare InterRMI.java
import java.rmi.*;
public interface InterRMI extends Remote {
    void pune(int i) throws RemoteException;
    int curent() throws RemoteException;
    void register(Inter ob) throws RemoteException;
}
```

```
// Unitatea de compilare Inter.java
import java.rmi.*;
public interface Inter extends Remote {
    public void raspuns(boolean b) throws RemoteException;
}

// Unitatea de compilare Client.java
import java.io.*; import java.rmi.*;
public class Client implements Inter, Serializable {
    public void raspuns(boolean b) {
        if (b) IO.println("O.K.");
        else IO.println("Insuficient");
    }
}

// Unitatea de compilare ClientRMI.java
import java.rmi.*; import java.rmi.server.*;
public class ClientRMI {
    public static void main(String[] sir) throws Exception {
        int val;
        InterRMI Ob = (InterRMI) Naming.lookup("rmi:///Ob");
        Client c = new Client();
        UnicastRemoteObject.exportObject(c);
        Ob.register(c);
        for ( ; ; ) {
            val = (int) IO.read(); if(val==0) break;
            Ob.pune(val);
        }
        IO.println("Val.curenta este : " + Ob.curent());
        System.exit(0);
    }
}

// Unitatea de compilare ServerRMI.java
import java.rmi.*; import java.rmi.server.*;
public class ServerRMI extends UnicastRemoteObject
    implements InterRMI {
    private int val; Inter ob;

    public ServerRMI(int i) throws RemoteException { val = i; }

    public void pune(int i) throws RemoteException {
        boolean b;
        b = val+i >= 0; if (b) val += i;
        IO.println("val : " +val); ob.raspuns(b);
    }

    public int curent() { return val; }

    public void register(Inter ob) { this.ob = ob; }

    public static void main(String[] sir) throws Exception {
        ServerRMI Ob = new ServerRMI(0);
        Naming.rebind("rmi:///Ob",Ob);
    }
}
```


Clasa Client trebuie și ea compilată cu `rmic`, deoarece implementează interfața `Inter`, care extinde `Remote`.

Serverul este pornit prin:
`java -cp dirclient;. ServerRMI`
 deoarece trebuie să aibă acces la `Client_Stub.class`.

Clientul este pornit, la fel ca în *Exemplul 2*, prin:
`java -cp dirserver;. ClientRMI`

14.5. Încărcarea dinamică a claselor

În acest subcapitol vom expune una dintre modalitățile de lucru pentru situația când client și serverul nu se află pe aceeași mașină. Problema principală ce trebuie rezolvată este cea a regăsirii claselor și încărcării lor.

O modalitate constă în a copia stub-ul pe client și a preciza în calea pentru cod locația acestei clase. Considerăm însă acest mod de lucru ca fiind restrictiv și nenatural. De aceea ne îndreptăm atenția asupra posibilității de a încărca *în mod dinamic* pe client clasele necesare aflate pe server.

În esență, există patru procese ce își desfășoară concomitent activitatea:

- clientul;
- serverul;
- registrul RMI;
- un server de Web ce se ocupă de încărcarea claselor și care va fi prezentat în continuare.

Vom presupune, pentru simplificare, că serverul de Web este lansat de pe aceeași mașină ca și serverul.

Serverul specifică o *bază pentru cod*, care este adresa URL a serverului de Web, apoi înregistrează obiectul (împreună cu baza pentru cod) în registrul RMI.

Clientul, prin invocarea metodei `Naming.lookup`, accesează registrul RMI și importă o instanță a stub-ului obiectului la distanță. În continuare, clientul cere serverului Web clasa stub (dacă această clasă nu se află în calea pentru cod). Pentru îndeplinirea acestor cereri, este necesar ca pe client să fie instalat un gestionar de securitate și să fie specificat un fișier de permisiuni.

• Baza pentru cod

În *Capitolul 4* am prezentat modul în care poate fi folosită variabila de mediu `CLASSPATH` pentru regăsirea claselor necesare executării aplicației. Comanda `SET CLASSPATH` și opțiunea `-classpath` din comenzile `java` și `javac` permit precizarea unei căi pentru clase, indicând locațiile unde vor fi căutate aceste clase.

Dacă dorim însă să regăsim clase aflate pe o altă mașină, modalitatea de mai sus nu mai poate fi folosită, deoarece calea pentru cod este valabilă doar pe mașina locală.

Baza pentru cod constituie sursa la care vor fi căutate clasele aflate pe o altă mașină. Această sursă este o locație URL.

La executarea unei aplicații pot fi specificate diferite proprietăți ale sistemului, printre care se numără și baza pentru cod. Specificarea se poate face fie din program, fie la lansarea interpretorului Java. Vom folosi a doua variantă, concretizată în inserarea în comanda `java` a opțiunii:

`-Djava.rmi.server.codebase="adresa"`
 unde adresa este o locație URL. Dacă locația URL este un director, după numele acestuia trebuie să apară `'/'`.

Când serverul RMI înregistrează un obiect la distanță (prin `Naming.bind` sau `Naming.rebind`), în registrul RMI este înscrisă și baza pentru cod.

Când clientul invocă metoda `Naming.lookup` pentru detectarea unui obiect la distanță pe baza numelui sub care a fost înscris în registru, se obține atât o referință la acel obiect (o instanță stub), cât și definiția clasei de tipul căreia este obiectul; definiția clasei este căutată la locația URL înscrisă odată cu obiectul în registrul RMI. Putem spune că baza pentru cod este folosită de clientul RMI pentru a executa un cod pe server.

Observații:

- căutarea codului începe prin cercetarea căii pentru cod (deci local). De aceea, pentru a evita obținerea unui cod nedorit, este indicată o setare corespunzătoare a variabilei de mediu `CLASSPATH`;
- dacă folosim un server de Web pentru încărcarea claselor, în baza pentru cod trebuie specificată locația URL a acestui server.

• Un miniserver de Web

Regăsirea cu succes a claselor necesare executării aplicației nu implică și încărcarea lor pe mașina locală. Dar știm că, de exemplu, clientul dorește nu numai regăsirea stub-ului, ci și încărcarea lui pe mașina gazdă. În acest scop este necesar să avem la dispoziție un server HTTP sau FTP; în continuare vom folosi un server HTTP.

La încercarea de încărcare dinamică a claselor, RMI va face apel la serverul de Web. Prezentăm în continuare un miniserver de Web ce reprezintă o variantă simplificată a celui din documentația de firmă.

Miniserverul trebuie pornit prin:

`java ClassFileServer port sursa/`

unde `port` este numele portului folosit, iar `sursa` specifică locația la care trebuie căutate stub-urile.

Pentru fiecare cerere este creat un socket și lansat un fir de execuție. Metoda run din clasa Conexiune realizează în principal următoarele:

- atașează socket-ului un flux de intrare și unul de ieșire;
- prin invocarea metodei getPath primește în fluxul de intrare o comandă GET și extrage din ea numele fișierului care trebuie transmis;
- prin invocarea metodei getBytes este obținut conținutul fișierului;
- acest conținut este transmis fluxului de ieșire atașat socket-ului, într-un format propriu limbajului HTTP.

```
import java.io.*; import java.net.*;
public class ClassFileServer {
    private String classpath;
    private int port;
    private static ServerSocket ss = null;
    ClassFileServer(int p, String s) {
        port = p; classpath = s;
    }
    public static void main(String args[]) throws IOException {
        int port; String classpath = "";
        port = Integer.parseInt(args[0]); classpath = args[1];
        ss = new ServerSocket(port);
        IO.println("ClassFileServer a pornit!");
        while(true) {
            Socket cs = ss.accept(); new Conexiune(cs, classpath);
        }
    }
}

class Conexiune extends Thread {
    Socket cs; DataOutputStream os; DataInputStream is;
    String classpath;
    public Conexiune(Socket cs, String s) {
        this.cs = cs; classpath = s;
        try {
            is = new DataInputStream(cs.getInputStream());
            os = new DataOutputStream(cs.getOutputStream());
        }
        catch(IOException e) { }
        this.start();
    }
}

public void run() {
    try {
        String path = getPath(is);
        byte[] content = getBytes(path);
        os.writeBytes(
            "HTTP/1.0 200 OK\r\n" +
            "Content-Length: " + content.length + "\r\n" +
            "Content-Type: application/java\r\n\r\n");
        os.write(content); os.flush();
    }
    catch(IOException e) { IO.println("IO Error"); }
    catch(ClassNotFoundException e)
```

```
{ IO.println("Class not found"); }
}

private String getPath(DataInputStream is)
    throws IOException {
    String line = is.readLine(), path = "";
    if (line.startsWith("GET /")) {
        line = line.substring(5, line.length()-1).trim();
        int index = line.indexOf(".class ");
        if (index != -1) {
            path = line.substring(0, index).replace('/', '.');
        }
    }
    else {
        IO.println("Cerere GET incorecta"); System.exit(0);
    }
    do { line = is.readLine(); }
    while ((line.length() != 0) &&
        (line.charAt(0) != '\r') &&
        (line.charAt(0) != '\n'));
    return path;
}

private byte[] getBytes(String path)
    throws IOException, ClassNotFoundException {
    IO.println("S-a cerut: " + path);
    File f = new File(classpath + File.separator +
        path.replace('.', File.separatorChar) +
        ".class");
    DataInputStream dis =
        new DataInputStream( new FileInputStream(f) );
    byte[] content = new byte[ (int) f.length() ];
    dis.readFully(content); return content;
}
}
```

• Gestionarul de securitate. Specificarea permisiunilor

În lucrul cu RMI apare necesitatea ca serverul și clientul să obțină unul de la altul diferite clase. Un exemplu îl constituie stub-ul, creat pe server dar folosit de client. De asemenea este vorba de clasele ce reprezintă tipul obiectelor care apar ca parametri în invocarea la distanță a metodelor sau care apar ca valori întoarse.

Dacă lucrăm pe o singură mașină, accesul la astfel de clase poate fi asigurat folosind calea pentru cod. Dacă lucrăm pe mai multe mașini, aceste clase trebuie încărcate prin transfer de pe o mașină pe alta. *Încărcarea este posibilă numai dacă a fost instalat un gestionar de securitate și au fost precizate permisiuni de acces corespunzătoare.* Bineînțeles, în baza pentru cod trebuie specificată și locația claselor respective,

RMI posedă un gestionar de securitate (*RMISecurity Manager*) implicit, dar utilizatorul poate să-și creeze propriul gestionar, prin extinderea clasei *SecurityManager* din pachetul *java.lang*.

Instalarea gestionarului de securitate RMI implicit se realizează în mod standard prin instrucțiunea:

```
if (System.getSecurityManager() == null)
    System.setSecurityManager(new RMISecurityManager());
```

RMI are propriul sistem de securitate, care este însă foarte restrictiv. Pentru a relaxa unele drepturi de acces, Java pune la dispoziția utilizatorilor facilitatea de a preciza o mulțime de permisiuni, numită *politică (strategie) de permisiuni*. Permisuniile reprezintă drepturile de acces la resursele sistemului gazdă pe care le acordăm codului executat de pe o altă mașină. Este natural să nu acordăm drepturi depline unui astfel de cod, pentru a împiedica de exemplu modificarea nedorită de fișiere, introducerea de viruși etc.

Strategia de permisiuni este specificată prin intermediul unui *fișier de strategie*, creat fie prin utilizarea unui editor de texte, fie cu ajutorul utilitarului *policytool.exe* aflat în același director cu *java*, *javac*, *rmic* etc.; a doua modalitate elimină necesitatea de a cunoaște sintaxa ce trebuie folosită la editarea fișierului.

Numele fișierului de strategie trebuie precizat în comanda *java* prin opțiunea:

```
-Djava.security.policy=nume
```

Un fișier de strategie este format din una sau mai multe *intrări*. Fiecare intrare are forma:

```
grant baza_cod {
    lista de permisiuni
};
```

Informația *baza_cod* este opțională. Ea are forma:

```
codeBase "nume"
```

unde *nume* este o locație URL. Permisuniile din intrare sunt acordate numai codului aflat la această locație. În lipsa acestei informații, permisuniile sunt generale și nu restricționate la o anumită locație URL.

Lista de permisiuni este o succesiune de permisiuni. Fiecare dintre ele este formată dintr-un tip, o țintă și una sau mai multe acțiuni. Ne vom mărgini la a prezenta câteva exemple.

Pentru accesul la fișiere putem folosi de exemplu:

```
permission java.io.FilePermission "FILE://c:/temp", "read";
prin care acordăm dreptul de acces prin citire la fișierele din directorul c:\temp
aflat pe mașina locală. Un alt exemplu îl constituie:
permission java.io.FilePermission
    "<<ALL FILES>>", "read, write";
```

prin care acordăm dreptul de acces, atât pentru citire cât și pentru scriere, la toate fișierele aflate pe mașina locală.

Permisiunea ce acordă dreptul de acces la rețea prin intermediul socket-urilor (folosite implicit de RMI) are forma generală:

```
permission java.net.SocketPermission "adresa:port", "lista";
```

unde *adresa* este adresa simbolică sau numerică a mașinii gazdă căreia îi acordăm permisiuni, *port* este portul folosit, iar *lista* conține opțiuni separate prin virgulă. Opțiunile puse la dispoziție sunt următoarele:

```
accept connect listen
```

Opțiunea *listen* are relevanță doar dacă *adresa* este *localhost*.

În cele ce urmează vom folosi însă varianta (nerecomandată!) de a acorda cele mai largi permisiuni codului aflat "oriunde", prin următorul fișier de strategie cu numele *policy*:

```
grant {
    permission java.security.AllPermission;
};
```

Exemplul 5. Reluăm *Exemplul 2*, descriind în continuare toate acțiunile care trebuie întreprinse pe mașinile client și server pentru încărcarea stub-ului și executarea aplicațiilor.

După cum am anunțat, miniserverul de Web va funcționa pe mașina server. Vom descrie în detaliu toate acțiunile care trebuie întreprinse. Fie *IP_server* adresa IP a acestei mașini.

În metodele principale din clasele *ClientRMI* și *ServerRMI* introducem instrucțiunea ce activează gestionarul de securitate:

```
if (System.getSecurityManager() == null)
    System.setSecurityManager(new RMISecurityManager());
```

În clasa *ClientRMI.java*, invocarea metodei *lookup* se va face prin:

```
Naming.lookup("rmi://IP_server/Ob");
```

Pe mașina client, plasăm într-un director *dirclient* oarecare unitățile de compilare *InterRMI.java*, *ClientRMI.java* și *IO.java*, fișierul *policy* de mai sus, precum și fișierul *client.bat* cu următorul conținut:

```
set classpath=.
java -Djava.security.policy=policy
    -Djava.rmi.server.codebase=http://IP_server:2002/
    ClientRMI
```

unde prin opțiunea *codebase* am precizat adresa IP și portul miniserverului care va asigura încărcarea claselor necesare (în cazul nostru al stub-ului). Din acest director executăm:

```
dirclient>javac ClientRMI.java
```

Pentru un plus de claritate, pe mașina server vom crea următoarea structură de directoare:

```
C:
  RMI
  S
  S_Web
```

În directorul C:\RMI\S plasăm unitățile de compilare InterRMI.java, ServerRMI.java și IO.java, fișierul policy de mai sus, precum și fișierul server.bat cu următorul conținut:

```
set classpath=.
java -Djava.security.policy=policy
      -Djava.rmi.server.codebase=http://IP_server:2002/
      ServerRMI
```

unde informația din opțiunea codebase este folosită la înscrierea obiectului la distanță în registru. Din acest director executăm:

```
C:\RMI\S>javac ServerRMI.java
C:\RMI\S>rmic ServerRMI
```

În directorul C:\RMI\S_Web plasăm unitățile de compilare IO.java și ClassFileServer.java. Din acest director executăm:

```
C:\RMI\S_Web>javac ClassFileServer.java
```

Trecem la descrierea executării aplicației propriu-zise.

Deschidem o fereastră pe mașina server în care, dintr-un director oarecare, pornim registrul de nume:

```
D:\>set classpath=
D:\>rmiregistry
```

Deschidem o a doua fereastră pe mașina server în care, din directorul S_Web, executăm:

```
C:\RMI\S_Web>set classpath=.
C:\RMI\S_Web>java ClassFileServer 2002 /C:/RMI/S/
```

prin care miniserverul este pornit pe mașina server, așteaptă cererile la portul 2002 și va căuta clasele în directorul C:\RMI\S. Drept urmare, pe ecran va apărea mesajul:

```
ClassFileServer a pornit
```

Deschidem o a treia fereastră pe mașina server în care, din directorul S, pornim serverul:

```
C:\RMI\S>server.bat
```

Pe ecran va apărea mesajul:

```
Serverul a pornit
```

iar în fereastra din care a fost pornit miniserverul de Web vor apărea mesajele:

```
S-a cerut: ServerRMI_Stub
S-a cerut: InterRMI
```

deoarece ClassFileServer a realizat încărcarea claselor respective.

Deschidem o fereastră pe mașina client în care, din dirclient, executăm:

```
dirclient>client.bat
```

În fereastra pentru miniserver va apărea mesajul:

```
S-a cerut: ServerRMI_Stub
```

ca urmare a încărcării stub-ului pe mașina client.

Din acest moment comunicarea decurge în modul cunoscut.

15 PE SCURT DESPRE APPLET-URI

În toate capitolele anterioare, programele Java au îmbrăcat forma unor aplicații: codul sursă este transformat de un compilator (javac în JDK) în fișiere byte-code, independente de platformă; apoi o mașină virtuală Java (java în JDK) permite executarea aplicației pe mașina locală.

Dar unul dintre principalele scopuri urmărite de Java este cel de a accesa programe prin intermediul Internet-ului.

Dezvoltarea comunicațiilor prin Web a deschis calea unei variante pentru aplicații și anume *applet*-urile. Ideea de bază este următoarea: folosind un *navigator* (browser) Web, accesăm o mașină aflată la distanță pentru a încărca pe mașina locală un fișier byte-code (applet) aflat pe acea mașină, după care browser-ul execută codul respectiv pe mașina locală.

Lucrul cu applet-uri necesită cunoștințe privind atât interfețele grafice, cât și lucru în rețea, ceea ce ne-a determinat să amânăm prezentarea applet-urilor până în acest ultim capitol.

15.1. Ce sunt applet-urile?

Un *applet* este un fișier byte-code aflat pe un server de Web. Applet-urile sunt gândite pentru a fi incluse într-o aplicație și nu pentru a fi executate de sine stătător, așa cum a fost cazul tuturor programelor scrise până acum.

De pe mașina gazdă este pornit un browser compatibil Java (de exemplu *Internet Explorer* sau *Netscape Communicator*) și acesta efectuează o cerere către un server de Web printr-o referință inclusă într-un fișier *HTML*; cererea constă în încărcarea codului applet-ului de pe server. O mașină virtuală Java activă în cadrul unui browser poate încărca fișiere byte-code pentru clase ce extind clasa `java.applet.Applet`, precum și alte clase necesare acestora. Pentru aceasta este folosit un încărcător de clase asociat serverului de Web pe care rezidă applet-ul. Odată obținut codul cerut, browser-ul pornește un fir de executare care execută acest cod, folosind resurse ale mașinii locale.

Întrucât clasa `Applet` extinde clasa `Panel`, sunt preluate caracteristicile implicite ale unui panou (de exemplu gestionarul de poziționare `FlowLayout`) și sunt moștenite metodele superclaselor (inclusiv `Component` și `Container`).

• Executarea applet-urilor

Applet-urile nu sunt pornite din linia de comandă, ci în modul descris mai sus (printr-o cerere adresată unui server Web) sau local (prin intermediul utilitarului `appletviewer`, aflat în același director ce include comenzile `java` și `javac`).

Forma cea mai simplă a comenzii `appletviewer` este:
`appletviewer nume.html`
 unde `nume` este numele unui fișier HTML.

Faptul că un applet nu este pornit din linia de comandă determină unele diferențe față de aplicațiile pornite în acest mod. De exemplu metoda principală `main` nu mai are semnificația cunoscută. Un applet nu are constructori. De asemenea terminarea unui applet nu poate fi realizată prin `System.exit(0)`; explicația constă în faptul că un applet există sub forma unui fir de executare al navigatorului.

Un applet are asociată o zonă în fereastra programului de navigare. Applet-ul nu trebuie să invoce metoda `setSize`, deoarece dimensiunile ferestrei sunt determinate de navigator și de fișierul HTML.

• Despre clasa `Applet` și ciclul de viață al unui applet

Principalele metode ale clasei `Applet` sunt:

```
void init()
void start()
void stop()
void destroy()
```

Nici una dintre aceste metode nu prevede vreo acțiune; pentru specificarea de acțiuni concrete ele trebuie redefinite în applet.

După ce applet-ul a fost încărcat pe sistem, este invocată automat metoda `init`. Este necesar să redefinim această metodă în situația în care trebuie efectuate anumite inițializări, ca de exemplu crearea de fire de executare sau acțiuni care în aplicațiile de sine stătătoare sunt incluse în constructori.

Executarea continuă cu invocarea metodei `start`. Această metodă mai este invocată și de fiecare dată când pagina Web este revizitată (deci rămân valabile inițializările efectuate anterior prin metoda `init`).

Când pagina Web care conține applet-ul este înlocuită cu o altă pagină, este invocată metoda `stop`. Așa cum am menționat, revenirea la pagina Web care conține applet-ul determină reluarea executării, prin invocarea metodei `start`.

La închiderea navigatorului, este invocată mai întâi metoda `stop`, iar apoi metoda `destroy`. Această din urmă metodă poate fi folosită pentru a anula anumite acțiuni prevăzute în metoda `init`; de exemplu, dacă prin `init` este creat un fir de executare, este normal ca în metoda `destroy` să includem oprirea firului.

Metoda `stop` este gândită ca fiind complementara lui `start`, iar metoda `destroy` ca fiind complementara lui `init`.

• Fișiere HTML

Applet-urile sunt folosite de navigatoare prin referințe inserate în fișiere HTML: browser-ele Web caută fișiere HTML.

Un *fișier HTML* (*HyperText Markup Language*) este un fișier text cu extensia `.html`, ce conține *marcaje* (*tag-uri*) de formatare. Nu se face distincție între literele mari și cele mici. Majoritatea marcajelor apar în perechi de forma:
`<marcaj atribute> . . . </marcaj>`

Vom lucra cu următoarea structură restrânsă a unui fișier HTML:

```
<HTML>
  <BODY>
    corp
  </BODY>
</HTML>
```

unde am folosit indentarea numai pentru un plus de claritate.

Corpul fișierului HTML este format la rândul său din marcaje.

Ne interesează în principal marcajul `APPLET`, pentru care precizăm deocamdată forma sa și atributele sale obligatorii:

```
<APPLET CODE="clasa" WIDTH=w HEIGHT=h>
</APPLET>
```

unde `w` și `h` specifică dimensiunile ferestrei în care este afișat applet-ul, iar `clasa` este o locație URL ce specifică fie fișierul byte-code atașat applet-ului, fie un director; în al doilea caz, serverul Web consideră un fișier cu nume și extensie implicite (de obicei `index.html`).

Exemplul 1. Exemplul este pur didactic, urmărind să pună în evidență ciclul de viață al unui applet. Vom folosi comanda `appletviewer`.

Să considerăm următoarea unitate de compilare `Ex1.java`:

```
import java.awt.*; import java.applet.Applet;

public class Ex1 extends Applet {
  TextArea t = new TextArea(8,10); char c = 'a';
```

```

void delay() {
    try { Thread.sleep( (int) (5000*Math.random()) ); }
    catch(InterruptedException e) { }
}

public void init() {
    add(t); t.append("init\n"); setVisible(true); delay();
}

public void start() {
    t.append("start " + ++c + "\n"); delay();
}

public void stop() { t.append("stop\n"); delay(); }
public void destroy() { t.append("destroy\n"); delay(); }
}

```

și următorul fișier HTML cu numele Ex1.html:

```

<HTML>
<BODY>
  <APPLET CODE="Ex1.class" WIDTH=220 HEIGHT=150>
  </APPLET>
</BODY>
</HTML>

```

Presupunem, pentru simplificare, că cele două fișiere sunt situate în același director. Atunci, după ce este compilat Ex1.java, la executarea comenzii:

```
appletviewer Ex1.html
```

pe ecran vor apărea în ordine:

- imaginea din figura 1, ca urmare a încărcării applet-ului;
- imaginea din figura 2, ca urmare a executării metodei init;
- imaginea din figura 3, ca urmare a executării metodei start.

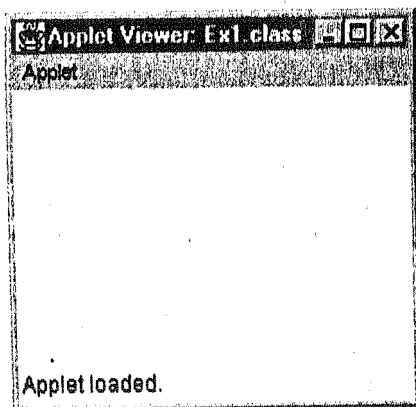


Fig. 1

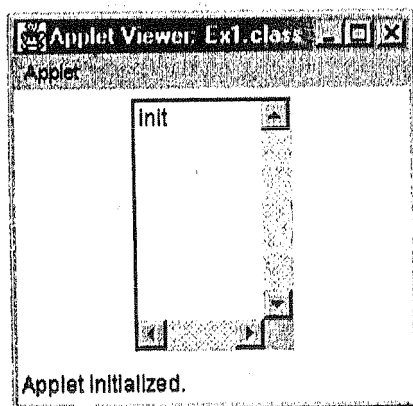


Fig. 2

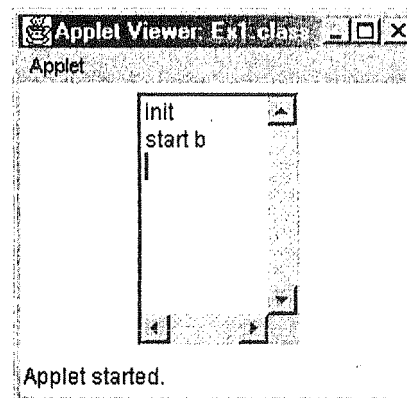


Fig. 3

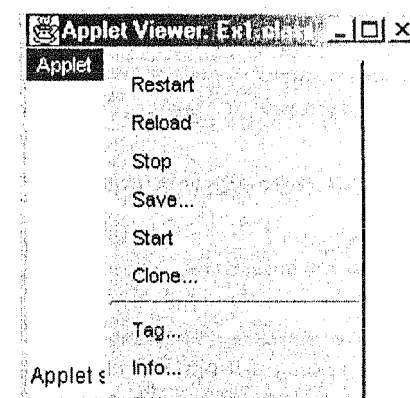


Fig. 4

Observăm că în dreapta sus apar butoanele de minimizare, maximizare și închidere obișnuite. În stânga mai apare lista de opțiuni cu alegere multiplă Applet.

Un clic pe Applet afișează opțiunile din listă, dintre care o parte apar în figura 4. Opțiunile permit repornirea, reîncărcarea, oprirea sau clonarea applet-ului, vizualizarea marcajului APPLET etc. Să considerăm de exemplu următoarea secvență de acțiuni:

- clic pe opțiunea Stop: panoul devine invizibil;
- clic pe opțiunea Start: este reafiat panoul cu aria de text, în care apar în

plus liniile:

```
stop
start c
```

- clic pe opțiunea Restart: în aria de text sunt adăugate liniile:

```
stop
destroy
init
start d
```

- clic pe opțiunea Tag: este afișat marcajul APPLET din fișierul HTML;

- clic pe opțiunea Info: este afișată lista parametrilor din marcajul APPLET al fișierul HTML; despre parametrii applet-urilor vom vorbi în subcapitolul următor.

15.2. Facilități de lucru cu applet-urile

• Tag-ul HREF din HTML

Dintr-un fișier HTML se poate face referire la un altul, inserând o legătură la acesta din urmă prin tag-ul HREF:

```
<A HREF="url"> text </A>
```

unde *url* este locația URL a unui fișier HTML. Ca efect textul *text* va apărea pe ecran marcat (subliniat). Dacă apăsăm pe acest text marcat (numit și *ancoră*), este afișat applet-ul al cărui fișier byte-code este specificat în tag-ul APPLET al fișierului HTML de la locația *url*.

• Alte attribute ale tag-ului APPLET

CODEBASE=*url*

specifică adresa URL ce reprezintă baza pentru cod, adică locațiile unde vor fi căutate fișierul byte-code al applet-ului și clasele referite de acesta;

<PARAM NAME=*nume* VALUE=*val*>

transmite applet-ului valoarea *val* pentru parametrul *nume*; *val* și *nume* sunt șiruri de caractere. Parametrul poate fi folosit de applet prin invocarea metodei *getParameter* ce va fi prezentată în continuare.

NAME=*nume*

asociază applet-ului șirul de caractere *nume*, pentru a putea fi folosit în referiri la applet, de exemplu pentru obținerea applet-ului prin invocarea metodei *getApplet* ce va fi prezentată în continuare.

Observații:

- prin utilizarea parametrilor putem interveni "în interiorul" unui applet. Altfel spus, avem posibilitatea de a *configura* applet-urile.
- necesitatea tag-ului CODEBASE rezultă din modul de pornire a unui applet: nu putem folosi comanda *java*, deci nu putem seta proprietatea *codebase*.

• Clasa Image

Clasa *Image* este superclasa tuturor claselor ce reprezintă imagini grafice (în mod standard de tipurile *.gif* și *.jpeg*). Ea apare în pachetul *java.awt* și este declarată prin:

```
public abstract class Image extends Object
```

• Interfața AudioClip

Această interfață din pachetul *java.applet* este o abstractizarea pentru reproducerea fișierelor audio (în mod standard de tipul *.au*) și este declarată prin:

```
public interface AudioClip
    public void start()
        pornește reproducerea sonoră a fișierului audio;
    public void loop()
        pornește reproducerea sonoră repetată a fișierului audio;
    public void stop()
        oprește reproducerea sonoră a fișierului audio.
```

• Interfața AppletContext

Interfața *AppletContext* din pachetul *java.applet* corespunde mediului applet-ului, format din applet împreună cu celelalte applet-uri din același fișier HTML. Dintre metode menționăm următoarele:

```
public Applet getApplet(String nume)
    întoarce applet-ul cu numele nume (vezi atributul NAME al tag-ului APPLET)
    din mediul applet-ului;
public void showDocument(URL url)
    înlocuiește pagina Web curentă cu cea specificată de url.
```

• Alte metode ale clasei Applet

```
URL getDocumentBase()
    întoarce locația URL a directorului în care apare fișierul HTML ce include
    applet-ul;
URL getCodebase()
    întoarce baza pentru cod a fișierului byte-code ce reprezintă applet-ul;
String getParameter(String nume)
    întoarce valoarea parametrului nume din atributul PARAM al tag-ului
    APPLET din fișierul HTML;
void resize(int w, int h)
    resetează dimensiunile zonei asociate applet-ului în fereastra browser-ului;
Image getImage(URL url, String nume)
    întoarce un obiect de tipul Image; imaginea respectivă poate fi afișată pe
    ecran; url și nume reprezintă locația URL, respectiv numele obiectului;
AudioClip getAudioClip(URL url, String nume)
    întoarce un obiect de tipul AudioClip (un fișier audio), ce poate fi reprodus
    din cadrul applet-ului; url și nume reprezintă locația URL, respectiv numele
    obiectului;
AppletContext getAppletContext()
    întoarce contextul applet-ului.
```

Exemplul 2. Reluăm *Exemplul 3* din capitolul 11, transformându-l într-un applet.

Reamintim că există două modalități de pornire a unui applet: utilizarea unui browser sau folosirea comenzii *appletviewer*. Bineînțeles, în prealabil este necesar ca, prin compilare, să obținem fișierul byte-code (cu extensia *.class*).

De această dată alegem ca pornirea applet-ului să se facă prin intermediul unui browser. Pentru exemplificare am folosit Internet Explorer. Drept adresă am specificat numele fișierului HTML (de exemplu *C:\Ex2.html*), al cărui conținut este:

```
<HTML>
<BODY>
  <APPLET CODE="Ex2.class" CODEBASE="d:/Ed_Tehn/15"
    WIDTH=300 HEIGHT=60>
  </APPLET>
</BODY>
</HTML>
```

în care atributele CODE și CODEBASE permit identificarea fișierului byte-code ca fiind D:\Ed_Tehn\15\Ex2.class.

Unitatea de compilare Ex2.java are următorul conținut:

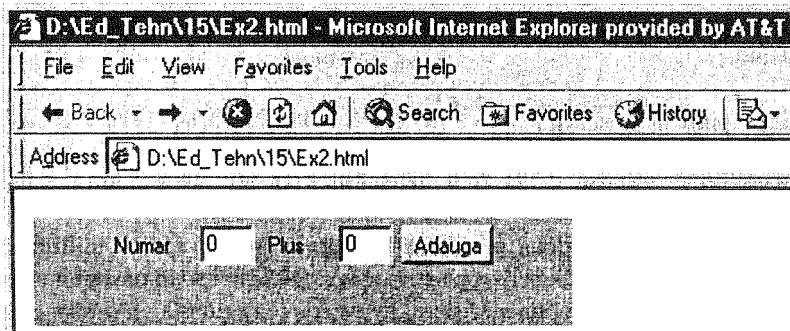
```
import java.awt.*; import java.awt.event.*;
import java.applet.Applet;

public class Ex2 extends Applet implements ActionListener {
    Label L1,L2; TextField T1,T2; Button B;
    int c1,c2;

    public void init() {
        L1 = new Label("Numar"); add(L1);
        T1 = new TextField("0"); add(T1);
        L2 = new Label("Plus "); add(L2);
        T2 = new TextField("0"); add(T2);
        B = new Button("Adauga"); add(B);
        B.addActionListener(this); setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        c2 = Integer.parseInt( T2.getText() );
        c1 += c2; T1.setText( Integer.toString(c1) );
    }
}
```

O porțiune din imaginea ce va apărea pe ecran este următoarea:



și vom putea acționa în modul cunoscut asupra butonului și câmpurilor de text.

15.3. Lucrul cu imagini și cu fișiere audio

Facilitățile Java pentru lucrul cu imagini și fișiere audio au fost prezentate în subcapitolul anterior. Ne propunem acum să le exemplificăm și să prezentăm și modalitatea prin care putem simula o animație prin afișarea unei succesiuni de imagini.

După cum știm, activitățile prevăzute în applet sunt executate în cadrul unui fir de executare al browser-ului. Pentru realizarea animației este recomandat să lăsăm acest fir să execute activitățile standard și să pornim un fir de executare distinct pentru afișarea succesivă a imaginilor.

Exemplul 3. Realizarea unei animații.

Vom afișa în mod repetat o succesiune de imagini. Numărul nrimag de imagini din secvență este comunicat applet-ului prin intermediul unui parametru inclus în fișierul HTML.

Imaginile sunt constituite fișierele cu numele fn0.gif, ..., fn14.gif, aflate în directorul curent (care conține applet-ul).

Pentru prelucrarea imaginilor și sunetelor este creat și pornit un fir de executare separat. Începutul și sfârșitul executării applet-ului sunt marcate de reproducerea fișierelor audio beep.au, respectiv ding.au, aflate în subdirectorul audio al directorului curent.

Imaginile sunt afișate folosind metodele paint și repaint, moștenite de la clasa Component.

Unitatea de compilare Ex3.java are forma:

```
import java.awt.*; import java.applet.Applet;
import java.net.*;

public class Ex3 extends Applet implements Runnable {
    int nrimag; int i;
    Image[] imag; Thread fir = null;

    public void init() {
        nrimag = Integer.parseInt( getParameter("nrimag") );
        imag = new Image[nrimag];
        for (int i = 0; i<nrimag; i++) {
            imag[i] = getImage(getCodeBase(),"fn"+i+".gif");
        }
        fir = new Thread(this);
    }

    public void paint(Graphics g) {
        g.drawImage(imag[i],0,0,this);
    }

    public void start() { fir.start(); }

    public void destroy() { fir.destroy(); }
```



```

public void run() {
    play(getCodeBase(), "audio/beep.au");
    for (int j=0; j<30; j++) {
        repaint();
        try { Thread.sleep(500); }
        catch (InterruptedException e) { }
        i = (i+1) % nrimag;
    }
    play(getCodeBase(), "audio/ding.au");
}
}

```

Conținutul fișierului HTML este următorul:

```

<HTML>
<BODY>
  <APPLET CODE="Ex3.class" WIDTH=350 HEIGHT=200>
    <PARAM name="nrimag" value="15">
  </APPLET>
</BODY>
</HTML>

```

15.4. Comunicarea între applet-uri

• Schimbul de informații între applet-uri din aceeași pagină Web

Problema generală a schimbului de informații între două applet-uri constituie un subiect delicat, datorită restricțiilor de securitate impuse lucrului cu applet-urile, restricții despre care vom vorbi în ultimul subcapitol.

În acest paragraf vom considera numai cadrul restrictiv în care applet-urile se află pe aceeași pagină Web și aceeași mașină gazdă.

O primă modalitate prin care două applet-uri își pot transmite informații constă în folosirea câmpurilor statice ale unei clase la care ambele au acces.

O a doua modalitate prin care putem realiza schimbul de informații constă în a utiliza facilitățile puse la dispoziție de interfața AppletContext și de clasa Applet. Ele permit ca dintr-un applet să obținem o referință la un alt applet și, pe baza acestei referințe, să invocăm o metodă a acestuia.

Exemplul următor ilustrează ambele modalități prezentate mai sus.

Exemplul 4. Doi clienți ai unei bănci fac tranzacții (depuneri și extrageri) cu acea bancă; valoarea din bancă nu poate deveni negativă. În plus fiecare îi poate cere celuilalt o sumă de bani cu restricția că, indiferent de suma cerută, cel care cere primește cel mult jumătate din suma de care dispune celălalt.

Suma disponibilă în bancă corespunde câmpul static al următoarei clase:

```

class Banca {
    static int val = 100;
}

```

Unitatea de compilare Client1.java corespunde primului client și are conținutul:

```

import java.awt.*; import java.applet.*;
import java.awt.event.*;
public class Client1 extends Applet
    implements ActionListener {
    Label L1 = new Label("Suma");
    TextField T1 = new TextField("50");
    Label L2 = new Label("Plus");
    TextField T2 = new TextField("0");
    Button B1 = new Button("Cere");
    Button B2 = new Button("Imprumuta");
    int suma=50, cerere, val; Client2 Ap;

    public void init() {
        add(L1); add(T1); add(L2); add(T2); add(B1); add(B2);
        setVisible(true);
    }

    public void start() {
        AppletContext AC = getAppletContext();
        Ap = (Client2) AC.getApplet("Client2");
        B1.addActionListener(this); B2.addActionListener(this);
    }

    public void actionPerformed(ActionEvent e) {
        cerere = Integer.parseInt(T2.getText());
        if( e.getActionCommand().equals("Cere") ) {
            if(Banca.val>=cerere) {
                Banca.val -= cerere; suma += cerere;
                T1.setText( Integer.toString(suma) );
            }
        }
        else {
            suma += Ap.imprumut(cerere);
            T1.setText( Integer.toString(suma) );
        }
    }

    public int imprumut(int i) {
        if(i<=0) return 0;
        int k = suma/2; if(i<k) k=i;
        suma -= k; T1.setText( Integer.toString(suma) );
        return k;
    }
}

```

Prin instrucțiunile:

```
AppletContext AC = getAppletContext();
Ap = (Client2) AC.getApplet("Client2");
```

primul client obține în Ap o referință la applet-ul corespunzător celui de al doilea client. Pe baza acestei referințe, el poate invoca (prin apăsarea butonului "Imprumuta") metoda imprumut și pe baza sumei obținute de la al doilea client efectuează actualizările corespunzătoare.

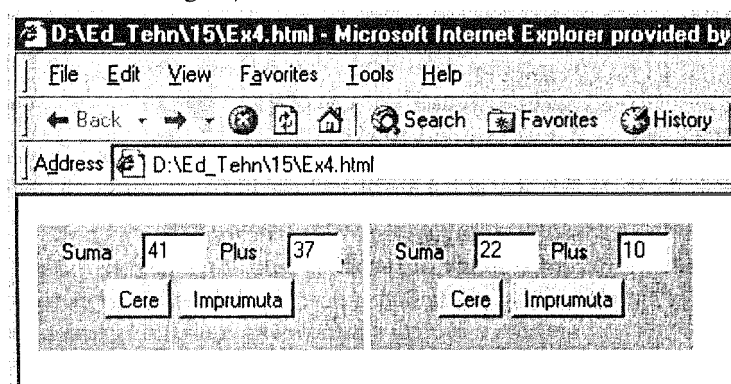
Prin apăsarea butonului "Cere", primul client realizează tranzacția dorită cu banca.

Unitatea de compilare Client2.java se obține din Client1.java prin interschimbarea în textul sursă a cuvintelor Client1 și Client2.

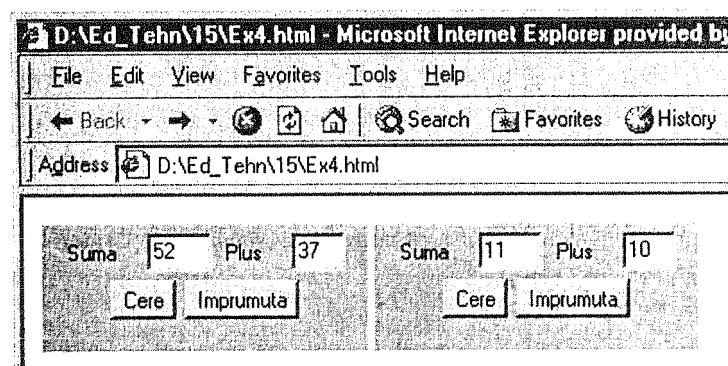
Conținutul fișierului HTML este următorul:

```
<HTML>
<BODY>
  <APPLET CODE="Client1.class" WIDTH=180 HEIGHT=70
    NAME="Client1">
  </APPLET>
  <APPLET CODE="Client2.class" WIDTH=180 HEIGHT=70
    NAME="Client2">
  </APPLET>
</BODY>
</HTML>
```

La executare, după diferite acțiuni ale celor doi clienți, putem ajunge la următoarea configurație:



Dacă în această situație primul client apasă pe butonul "Imprumuta", se va trece la configurația:



• Conectarea la o altă pagină Web

În acest paragraf prezentăm două modalități prin care dintr-o pagină Web putem activa și afișa o altă pagină Web:

- utilizarea în codul sursă a URL-ului celei de a doua pagini;
- folosirea marcatului HREF, indicând spre a doua pagină, în fișierul HTTP folosit pentru afișarea primei pagini.

Întrucât facilitățile Java necesare au fost prezentate anterior, trecem direct la prezentarea unui exemplu care ilustrează ambele modalități.

Exemplul 5.

În directorul D:\Ed_Tehn\15\1 apar unitatea de compilare C1.java:

```
import java.awt.*; import java.awt.event.*;
import java.applet.*; import java.net.*;
public class C1 extends Applet implements ActionListener {
    Label L; Button B; URL url; AppletContext AC;

    public void init() {
        L = new Label("Pagina 1"); add(L);
        B = new Button("Pag. 2"); add(B);
        B.addActionListener(this); AC = getAppletContext();
        try {
            url = new URL("FILE:///D:Ed_Tehn/15/2/2.html");
        }
        catch(MalformedURLException e) { }
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        AC.showDocument(url);
    }
}
```

și fișierul 1.html:

```
<HTML>
  <BODY>
    <APPLET CODE="C1.class" WIDTH=100 HEIGHT=70>
  </APPLET>
</BODY>
</HTML>
```

În directorul D:\Ed_Tehn\15\2 apar unitatea de compilare C2.java:

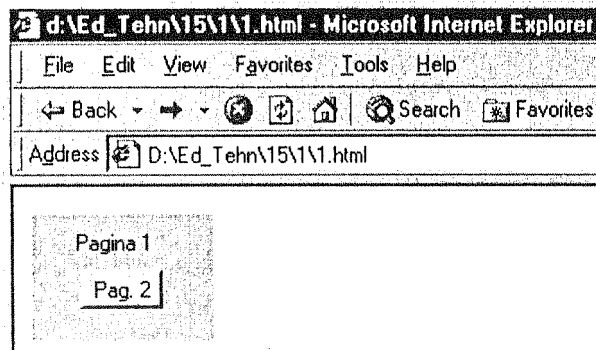
```
import java.awt.*; import java.applet.*;
public class C2 extends Applet {
  Label L;

  public void init() {
    L = new Label("Pagina 2"); add(L);
    setVisible(true);
  }
}
```

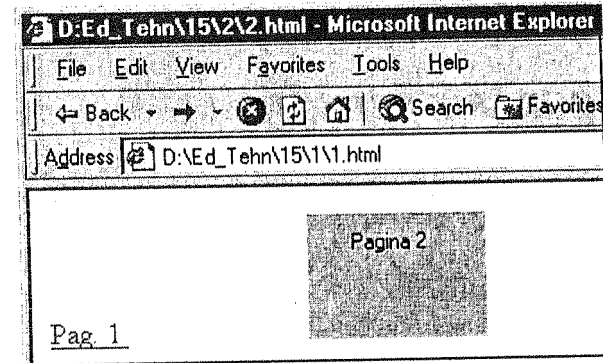
și fișierul 2.html:

```
<HTML>
  <BODY>
    <A HREF="d:/Ed_Tehn/15/1/1.html"> Pag. 1 </A>
    <APPLET CODE="C2.class" WIDTH=100 HEIGHT=70 HSPACE=100>
  </APPLET>
</BODY>
</HTML>
```

După pornirea browser-ului și indicarea adresei fișierului 1.html, applet-ul corespunzător clasei C1.class este pornit și pe ecran va apărea:



Dacă apăsăm pe butonul "Pag. 2", va fi încărcat applet-ul corespunzător clasei C2 și pe ecran va apărea:



Revenirea la pagina anterioară se poate realiza apăsând pe ancora "Pag. 1" sau pe butonul "Back" al browser-ului.

15.5. Restricții în lucrul cu applet-uri

Lucrul cu applet-uri este supus mai multor restricții. Fiecare browser implementează o serie de politici de securitate, care pot diferi de la browser la browser. Prin executarea pe mașina locală a unui applet al cărui cod este în definitiv necunoscut și eventual scris de persoane răuvoitoare, se pot introduce viruși sau distruge fișiere.

Din motive de securitate, applet-urile nu pot efectua în general acțiuni ca:

- accesarea (pentru scriere, citire și ștergere) a fișierelor de pe mașina client;
- realizarea de conexiuni în rețea, cu excepția conexiunilor cu serverul unde se află applet-ul;
- pornirea altor programe pe mașina client;
- conectarea în rețea la altă mașină decât serverul.

Nu ne propunem prezentarea tuturor restricțiilor și nici a modalităților prin care unele dintre ele pot fi înlăturate. Pentru mai multe amănunte recomandăm cititorului să consulte documentația de firmă.

BIBLIOGRAFIE SELECTIVĂ

- [1] Arnold K., Gosling J. : *The Java Programming Language*, Addison-Wesley, 1996
- [2] Athanasiu Irina, Costinescu B., Drăgoi O.A., Popovici F.I. : *Limbajul Java. O perspectivă pragmatică*, Computer Libris Agora, 1998
- [3] Budd T. : *Understanding Object-oriented Programming with Java*, Addison-Wesley, 1998
- [4] Eckel B. : *Thinking in Java*, Prentice-Hall, 2001
- [5] Georgescu H. : *Programare concurentă. Teorie și aplicații*, Editura Tehnică, 1996
- [6] Gosling J., Joy B., Steele G. : *The Java Language Specification*, Addison-Wesley, 1996
- [7] Hartley S.J. : *Concurrent Programming*, Oxford University Press, 1998
- [8] Hunt J. : *Java and Object Orientation*, Springer, 1998
- [9] Jurcă I. : *Programarea rețelelor de calculatoare*, Editura de Vest, 2000
- [10] Lea D. : *Concurrent Programming in Java*, Addison Wesley, 1996
- [11] Lee L. : *Java: Data Structures and Programming*, Springer, 1998
- [12] Mahmoud Q.H. : *Distributed Programming with Java*, Manning, 2000
- [13] Oaks, S. : *Java Threads*, O'Reilly, 1997
- [14] Poo, D., Kiong D. : *Object-oriented Programming and Java*, Springer, 1998
- [15] Walnum, C. : *Java by Example*, Que, 1996
- [16] Winston, P.H., Narasimhan S. : *On to Java*, Addison-Wesley, 1998

INDEX

A

acces, 60
 adresă numerică, 247
 adresă simbolică, 247
 analiză lexicală, 187
 apeluri inverse, 273
 aplicație, 13
 applet, 13, 283
 ciclul de viață, 284
 comunicare între applet-uri, 292
 conectare la o pagină Web, 295
 context, 290
 restricții, 297
 Applet, 289
 appletviewer, 284
 arbori
 de sortare, 40
 parcurs pe niveluri, 93
 arie de text, 201, 211
 aritmetica numerelor mari, 112
 ascunderea câmpurilor, 67
 atribut, 285, 288
 AWT, 194
 AWTEvent, 199

B

bară de defilare, 201, 218
 baza pentru cod (codebase), 276
 bloc, 60
 bloc de inițializare, 49, 67

browser, 283
 buffer, 171, 172, 182
 BufferedInputStream, 172
 Button, 201, 210
 byte-code, 12, 283

C

cadru, 196, 209
 calea pentru cod (classpath), 58
 Canvas, 222
 caractere, 27
 catch, 100
 câmp de text, 197
 câmpuri, 17
 căsuță de opțiuni, 201, 214
 chat, 253
 Checkbox, 214
 Choice, 216
 clase, 12, 17
 abstracte, 80
 anonime, 95
 extinse, 65
 interne, 92, 96
 înfășurătoare, 110
 lucrul cu clase, 128
 Class, 128
 Client - Server (modelul), 246, 263
 clonare, 89
 colectorul de reziduuri, 24, 128, 271
 Collections, 126
 comentarii, 29

compilare,
 javac, 15, 58
 rmic, 268
 Component, 196, 208
 componente elementare, 200
 constante, 50
 constructori, 17, 67
 Container, 196, 208
 conversii, 24, 106, 117
 culori, 208
 cuvinte cheie, 28

D

DataInputStream, 163, 170
 demon, 159
 Dialog, 200, 220

E

enumerare, 120
 etichete, 38
 etichete grafice, 197, 201, 210
 evenimente, 201, 230
 excepții, 13, 96
 Exception, 105
 excludere reciprocă, 147
 executare (java), 15, 58
 extinderea,
 claselor, 65, 240
 interfețelor, 83

F

ferestre, 196
 de defilare, 200, 220
 de dialog, 200, 220
 File, 192
 FileInputStream, 163, 169
 fire de executare, 134, 283
 creare, 135
 demon, 159
 grup de fire, 159

 stări, 135
 fișiere,
 byte-code, 283
 cu acces direct, 190
 de strategie, 280
 de imagini, 288, 291
 de sunet, 291
 HTML, 283, 285
 FlowLayout, 201, 225
 fluxuri,
 de intrare/ieșire, 160
 la nivel de caracter, 178
 la nivel de octet, 163
 suprapunere de fluxuri, 160
 font, 207
 funcții matematice, 109

G

generarea (n,k)-combinărilor, 51
 gestionari,
 de evenimente, 199, 202, 203
 de poziționare, 197, 201, 225, 284
 de securitate, 279
 graf,
 asociat unui obiect, 236
 parcursere în adâncime, 46
 parcursere pe niveluri, 122
 GridBagLayout, 201, 227
 GridLayout, 201, 226

H

HTML (limbaj), 283, 285
 HTTP (protocol), 254

I

identificatori, 28
 implementarea interfețelor, 85, 239
 import, 14, 55
 InetAddress, 247

inițializarea,
 claselor, 50
 interfețelor, 86
 InputStream, 162, 271
 instanceof, 108
 instrucțiuni, 36
 interfață, 81
 interfață grafică, 194
 interpretor, 12
 invocarea,
 metodelor, constructorilor, 17
 la distanță (RMI), 263
 IO (intrări/ieșiri standard), 15

I

încărcarea dinamică a claselor, 276
 încărcător de clase, 283

L

Label, 210
 legare dinamică, 75
 literali, 28
 List, 217
 listă înlănțuită, 22
 listă de opțiuni,
 cu alegere unică, 201, 216
 cu alegere multiplă, 201, 217
 locație URL, 251, 277

M

Math, 109
 marcaj APPLET, 285
 mașina virtuală Java, 12
 membru al unei entități, 76
 Method, 128
 metoda arborelui binar, 141
 metoda principală main, 14
 metode, 14, 17
 abstracte, 80
 statice, 20, 63

modelul aleator, 144
 modificatori, 14, 30, 62, 76
 modificatori de acces, 63, 77
 monitor, 148
 moștenire, 66, 67
 moștenire multiplă, 67, 81, 87

N

Naming, 270
 notify, notifyAll, 150
 Number, 112

O

obiect, 12
 obiect la distanță, 263
 ObjectInputStream, 238
 operatori, 32
 precedență, 35
 asociativitate, 35
 organizări ale datelor, 119
 OutputStream, 162

P

pachet Internet, 246
 pachete, 16, 55
 package, 55
 Panel, 196, 210, 284
 parametri în applet-uri, 288
 parametrii metodelor și
 constructorilor, 61
 permisiuni, 280
 PipedInputStream, 171, 175
 polimorfism, 70, 74
 port, 247
 PrintStream, 177
 problema
 filozofilor chinezi, 156
 grădinilor ornamentale, 146
 Producător-Consumator, 146,
 151, 176

proces, 134
programare concurentă, 144
programare distribuită, 244
programare orientată pe obiecte, 24
programare paralelă, 139
proprietăți ale sistemului, 271, 277
protocol, 246

R

Random, 110
RandomAccessFile, 190
recursivitate, 39
Reader, 162
redefinirea metodelor, 67
referințe, 18
registrul de nume RMI, 266
rețele, 244
RMI, 266

S

Scrollbar, 218
semafoare, 153
semnarea digitală a mesajelor, 259
semnificația unui nume, 64
serializare, 233
server HTTP, 255, 277
ServerSocket, 251
signatură, 16
sincronizare, 138
skeleton, 266
Socket, 249, 251
spații albe, 27
stive, 103, 124, 268
StreamTokenizer, 187
String, 113
StringTokenizer, 125
stub, 266
subclasă, 64
super, 64, 67, 73
superclasă, 64, 241
suprafețe de desenare, 201, 222

System, 132

Ș

șiruri de caractere, 113

T

tablouri, 43
 prelucrări ale tablourilor, 117
tag-ul APPLET, 285
terminator de linie, 27
TextArea, 211
TextField, 211
this, 51, 63
Thread, 135
throw, throws, 99
tip declarat, 67
tip real, 67
tipuri primitive, 31
try, 100

U

unitate de compilare 14, 55
URL, 251
UnicastRemoteObject, 270

V

variabile, 18, 30
variabile locale, 60
Vector, 120
vizibilitate, 60

W

wait, 250
Writer, 162
Window, 196